

Antmicro

Kenning

2026-11-10

CONTENTS

1 Introduction **1**

2 Kenning **2**
2.1 Introduction . . . 3
2.2 Kenning installation . . . 3
2.3 Kenning structure . . . 5
2.4 Kenning usage . . . 7
2.5 Using Kenning as a library in Python scripts . . . 15
2.6 Adding new implementations . . . 16

3 Deep Learning deployment stack **18**
3.1 From training to deployment . . . 18
3.2 Dataset preparation . . . 18
3.3 Model preparation and training . . . 19
3.4 Model optimization . . . 20
3.5 Model compilation and deployment . . . 20

4 Defining optimization pipelines in Kenning **21**
4.1 JSON specification . . . 21
4.2 Model evaluation using its native framework . . . 22
4.3 Model training . . . 24
4.4 Optimizing and running a model on a single device . . . 25
4.5 Compiling a model and running it remotely . . . 27

5 Using Kenning via command line arguments **30**
5.1 Command-line arguments for classes . . . 30
5.2 Autocompletion for command line interface . . . 30
5.3 Model training . . . 30
5.4 In-framework inference performance measurements . . . 32
5.5 Testing inference on target hardware . . . 33
5.6 Running inference . . . 38
5.7 Probing . . . 39
5.8 Generating performance reports . . . 41
5.9 Displaying information about available classes . . . 41

6 Kenning gallery of use cases **43**
6.1 Generating anomaly detection models for the MAX32690 Evaluation Kit with AutoML . . . 43

6.2 Anomaly detection model training and deployment on the MAX32690 Evaluation Kit . . . 49
6.3 Evaluation and optimization of DINOv2 for depth estimation . . . 58
6.4 Displaying information about available classes . . . 66
6.5 Using Kenning with ROS 2 for evaluation, optimization and deployment . . . 71
6.6 Evaluating models on hardware using Kenning Zephyr Runtime . . . 75
6.7 Large Language Models with Safetensors . . . 79
6.8 Visualizing Kenning data flows with Pipeline Manager . . . 82
6.9 Structured pruning for PyTorch models . . . 84
6.10 Model quantization and compilation using LiteRT and TVM . . . 88
6.11 Unstructured Pruning of TensorFlow Models . . . 99
6.12 Optimizing and comparing an instance segmentation model . . . 104
6.13 Transfer Learning and Optimization for Instance Segmentation Model - YOLACT 114

7 Kenning environment variables 122
7.1 KENNING_CACHE_DIR . . . 122
7.2 KENNING_MAX_CACHE_SIZE . . . 122
7.3 KENNING_USE_DEFAULT_EXCEPTHOOK . . . 122
7.4 KENNING_DOCS_VENV . . . 122
7.5 KENNING_ENABLE_ALL_LOGS . . . 122
7.6 KENNING_DISABLE_IO_VALIDATION . . . 123

8 Kenning measurements 124
8.1 Performance metrics . . . 124

9 Choosing optimal optimization pipeline 126
9.1 Optimization config specification . . . 126
9.2 Output details . . . 128

10 Sample autogenerated report 130
10.1 Pet Dataset classification using TVM-compiled TensorFlow model . . . 130
10.2 Inference performance metrics for build.local-cpu-tvm-tensorflow-classification.json . . . 133
10.3 Inference quality metrics for build.local-cpu-tvm-tensorflow-classification.json . . . 136

11 Sample AutoML report 138
11.1 AutoML statistics . . . 138
11.2 Classification comparison . . . 140
11.3 Inference comparison . . . 147

12 Sample RISC-V Zephyr Tracing Report 148
12.1 Sample RISC-V Zephyr Tracing Report . . . 148

13 Creating applications with Kenning 179
13.1 JSON structure . . . 179
13.2 KenningFlow execution . . . 182
13.3 Implemented Runners . . . 182

14 Kenning Zephyr Runtime 183
14.1 Usage . . . 184
14.2 General code structure . . . 194

15 Developing Kenning blocks 198

15.1 Model and I/O metadata . . . 198
15.2 Implementing a new Kenning component . . . 201
15.3 Implementing Kenning runtime blocks . . . 212
15.4 Adjusting ModelWrapper for AutoML flow . . . 213

16 Kenning resources **219**
16.1 Accessing resources . . . 219
16.2 Caching . . . 220
16.3 Custom URI schemes . . . 220
16.4 CLI commands . . . 221

17 Kenning protocols **223**
17.1 BytesBasedProtocol . . . 224
17.2 KenningProtocol . . . 235
17.3 NetworkProtocol . . . 245
17.4 UARTProtocol . . . 247
17.5 ROS2Protocol . . . 248

18 Kenning platforms **254**
18.1 Specification . . . 254
18.2 Generating definitions for boards supported by Zephyr RTOS . . . 258

19 Integration with ROS 2 **260**
19.1 Dependencies . . . 260
19.2 Installation . . . 260
19.3 Running the container . . . 261
19.4 Running Kenning together with ROS 2 . . . 261
19.5 Setting Kenning parameters . . . 262
19.6 Summary . . . 263

20 Kenning API **264**
20.1 Deployment API overview . . . 264
20.2 KenningFlow . . . 266
20.3 Runner . . . 267
20.4 Dataset . . . 268
20.5 ModelWrapper . . . 275
20.6 Optimizer . . . 281
20.7 Runtime . . . 286
20.8 Platform . . . 292
20.9 AutoML . . . 293
20.10 Protocol . . . 296
20.11 DataConverter . . . 305
20.12 Measurements . . . 306
20.13 DataProvider . . . 312
20.14 OutputCollector . . . 313
20.15 ArgumentsHandler . . . 314
20.16 ResourceManager . . . 319
20.17 ResourceURI . . . 321
20.18 ModelConverter . . . 322

Python Module Index **325**

Index **326**

CHAPTER

ONE

INTRODUCTION

Kenning provides an API for deploying deep learning applications on edge devices using various model training and compilation frameworks.

This documentation consists of the following chapters:

- *Kenning* section provides a project description, installation steps and a quick start guide,
- *Deep Learning deployment stack* section describes a typical model deployment flow on edge devices,
- *Defining optimization pipelines in Kenning* section describes a way to create advanced optimization scenarios with JSON config,
- *Using Kenning via command-line arguments* section describes executable scripts available in Kenning,
- *Kenning gallery* contains use cases with models optimization and integration with ROS2, Renode and Pipeline Manager,
- *Kenning environment variables* section describes specific variables which can influence how the Kenning works,
- *Kenning measurements* section describes data gathered during the compilation and evaluation process,
- *Choosing optimal optimization pipeline* section describes a way to run multiple pipelines and search for the one that achieves the best result,
- *Sample autogenerated report* section provides a sample report generated using Kenning,
- *Kenning Zephyr Runtime* section describes a Zephyr RTOS application that can be used to evaluate models on embedded platforms,
- *Sample RISC-V Zephyr Tracing Report* section provides a sample report with detailed runtime analysis for a model executed on a HiFive Unmatched board simulated in Renode using Kenning Zephyr Runtime and Zephelin libraries,
- *Creating applications with Kenning* section describes KenningFlow and its usage,
- *Developing Kenning blocks* section describes a way to develop new Kenning components,
- *Kenning resources* section describes how Kenning deals with its resources (pretrained models, datasets etc.),
- *ROS 2 Integration* section describes how Kenning can be used together with ROS 2 nodes,
- *Kenning API* section provides an in-depth description of the Kenning API.

CHAPTER

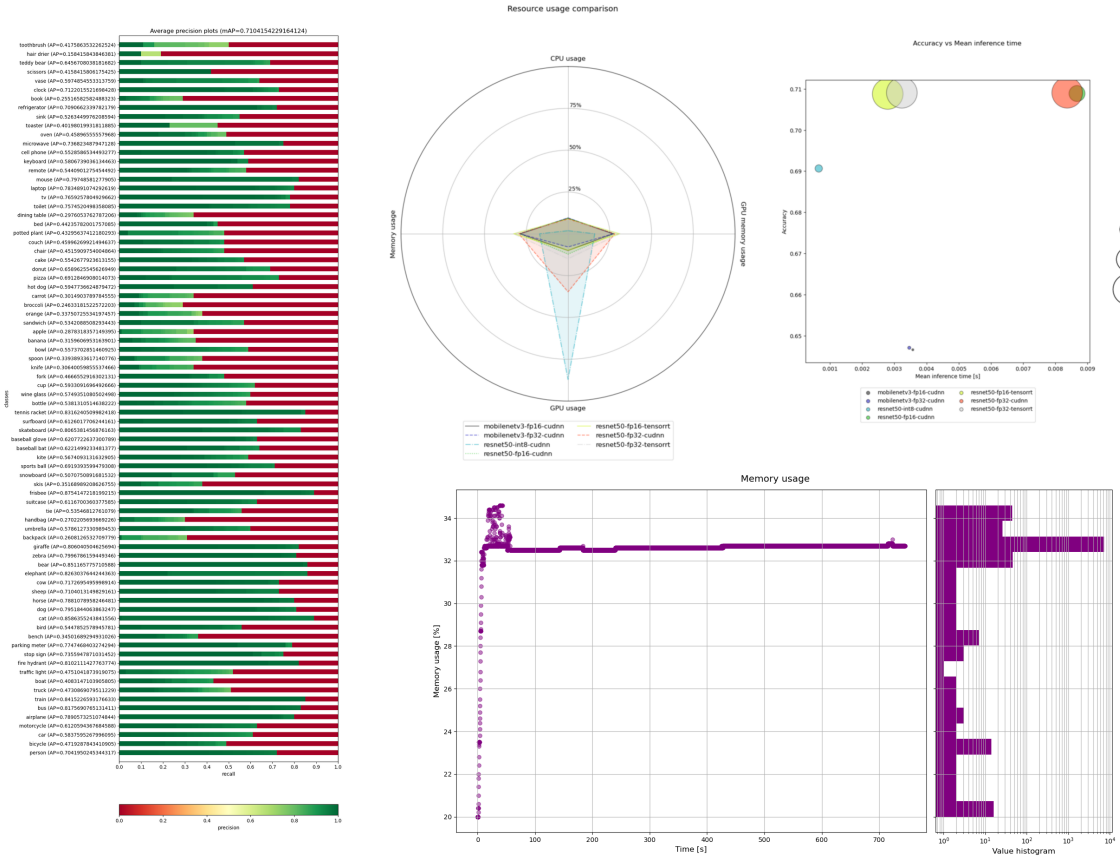
TWO

KENNING

Copyright (c) 2020-2026 Antmicro

Kenning is a framework for creating deployment flows and runtimes for Deep Neural Network applications on various target hardware.

Website | Documentation | Core API | Tutorials



Contents:

- *Introduction*
- *Kenning installation*
- *Kenning structure*
- *Kenning usage*
- *Using Kenning as a library in Python scripts*
- *Adding new implementations*

2.1 Introduction

Kenning aims towards providing modular execution blocks for:

- dataset management,
- model training,
- model optimization and compilation for a given target hardware,
- running models using efficient runtimes on target device,
- model evaluation and performance reports.

These can be used seamlessly regardless of underlying frameworks for the above-mentioned steps.

Kenning's aim is not to bring yet another training or compilation framework for deep learning models - there are lots of mature and versatile frameworks that support certain models, training routines, optimization techniques, hardware platforms and other components crucial to the deployment flow. Still, there is no framework that would support all of the models or target hardware devices - especially the support matrix between compilation frameworks and target hardware is extremely sparse. This means that any change in the application, especially in hardware, may end up in a necessity to change the entirety or a significant part of the application flow.

Kenning addresses this issue by providing a unified API that focuses on deployment tasks rather than their implementation - the developer decides which implementation should be used for each task, and with Kenning, it is possible to do in a seamless way. This way, switching to another target platform results, in most cases, in a very small change in the code, instead of reimplementing larger parts of a project. This is how Kenning can get the most out of the existing Deep Neural Network training and compilation frameworks.

Seamless nature of Kenning also allows developers to quickly evaluate the model on various stages of optimizations and compare them as shown in *Kenning usage*.

2.2 Kenning installation

2.2.1 Module installation with pip

`NOTE`: Kenning supports Python from 3.11 to 3.13.

To install Kenning with its basic dependencies with `pip`, run:

```
uv pip install -U git+https://github.com/antmicro/kenning.git
```

Since Kenning can support various frameworks, and not all of them are required for users' particular use cases, some of the requirements are optional. We can distinguish the following groups of extra requirements:

- `tensorflow` - modules for work with TensorFlow models (ONNX conversions, addons, and TensorFlow framework),
- `torch` - modules for work with PyTorch models,
- `nvidia_perf` - modules for performance measurements for NVIDIA GPUs,
- `object_detection` - modules for work with YOLOv3 object detection and the Open Images Dataset V6 computer vision dataset,
- `speech_to_text` - modules for work with audio samples and speech-to-text models,
- `iree` - modules for IREE compilation and runtime,
- `tvm` - modules for Apache TVM compilation and runtime (Relay-based TVM 0.18, for other TVM variants see *Dependency groups*),
- `onnxruntime` - modules for ONNX Runtime with CPU support,
- `onnxruntime_gpu` - modules for ONNX Runtime with GPU support,
- `nni` - modules for Neural Network Intelligence optimizers,
- `docs` - modules for generating documentation,
- `test` - modules for Kenning framework testing,
- `real_time_visualization` - modules for real time visualization runners,
- `pipeline_manager` - modules for communication with visual editor,
- `reports` - modules for generating reports and comparisons,
- `uart` - modules for work with serial ports,
- `renode` - modules for work with Renode,
- `fuzzy` - modules for fuzzy search,
- `llm` - modules for llm optimization,
- `sparsity-aware-kernel` - modules for SparseGPT.

To install the extra requirements, e.g. `tensorflow`, run:

```
sudo pip install git+https://github.com/antmicro/kenning.git
↪#egg=kenning[tensorflow]
```

or, in newer `pip` releases:

```
pip install "kenning[tensorflow] @ git+https://github.com/antmicro/kenning.git"
```

or, with `uv`:

```
uv pip install "kenning[tensorflow] @ git+https://github.com/antmicro/kenning.git"
```

2.2.2 Working directly with the repository

For development purposes, and to use additional resources (such as sample scripts), clone the repository with:

```
git clone https://github.com/antmicro/kenning.git
cd kenning/
```

Then install using:

```
uv sync --extra tensorflow,litert,tvm,reports
```

2.2.3 Dependency groups

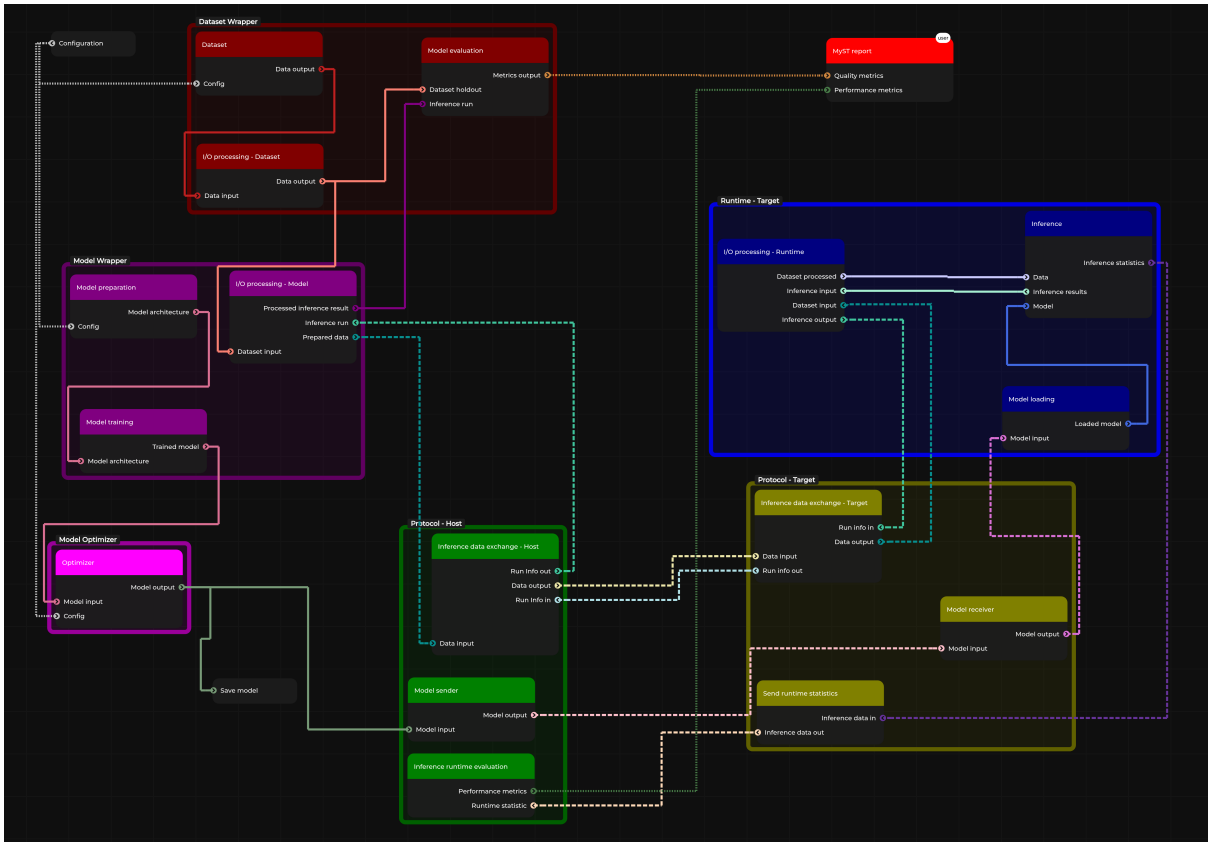
Some dependencies are provided as dependency groups instead of extras, so that they are not pulled in by `--all-extras`. They are available only when working directly with the repository:

- `tvm-cuda` - Apache TVM 0.18 with CUDA support, used instead of the `tvm` extra,
- `tvm-relax` - Relax-based Apache TVM compilation and runtime, conflicts with the `tvm` extra.

To install a dependency group, e.g. `tvm-relax`, run:

```
uv sync --extra tensorflow,tflite,reports --group tvm-relax
```

2.3 Kenning structure



The `kenning` module consists of the following submodules:

- `core` - provides interface APIs for datasets, models, optimizers, runtimes, protocols, and data converters
- `datasets` - provides implementations for datasets
- `modelwrappers` - provides implementations for models for various problems implemented in various frameworks,
- `optimizers` - provides implementations for compilers and optimizers for deep learning models,
- `runtimes` - provides implementations of runtime on target devices,
- `interfaces` - provides interface classes to group related methods used in Kenning `core` classes,
- `protocols` - provides implementations for communication protocols between host and tested target,
- `dataproviders` - provides implementations for reading input data from various sources, such as camera, directories or TCP connections,
- `dataconverters` - provides implementations for data converters for various data types
- `outputcollectors` - provides implementations for processing outputs from models, i.e. saving results to file, or displaying predictions on screen.
- `runners` - provide implementations for runners that can be used in runtime,
- `drawing` - provides methods for rendering plots for reports,
- `resources` - contains project's resources, like RST templates, or trained models,
- `scenarios` - contains executable scripts for running training, inference, benchmarks and other tests on target devices,
- `utils` - various functions and classes used in all above-mentioned submodules,
- `tests` - submodules for framework testing,
- `pipeline_manager` - contains tools for integrating with Pipeline Manager visualizer
- `cli` - provides tools and methods for creating CLI tools based on Kenning

`core` classes used throughout the entire Kenning framework:

- `Dataset` class - performs dataset download, preparation, dataset-specific input preprocessing (i.e. input file opening, normalization), output postprocessing and model evaluation,
- `ModelWrapper` class - trains the model, prepares the model, performs model-specific input preprocessing and output postprocessing, runs inference on host using native framework,
- `Optimizer` class - optimizes and compiles the model,
- `Runtime` class - loads the model, performs inference on compiled model, runs target-specific processing of inputs and outputs, and runs performance benchmarks,
- `Protocol` class - implements the communication protocol between the host and the target,
- `DataProvider` class - implements data provision from such sources as camera, TCP connection or others for inference,
- `OutputCollector` class - implements parsing and utilization of data from inference (such as displaying the visualizations, sending the results to via TCP),

- `DataConverter` class - performs data conversion from dataset-specific format to protocol-specific format and vice versa,
- `Runner` class - represents single runtime processing block.

2.4 Kenning usage

There are several ways to use Kenning:

- Using executable scripts from the `scenarios` submodule, configurable via JSON files (recommended approach);
- Using executable scripts from the `scenarios` submodule, configurable via command-line arguments;
- Using Kenning as a Python module.

Kenning scenarios are executable scripts that can be used for:

- Model training and benchmarking using its native framework (`kenning.scenarios.model_training`),
- Model optimization and compilation for target hardware (`kenning.scenarios.inference_tester`),
- Model benchmarking on target hardware (`kenning.scenarios.inference_tester` and `kenning.scenarios.inference_server`),
- Rendering performance and quality reports from benchmark data (`kenning.scenarios.render_report`),
- and more.

They are available through the `kenning` executable as subcommands. To get the current list of subcommands, run:

```
kenning -h
```

The available subcommands are:

- `train` - trains the given model (`kenning.scenarios.model_training`).
- `optimize` - optimizes and compiles the model for a given target device (`kenning.scenarios.inference_tester`).
- `test` - runs benchmark and evaluation of the model on the target device (`kenning.scenarios.inference_tester`).
- `report` - creates Markdown and HTML files summarizing the quality of the model in terms of performance and predictions.
- `flow` - runs Kenning-based applications.
- `visual-editor` - runs a graphical interface letting you represent, edit and run optimizations and applications in-browser using Pipeline Manager.
- `fine-tune-optimizers` - runs a search for the best optimizations for a given target platform based on selected optimizers, runtimes and models, along with their settings.
- `server` - runs a benchmark and evaluation server on target device.

- `info` - provides information about a given Kenning class.
- `list` - lists available Kenning modules for optimization and runtime.
- `fuzzy-search` - searches for class path across Kenning and returns it.
- `cache` - manages Kenning cache used for models and datasets.
- `completion` - configures autocompletion feature for Kenning CLI.

2.4.1 Running Kenning

Let's start off with installing the module and its necessary dependencies:

```
uv pip install "kenning[tensorflow,litert,tvm,pipeline_manager,reports] @␣
↪git+https://github.com/antmicro/kenning.git"
```

Kenning provides two tools for displaying information about available classes: `kenning list` and `kenning info`.

The `kenning list` tool lists all available modules used to form optimization and runtime pipelines in Kenning. Running:

```
kenning list
```

Should list:

```
Optimizers (in kenning.optimizers):

    kenning.optimizers.nni_pruning.NNIPruningOptimizer
    kenning.optimizers.onnx.ONNXCompiler
    kenning.optimizers.tensorflow_pruning.TensorFlowPruningOptimizer
    kenning.optimizers.model_inserter.ModelInserter
    kenning.optimizers.tvm.TVMCompiler
    kenning.optimizers.iree.IREECompiler
    kenning.optimizers.tensorflow_clustering.TensorFlowClusteringOptimizer
    kenning.optimizers.litert.LiteRTCompiler

Datasets (in kenning.datasets):

    kenning.datasets.pet_dataset.PetDataset
    kenning.datasets.visual_wake_words_dataset.VisualWakeWordsDataset
    kenning.datasets.random_dataset.RandomizedDetectionSegmentationDataset
    kenning.datasets.open_images_dataset.OpenImagesDatasetV6
    kenning.datasets.random_dataset.RandomizedClassificationDataset
    kenning.datasets.common_voice_dataset.CommonVoiceDataset
    kenning.datasets.magic_wand_dataset.MagicWandDataset
    kenning.datasets.coco_dataset.COCODataset2017
    kenning.datasets.imagenet_dataset.ImageNetDataset

Modelwrappers (in kenning.modelwrappers):

    kenning.modelwrappers.instance_segmentation.yolact.YOLACT
    kenning.modelwrappers.classification.tflite_magic_wand.MagicWandModelWrapper
```

(continues on next page)

(continued from previous page)

```
    kenning.modelwrappers.classification.pytorch_pet_dataset.
↪PyTorchPetDatasetMobileNetV2
    kenning.modelwrappers.object_detection.darknet_coco.TVMDarknetCOCOYOLOV3
    kenning.modelwrappers.instance_segmentation.yolact.YOLACTWithPostprocessing
    kenning.modelwrappers.classification.tensorflow_imagenet.TensorFlowImageNet
    kenning.modelwrappers.classification.tflite_person_detection.
↪PersonDetectionModelWrapper
    kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2
    kenning.modelwrappers.instance_segmentation.pytorch_coco.PyTorchCOCOMaskRCNN
    kenning.modelwrappers.object_detection.yolov4.ONNXYOLOV4

...
```

To list available compilers, run:

```
kenning list optimizers
```

For more verbose information, use the `-v` and `-vv` flags (module dependencies, description, supported formats and more)

The `kenning info` tool provides more detailed information on a given class, e.g. for:

```
kenning info kenning.optimizers.litert.LiteRTCompiler
```

We should get something like:

```
                                        Class LiteRTCompiler

The LiteRT and EdgeTPU compiler.



                                             Dependencies

 • numpy
 • tensorflow
 • ai_edge_litert.tools.flatbuffer_utils
 • importlib_resources.path
 • jinja2.Template
 • tensorflow_model_optimization
 • tensorflow.lite.python.schema_py_generated
 • ai_edge_litert.interpreter
 • ai_edge_litert.schema_py_generated


                                       Block input/output formats

                                             Input formats

 • keras
 • onnx
```

(continues on next page)

(continued from previous page)

```
• any

                                          Output formats

• litert


                                     Arguments' specification

                                          model_framework

• The input type of the model, framework-wise
• argparse flag: --model-framework
• default: any
• enum
   • keras
   • onnx
   • any
   • any

                                                target

• The LiteRT target device scenario
• argparse flag: --target
• default: default
• enum
   • default
   • int8
   • float16
   • edgetpu

                                        inference_input_type

• Data type of the input layer
• argparse flag: --inference-input-type
• default: float32
• enum
   • float32
   • int8
   • uint8

                                        inference_output_type

• Data type of the output layer
• argparse flag: --inference-output-type
• default: float32
• enum
   • float32
   • int8
```

(continues on next page)

(continued from previous page)

```
    • uint8

                                          dataset_percentage

• Tells how much data from dataset (from 0.0 to 1.0) will be used for␣
↪calibration dataset
• argparse flag: --dataset-percentage
• convert-type: builtins.float
• type: number
• default: 0.25

                                       quantization_aware_training

• Enable quantization aware training
• argparse flag: --quantization-aware-training
• convert-type: builtins.bool
• type: boolean
• default: False

                                           use_tf_select_ops

• Enable Tensorflow ops in model conversion (via SELECT_TF_OPS)
• argparse flag: --use-tf-select-ops
• convert-type: builtins.bool
• type: boolean
• default: False

                                       resolver_template_path

•
          Path to the custom template for creating C-based file with␣
↪`tflite::MicroMutableOpResolver`
    with used ops for a given model.

          When provided, it will be used instead of the default template located␣
↪in
    `kenning.resources.templates` module
          (as `tflite_ops_resolver.h.template`).

          The template is a Jinja2 template, where names of used ops are provided␣
↪via `opcode_names`
    list.

          Number of used ops can be established with `{{opcode_names|length}}`.
          Ops can be added to resolver like so:
          ```
          // ...
          tflite::MicroMutableOpResolver<{{opcode_names|length}}> resolver;
          // ...
```

(continues on next page)
```

(continued from previous page)

```
          {%- for opcode in opcode_names %}
              g_tflite_resolver.Add{{opcode}}();
          {%- endfor %}
          // ...
          ```

• argparse flag: --resolver-template-path
• convert-type: kenning.utils.resource_manager.ResourceURI
• type
   • string
   • null
• default: None

                                          resolver_output_path

• Path where a C-based file with tflite::MicroMutableOpResolver with used ops␣
↪for a given model should
  be saved
• argparse flag: --resolver-output-path
• convert-type: kenning.utils.resource_manager.ResourceURI
• type
   • string
   • null
• default: None

                                                epochs

• Number of epochs for the training
• argparse flag: --epochs
• convert-type: builtins.int
• type: integer
• default: 3

                                              batch_size

• The size of a batch for the training
• argparse flag: --batch-size
• convert-type: builtins.int
• type: integer
• default: 32

                                               optimizer

• Optimizer used during the training
• argparse flag: --optimizer
• convert-type: builtins.str
• type: string
• default: adam
• enum
```

(continues on next page)
```

(continued from previous page)

```
    • adam
    • SGD
    • RMSprop

                                          disable_from_logits

• Determines whether output of the model is normalized
• argparse flag: --disable-from-logits
• convert-type: builtins.bool
• type: boolean
• default: False

                                              save_to_zip

• Determines whether optimized model should additionally be saved in ZIP format
• argparse flag: --save-to-zip
• convert-type: builtins.bool
• type: boolean
• default: False

                                          compiled_model_path

• The path to the compiled model output
• argparse flag: --compiled-model-path
• convert-type: kenning.utils.resource_manager.ResourceURI
• type: string
• required: True

                                               location

• Specifies where optimization should be performed in client-server scenario
• argparse flag: --location
• default: host
• enum
    • host
    • target
```

If a certain class does not have necessary dependencies installed, Kenning suggests what modules need to be installed, e.g.:

```
This method requires additional dependencies, please use `pip install
↪"kenning[tensorflow]"` to install them.
```

The classes described above are used to form optimization and evaluation scenarios in Kenning. Some of them can also be used to quickly prototype simple applications.

With Kenning, we can combine multiple compilers and optimizers to create a fast and small model, including embedded devices. The optimization flow can be defined in JSON format, as follows:

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
    "parameters": {
      "model_name": "tvm-avx2-int8",
      "model_path": "kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5"
    }
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
    "parameters": {
      "dataset_root": "./build/PetDataset"
    }
  },
  "optimizers": [
    {
      "type": "kenning.optimizers.litert.LiteRTCompiler",
      "parameters": {
        "target": "int8",
        "compiled_model_path": "./build/int8.tflite",
        "inference_input_type": "int8",
        "inference_output_type": "int8",
        "dataset_percentage": 0.01
      }
    },
    {
      "type": "kenning.optimizers.tvm.TVMCompiler",
      "parameters": {
        "target": "llvm -mcpu=core-avx2",
        "conv2d_data_layout": "NCHW",
        "compiled_model_path": "./build/int8_tvm.tar"
      }
    }
  ],
  "runtime": {
    "type": "kenning.runtimes.tvm.TVMRuntime",
    "parameters": {
      "save_model_path": "./build/int8_tvm.tar"
    }
  }
}
```

The above scenario takes the MobileNetV2 model trained for classification of cat and dog breeds, and passes it to:

- `kenning.optimizers.litert.LiteRTCompiler` to quantize the weights of the model
- `kenning.optimizers.tvm.TVMCompiler` to create an optimized runtime of the model, utilizing AVX2 vector instructions.

The optimization of the model, its evaluation and report generation can be run with one command:

```
kenning optimize test report --json-cfg sample.json --measurements out.json --
↪report-path report.md --to-html report-html
```

The generated report can be found in the `report-html` directory.

2.4.2 Kenning tutorials

For more examples on Kenning usage, check Kenning tutorials.

2.5 Using Kenning as a library in Python scripts

Kenning is also a regular Python module - after pip installation it can be used in Python scripts. The example compilation of the model can look as follows:

```
from pathlib import Path
from kenning.datasets.pet_dataset import PetDataset
from kenning.modelwrappers.classification.tensorflow_pet_dataset import (
    TensorFlowPetDatasetMobileNetV2,
)
from kenning.optimizers.litert import LiteRTCompiler
from kenning.runtimes.litert import LiteRTRuntime
from kenning.core.measurements import MeasurementsCollector
from kenning.utils.resource_manager import ResourceURI

dataset = PetDataset(root=Path("./build/pet-dataset/"))
model = TensorFlowPetDatasetMobileNetV2(
    model_path=ResourceURI(
        "kenning:///models/classification/tensorflow_pet_dataset_mobilenetv2.h5"
    ),
    dataset=dataset,
)
model.save_io_specification(model.model_path)
compiler = LiteRTCompiler(
    dataset=dataset,
    compiled_model_path=Path("./build/compiled-model.tflite"),
    modelframework="keras",
    target="default",
    inferenceinputtype="float32",
    inferenceoutputtype="float32",
)
compiler.compile(
    input_model_path=ResourceURI(
        "kenning:///models/classification/tensorflow_pet_dataset_mobilenetv2.h5"
    )
)
```

The above script downloads the dataset and compiles the model with FP32 inputs and outputs using TensorFlow Lite.

To get a quantized model, replace `target`, `inferenceinputtype` and `inferenceoutputtype` to `int8`:

```
compiler = LiteRTCompiler(
    dataset=dataset,
    compiled_model_path=Path("./build/compiled-model.tflite"),
    modelframework="keras",
    target="int8",
    inferenceinputtype="int8",
    inferenceoutputtype="int8",
    dataset_percentage=0.3,
)
compiler.compile(
    input_model_path=ResourceURI(
        "kenning:///models/classification/tensorflow_pet_dataset_mobilenetv2.h5"
    )
)
```

To check how the compiled model is performing, create `LiteRTRuntime` object and run local model evaluation:

```
runtime = LiteRTRuntime(
    protocol=None, model_path=Path("./build/compiled-model.tflite")
)

runtime.run_locally(dataset, model, Path("./build/compiled-model.tflite"))
MeasurementsCollector.save_measurements("out.json")
```

The `runtime.run_locally` method runs benchmarks of the model on the current device.

The `MeasurementsCollector` class collects all benchmarks' data for model inference and saves it in JSON format that can be later used to render reports with the `kenning.scenarios.render_report` script.

As it can be observed, all classes accessible from JSON files in these scenarios share their configuration a with the classes in the Python scripts mentioned above.

2.6 Adding new implementations

`Dataset`, `ModelWrapper`, `Optimizer`, `Protocol`, `Runtime` and other classes from the `kenning.core` module have dedicated directories for their implementations. Each method in the base classes that requires implementation raises an `NotImplementedError` exception. They can be easily implemented or extended, but they need to conform to certain rules, usually described in the source documentation.

For more details and examples on how the Kenning framework can be adjusted and enhanced, follow the Kenning documentation. Implemented methods can be also overridden, if necessary.

Most of the base classes implement `form_argparse` and `from_argparse` methods. The former creates an argument parser and a group of arguments specific to the base class. The latter creates an object of the class based on the arguments from argument parser.

Inheriting classes can modify `form_argparse` and `from_argparse` methods to provide better control over their processing, but they should always be based on the results of their base

implementations.

CHAPTER

THREE

DEEP LEARNING DEPLOYMENT STACK

This chapter lists and describes typical actions performed on deep learning models before deployment on target devices.

3.1 From training to deployment

A deep learning application deployed on IoT devices usually goes through the following process:

- a dataset is prepared for a deep learning process,
- evaluation metrics are specified based on a given dataset and outputs,
- data in the dataset undergoes analysis, data loaders that perform the preprocessing are implemented,
- the deep learning model is either designed from scratch or a baseline is selected from a wide selection of existing pre-trained models for a given deep learning application (classification, detection, semantic segmentation, instance segmentation, etc.) and adjusted to a particular use case,
- a loss function and a learning algorithm are specified along with the deep learning model,
- the model is trained, evaluated and improved,
- the model is compiled to a representation that is applicable to a given target,
- the model is executed on a target device.

3.2 Dataset preparation

If a model is not available or it is trained for a different use case, the model needs to be trained or re-trained.

Each model requires a dataset - a set of sample inputs (audio signals, images, video sequences, OCT images, other sensors) and, usually, also outputs (association to class or classes, object location, object mask, input description). Datasets are usually split into the following categories:

- training dataset - the largest subset that is used to train a model,
- validation dataset - a relatively small set that is used to verify model performance after each training epoch (the metrics and loss function values show if any overfitting occurred during the training process),
- test dataset - the subset that acts as the final evaluation of a trained model.

It is required that the test dataset and the training dataset are mutually exclusive, so that the evaluation results are not biased in any way.

Datasets can be either designed from scratch or found in e.g.:

- Kaggle datasets,
- Google Dataset Search,
- Dataset list,
- Universities' pages,
- Open Images Dataset,
- Common Voice Dataset.

3.3 Model preparation and training

Currently, the most popular approach is to find an existing model that fits a given problem and perform transfer learning to adapt the model to the requirements. In transfer learning, the existing model's final layers are slightly modified to adapt to a new problem. These updated final layers of the model are trained using the training dataset. Finally, some additional layers are unfrozen and the training is performed on a larger number of parameters at a very small learning rate - this process is called fine-tuning.

Transfer learning provides a better starting point for the training process, allows to train a correctly performing model with smaller datasets and reduces the time required to train a model. The intuition behind this is that there are multiple common features between various objects in real-life environments, and the features learned from one deep learning scenario can be then reused in another scenario.

Once a model is selected, it requires adequate data input preprocessing in order to perform valid training. The input data should be normalized and resized to fit input tensor requirements. In case of the training dataset, especially if it is quite small, applying reasonable data augmentations like random brightness, contrast, cropping, jitters or rotations can significantly improve the training process and prevent the network from overfitting.

In the end, a proper training procedure needs to be specified. This step includes:

- loss function specification for the model. Some weights regularizations can be specified, along with the loss function, to reduce the chance of overfitting
- optimizer specification (like Adam, Adagrad). This involves setting hyperparameters properly or adding schedules and automated routines to set those hyperparameters (i.e. scheduling the learning rate value, or using LR-Finder to set the proper learning rate for the scenario)
- number of epochs specification or scheduling, e.g. early stopping can be introduced.
- providing some routines for quality metrics measurements
- providing some routines for saving intermediate models during training (periodically, or the best model according to a particular quality measure)

3.4 Model optimization

A successfully trained model may require some optimizations in order to run on given IoT hardware. The optimizations may involve precision of weights, computational representation, or model structure.

Models are usually trained with FP32 precision or mixed precision (FP32 + FP16, depending on the operator). Some targets, on the other hand, may significantly benefit from changing the precision from FP32 to FP16, INT8 or INT4. The optimizations here are straightforward for the FP16 precision, but the integer-based quantizations require dataset calibration to reduce precision without a significant loss in a model's quality.

Other optimizations change the computational representation of the model by e.g. layer fusion or specialized operators for convolutions of a particular shape, among others.

In the end, there are algorithmic optimizations that change the entire model structure, like weights pruning, conditional computation, model distillation (the current model acts as a teacher that is supposed to improve the quality of a much smaller model).

If these model optimizations are applied, the optimized models should be evaluated using the same metrics as the original model. This is required in order to find any drops in quality.

3.5 Model compilation and deployment

Deep learning compilers can transform model representation to:

- a source code for a different programming language, e.g. Halide, C, C++, Java, that can be later used on a given target,
- a machine code utilizing available hardware accelerators with e.g. OpenGL, OpenCL, CUDA, TensorRT, ROCm libraries,
- an FPGA bitstream,
- other targets.

Those compiled models are optimized to perform as efficiently as possible on given target hardware.

In the final step, the models are deployed on a hardware device.

CHAPTER

FOUR

DEFINING OPTIMIZATION PIPELINES IN KENNING

Kenning blocks (specified in the *Kenning API*) can be configured either via command line (see *Using Kenning via command line arguments*), or via configuration files, specified in JSON format. The latter approach allows the user to create more advanced and easy-to-reproduce scenarios for model deployment. Most notably, various optimizers available through Kenning can be chained to utilize various optimizations and get better performing models.

One of the scenarios most commonly used in Kenning is model optimization and compilation. It can be done using `kenning.scenarios.inference_tester`.

To run below examples it is required to install Kenning with dependencies as follows:

```
uv pip install "kenning[tensorflow,litert,tvm] @ git+https://github.com/antmicro/
↪kenning.git"
```

4.1 JSON specification

The `kenning.scenarios.inference_tester` takes the specification of optimization and the testing flow in a JSON format. The root element of the JSON file is a dictionary that can have the following keys:

- `model_wrapper` - **mandatory field**, accepts dictionary as a value that defines the *ModelWrapper* object for the deployed model (provides I/O processing, optionally model).
- `dataset` - **mandatory field**, accepts dictionary as a value that defines the *Dataset* object for model optimization and evaluation.
- `optimizers` - *optional field*, accepts a list of dictionaries specifying the sequence of *Optimizer*-based optimizations applied to the model.
- `protocol` - *optional field*, defines the *Protocol* object used to communicate with a remote target platform.
- `runtime` - *optional field* (**required** when `optimizers` are provided), defines the *Runtime*-based object that will infer the model on target device.

Each dictionary in the fields above consists of:

- `type` - appropriate class for the key,
- `parameters` - `type`-specific arguments for an underlying class (see *Defining arguments for core classes*).

4.2 Model evaluation using its native framework

The simplest JSON configuration looks as follows:

Listing 4.1: mobilenetv2-tensorflow-native.json

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
    "parameters": {
      "model_name": "native",
      "model_path": "kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5",
      "batch_size": 32,
      "learning_rate": 0.0001,
      "num_epochs": 50,
      "logdir": "build/logs"
    }
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
    "parameters": {
      "dataset_root": "./build/PetDataset"
    }
  }
}
```

It only takes `model_wrapper` and `dataset`. This way, the model will be loaded and evaluated using its native framework.

The *ModelWrapper* used is `TensorFlowPetDatasetMobileNetV2`, which is a MobileNetV2 model trained to classify 37 breeds of cats and dogs. In the `type` field, we specify the full "path" to the class by specifying the module it is implemented in (`kenning.modelwrappers.classification.tensorflow_pet_dataset`) and the name of the class (`TensorFlowPetDatasetMobileNetV2`) in a Python-like format (dot-separated).

In `parameters`, arguments specific to `TensorFlowPetDatasetMobileNetV2` are provided. The following parameters are available based on the argument specification:

```
# this argument structure is taken from kenning.core.model - it is inherited by␣
↪child classes
arguments_structure = {
    "modelpath": {
        "argparse_name": "--model-path",
        "description": "Path to the model",
        "type": Path,
        "required": True,
    }
}
```

The only mandatory parameter here is `model_path`, which points to a file containing the model. It is a required argument.

The `dataset` used here, is `PetDataset`. Like previously, it is provided in a module-like format (`kenning.datasets.pet_dataset.PetDataset`). The parameters here are specified in `kenning.core.dataset.Dataset` (inherited) and `kenning.core.dataset.PetDataset`:

```
arguments_structure = {
    # coming from kenning.core.dataset.Dataset
    "root": {
        "argparse_name": "--dataset-root",
        "description": "Path to the dataset directory",
        "type": Path,
        "required": True,
    },
    "batch_size": {
        "argparse_name": "--inference-batch-size",
        "description": "The batch size for providing the input data",
        "type": int,
        "default": 1,
    },
    "download_dataset": {
        "description": "Downloads the dataset before taking any action",
        "type": bool,
        "default": False,
    },
    # coming from kenning.datasets.pet_dataset.PetDataset
    "classify_by": {
        "argparse_name": "--classify-by",
        "description": "Determines if classification should be performed by␣
↪species or by breeds",
        "default": "breeds",
        "enum": ["species", "breeds"],
    },
    "image_memory_layout": {
        "argparse_name": "--image-memory-layout",
        "description": "Determines if images should be delivered in NHWC or NCHW␣
↪format",
        "default": "NHWC",
        "enum": ["NHWC", "NCHW"],
    },
}
```

As visible, the parameters allow the user to:

- specify the dataset's location,
- download the dataset,
- configure data layout and batch size,
- configure anything specific to the dataset.

Note

For more details on defining parameters for Kenning core classes, check *Defining arguments*

for core classes.

If `optimizers` or `runtime` are not specified, the model is executed using the *ModelWrapper*'s `run_inference` method. The dataset test data is passed through the model and evaluation metrics are collected.

To run the defined pipeline (assuming that the JSON file is under `pipeline.json`), run:

```
kenning test \
    --json-cfg mobilenetv2-tensorflow-native.json \
    --measurements measurements.json \
    --verbosity INFO
```

The `measurements.json` file is the output of the `kenning.scenarios.inference_tester` providing measurement data. It contains information such as:

- the JSON configuration defined above,
- versions of core class packages used (e.g. `tensorflow`, `torch`, `tvm`),
- available resource usage readings (CPU usage, GPU usage, memory usage),
- data necessary for evaluation, such as predictions, confusion matrix, etc.

This information can be later used for *Generating performance reports*.

Note

Check *Kenning measurements* for more information.

4.3 Model training

Provided that training is supported by a given model, you can specify parameters as follows:

Listing 4.2: mobilenetv2-tensorflow-native.json

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
    "parameters": {
      "model_name": "native",
      "model_path": "kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5",
      "batch_size": 32,
      "learning_rate": 0.0001,
      "num_epochs": 50,
      "logdir": "build/logs"
    }
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
```

(continues on next page)

(continued from previous page)

```
    "parameters": {
      "dataset_root": "./build/PetDataset"
    }
  }
}
```

To train the model, simply run:

```
kenning train \
    --json-cfg mobilenetv2-tensorflow-native.json \
    --verbosity INFO
```

Furthermore, the configuration is shared among subcommands:

```
kenning train test \
    --json-cfg train-test.json \
    --measurements output.json \
    --verbosity INFO
```

File with measurements also stores training information, which can be viewed directly or displayed in a generated report.

4.4 Optimizing and running a model on a single device

Model optimization and deployment can be performed directly on target device, if the device is able to perform the optimization steps. It can also be used to check the outcome of certain optimizations on a desktop platform before deployment.

Optimizations and compilers used in a scenario are defined in the `optimizers` field. This field accepts a list of optimizers - they are applied to the model in the same order in which they are defined in the `optimizers` field.

For example, a model can be subjected to the following optimizations:

- Quantization of weights and activations using TensorFlow Lite.
- Conversion of data layout from NHWC to NCHW format using Apache TVM
- Compilation to x86 runtime with AVX2 vector extensions using Apache TVM.

Such case will result is the following scenario:

Listing 4.3: mobilenetv2-tensorflow-tvm-avx-int8.json

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
    "parameters": {
      "model_name": "tvm-avx2-int8",
      "model_path": "kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5"
    }
```

(continues on next page)

(continued from previous page)

```
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
    "parameters": {
      "dataset_root": "./build/PetDataset"
    }
  },
  "optimizers": [
    {
      "type": "kenning.optimizers.litert.LiteRTCompiler",
      "parameters": {
        "target": "int8",
        "compiled_model_path": "./build/int8.tflite",
        "inference_input_type": "int8",
        "inference_output_type": "int8"
      }
    },
    {
      "type": "kenning.optimizers.tvm.TVMCompiler",
      "parameters": {
        "target": "llvm -mcpu=core-avx2",
        "opt_level": 3,
        "conv2d_data_layout": "NCHW",
        "compiled_model_path": "./build/int8_tvm.tar"
      }
    }
  ],
  "runtime": {
    "type": "kenning.runtimes.tvm.TVMRuntime",
    "parameters": {
      "save_model_path": "./build/int8_tvm.tar"
    }
  }
}
```

As emphasized above, the `optimizers` list is added, with two entries:

- a `kenning.optimizers.litert.LiteRTCompiler` type block, quantizing the model,
- a `kenning.optimizers.tvm.TVMCompiler` type block, performing remaining optimization steps.

In the `runtime` field, a TVM-specific `kenning.runtimes.tvm.TVMRuntime` type is used.

The first optimizer on the list reads the input model path from the *ModelWrapper*'s `model_path` field. Each consecutive *Optimizer* reads the model from a file saved by the previous *Optimizer*. In the simplest scenario, the model is saved to `compiled_model_path` in each optimizer, and is fetched by the next *Optimizer*.

In case the default output file type of the previous *Optimizer* is not supported by the next *Optimizer*, the first common supported model format is determined and used to pass the model between optimizers.

In case no such format exists, the `kenning.scenarios.inference_tester` returns an error.

> **Note**
>
> More details on input/output formats between *Optimizer* objects can be found in *Developing Kenning blocks*.

The scenario can be executed as follows:

```
kenning optimize test --json-cfg mobilenetv2-tensorflow-tvm-avx-int8.json --
↪measurements output.json
```

4.5 Compiling a model and running it remotely

For some platforms, we cannot run a Python script to evaluate or run the model to check its quality - the dataset is too large to fit in the storage, no libraries or compilation tools are available for the target platform, or the device does not have an operating system to run Python on.

In such cases, it is possible to evaluate the system remotely using the *Protocol* and the `kenning.scenarios.inference_server` scenario.

For this use case, we need two JSON files - one for inference server configuration, and another one for the `kenning.scenarios.inference_tester` configuration, which acts as a runtime client.

The client and the server may communicate via different means, protocols and interfaces - we can use TCP communication, UART communication or other. It depends on the *Protocol* used.

In addition, in such scenario optimizers can be executed either on host (which is default behavior) or on target device. To specify it, you can use `location` parameter of the *Optimizer*.

To create client/server scenario configuration it is required to add a `protocol` entry:

Listing 4.4: tflite-tvm-classification-client-server.json

```
{
    "platform": {
        "type": "LocalPlatform"
    },
    "model_wrapper": {
        "type": "TensorFlowPetDatasetMobileNetV2",
        "parameters": {
            "model_path": "kenning:///models/classification/tensorflow_pet_
↪dataset_mobilenetv2.h5"
        }
    },
    "dataset": {
        "type": "PetDataset",
        "parameters": {
            "dataset_root": "./build/PetDataset"
        }
    },
```

(continues on next page)

(continued from previous page)

```
    "optimizers": [
        {
            "type": "LiteRTCompiler",
            "parameters": {
                "target": "default",
                "compiled_model_path": "./build/compiled_tflite.tflite",
                "inference_input_type": "float32",
                "inference_output_type": "float32"
            }
        },
        {
            "type": "TVMCompiler",
            "parameters": {
                "target": "llvm -mcpu=core-avx2",
                "compiled_model_path": "./build/compiled_tvm.tar",
                "opt_level": 3,
                "location": "target"
            }
        }
    ],
    "runtime": {
        "type": "TVMRuntime",
        "parameters": {
            "save_model_path": "./build/compiled_model.tar"
        }
    },
    "protocol": {
        "type": "NetworkProtocol",
        "parameters": {
                "host": "localhost",
                "port": 12345,
                "packet_size": 4096
        }
    }
}
```

In the `protocol` entry, we specify a `kenning.protocols.network.NetworkProtocol` and provide a server address (`host`), an application port (`port`) and packet size (`packet_size`)

The server parses only `runtime` and `protocol` from the configuration, so any changes to the other of the blocks does not require server restart. The server uses `protocol` to receive requests from clients and `runtime` to run the tested models.

The remaining things are provided by the client - input data and model. Direct outputs from the model are sent as is to the client, so it can postprocess them and evaluate the model using the dataset. The server also sends measurements from its sensors in JSON format as long as it is able to collect and send them.

First, run the server, so that it is available for the client:

```
kenning server \
```

(continues on next page)

(continued from previous page)

```
    --json-cfg tflite-tvm-classification-client-server.json \
    --verbosity INFO &
```

Then, run the client:

```
kenning optimize test \
    --json-cfg tflite-tvm-classification-client-server.json \
    --measurements ./build/tflite-tvm-classification.json \
    --verbosity INFO
```

The rest of the flow is automated.

To execute one of the optimizers on the target-side, simply add the `location` parameter as follows:

```
"optimizers":
[
    {
        "type": "kenning.optimizers.litert.LiteRTCompiler",
        "parameters":
        {
            "target": "int8",
            "compiled_model_path": "./build/int8.tflite",
            "inference_input_type": "int8",
            "inference_output_type": "int8"
        }
    },
    {
        "type": "kenning.optimizers.tvm.TVMCompiler",
        "parameters": {
            "target": "llvm -mcpu=core-avx2",
            "opt_level": 3,
            "conv2d_data_layout": "NCHW",
            "compiled_model_path": "./build/int8_tvm.tar",
            "location": "target"
        }
    }
],
```

and start the client the same as above (it is not required to restart server).

CHAPTER

FIVE

USING KENNING VIA COMMAND LINE ARGUMENTS

Kenning provides several scripts for training, compilation and benchmarking of deep learning models on various target hardware. The executable scripts are present in the kenning.scenarios module. Sample bash scripts using the scenarios are located in the scripts directory in the repository.

Runnable scripts in scenarios require implemented classes to be provided from the `kenning.core` module to perform such actions as in-framework inference, model training, model compilation and model benchmarking on target.

To run below examples it is required to install Kenning with dependencies as follows:

```
uv pip install "kenning[tensorflow,tvm] @ git+https://github.com/antmicro/kenning.
↪git"
```

5.1 Command-line arguments for classes

Each class (*Dataset*, *ModelWrapper*, *Optimizer* and other) provided to the runnable scripts in scenarios can provide command-line arguments that configure the work of an object of the given class.

Each class in `kenning.core` implements `form_argparse` and `from_argparse` methods. The former creates an `argparse` group for a given class with its parameters. The latter takes the arguments parsed by `argparse` and returns the object of a class.

5.2 Autocompletion for command line interface

Kenning provides autocompletion for its command line interface. This feature requires additional configuration to work properly, which can be done using `kenning completion` command. Optionally, it can be configured as described in argcomplete documentation.

5.3 Model training

`kenning.scenarios.model_training` performs model training using Kenning's *ModelWrapper* and *Dataset* objects. To get the list of training parameters, select the model and training dataset to use (i.e. `TensorFlowPetDatasetMobileNetV2` model and `PetDataset` dataset) and run:

```
kenning train \
    --modelwrapper-cls kenning.modelwrappers.classification.tensorflow_pet_
↪dataset.TensorFlowPetDatasetMobileNetV2 \
```

(continues on next page)

(continued from previous page)

```
    --dataset-cls kenning.datasets.pet_dataset.PetDataset \
    -h
```

To simplify usage of internal implementations of models, optimizations and other, we also support referring to classes just by their name, without module path:

```
kenning train \
    --modelwrapper-cls TensorFlowPetDatasetMobileNetV2 \
    --dataset-cls PetDataset \
    -h
```

This will list the possible parameters that can be used to configure the dataset, the model, and the training parameters. For the above call, the output is as follows:

```
common arguments:
  -h, --help            show this help message and exit
  --verbosity {DEBUG,INFO,WARNING,ERROR,CRITICAL}
                        Verbosity level

'train' arguments:
  --modelwrapper-cls MODELWRAPPER_CLS
                        ModelWrapper-based class with inference implementation to␣
↪import
  --dataset-cls DATASET_CLS
                        Dataset-based class with dataset to import
  --batch-size BATCH_SIZE
                        The batch size for training
  --learning-rate LEARNING_RATE
                        The learning rate for training
  --num-epochs NUM_EPOCHS
                        Number of epochs to train for
  --logdir LOGDIR       Path to the training logs directory

ModelWrapper arguments:
  --model-path MODEL_PATH
                        Path to the model

PetDataset arguments:
  --classify-by {species,breeds}
                        Determines if classification should be performed by␣
↪species or by breeds
  --image-memory-layout {NHWC,NCHW}
                        Determines if images should be delivered in NHWC or NCHW␣
↪format

Dataset arguments:
  --dataset-root DATASET_ROOT
                        Path to the dataset directory
  --inference-batch-size INFERENCE_BATCH_SIZE
                        The batch size for providing the input data
```

(continues on next page)

(continued from previous page)

```
  --download-dataset    Downloads the dataset before taking any action. If the␣
↪dataset files are already downloaded and
                        the checksum is correct then they are not downloaded␣
↪again. Is enabled by default.
  --force-download-dataset
                        Forces dataset download
  --external-calibration-dataset EXTERNAL_CALIBRATION_DATASET
                        Path to the directory with the external calibration␣
↪dataset
  --split-fraction-test SPLIT_FRACTION_TEST
                        Default fraction of data to leave for model testing
  --split-fraction-val SPLIT_FRACTION_VAL
                        Default fraction of data to leave for model validation
  --split-seed SPLIT_SEED
                        Default seed used for dataset split
```

Note

The list of options depends on *ModelWrapper* and *Dataset*.

At the end, the training can be configured as follows:

```
kenning train \
    --modelwrapper-cls TensorFlowPetDatasetMobileNetV2 \
    --dataset-cls PetDataset \
    --logdir build/logs \
    --dataset-root build/pet-dataset \
    --model-path build/trained-model.h5 \
    --batch-size 32 \
    --learning-rate 0.0001 \
    --num-epochs 50
```

This will train the model with a 0.0001 learning rate and batch size 32 for 50 epochs. The trained model will be saved as `build/trained-model.h5`.

5.4 In-framework inference performance measurements

The `kenning.scenarios.inference_performance` script runs inference on a given model in a framework it was trained on. It requires you to provide:

- a *ModelWrapper*-based object wrapping the model to be tested,
- a *Dataset*-based object wrapping the dataset applicable to the model,
- a path to the output JSON file with performance and quality metrics gathered during inference by the *Measurements* object.

The example call for the method is as follows:

```
kenning test \
    --modelwrapper-cls TensorFlowPetDatasetMobileNetV2 \
    --dataset-cls PetDataset \
    --measurements build/tensorflow_pet_dataset_mobilenetv2.json \
    --model-path kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5 \
    --dataset-root build/pet-dataset/
```

The script downloads the dataset to the `build/pet-dataset` directory, loads the `tensorflow_pet_dataset_mobilenetv2.h5` model, runs inference on all images from the dataset and collects performance and quality metrics throughout the run. The performance data stored in the JSON file can be later rendered using *Generating performance reports*.

5.5 Testing inference on target hardware

The `kenning.scenarios.inference_tester` and `kenning.scenarios.inference_server` are used for inference testing on target hardware. The `inference_tester` loads the dataset and the model, compiles the model and runs inference either locally or remotely using `inference_server`.

The `inference_server` receives the model and input data, and sends output data and statistics.

Both `inference_tester` and `inference_server` require *Runtime* to determine the model execution flow. Both scripts communicate using the communication protocol implemented in the *Protocol*.

At the end, the `inference_tester` returns the benchmark data in the form of a JSON file extracted from the *Measurements* object.

The `kenning.scenarios.inference_tester` requires:

- a *ModelWrapper*-based class that implements model loading, I/O processing and optionally model conversion to ONNX format,
- a *Runtime*-based class that implements data processing and the inference method for the compiled model on the target hardware,
- a *Dataset*-based class that implements data sample fetching and model evaluation,
- a path to the output JSON file with performance and quality metrics.

An *Optimizer*-based class can be provided to compile the model for a given target if needed.

Optionally, it requires a *Protocol*-based class when running remotely to communicate with the `kenning.scenarios.inference_server`.

To print the list of required arguments, run:

```
kenning optimize test \
    --modelwrapper-cls TensorFlowPetDatasetMobileNetV2 \
    --runtime-cls TVMRuntime \
    --dataset-cls PetDataset \
    --compiler-cls TVMCompiler \
    --protocol-cls NetworkProtocol \
    -h
```

With the above classes, the help can look as follows:

```
common arguments:
  -h, --help            show this help message and exit
  --verbosity {DEBUG,INFO,WARNING,ERROR,CRITICAL}
                        Verbosity level
  --convert-to-onnx CONVERT_TO_ONNX
                        Before compiling the model, convert it to ONNX and use in␣
↪compilation (provide a path to save here)
  --measurements MEASUREMENTS
                        The path to the output JSON file with measurements

Inference configuration with JSON:
  Configuration with pipeline defined in JSON file. This section is not␣
↪compatible with 'Inference configuration with flags'. Arguments with '*' are␣
↪required.

  --json-cfg JSON_CFG   * The path to the input JSON file with configuration of␣
↪the inference

Inference configuration with flags:
  Configuration with flags. This section is not compatible with 'Inference␣
↪configuration with JSON'. Arguments with '*' are required.

  --modelwrapper-cls MODELWRAPPER_CLS
                        * ModelWrapper-based class with inference implementation␣
↪to import
  --dataset-cls DATASET_CLS
                        * Dataset-based class with dataset to import
  --compiler-cls COMPILER_CLS
                        * Optimizer-based class with compiling routines to import
  --runtime-cls RUNTIME_CLS
                        Runtime-based class with the implementation of model␣
↪runtime
  --protocol-cls PROTOCOL_CLS
                        Protocol-based class with the implementation of␣
↪communication between inference
                        tester and inference runner

ModelWrapper arguments:
  --model-path MODEL_PATH
                        Path to the model

PetDataset arguments:
  --classify-by {species,breeds}
                        Determines if classification should be performed by␣
↪species or by breeds
  --image-memory-layout {NHWC,NCHW}
                        Determines if images should be delivered in NHWC or NCHW␣
↪format
```

(continues on next page)

(continued from previous page)

```
Dataset arguments:
  --dataset-root DATASET_ROOT
                        Path to the dataset directory
  --inference-batch-size INFERENCE_BATCH_SIZE
                        The batch size for providing the input data
  --download-dataset    Downloads the dataset before taking any action. If the␣
↪dataset files are already downloaded and the checksum is correct then they are␣
↪not downloaded again. Is enabled by default.
  --force-download-dataset
                        Forces dataset download
  --external-calibration-dataset EXTERNAL_CALIBRATION_DATASET
                        Path to the directory with the external calibration␣
↪dataset
  --split-fraction-test SPLIT_FRACTION_TEST
                        Default fraction of data to leave for model testing
  --split-fraction-val SPLIT_FRACTION_VAL
                        Default fraction of data to leave for model validation
  --split-seed SPLIT_SEED
                        Default seed used for dataset split

TVMRuntime arguments:
  --save-model-path SAVE_MODEL_PATH
                        Path where the model will be uploaded
  --target-device-context {llvm,stackvm,cpu,c,test,hybrid,composite,cuda,nvptx,cl,
↪opencl,sdaccel,aocl,aocl_sw_emu,vulkan,metal,vpi,rocm,ext_dev,hexagon,webgpu}
                        What accelerator should be used on target device
  --target-device-context-id TARGET_DEVICE_CONTEXT_ID
                        ID of the device to run the inference on
  --runtime-use-vm      At runtime use the TVM Relay VirtualMachine

Runtime arguments:
  --disable-performance-measurements
                        Disable collection and processing of performance metrics

TVMCompiler arguments:
  --model-framework {keras,onnx,darknet,torch,tflite}
                        The input type of the model, framework-wise
  --target TARGET       The kind or tag of the target device
  --target-host TARGET_HOST
                        The kind or tag of the host (CPU) target device
  --opt-level OPT_LEVEL
                        The optimization level of the compilation
  --libdarknet-path LIBDARKNET_PATH
                        Path to the libdarknet.so library, for darknet models
  --compile-use-vm      At compilation stage use the TVM Relay VirtualMachine
  --output-conversion-function {default,dict_to_tuple}
                        The type of output conversion function used for PyTorch␣
↪conversion
  --conv2d-data-layout CONV2D_DATA_LAYOUT
```

(continues on next page)

(continued from previous page)

```
                        Configures the I/O layout for the CONV2D operations
  --conv2d-kernel-layout CONV2D_KERNEL_LAYOUT
                        Configures the kernel layout for the CONV2D operations
  --use-fp16-precision  Applies conversion of FP32 weights to FP16
  --use-int8-precision  Applies conversion of FP32 weights to INT8
  --use-tensorrt        For CUDA targets: delegates supported operations to␣
↪TensorRT
  --dataset-percentage DATASET_PERCENTAGE
                        Tells how much data from the calibration dataset␣
↪(training or external) will be used for calibration dataset

Optimizer arguments:
  --compiled-model-path COMPILED_MODEL_PATH
                        The path to the compiled model output

NetworkProtocol arguments:
  --host HOST           The address to the target device
  --port PORT           The port for the target device

BytesBasedProtocol arguments:
  --packet-size PACKET_SIZE
                        The maximum size of the received packets, in bytes.
  --endianness {big,little}
                        The endianness of data to transfer
```

The `kenning.scenarios.inference_server` requires only:

- a *Protocol*-based class for the implementation of the communication,
- a *Runtime*-based class for the implementation of runtime routines on device.

Both classes may require some additional arguments that can be listed with the `-h` flag.

An example script for the `inference_tester` is:

```
kenning optimize test \
    --modelwrapper-cls TensorFlowPetDatasetMobileNetV2 \
    --runtime-cls LiteRTRuntime \
    --dataset-cls PetDataset \
    --measurements ./build/google-coral-devboard-tflite-tensorflow.json \
    --compiler-cls LiteRTCompiler \
    --protocol-cls NetworkProtocol \
    --model-path kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5 \
    --model-framework keras \
    --target "edgetpu" \
    --compiled-model-path build/compiled-model.tflite \
    --inference-input-type int8 \
    --inference-output-type int8 \
    --host 192.168.188.35 \
    --port 12344 \
```

(continues on next page)

(continued from previous page)

```
    --packet-size 32768 \
    --save-model-path /home/mendel/compiled-model.tflite \
    --dataset-root build/pet-dataset \
    --inference-batch-size 1 \
    --verbosity INFO
```

The above runs with the following `inference_server` setup:

```
kenning server \
    --protocol-cls NetworkProtocol \
    --runtime-cls LiteRTRuntime \
    --host 0.0.0.0 \
    --port 12344 \
    --packet-size 32768 \
    --save-model-path /home/mendel/compiled-model.tflite \
    --delegates-list libedgetpu.so.1 \
    --verbosity INFO
```

Note

This run was tested on a Google Coral Devboard device.

`kenning.scenarios.inference_tester` can be also executed locally - in this case, the `--protocol-cls` argument can be skipped. The example call is as follows:

```
kenning optimize test \
    --modelwrapper-cls TensorFlowPetDatasetMobileNetV2 \
    --runtime-cls TVMRuntime \
    --dataset-cls PetDataset \
    --measurements ./build/local-cpu-tvm-tensorflow-classification.json \
    --compiler-cls TVMCompiler \
    --model-path kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5 \
    --model-framework keras \
    --target "llvm" \
    --compiled-model-path ./build/compiled-model.tar \
    --opt-level 3 \
    --save-model-path ./build/compiled-model.tar \
    --target-device-context cpu \
    --dataset-root ./build/pet-dataset/ \
    --inference-batch-size 1 \
    --verbosity INFO
```

Note

For more examples of running `inference_tester` and `inference_server`, check the kenning/scripts directory. Directories with scripts for client and server calls for various target devices, deep learning frameworks and compilation frameworks can be found in the

kenning/scripts/edge-runtimes directory.

5.6 Running inference

`kenning.scenarios.inference_runner` is used to run inference locally on a pre-compiled model.

`kenning.scenarios.inference_runner` requires:

- a *ModelWrapper*-based class that performs I/O processing specific to the model,
- a *Runtime*-based class that runs inference on target using the compiled model,
- a *DataProvider*-based class that implements fetching of data samples from various sources,
- a list of *OutputCollector*-based classes that implement output processing for the specific use case.

To print the list of required arguments, run:

```
python3 -m kenning.scenarios.inference_runner \
    kenning.modelwrappers.object_detection.darknet_coco.TVMDarknetCOCOYOLOV3 \
    TVMRuntime \
    kenning.dataproviders.camera_dataprovider.CameraDataProvider \
    --output-collectors kenning.outputcollectors.name_printer.NamePrinter \
    -h
```

With the above classes, the help can look as follows:

```
positional arguments:
  modelwrappercls       ModelWrapper-based class with inference implementation to␣
↪import
  runtimecls            Runtime-based class with the implementation of model␣
↪runtime
  dataprovidercls       DataProvider-based class used for providing data
optional arguments:
  -h, --help            show this help message and exit
  --output-collectors OUTPUT_COLLECTORS [OUTPUT_COLLECTORS ...]
                        List to the OutputCollector-based classes where the␣
↪results will be passed
  --verbosity {DEBUG,INFO,WARNING,ERROR,CRITICAL}
                        Verbosity level
Inference model arguments:
  --model-path MODEL_PATH
                        Path to the model
  --classes CLASSES     File containing Open Images class IDs and class names in␣
↪CSV format to use (can be generated using
                        kenning.scenarios.open_images_classes_extractor) or class␣
↪type
Runtime arguments:
  --disable-performance-measurements
                        Disable collection and processing of performance metrics
```

(continues on next page)

(continued from previous page)

```
  --save-model-path SAVE_MODEL_PATH
                        Path where the model will be uploaded
  --target-device-context {llvm,stackvm,cpu,c,cuda,nvptx,cl,opencl,aocl,aocl_sw_
↪emu,sdaccel,vulkan,metal,vpi,rocm,ext_dev,hexagon,webgpu}
                        What accelerator should be used on target device
  --target-device-context-id TARGET_DEVICE_CONTEXT_ID
                        ID of the device to run the inference on
  --input-dtype INPUT_DTYPE
                        Type of input tensor elements
  --runtime-use-vm      At runtime use the TVM Relay VirtualMachine
  --use-json-at-output  Encode outputs of models into a JSON file with base64-
↪encoded arrays
DataProvider arguments:
  --video-file-path VIDEO_FILE_PATH
                        Video file path (for cameras, use /dev/videoX where X is␣
↪the device ID eg. /dev/video0)
  --image-memory-layout {NHWC,NCHW}
                        Determines if images should be delivered in NHWC or NCHW␣
↪format
  --image-width IMAGE_WIDTH
                        Determines the width of the image for the model
  --image-height IMAGE_HEIGHT
                        Determines the height of the image for the model
OutputCollector arguments:
  --print-type {detector,classificator}
                        What is the type of model that will input data to the␣
↪NamePrinter
```

An example script for `inference_runner`:

```
python3 -m kenning.scenarios.inference_runner \
    kenning.modelwrappers.object_detection.darknet_coco.TVMDarknetCOCOYOLOV3 \
    TVMRuntime \
    kenning.dataproviders.camera_dataprovider.CameraDataProvider \
    --output-collectors kenning.outputcollectors.detection_visualizer.
↪DetectionVisualizer kenning.outputcollectors.name_printer.NamePrinter \
    --disable-performance-measurements \
    --model-path kenning:///models/object_detection/yolov3.weights \
    --save-model-path ../compiled-model.tar \
    --target-device-context "cuda" \
    --verbosity INFO \
    --video-file-path /dev/video0
```

5.7 Probing

`kenning.scenarios.probe` runs a probe server that monitors machine resources (CPU, memory, GPU, etc.) for a specific process and sends the statistics to requesting clients. One of such clients is `RemoteModelWrapper`, defining a model that runs on a remote server.

The `probe` server requires:

- a *Protocol*-based class that implements the communication protocol between the probe server and clients,
- the PID of the process to monitor.

Optionally, it allows configuring the prefix added to measurements and the sampling interval.

To print the list of required arguments, run:

```
kenning probe \
    --protocol-cls NetworkProtocol \
    --pid 1234 \
    -h
```

With the above class, the help can look as follows:

```
common arguments:
  -h, --help            show this help message and exit
  --verbosity {NOTSET,DEBUG,INFO,WARNING,ERROR,CRITICAL}
                        Verbosity level

Probe configuration with JSON/YAML file:
  Configuration with data defined in JSON/YAML file. This section is not␣
↪compatible with 'Probe configuration with flags'. Arguments with '*' are␣
↪required

Probe configuration with flags:
  Configuration with flags. This section is not compatible with 'Probe␣
↪configuration with JSON/YAML file'. Arguments with '*' are required.

  --protocol-cls PROTOCOL_CLS
                        * Protocol-based class with the implementation of␣
↪communication between server and host
  --pid PID             * PID of the monitored process
  --prefix PREFIX       * Prefix added to measurements
  --step STEP           * Interval of sampling measurements
```

Also `multi-client` flag can be passed to enable multiple clients to connect to the same probe reporting resource usage metrics. An example script for `probe`:

```
kenning probe \
    --protocol-cls NetworkProtocol \
    --pid 1234 \
    --prefix session_utilization \
    --step 0.1 \
    --host 0.0.0.0 \
    --multi-client \
    --port 12345 \
    --verbosity INFO
```

The above starts a probe server that monitors the process with PID `1234`, sampling resource usage every 0.1 seconds. The collected statistics are sent to clients connecting via the network protocol.

5.8 Generating performance reports

`kenning.scenarios.inference_performance` and `kenning.scenarios.inference_tester` return a JSON file as the result of benchmarks. They contain both performance metrics data, and quality metrics data.

The data from JSON files can be analyzed, processed and visualized by the `kenning.scenarios.render_report` script. This script parses the information in JSON files and returns an RST file with the report, along with visualizations.

It requires:

- a JSON file with benchmark data,
- a report name for use in the RST file and for creating Sphinx refs to figures,
- an RST output file name,
- `--root-dir` specifying the root directory of the Sphinx documentation where the RST file will be embedded (it is used to compute relative paths),
- `--img-dir` specifying the path where the figures should be saved,
- `--report-types`, which is a list describing the types the report falls into.

An example call and the resulting RST file can be observed in *Sample autogenerated report*.

As for now, the available report types are:

- `performance` - this is the most common report type that renders information about overall inference performance metrics, such as inference time, CPU usage, RAM usage, or GPU utilization,
- `classification` - this report is specific to the classification task, it renders classification-specific quality figures and metrics, such as confusion matrices, accuracy, precision, G-mean,
- `detection` - this report is specific to the detection task, it renders detection-specific quality figures and metrics, such as recall-precision curves or mean average precision.

5.9 Displaying information about available classes

`kenning.scenarios.list_classes` and `kenning.scenarios.class_info` provide useful information about classes and can help in creating JSON scenarios.

`kenning.scenarios.list_classes` will list all available classes by default, though the output can be limited by providing positional arguments representing groups of modules: `optimizers`, `runners`, `dataproviders`, `datasets`, `modelwrappers`, `outputcollectors`, `runtimes`. The amount of information displayed can be controlled using `-v` and `-vv` flags.

To print available arguments run `python -m kenning.scenarios.list_classes -h`.

`kenning.scenarios.class_info` provides information about a class given in an argument. More precisely, it will display:

- module and class docstrings,
- dependencies along with the information whether they are available in the current python environment,

- supported input and output formats,
- arguments structure used in JSON configurations.

The script accepts class names, e.g.: `LiteRTRuntime`.

However, module-like paths to the files (e.g. `kenning.runtimes.litert`) can be used, too. Additionally, a class can be added to the path like so: `kenning.runtimes.litert.LiteRTRuntime`. To get more detailed information, an optional `--load-class-with-args` argument can be passed. This needs all required class arguments to be provided, and that all dependencies are available.

For more detail, check `python -m kenning.scenarios.class_info -h`.

CHAPTER

SIX

KENNING GALLERY OF USE CASES

This section contains step-by-step examples of Kenning use cases.

6.1 Generating anomaly detection models for the MAX32690 Evaluation Kit with AutoML

This example demonstrates how to find anomaly detection models and evaluate them on a simulated MCU using Kenning, Zephyr RTOS and Renode. The models are generated with the Auto-PyTorch AutoML framework. The platform used in this example is the MAX32690 Evaluation Kit. The demo uses Kenning Zephyr Runtime and Zephyr RTOS for execution of the model on simulated hardware.

6.1.1 Prepare an environment for development

Assuming `git` and `docker` are available in the system, first let's clone the repository:

```
git clone https://github.com/antmicro/kenning-zephyr-runtime-example-app.git␣
↪sample-app
```

Then, let's build the Docker image based on `ghcr.io/antmicro/kenning-zephyr-runtime:latest` for quicker environment setup:

```
docker build -t kenning-automl ./sample-app/environments
```

After successful build of the image, run:

```
docker run --rm -it --name automl -w $(realpath sample-app) -v $(pwd):$(pwd)␣
↪kenning-automl:latest bash
```

Then, in the Docker container, initialize the Zephyr application and Kenning Zephyr Runtime as follows:

```
pip install --break-system-packages uv
uv venv .venv
source .venv/bin/activate
west init -l app
west update
west zephyr-export
pushd ./kenning-zephyr-runtime
uv pip install -r requirements.txt -r ../zephyr/scripts/requirements-base.txt
```

(continues on next page)

(continued from previous page)

```
./scripts/prepare_modules.sh
popd
```

Then install Zephyr SKD:

```
west sdk install --toolchains x86_64-zephyr-elf aarch64-zephyr-elf arm-zephyr-
↪eabi riscv64-zephyr-elf
```

Note

To make sure you have the latest version of the Kenning with AutoML features, run:

```
uv pip install "kenning[iree,tvm,torch,anomaly_detection,auto_pytorch,
↪tensorflow,litert,reports,renode,uart] @ git+https://github.com/antmicro/
↪kenning.git"
```

With the configured environment, you can now run the AutoML flow.

To create a `workspace` directory where intermediate results of command executed further will be stored, run:

```
mkdir -p workspace
```

Note

For more step-by-step instructions on how to set up the environment locally, see Kenning Zephyr Runtime build instructions.

6.1.2 Run AutoML flow

The AutoML flow can be configured using the YAML below.

```
# This scenario demonstrates AutoML flow on example of Anomaly Detection in time␣
↪series
# for MAX32960 Evaluation Kit.
#
# Configures AutoML flow
automl:
  type: AutoPyTorchML
  parameters:
    # Time limit for AutoML task (in minutes)
    time_limit: 20
    # List of model architectures used for AutoML,
    # represented by ModelWrapper (has to implement AutoMLModel class)
    use_models:
      - PyTorchAnomalyDetectionVAE
    # Folder storing AutoML results
    output_directory: ./workspace/automl-results
```

(continues on next page)

(continued from previous page)

```
    # Maximum number of models returned by the flow
    n_best_models: 5

    # AutoPyTorch specific options
    # Chosen metric to optimize
    optimize_metric: f1
    # Type of budget for training models, either epochs or time limit
    budget_type: epochs
    # Lower and upper limit of the budger
    min_budget: 1
    max_budget: 5
    # Size of the application that will use generated models
    application_size: 75.5

# Chooses the platform to run
platform:
  type: ZephyrPlatform
  parameters:
    # Chooses MAX32690 Evaluation Kit
    name: max32690evkit/max32690/m4
    # Use Renode to simulate the platform
    simulated: True

# Defines dataset for anomaly detection
dataset:
  type: AnomalyDetectionDataset
  parameters:
    dataset_root: ./workspace/CATS
    csv_file: kenning:///datasets/anomaly_detection/cats_nano.csv
    split_fraction_test: 0.1
    split_seed: 12345
    inference_batch_size: 1

# Remaining configurations, despite not being directly
# used for a AutoML flow, are copied to resulting scenarios,
# and influences the standard Kenning flow run with generated models

optimizers:
- type: LiteRTCompiler
  parameters:
    target: default
    compiled_model_path: ./workspace/automl-results/vae.tflite
    inference_input_type: float32
    inference_output_type: float32

runtime_builder:
  type: ZephyrRuntimeBuilder
  parameters:
    workspace: .
```

(continues on next page)

(continued from previous page)

```
    # venv_dir: ../.venv
    output_path: ./workspace/kzr_build
    run_west_update: false
    extra_targets: [board-repl]
```

The introduced `automl` entry provides an implementation of the AutoML flow, as well as its parameters. The main parameters affecting the model search process are:

- `time_limit` - determines how much time the AutoML algorithm will spend looking for the models
- `application_size` - determines how much of the space is consumed by the app excluding the model. By taking into account the provided size of the application ran on the board and the size of RAM (received from the platform class), the flow will automatically reject (before training) all models that do not fit into available space. This ensures than time is not wasted on models that cannot be run on the hardware (or its simulation).
- `use_models` - provides models for the algorithm to take into account. Models provided in the list contribute to the search space of available configurations, providing their hyperparameters with acceptable ranges and structure definition. It is possible to override default ranges specifying them right after chosen model:

  ```
  use_models:
    - PyTorchAnomalyDetectionVAE:
        encoder_neuron_list:
          list_range: [4, 15]
          item_range: [6, 128]
        dropout_rate:
          item_range: [0.1, 0.4]
        output_activation:
          enum: [tanh]
  ```

  The list of AutoML specific options is available in *Defining arguments for classes section*.

Note

To use the microTVM runtime, change the optimizer to:

```
optimizers:
- type: TVMCompiler
  parameters:
    compiled_model_path: ./workspace/vae.graph_data
```

Depending on runtime selection, application size may vary greatly, so to generate models with correct size, make sure to adjust this value.

To run the full flow, use this command:

```
kenning automl optimize test report \
  --cfg ./kenning-zephyr-runtime/kenning-scenarios/renode-auto-tflite-automl-vae-
↪max32690.yml \
```

(continues on next page)

(continued from previous page)

```
  --report-path ./workspace/automl-report/report.md \
  --allow-failures --to-html --ver INFO \
  --skip-general-information
```

The command above:

- Runs an AutoML search for models for the amount of time specified in the `time_limit`
- Optimizes best-performing using given optimization pipeline
- Runs evaluation of the compiled models in Renode simulation
- Generates a full comparison report for the models so that user can pick the best one (located in `workspace/automl-report/report/report.html`)
- Generates optimized models (`vae.<id>.tflite`), their AutoML-derived configuration (`automl_conf_<id>.yml`) and IO specification files for Kenning (`vae.<id>.tflite.json`) under `workspace/automl-results/`.

6.1.3 Run AutoML flow with quantization

The AutoML flow can also take into account the model size after quantization. To make it possible, a scenario has to define at least one Optimizer which will quantize the model:

Listing 6.1: automl_quantization.yml

```yaml
# This scenario demonstrates AutoML flow with model quantization
# on example of Anomaly Detection in time series for MAX32960 Evaluation Kit.

automl:
  type: AutoPyTorchML
  parameters:
    output_directory: ./workspace/automl-quant-results
    time_limit: 3
    use_models:
      - PyTorchAnomalyDetectionCNN:
          # Set custom ranges for CNN parameters
          conv_stride:
            item_range: [1, 1]
          conv_dilation:
            item_range: [1, 1]
          pool_stride:
            item_range: [1, 1]
          pool_dilation:
            item_range: [1, 1]
    n_best_models: 5
    # AutoPyTorch specific options
    optimize_metric: f1
    budget_type: epochs
    min_budget: 1
    max_budget: 5
    # Increase size of the reduced dataset for quantization
    callback_max_samples: 200
```

(continues on next page)

(continued from previous page)

```
    # Size of the application that will use generated models
    application_size: 75.5
    # To use GPU uncomment line below
    # use_cuda: True

optimizers:
- type: LiteRTCompiler
  parameters:
    # Quantize model to 8-bit integers
    target: int8
    compiled_model_path: ./workspace/automl-quant-results/cnn_int8.tflite
    inference_input_type: int8
    inference_output_type: int8

# Chooses MAX32690 Evaluation Kit
platform:
  type: ZephyrPlatform
  parameters:
    name: max32690evkit/max32690/m4
    simulated: True

dataset:
  type: AnomalyDetectionDataset
  parameters:
    dataset_root: ./workspace/CATS
    csv_file: kenning:///datasets/anomaly_detection/cats_nano.csv
    split_fraction_test: 0.1
    split_seed: 12345
    inference_batch_size: 1

runtime_builder:
  type: ZephyrRuntimeBuilder
  parameters:
    output_path: ./workspace/kzr_quant_build
    workspace: ./kenning-zephyr-runtime
    venv_dir: ../.venv
    run_west_update: false
    extra_targets: [board-repl]
```

In order to avoid training models that will not fit into available space, the flow triggers quantization on initialized models and rejects ones that are too large. If model fits in the memory, the flow proceeds with training of default (non-quantized) models. In the end, it quantizes and evaluates the best models based on the training process. Such flow can be run with the following command, assuming *the scenario* is saved as `automl_quantization.yml`:

```
kenning automl optimize test report \
  --cfg ./automl_quantization.yml \
  --report-path ./workspace/automl-report/report.md \
  --allow-failures --to-html --ver INFO \
  --skip-general-information
```

6.1.4 Run sample app with a chosen model

Once the best model is selected (e.g. `workspace/automl-results/vae.0.tflite`), let's compile the sample app with it:

```
west build \
  -p always \
  -b max32690evkit/max32690/m4 app -- \
  -DEXTRA_CONF_FILE="litert.conf" \
  -DCONFIG_KENNING_MODEL_PATH=\"$(realpath workspace/automl-results/vae.0.tflite)\
↪"
west build -t board-repl
```

Note

To use microTVM runtime, change `-DEXTRA_CONF_FILE` to `"tvm.conf"` and `-DCONFIG_KENNING_MODEL_PATH` to a chosen model compiled with TVM (usually with `.graph_data` extension).

In the end, the app with the model can be simulated with:

```
python kenning-zephyr-runtime/scripts/run_renode.py --no-kcomm
```

To end the simulation, press `Ctrl+C`.

6.2 Anomaly detection model training and deployment on the MAX32690 Evaluation Kit

This example demonstrates how to train an example anomaly detection model and deploys it on an MCU using Kenning and Zephyr RTOS. The platform used in this example is the Analog Devices MAX32690 Evaluation Kit. This demo uses Kenning Zephyr Runtime and Zephyr RTOS for execution of the model on hardware.

6.2.1 Prepare an environment for development

This example uses a pre-built Docker image from Kenning Zephyr Runtime. To get started, create a Docker container with the necessary environment:

```
docker run --rm -it -v $(pwd):$(pwd) -w $(pwd) ghcr.io/antmicro/kenning-zephyr-
↪runtime:latest /bin/bash
```

Then, create a workspace directory in the container:

```
mkdir -p zephyr-workspace && cd zephyr-workspace
```

In case the MAX32690 Evaluation Kit is connected to the desktop PC using a MAX32625PICO debugger, the Docker container needs to be started in privileged mode, with UART and ACM devices forwarded to it. Assuming that the programmer is available as `/dev/ttyACM0`, and UART as `/dev/ttyUSB0` in the currently running system, run:

```
docker run --privileged --device /dev/ttyACM0 --device /dev/ttyUSB0 --rm -it -v
↪$(pwd):$(pwd) -v /dev/serial/by-id/:/dev/serial/by-id/ -w $(pwd) ghcr.io/
↪antmicro/kenning-zephyr-runtime:latest /bin/bash
```

Then, in the Docker container, clone the Kenning Zephyr Runtime repository and install the latest Zephyr SDK:

```
pip install --break-system-packages uv
git clone https://github.com/antmicro/kenning-zephyr-runtime.git
cd kenning-zephyr-runtime/
uv venv .venv
./scripts/prepare_zephyr_env.sh
./scripts/prepare_modules.sh
source .venv/bin/activate
```

Note

Optionally, to use the latest Kenning version, install it in the image and reload the virtual environment:

```
uv pip install "kenning[iree,tvm,torch,anomaly_detection,tensorflow,litert,
↪reports,renode,uart,zephyr,auto_pytorch] @ git+https://github.com/antmicro/
↪kenning.git"
```

An environment configured this way will allow you to work with Kenning and Zephyr RTOS. For more step-by-step instructions on how to set up the environment locally, see Kenning Zephyr Runtime build instructions.

Now create a `workspace` director where intermediate results of commands executed further will be stored:

```
mkdir -p workspace
```

6.2.2 Install software necessary for flashing the MAX32690 Evaluation Kit

To flash the MAX32690 Evaluation Kit, a Maxim Micros SDK is needed.

Within the Docker image, run:

```
mkdir -p /usr/share/applications
wget -O ./MaximMicrosSDK_linux.run https://github.com/analogdevicesinc/msdk/
↪releases/download/v2024_10/MaximMicrosSDK_linux.run
chmod +x ./MaximMicrosSDK_linux.run
./MaximMicrosSDK_linux.run install
export PATH=/root/MaximSDK/Tools/OpenOCD/:$PATH
```

Now follow along, answering prompts in the installation process. Once everything is installed successfully, it will be possible to flash the device using the evaluation app from Kenning Zephyr Runtime.

Note

During installation, the script may notify you that it was unable to install libncurses5.

In such a case, answer `Ignore` and proceed with the installation - it is not needed for this demo.

6.2.3 Train the sample anomaly detection model (optional)

Note

The pretrained model is available at `https://dl.antmicro.com/kenning/models/anomaly_detection/vae_cats.pth`. In Kenning, such files are obtainable using the `kenning://` scheme, e.g. `kenning:///models/anomaly_detection/vae_cats.pth`.

To skip the training step, run:

```
wget https://dl.antmicro.com/kenning/models/anomaly_detection/vae_cats.pth
wget https://dl.antmicro.com/kenning/models/anomaly_detection/vae_cats.pth.json
```

And proceed to the next section.

Once the environment is set up, the sample model can be trained. In this demo, a Variational AutoEncoder (VAE) will be used. In Kenning, there is a PytorchAnomalyDetectionVAE `ModelWrapper` encapsulating the model.

Controlled Anomalies Time Series (CATS) will be used for a dataset. It provides telemetry readings of a simulated complex dynamical system with external stimuli. It provides a representative set of time series for sensors with anomalies.

Note

Check *Specification of the kenning.datasets.anomaly_detection_dataset module* for details on the expected structure of the dataset's CSV file to understand how to create custom dataset compliant with the class.

The model can be trained using the following command:

```
kenning train test \
    --dataset-cls AnomalyDetectionDataset \
    --dataset-root ./dataset/ \
    --csv-file https://zenodo.org/records/8338435/files/data.csv \
    --modelwrapper-cls PyTorchAnomalyDetectionVAE \
    --model-path ./vae_cats.pth \
    --batch-size 256 --learning-rate 0.00002 --num-epochs 4 \
    --verbosity INFO \
    --measurements workspace/vae-torch-native.json \
    --inference-batch-size 256 \
    --batch-norm --loss-beta 0.2 --loss-capacity 0.1
```

This command:

- Downloads the CATS dataset from https://zenodo.org/records/8338435/files/data.csv (if it has not been downloaded yet)
- Creates the `AnomalyDetectionDataset` class with dataset data taken from the downloaded CSV
- Creates the `PyTorchAnomalyDetectionVAE` model wrapper encapsulating the VAE model, providing necessary methods for input preprocessing, output postprocessing, model training and more
- Trains the model using the provided batch size, learning rate, number of epochs and other exposed training parameters for the model
- Evaluates the model at the end, providing evaluation results in `./workspace/vae-torch-native.json`
- Saves the trained model as the `./vae_cats.pth` file.

6.2.4 Deploy the VAE model with TensorFlow Lite Micro

Once the VAE model is trained and available under `./vae_cats.pth`, it can be compiled with `kenning optimize` command and deployed on device using either TensorFlow Lite Micro or microTVM. This section focuses on TensorFlow Lite Micro.

Let's consider the following scenario:

```
# This scenario demonstrates deployment of Anomaly Detection in time series on␣
↪MAX32960 Evaluation Kit
# It provides two variants of execution - a deployment on an actual hardware, and␣
↪simulation in Renode
# This scenario should be executed within kenning-zephyr-runtime project, with␣
↪built evaluation app for
# TFLite Micro runtime:
#
# west build -p always -b max32690evkit/max32690/m4 app -- -DEXTRA_CONF_FILE=
↪"litert.conf"
platform:
  type: ZephyrPlatform
  parameters:
    name: max32690evkit/max32690/m4
    # to run inference on actual hardwarem change simulated to false
    simulated: true
    zephyr_build_path: ./build/
    uart_port: /dev/ttyUSB0

# model wrapper for the VAE anomaly detection model
model_wrapper:
  type: PyTorchAnomalyDetectionVAE
  parameters:
    model_path: ../vae_cats.pth
    encoder_neuron_list: [16, 8]
    batch_norm: true

# A dataset used for evaluating the model
```

(continues on next page)

(continued from previous page)

```
dataset:
  type: AnomalyDetectionDataset
  parameters:
    dataset_root: ./workspace/dataset
    csv_file: https://zenodo.org/records/8338435/files/data.csv
    split_fraction_test: 0.0005
    inference_batch_size: 1
    split_seed: 12345

# run TFLite conversion from above PyTorch model to TFLite Flatbuffers
optimizers:
  - type: LiteRTCompiler
    parameters:
      target: default
      compiled_model_path: ./workspace/vae_cats.tflite
      inference_input_type: float32
      inference_output_type: float32
```

The scenario contents are as follows:

- `platform` - specifies the target platform where the model will be deployed. In this case, the target platform is called `max32690evkit/max32690/m4`. `simulated` boolean tells whether the board should be simulated or not.
- `model_wrapper` - provides a class that encapsulates necessary preprocessing and postprocessing functions for input and output data.
- `dataset` - provides a class implementing methods around dataset management
- `optimizers` - provides a list of optimizations that are used for optimizing and/or compiling the model.

First, compile the model using `kenning optimize`:

```
kenning optimize --cfg ./kenning-scenarios/vae-inference/tflite-max32690.yml
```

This scenario will create a `./workspace/vae_cats.tflite` file with an optimized model.

Now, to test the model in simulation or an actual hardware, the evaluation app needs to be compiled. To do so, use the `west build` command with a `litert.conf` configuration for the selected board. However, this will build TensorFlow Lite Micro runtime with a limited set of operators. To get the runtime with just the necessary set of operators for a given model, you can provide the model that was just created.

To build the evaluation application, run:

```
west build -p always -b max32690evkit/max32690/m4 app -- \
      -DEXTRA_CONF_FILE=litert.conf \
      -DCONFIG_KENNING_MODEL_PATH=\"./workspace/vae_cats.tflite\"
west build -t board-repl
```

Once the model and evaluation app are ready, you can simulate the board in Renode with:

```
kenning test --cfg ./kenning-scenarios/vae-inference/tflite-max32690.yml --
↪measurements workspace/vae-tflite-renode.json --verbosity INFO
```

The produced `workspace/vae-tflite.json` is a file with raw measurements regarding the model's performance and predictions.

It can be parsed and rendered into a Markdown-based or HTML-based report using the `kenning report` command:

```
kenning report --measurements workspace/vae-tflite-renode.json --report-path␣
↪reports/vae-tflite-renode/report.md --to-html
```

Lastly, the model can be evaluated on actual MAX32690 Evaluation Kit. To do so, first flash the board with an evaluation app:

```
/root/MaximSDK/Tools/OpenOCD/openocd \
    -s /root/MaximSDK/Tools/OpenOCD/scripts/ \
    -c 'source [find interface/cmsis-dap.cfg]' \
    -c 'source [find target/max32690.cfg]' \
    -c 'init' -c 'targets' -c 'reset init' \
    -c 'flash write_image erase ./build/zephyr/zephyr.hex' \
    -c 'reset run' \
    -c 'shutdown'
```

Logs from the flashing process should look as follows to ensure successful flashing:

```
Open On-Chip Debugger (Analog Devices 0.12.0-1.0.0-7)  OpenOCD 0.12.0 (2023-09-27-
↪07:53)
Licensed under GNU GPL v2
Report bugs to <processor.tools.support@analog.com>
Info : CMSIS-DAP: SWD supported
Info : CMSIS-DAP: Atomic commands supported
Info : CMSIS-DAP: Test domain timer supported
Info : CMSIS-DAP: FW Version = 2.0.0
Info : CMSIS-DAP: Serial# = 0409170272c2c8d8000000000000000000000097969906
Info : CMSIS-DAP: Interface Initialised (SWD)
Info : SWCLK/TCK = 1 SWDIO/TMS = 1 TDI = 0 TDO = 0 nTRST = 0 nRESET = 1
Info : CMSIS-DAP: Interface ready
Info : clock speed 2000 kHz
Info : SWD DPIDR 0x2ba01477
Info : [max32xxx.cpu] Cortex-M4 r0p1 processor detected
Info : [max32xxx.cpu] target has 6 breakpoints, 4 watchpoints
Info : starting gdb server for max32xxx.cpu on 3333
Info : Listening on port 3333 for gdb connections
    TargetName         Type       Endian TapName            State
--  ------------------ ---------- ------ ------------------ ------------
 0* max32xxx.cpu       cortex_m   little max32xxx.cpu       running

[max32xxx.cpu] halted due to debug-request, current mode: Thread
xPSR: 0x61000000 pc: 0x10016372 psp: 0x2000afa0
Info : SWD DPIDR 0x2ba01477
```

(continues on next page)

(continued from previous page)

```
[max32xxx.cpu] halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0x0000fff4 msp: 0x2000a900
auto erase enabled
wrote 131072 bytes from file ./build/zephyr/zephyr.hex in 3.210366s (39.871 KiB/s)

Info : SWD DPIDR 0x2ba01477
shutdown command invoked
```

Once `./build/zephyr/zephyr.hex` is successfully written to the device, the model can be tested directly on hardware platform.

In order to do so, set `simulated` to `false` in the `./kenning-scenarios/vae-inference/tflite-max32690.yml` file. Finally, use a single-command approach, where `kenning test report` are invoked all at once (model is already compiled, hence lack of `optimize`):

```
kenning test report --cfg ./kenning-scenarios/vae-inference/tflite-max32690.yml \
    --measurements workspace/vae-tflite-hw.json \
    --report-path reports/vae-tflite-hw/report.md --to-html \
    --verbosity INFO
```

The `workspace/vae-tflite-hw.json` will hold the collected performance and quality data, and `reports/vae-tflite-hw/report/report.html` will demonstrate the work of the model on hardware.

6.2.5 Deploy the VAE model with microTVM

For microTVM, the deployment looks similar to the scenario above. The only difference are the optimizers and runtimes used:

```
# This scenario demonstrates deployment of Anomaly Detection in time series on␣
↪MAX32960 Evaluation Kit
# It provides two variants of execution - a deployment on an actual hardware, and␣
↪simulation in Renode
# This scenario should be executed within kenning-zephyr-runtime project, with␣
↪built evaluation app for
# microTVM runtime:
#
# west build -p always -b max32690evkit/max32690/m4 app -- -DEXTRA_CONF_FILE='tvm.
↪conf;boards/max32690evkit_max32690_m4.conf' -DKENNING_MODEL_PATH=`realpath ./
↪vae_cats.pth`
platform:
  type: ZephyrPlatform
  parameters:
    name: max32690evkit/max32690/m4
    # to run inference on actual hardwarem change simulated to false
    simulated: true
    zephyr_build_path: ./build/
    uart_port: /dev/ttyUSB0

# model wrapper for the VAE anomaly detection model
model_wrapper:
```

(continues on next page)

(continued from previous page)

```
  type: PyTorchAnomalyDetectionVAE
  parameters:
    model_path: ./vae_cats.pth
    encoder_neuron_list: [16, 8]
    batch_norm: true

# A dataset used for evaluating the model
dataset:
  type: AnomalyDetectionDataset
  parameters:
    dataset_root: ./workspace/dataset
    csv_file: https://zenodo.org/records/8338435/files/data.csv
    split_fraction_test: 0.0005
    inference_batch_size: 1
    split_seed: 12345

# run TVM conversion from above PyTorch model
optimizers:
- type: TVMCompiler
  parameters:
    compiled_model_path: ./workspace/vae_cats.graph_data
```

To begin evaluation, compile the model for microTVM:

```
kenning optimize --cfg ./kenning-scenarios/vae-inference/tvm-max32690.yml
```

Then compile the evaluation app using microTVM as the runtime. Since microTVM requires building with the model, we provide path to the model and set `CONFIG_KENNING_TVM_MODEL_PRE_GEN=y` (since the model is pre-generated by Kenning).

```
west build -p always -b max32690evkit/max32690/m4 app -- -DEXTRA_CONF_FILE='tvm.
↪conf;boards/max32690evkit_max32690_m4.conf' -DCONFIG_KENNING_MODEL_PATH=\"
↪$(realpath ./workspace/vae_cats.graph_data)\" -DCONFIG_KENNING_TVM_MODEL_PRE_
↪GEN=y
west build -t board-repl
```

Then, run:

```
kenning test report --cfg ./kenning-scenarios/vae-inference/tvm-max32690.yml \
    --measurements workspace/vae-tvm-renode.json \
    --report-path reports/vae-tvm-renode/report.md --to-html \
    --verbosity INFO
```

This performs all actions at once - model optimization, model evaluation and report generation.

To test the model on hardware, set `simulated` in the scenario to `false` and then flash the device with the microTVM-based app:

```
/root/MaximSDK/Tools/OpenOCD/openocd \
    -s /root/MaximSDK/Tools/OpenOCD/scripts/ \
    -c 'source [find interface/cmsis-dap.cfg]' \
```

(continues on next page)

(continued from previous page)

```
    -c 'source [find target/max32690.cfg]' \
    -c 'init' -c 'targets' -c 'reset init' \
    -c 'flash write_image erase ./build/zephyr/zephyr.hex' \
    -c 'reset run' \
    -c 'shutdown'
```

Run tests on device (`optimize` is not necessary, since compilation was done before simulation in Renode):

```
kenning test report --cfg ./kenning-scenarios/vae-inference/tvm-max32690.yml \
    --measurements workspace/vae-tvm-hw.json \
    --report-path reports/vae-tvm-hw/report.md --to-html \
    --verbosity INFO
```

6.2.6 Comparing performance and quality of runtimes and models

Based on the results for microTVM and TensorFlow Lite Micro runtimes, you can generate comparison reports that will show the differences in performance and quality between the two.

To generate a report for Renode simulations, run:

```
kenning report --measurements \
    workspace/vae-tflite-renode.json \
    workspace/vae-tvm-renode.json \
    --report-path reports/vae-tflite-tvm-comparison-renode/report.md --to-html \
    --verbosity INFO
```

For comparison of execution on hardware, run:

```
kenning report --measurements \
    workspace/vae-tflite-hw.json \
    workspace/vae-tvm-hw.json \
    --report-path reports/vae-tflite-tvm-comparison-hw/report.md --to-html \
    --verbosity INFO
```

The reports in HTML format will be available here:

- `reports/vae-tflite-tvm-comparison-renode/report/report.html`
- `reports/vae-tflite-tvm-comparison-hw/report/report.html`

6.2.7 Specification of the kenning.datasets.anomaly_detection_dataset module

Class AnomalyDetectionDataset

Generic dataset for anomaly detection in time series problem.

It reads data from provided CSV file and prepares sequences of data.

CSV file has to follow the schema:

Timestamp column	Param 1 name	Param 2 name	...	Param N name	Label
timestamps	Numerical values	Numerical values	...	Numerical values	Integer values

Kenning automatically discards the timestamp column, as well as the header row.

The numerical values of parameters are used as signals or data from sensors, whereas the labels specify anomaly occurrence (values greater than 0).

Each label describes whether an anomaly has been observed within window_size previous samples.

This results with final version of dataset where one entry looks like:

X			Y
Param 1 value from t - window_size + 1	...	Param N value from t - window_size + 1	
...	...	...	
Param 1 value from t - 1	...	Param N value from t - 1	
Param 1 value from t	...	Param N value from t	0 (no anomaly) or 1 (anomaly)

6.3 Evaluation and optimization of DINOv2 for depth estimation

This example demonstrates how to run, optimize and evaluate a monocular depth estimation model with Kenning.

This demo uses:

- DINOv2 model with a regression head trained for monocular depth estimation,
- NYUDepthV2 dataset, which includes RGB images of interior scenes with accompanying depth maps.

6.3.1 Prepare an environment for development

First, make sure that Kenning is installed in your virtual environment with all the required dependencies.

CPU

For CPU-only setup, simply run this command in your virtual environment:

```
uv pip install "kenning[onnxruntime,tensorflow,object_detection,reports,iree] @␣
↪git+https://github.com/antmicro/kenning.git"
```

GPU

For GPU setup, run this command in your virtual environment:

```
uv pip install "kenning[onnxruntime-gpu,tensorflow,object_detection,reports,iree]␣
↪@ git+https://github.com/antmicro/kenning.git"
```

Please note that by default the ONNX Runtime expects a system-wide NVIDIA cuBLAS installation. If you don't want to install it system-wide, you can instead run:

```
uv pip install nvidia-cublas
export LD_LIBRARY_PATH=<path_to_your_dir>/.venv/lib/python3.12/site-packages/
↪nvidia/cu13/lib:$LD_LIBRARY_PATH
```

replacing the python and CUDA versions if necessary.

6.3.2 Evaluation in ONNX framework

Running the model

We will begin by running and evaluating the model in its native ONNX framework. For that we can use a script provided in `scripts/configs/dinov2-onnx-video.yml`

```
dataset:
  type: NYUDepthDatasetV2
  parameters:
    dataset_root: build/nyu_depth
    split_fraction_test: 1
model_wrapper:
  type: Dinov2ONNX
  parameters:
    model_path: kenning:///models/depth_estimation/dinov2.onnx
    input_width: 640
    input_height: 480
runtime:
  type: ONNXRuntime
  parameters:
    save_model_path: kenning:///models/depth_estimation/dinov2.onnx
    execution_providers:
    - CUDAExecutionProvider
    - CPUExecutionProvider
```

First, we specify the dataset we want to use for the pipeline. We also set two dataset parameters:

- `dataset_root` - path where we want to save the downloaded dataset files.
- `split_fraction_test` - what percentage of the dataset we want to use for evaluation. Since we are not training the model in this pipeline, we can use the entire dataset for evaluation, so we set the parameter to `1` (100%).

Than we specify the modelwrapper that we want to use and set its parameters:

- `model_path` - path to the saved model weights. By using a special path beginning with `kenning:///` we can automatically download the weights prepared for this `ModelWrapper`.
- `input_width` and `input_height` - the width and height of images the model expects as its input. The dataset images will be automatically resized to these dimensions before being passed to the model.

Finally, we specify the runtime for the model. The runtime parameters are:

- `save_model_path` - the path from which the runtime will load the model,
- `execution_providers` - a list of ONNX Runtime Execution Providers. The runtime will select the first available provider from the list. In this case we first try to run the model on GPU with CUDA, and if it's unavailable we fall back to CPU.

Once the pipeline config is ready, we can run it with:

```
kenning test \
    --cfg scripts/configs/dinov2-onnx-nyu-depth.yml \
    --measurements measurements.json
```

Which will run inference on all the dataset images and save the results in a `measurements.json` file.

The file contains data which is not aggregated in any way and is not easily human readable, which is why Kenning provides reporting and visualization features.

Evaluation and reports

To generate the report, we need to extend the `kenning` command with extra arguments:

```
kenning test report \
    --cfg scripts/configs/dinov2-onnx-nyu-depth.yml \
    --measurements measurements.json \
    --report-path reports/report.md \
    --to-html
```

First, we add the `report` subcommand to enable reporting features. Than we specify the path for the markdown report. Finally, we add the `--to-html` flag which turns the markdown into HTML with beautifully rendered charts.

`NYUDepthDatasetV2` also allows us to set the number of best and worst predictions for visualization. Let's extend our dataset config:

```
dataset:
  type: NYUDepthDatasetV2
  parameters:
    dataset_root: build/dataset
    split_fraction_test: 1
    save_n_best: 1
    save_n_worst: 1
```

These values default to `5`, so you don't need to set them manually, but here we will lower the values to make the example more concise.

Example report

The report contains many generic sections about model size, performance etc. The two sections specific to depth estimation are:

- a table showing relevant metrics, such as Mean Absolute Relative Error, Threshold Accuracy (denoted δ), as well as regular regression metrics,

- a visualization of model predictions, including an original RGB image and heatmaps for original depth labels, model predictions, as well as the model's absolute relative error.

Following is a part of an example report:

Depth estimation metrics

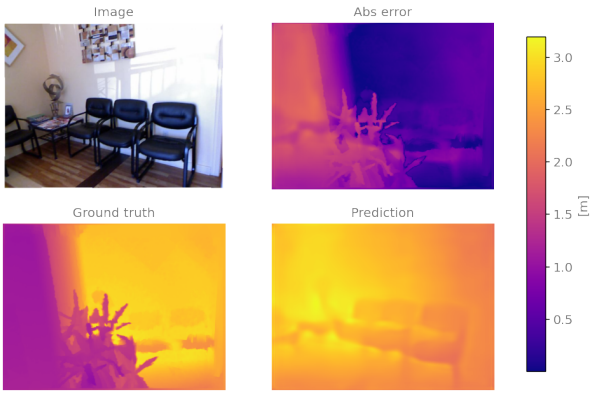
Table 6.1: Depth estimation metrics

Statistic	Value
Mean Average Error	0.3498284121534387
Root Mean Squared Error	0.521750912261734
Mean Absolute Relative Error	0.1380133881157147
$\delta^{0.125}$	0.15234914389593399
$\delta^{1.0}$	0.8476118767253278
$\delta^{2.0}$	0.9766854026483781
$\delta^{3.0}$	0.994993038032695

Best sample predictions



Worst sample predictions



6.3.3 Optimization with IREE

Kenning allows us to optimize and evaluate the model. One of the options for model optimization is compiling it with the IREE compiler.

IREE is a MLIR-based compiler which lowers AI models to a unified Intermediate Representation, which can be further compiled to an executable bytecode for running on various platforms such as desktop, microcontrollers or server environments.

Optimization with IREE for CPU

Let's define an optimization pipeline for compiling DINO for our host CPU:

```
dataset:
  type: NYUDepthDatasetV2
  parameters:
    dataset_root: build/nyu_depth
    split_fraction_test: 1
model_wrapper:
  type: Dinov2ONNX
  parameters:
    model_path: kenning:///models/depth_estimation/dinov2.onnx
    input_width: 640
    input_height: 480
optimizers:
- type: IREECompiler
  parameters:
    compiled_model_path: ./build/iree-compiled
    backend: llvm-cpu
    compiler_args:
    - iree-hal-target-device=local
    - iree-llvmcpu-target-cpu=host
    - iree-opt-level=O3
runtime:
   type: IREERuntime
   parameters:
     save_model_path: ./build/iree-compiled
     driver: local-task
     runtime_args:
     - task_topology_group_count=8
```

The dataset and model configurations stay the same.

After that we introduce the list of optimizers we want to run on the model - in our case only an IREECompiler. The most important compiler parameters are:

- `backend` - what backend we want to use for actual code generation, in our case LLVM for the cpu
- `compiler-args` - arguments passed directly to the IREE compiler, in our case:
  - `iree-hal-target-device=local` - we are compiling for the local machine
  - `iree-llvmcpu-target=host` - use CPU/architecture features (such as vector extensions) which are supported by the local host CPU

  - `iree-opt-level=O3` - enable the highest level of optimization

We also specify the runtime and configure it's parameters:

- `driver` - what IREE Runtime HAL driver we want to use. `local-task` is an async multi-threaded CPU driver
- `runtime_args` - here we specify `task_topology_group_count=8` which instructs the IREE Runtime to use 8 threads for model execution. While the runtime tries to guess the optimal thread count for a given model and host configuration, it often guesses incorrectly so it's best to experiment with this parameter.

Note

Additional optimizations can be enabled by adding an `iree-opt-data-tiling` argument to the compiler. However, as of writing, this argument breaks the DINOv2 compilation.

To run the optimization pipeline and evaluate the resulting model, we just need to add a new `optimize` subcommand and change the config name:

```
kenning optimize test report \
    --cfg scripts/configs/dinov2-iree-cpu-nyu-depth.yml \
    --measurements measurements.json \
    --report-path reports/report.md \
    --to-html
```

We can than inspect the report to see if the model performance has improved without significant degradation of the metrics.

Caution

By default the `NYUDepthDatasetV2` saves several best and worst prediction images and depth maps. The saving can be resource intensive and can introduce random latency spikes in the inference loop. Because of that, when evaluating model performance, it's best to set `save_n_best` and `save_n_worst` parameters to `0`.

Optimization with IREE for GPU

To compile the model for a CUDA-enabled GPU, we can use the following config:

```
dataset:
  type: NYUDepthDatasetV2
  parameters:
    dataset_root: build/nyu_depth
    split_fraction_test: 1
model_wrapper:
  type: Dinov2ONNX
  parameters:
    model_path: kenning:///models/depth_estimation/dinov2.onnx
    input_width: 640
    input_height: 480
```

(continues on next page)

(continued from previous page)

```
optimizers:
- type: IREECompiler
  parameters:
    compiled_model_path: ./build/iree-compiled
    backend: cuda
    compiler_args:
    - iree-hal-target-device=cuda
    - iree-cuda-target=sm_86
    - iree-opt-level=O3
    - iree-preprocessing-pass-pipeline=builtin.module(util.func(iree-global-opt-
↪raise-special-ops, iree-flow-canonicalize), iree-preprocessing-transpose-
↪convolution-pipeline, iree-preprocessing-pad-to-intrinsics, util.func(iree-
↪preprocessing-generalize-linalg-matmul-experimental),iree-preprocessing-convert-
↪conv-to-channels-last,iree-preprocessing-convert-conv-filter-to-channels-last
↪{filter-layout=fhwc})
runtime:
  type: IREERuntime
  parameters:
    save_model_path: ./build/iree-compiled
    driver: cuda
```

The main differences between the CPU config include:

- `backend`, `iree-hal-target-device`, and `driver` changed to `cuda`
- `iree-cuda-target` - the compute capability of the GPU (`sm_86` stands for compute capability 8.6)
- `iree-preprocessing-pass-pipeline` - custom arrangement of compiler preprocessing passes tailored for the DINOv2 model that reorganize convolution layout for faster GPU inference

Attention

You should adjust the `iree-cuda-target` according to your GPU compute capability. You can check your GPU compute capability in the NVIDIA documentation.

To run the CUDA optimization pipeline, it's enough to change the config name:

```
kenning optimize test report \
    --cfg scripts/configs/dinov2-iree-cuda-nyu-depth.yml \
    --measurements measurements.json \
    --report-path reports/report.md \
    --to-html
```

6.3.4 Comparing different models

Kenning also provided support for comparing different model performance and metrics.

Saving measurements files for comparison

To compare different models, we first need to save measurement files for multiple models. Let's collect them for the native ONNX Runtime pipeline and for the model compiled with IREE for the CPU:

```
kenning test \
    --cfg scripts/configs/dinov2-onnx-nyu-depth.yml \
    --measurements onnx-measurements.json
```

```
kenning optimize test \
    --cfg scripts/configs/dinov2-iree-cpu-nyu-depth.yml \
    --measurements iree-cpu-measurements.json
```

Please note the different paths of the measurement files.

Generating the comparison report

Later we can generate a comparison report using:

```
kenning report \
    --measurements onnx-measurements.json \
    --measurements iree-cpu-measurements.json \
    --report-path reports/comparison_report.md \
    --to-html
```

The generated report now includes many sections with comparison charts. Below is an example of a bubble chart showing:

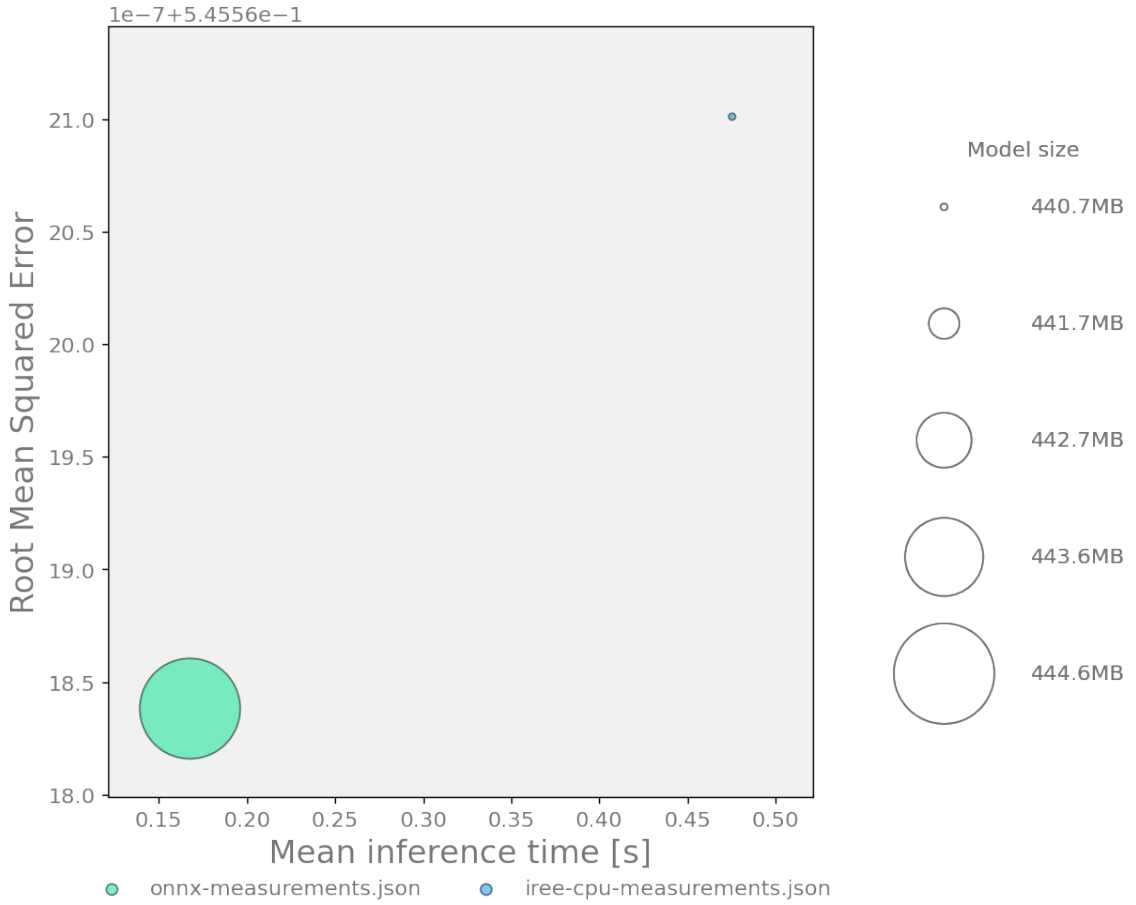
- mean inference time (x axis),
- selected model metric - in this case RMSE (y axis, see below for customizing this metric),
- model size (bubble size).

We can see that the IREE model in this case is actually slower but smaller, while achieving the same metric values (down to rounding precision errors).

Customizing the compared metric

The models are compared based on a selected metric - RMSE by default for depth estimation. We can change this metric by adding an extra command line argument to the report generation command:

```
kenning report \
    --measurements onnx-measurements.json \
    --measurements iree-cpu-measurements.json \
    --main-quality-metric=mare \
    --report-path reports/comparison_report.md \
    --to-html
```



6.4 Displaying information about available classes

The Kenning project provides several scripts for assessing information about classes (such as *Dataset*, *ModelWrapper*, *Optimizer*).

Below, we provide an overview of means to display this information.

First, make sure that Kenning is installed:

```
uv pip install "kenning @ git+https://github.com/antmicro/kenning.git"
```

6.4.1 Kenning list

`kenning list` lists all available classes, grouping them by the base class (group of modules).

The script can be executed as follows:

```
kenning list
```

This will return a list similar to the one below:

```
Optimizers (in kenning.optimizers):

    kenning.optimizers.onnx.ONNXCompiler
    kenning.optimizers.tensorflow_optimizers.TensorFlowOptimizer
    kenning.optimizers.tvm.TVMCompiler
```

(continues on next page)

(continued from previous page)

```
    kenning.optimizers.iree.IREECompiler
    kenning.optimizers.tensorflow_pruning.TensorFlowPruningOptimizer
    kenning.optimizers.tensorflow_clustering.TensorFlowClusteringOptimizer
    kenning.optimizers.nni_pruning.NNIPruningOptimizer
    kenning.optimizers.litert.LiteRTCompiler
    kenning.optimizers.model_inserter.ModelInserter

Datasets (in kenning.datasets):

    kenning.datasets.random_dataset.RandomizedClassificationDataset
    kenning.datasets.coco_dataset.COCODataset2017
    kenning.datasets.open_images_dataset.OpenImagesDatasetV6
    kenning.datasets.helpers.detection_and_segmentation.
↪ObjectDetectionSegmentationDataset
    kenning.datasets.magic_wand_dataset.MagicWandDataset
    kenning.datasets.common_voice_dataset.CommonVoiceDataset
    kenning.datasets.pet_dataset.PetDataset
    kenning.datasets.random_dataset.RandomizedDetectionSegmentationDataset
    kenning.datasets.imagenet_dataset.ImageNetDataset
    kenning.datasets.visual_wake_words_dataset.VisualWakeWordsDataset

Modelwrappers (in kenning.modelwrappers):

    kenning.modelwrappers.instance_segmentation.pytorch_coco.PyTorchCOCOMaskRCNN
    kenning.modelwrappers.object_detection.darknet_coco.TVMDarknetCOCOYOLOV3
    kenning.modelwrappers.instance_segmentation.yolact.YOLACTWithPostprocessing
    kenning.modelwrappers.classification.tensorflow_imagenet.TensorFlowImageNet
    kenning.modelwrappers.instance_segmentation.yolact.YOLACTWrapper
    kenning.modelwrappers.object_detection.yolo_wrapper.YOLOWrapper
    kenning.modelwrappers.frameworks.tensorflow.TensorFlowWrapper
    kenning.modelwrappers.classification.tflite_magic_wand.MagicWandModelWrapper
    kenning.modelwrappers.classification.tflite_person_detection.
↪PersonDetectionModelWrapper
    kenning.modelwrappers.instance_segmentation.yolact.YOLACT
    kenning.modelwrappers.frameworks.pytorch.PyTorchWrapper
    kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2
    kenning.modelwrappers.object_detection.yolov4.ONNXYOLOV4
    kenning.modelwrappers.classification.pytorch_pet_dataset.
↪PyTorchPetDatasetMobileNetV2

...
```

The output of the command can be limited by providing one or more positional arguments representing module groups:

- `optimizers`,
- `runners`,

- dataproviders,
- datasets,
- modelwrappers,
- outputcollectors,
- runtimes.

The command can also be used to list available runtimes:

```
kenning list runtimes
```

Which will return a list similar to the one below::

```
Runtimes (in kenning.runtimes):

    kenning.runtimes.iree.IREERuntime
    kenning.runtimes.litert.LiteRTRuntime
    kenning.runtimes.pytorch.PyTorchRuntime
    kenning.runtimes.tvm.TVMRuntime
    kenning.runtimes.onnx.ONNXRuntime
```

More verbose information is available with `-v` and `-vv` flags. They will display dependencies, descriptions and other information for each class.

6.4.2 Kenning info

`kenning info` displays more detailed information about a particular class. This information is especially useful when creating JSON scenario configurations. The command displays the following:

- docstrings
- dependencies, along with information on availability in the current Python environment
- supported input and output formats
- argument structure used in JSON

Let's consider a scenario where we want to compose a Kenning flow utilizing a YOLOv4 ModelWrapper. Execute the following command:

```
kenning info kenning.modelwrappers.object_detection.yolov4.ONNXYOLOV4
```

This will display all the necessary information about the class:

```
Class: ONNXYOLOV4

Input/output specification:
* input
  * shape: (1, 3, keyparams['width'], keyparams['height'])
  * dtype: float32
* output
  * shape: (1, 255, (keyparams['width'] // (8 * (2 ** 0))), (keyparams['height'] /
↪/ (8 * (2 ** 0))))
```

(continues on next page)

(continued from previous page)

```
  * dtype: float32
* output.3
  * shape: (1, 255, (keyparams['width'] // (8 * (2 ** 1))), (keyparams['height'] /
↪/ (8 * (2 ** 1))))
  * dtype: float32
* output.7
  * shape: (1, 255, (keyparams['width'] // (8 * (2 ** 2))), (keyparams['height'] /
↪/ (8 * (2 ** 2))))
  * dtype: float32
* detection_output
  * type: List[DetectObject]

Dependencies:
* torch
* numpy
* onnx
* torch.nn.functional

Arguments specification:
* classes
  * argparse_name: --classes
  * convert-type: builtins.str
  * type
    * string
  * description: File containing Open Images class IDs and class names in CSV␣
↪format to use (can be generated using kenning.scenarios.open_images_classes_
↪extractor) or class type
  * default: coco
* model_path
  * argparse_name: --model-path
  * convert-type: kenning.utils.resource_manager.ResourceURI
  * type
    * string
  * description: Path to the model
  * required: True
```

Note

By default, the command only performs static code analysis. For example, some values in the input/output specification are expressions because the command did not import or evaluate any values. This is done to allow for missing dependencies.

Loading a class with arguments

To gain access to more detailed information, the `--load-class-with-args` argument can be used. Provided that all dependencies are satisfied, the script will load the verified module to collect more detailed information about available settings.

In the example above, the ONNXYOLOV4 configuration specifies that the `model_path` argument is required. All dependencies are available as there is no warning message.

To load a class with arguments, run this command:

```
kenning info kenning.modelwrappers.object_detection.yolov4.ONNXYOLOV4 \
  --load-class-with-args \
  --model-path kenning:///models/detection/yolov4.onnx
```

```
Class: ONNXYOLOV4

Input/output specification:
* input
  * shape: (1, 3, 608, 608)
  * dtype: float32
* output
  * shape: (1, 255, 76, 76)
  * dtype: float32
* output.3
  * shape: (1, 255, 38, 38)
  * dtype: float32
* output.7
  * shape: (1, 255, 19, 19)
  * dtype: float32
* detection_output
  * type: List[DetectObject]

Dependencies:
* onnx
* numpy
* torch.nn.functional
* torch

Arguments specification:
* classes
  * argparse_name: --classes
  * convert-type: builtins.str
  * type
    * string
  * description: File containing Open Images class IDs and class names in CSV␣
↪format to use (can be generated using kenning.scenarios.open_images_classes_
↪extractor) or class type
  * default: coco
* model_path
  * argparse_name: --model-path
  * convert-type: kenning.utils.resource_manager.ResourceURI
  * type
    * string
  * description: Path to the model
  * required: True
```

6.5 Using Kenning with ROS 2 for evaluation, optimization and deployment

This example demonstrates how to optimize, run and evaluate an instance segmentation model using Kenning and ROS 2 nodes.

For this task, YOLACT (You Only Look At CoefficienTs) model will be used. The model will be deployed on a CPU or GPU using Kenning's TVMCompiler, which is a wrapper for the TVM Deep Neural Network Compiler.

6.5.1 Dependencies

For this example you need:

1. Software:
   - repo tool to clone all necessary repositories
   - Docker to use a prepared environment
   - nvidia-container-toolkit to provide access to the GPU in the Docker container (**optional**)
2. Hardware:
   - A camera for streaming frames
   - A CUDA-enabled NVIDIA GPU for inference acceleration (**optional**)

6.5.2 Installation

To simplify the installation, a Docker image (**Ubuntu 24.04, Python 3.12**) containing all the dependencies required to run the environment has been prepared. You can either pull the pre-built image (**GPU, CUDA only**) or build it from scratch yourself. Currently, following platforms are supported:

- x86_64 / arm64 (CPU)
- x86_64 / arm64 (GPU, CUDA)
- NVIDIA Jetson

  NOTE

  See README.md for more information about supported platforms. The resulting image comes with UV ready to use inside the container (venv is activated), for example `uv pip install torch` can be used.

Download the demo

Create a workspace directory, where all downloaded repositories will be stored:

```
mkdir kenning-ros2-demo && cd kenning-ros2-demo
```

Then, download all dependencies using the `repo` tool:

```
repo init -u https://github.com/antmicro/ros2-gui-node.git -m examples/kenning-
↪instance-segmentation/manifest.xml
repo sync -j`nproc`
```

NOTE

Before executing `repo` command you may need to set up git credential by typing into terminal:

```
git config --global user.email "<e-mail address>"
git config --global user.name "Name Surname"
```

It downloads the following repositories:

- Kenning for model optimization and runtime, in the `kenning` directory
- ROS 2 Camera node for obtaining frames from the camera and serving its parameters as ROS 2 parameters, in the `src/camera_node` directory
- Kenning's ROS 2 messages and services for computer vision, in the `src/computer_vision_msgs` directory
- ROS 2 GUI Node, in the `src/gui_node` directory

Prepare the Docker environment

By default, running `./build-docker.sh <platform>` does not build TVM. Since this tutorial compiles the YOLACT model with TVM, you'll need to either install your own TVM wheel after the build (see the NOTE below), or build TVM from source by passing the `--build-tvm` flag.

CPU

```
./src/gui_node/environments/build-docker.sh cpu
```

GPU

```
./src/gui_node/environments/build-docker.sh gpu
```

Jetson

```
./src/gui_node/environments/build-docker.sh jetson
```

NOTE

Omitting `--build-tvm` is faster to build, but then TVM has to be installed manually afterwards via `uv pip install "./kenning[tvm]"` (CPU) or `uv pip install "./kenning[tvm-cuda]"` (GPU) before running the steps below. For more details on how to use this script and what it does, refer to: README.md

Running the container

Allow non-network local connections to X11 so that the GUI can be started from the Docker container:

```
xhost +local:
```

The pulled or built image can be run with the following command (you need to pass the appropriate platform argument):

CPU

```
./src/gui_node/environments/run-docker.sh cpu
```

GPU

```
./src/gui_node/environments/run-docker.sh gpu
```

Jetson

```
./src/gui_node/environments/run-docker.sh jetson
```

NOTE

For more details on how to use this script and what it does, refer to: README.md

6.5.3 Install Kenning

Install Kenning with necessary dependencies:

```
uv pip install "./kenning[object_detection, torch, tvm, reports]"
```

6.5.4 Compiling the model

TVM compilation involves converting the network into an **Intermediate Representation (IR)**, where computational graph optimizations are performed, including operation fusion (combining consecutive layers into one, which reduces memory transfers). The compiler then generates machine code optimized for a specific hardware architecture (e.g., **x86, ARM, RISC-V, CUDA**), utilizing its specific instructions (e.g., **AVX-512, NEON, or Tensor Cores**) to maximize inference performance.

In Kenning, we can do this incredibly easily. What's more, in this particular example, we don't need to configure anything, since separate scripts have been prepared for each platform. Simply run:

CPU

```
kenning optimize --cfg src/gui_node/examples/kenning-instance-segmentation/yolact-
↪tvm-cpu-optimization.yaml
```

GPU

```
kenning optimize --cfg src/gui_node/examples/kenning-instance-segmentation/yolact-
↪tvm-gpu-optimization.yaml
```

Jetson

```
kenning optimize --cfg src/gui_node/examples/kenning-instance-segmentation/yolact-
↪tvm-gpu-optimization.yaml
```

6.5.5 Evaluation

To evaluate the model above, we can either use a YAML configuration file, or specify the required arguments for the test scenario directly in the CLI (*Using Kenning via command-line arguments*):

CPU

```
kenning test report --cfg src/gui_node/examples/kenning-instance-segmentation/
↪yolact-tvm-cpu-optimization.yaml
```

GPU

```
kenning test report --cfg src/gui_node/examples/kenning-instance-segmentation/
↪yolact-tvm-gpu-optimization.yaml
```

Jetson

```
kenning test report --cfg src/gui_node/examples/kenning-instance-segmentation/
↪yolact-tvm-gpu-optimization.yaml
```

This command will evaluate the model on the dataset, collect performance and quality metrics into the file specified by `--measurements`, and then generate a Markdown report from them (as well as HTML).

6.5.6 Running the demo

ROS2 GUI Node is a project created for visualizing data from ROS 2. ROS 2 itself is a robotics middleware based on a publish-subscribe (pub/sub) pattern and a node-based architecture. Conceptually, it functions much like a microservices framework, enabling the development of efficient and modular applications for edge devices. In this architecture, every module - from the camera, through individual AI models, to the graphical user interface (GUI) - runs as a separate, independent node.

First of all, load the `setup.sh` script for ROS 2 tools:

```
source /opt/ros/$ROS_DISTRO/setup.sh
```

Then, build the GUI node and the Camera node with:

```
colcon build --base-paths src --cmake-args -DBUILD_KENNING_YOLACT_DEMO=y -
↪DPython3_EXECUTABLE=/opt/venv/bin/python3
```

Next, load the ROS 2 environment including the newly built packages:

```
source install/setup.sh
```

Finally, launch Kenning, Camera node, and GUI node using the launch file:

CPU

```
ros2 launch gui_node kenning-instance-segmentation-cpu.py use_gui:=true
```

GPU

```
ros2 launch gui_node kenning-instance-segmentation.py use_gui:=true
```

Jetson

```
ros2 launch gui_node kenning-instance-segmentation.py use_gui:=true
```

If you don't want to use the GUI at all, run without `use_gui:=true`:

CPU

```
ros2 launch gui_node kenning-instance-segmentation-cpu.py
```

GPU

```
ros2 launch gui_node kenning-instance-segmentation.py
```

Jetson

```
ros2 launch gui_node kenning-instance-segmentation.py
```

Lastly, a GUI should appear, with:

- Direct view from Camera node
- Instance segmentation view based on predictions from Kenning (started using `kenning flow` with `./kenning-instance-segmentation.yaml` or `kenning-instance-segmentation-cpu.yaml` if you are not using a GPU)
- A widget visualizing a list of detected objects, with a possibility to filter out not interesting classes

6.5.7 Summary

In this example, we used Kenning together with ROS 2 to optimize, evaluate, and deploy an instance segmentation model. Starting from the YOLACT model, we compiled it using TVM for CPU, GPU, and Jetson platforms, evaluated its performance and quality with `kenning test`, and finally ran a live demo streaming camera frames through Kenning and visualizing the detected instances in the ROS 2 GUI Node. The same workflow can be easily adapted to other models and datasets supported by Kenning.

6.6 Evaluating models on hardware using Kenning Zephyr Runtime

This section contains tutorial of evaluating models on microcontrollers using Kenning Zephyr Runtime and Renode.

6.6.1 Preparing the Zephyr environment

First, we need to setup environment for building the runtime. Start with installing dependencies:

- Zephyr dependencies
- `jq`
- `curl`
- `west`
- `CMake`
- `python` with `uv`

On Debian-based Linux distributions, the above-listed dependencies can be installed as follows:

```
apt update

apt install -y --no-install-recommends ccache curl device-tree-compiler dfu-util␣
↪file \
  g++-multilib gcc gcc-multilib git jq libmagic1 libsdl2-dev make ninja-build \
  python3-dev python3-setuptools python3-tk python3-wheel python3-venv \
  mono-complete wget xxd xz-utils patch
```

Next, create a Zephyr workspace directory and clone there Kenning Zephyr Runtime repository:

```
mkdir -p zephyr-workspace && cd zephyr-workspace
git clone https://github.com/antmicro/kenning-zephyr-runtime.git
cd kenning-zephyr-runtime
```

Then, initialize Zephyr workspace, and install Python dependencies with:

```
uv venv .venv
source .venv/bin/activate
uv pip install west
west init -l .
west update
west zephyr-export
uv pip install -r requirements.txt -r ../zephyr/scripts/requirements-base.txt
```

Install latest Zephyr SDK with:

```
west sdk install --toolchains x86_64-zephyr-elf aarch64-zephyr-elf arm-zephyr-
↪eabi riscv64-zephyr-elf
```

Finally, prepare additional Zephyr modules:

```
./scripts/prepare_modules.sh
```

6.6.2 Installing Kenning with Renode support

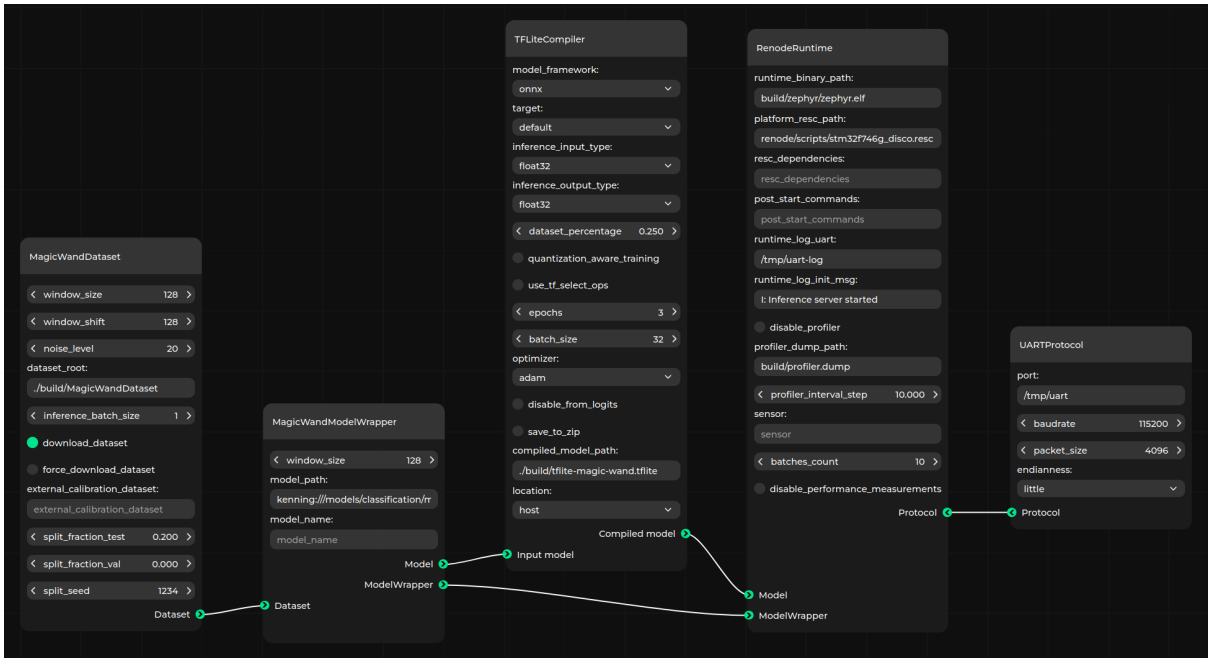
Evaluating models using Kenning Zephyr Runtime requires Kenning with Renode support. Use uv to install it:

```
uv pip install "kenning[tvm,tensorflow,reports,renode,uart,tflite,torch] @␣
↪git+https://github.com/antmicro/kenning.git"
```

To use Renode, either follow Renode documentation or use the script:

```
source ./scripts/prepare_renode.sh
```

6.6.3 Building and evaluating Magic Wand model using LiteRT backend



Let's build the Kenning Zephyr Runtime with LiteRTMicro as model executor for `stm32f746g_disco` board. Run:

```
west build --board stm32f746g_disco app -- -DEXTRA_CONF_FILE=litert.conf
west build -t board-repl
```

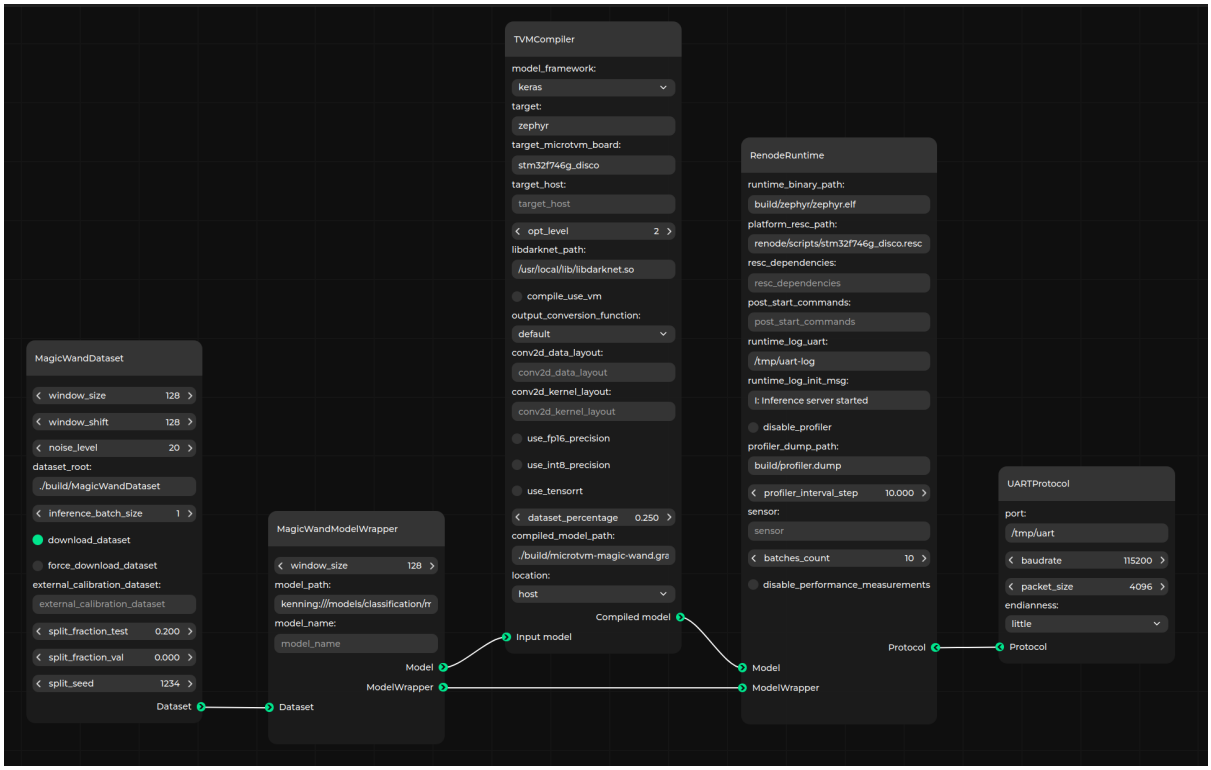
The built binary can be found in `build/zephyr/zephyr.elf`.

To evaluate the Magic Wand model using built runtime, run:

```
kenning optimize test \
    --cfg kenning-scenarios/magic-wand-inference/tflite/renode-auto-stm32f746g.
↪yml \
    --measurements build/zephyr-stm32-tflite-magic-wand.json --verbosity INFO \
    --verbosity INFO
```

The evaluation results would be saved at `build/zephyr-stm32-tflite-magic-wand.json`.

6.6.4 Building and evaluating Magic Wand model using microTVM backend



Building the Kenning Zephyr Runtime with microTVM support for the same board requires changing only `-DEXTRA_CONF_FILE` value. Run:

```
west build --board stm32f746g_disco app -- -DEXTRA_CONF_FILE=tvm.conf
west build -t board-repl
```

And now, as previously, run evaluation using Kenning:

```
kenning optimize test \
    --cfg kenning-scenarios/magic-wand-inference/tvm/renode-auto-stm32f746g.yml \
    --measurements build/zephyr-stm32-tvm-magic-wand.json --verbosity INFO \
    --verbosity INFO
```

The evaluation results would be saved at `zephyr-stm32-tvm-magic-wand.json`.

6.6.5 Comparing the results

To generate comparison report run:

```
kenning report \
    --measurements \
      build/zephyr-stm32-tflite-magic-wand.json \
      build/zephyr-stm32-tvm-magic-wand.json \
    --report-path build/zephyr-stm32-tflite-tvm-comparison.md \
    --to-html
```

The HTML version of the report should be saved at `build/zephyr-stm32-tflite-tvm-comparison/zephyr-stm32-tflite-tvm-comparison.html`.

6.6.6 Collecting ML-aware traces using Zephelin and generating a trace report

Kenning is integrated with Zephelin - a tool for ML-aware tracing of Zephyr applications. This integration allows for seamless tracing of models tested in Kenning Zephyr Runtime. As an example, we will collect traces using GDB during inference of Magic Wand model with microTVM on `stm32f746g_disco`.

First, build Kenning Zephyr Runtime with Zephelin configured for GDB tracing:

```
west build -p -b stm32f746g_disco app -- -DEXTRA_CONF_FILE="tvm.conf;$(realpath ./
↪zpl.conf);zpl_gdb.conf"
west build -t board-repl
```

Second, install Zephelin's Python dependencies and the GDB debugger with support for multiple architectures:

```
uv pip install -r ../zephelin/requirements.txt
apt install -y gdb-multiarch
```

Then run optimization, inference and report generation:

```
kenning optimize test report \
    --cfg kenning-scenarios/magic-wand-inference/tvm/renode-zephelin-gdb-
↪stm32f746g.yml \
    --measurements build/zephelin-zephyr-stm32-tvm-magic-wand.json --verbosity␣
↪INFO \
    --report-path build/zephelin-zephyr-stm32-tvm.md \
    --to-html
```

Note

Platform parameter `enable_zephelin` needs to be set to `true` (as you can see inside the `kenning-scenarios/zephelin-renode-zephyr-tvm-magic-wand-inference.yml` file), to enable automatic trace collection by Kenning. For manual trace collection, please refer to Zephelin documentation.

6.7 Large Language Models with Safetensors

This example demonstrates how to optimize large language models using the safetensors format. This is useful when deploying quantized LLMs by quickly switching between different quantization methods, such as GPTQ and AWQ. For the purpose of demonstration, SmolLM2 is used for quick optimization and report generation.

6.7.1 Setup

To get started, clone the Kenning repository:

```
git clone https://github.com/antmicro/kenning.git
cd kenning/
```

and install it with

```
uv pip install ".[llm,sparsity-aware-kernel,reports]"
```

Info

This example requires a CUDA-enabled GPU.

Depending on the example, you may also need:

- CUDA Toolkit (13.0 recommended), including NVCC and NVRTC, to compile CUDA code.
- cuDNN (9 recommended) for operations that use cuDNN.
- NVML to generate GPU reports.

Check the example to see which of these components are needed.

6.7.2 AWQ Optimization

The first example shows how to use AWQ optimization to protect important weights in order to maintain model accuracy. The configuration specifies the `model_framework` as `safetensors` to ensure compatible loading.

Use the following configuration:

Listing 6.2: llm-smol-awq.yaml

```yaml
dataset:
  type: kenning.datasets.cnn_dailymail.CNNDailymailDataset
  parameters:
    dataset_root: ./build/CNNDailymailDataset
    inference_batch_size: 128
    gather_predictions: true
    split_fraction_test: 0.001
model_wrapper:
  type: kenning.modelwrappers.llm.smol.SmolLM2
  parameters:
    model_path: hf://HuggingFaceTB/SmolLM2-135M-Instruct
optimizers:
  - type: kenning.optimizers.awq.AWQOptimizer
    parameters:
      compiled_model_path: ./build/smollm2_awq_4bit
      target_precision: 4
      group_size: 64
runtime:
  type: kenning.runtimes.vllm.VLLMRuntime
  parameters:
    save_model_path: ./build/smollm2_awq_4bit
    max_tokens: 8192
```

Run the optimization flow:

```
kenning optimize --cfg llm-smol-awq.yaml
```

6.7.3 SparseGPTQ Optimization

Kenning supports SparseGPTQ which combines quantization with structured pruning. It is useful when reduction in both the memory footprint and computational cost is required. This approach allows the LLM to run on hardware very limited RAM and processing power.

Use the following configuration:

Listing 6.3: llm-smol-sparsegptq.yaml

```yaml
dataset:
  type: kenning.datasets.cnn_dailymail.CNNDailymailDataset
  parameters:
    dataset_root: ./build/CNNDailymailDataset
    inference_batch_size: 128
    gather_predictions: true
    split_fraction_test: 0.001
model_wrapper:
  type: kenning.modelwrappers.llm.smol.SmolLM2
  parameters:
    model_path: hf://HuggingFaceTB/SmolLM2-135M-Instruct
optimizers:
  - type: kenning.optimizers.awq.GPTQSparseGPTOptimizer
    parameters:
      compiled_model_path: ./build/smollm2_sparsegptq_4bit
      group_size: 64
runtime:
  type: kenning.runtimes.vllm.VLLMRuntime
  parameters:
    save_model_path: ./build/smollm2_sparsegptq_4bit
    max_tokens: 8192
    sparse_gptq_kernel: true
    max_model_len: 1024
```

Run the optimization flow:

```
kenning optimize --cfg llm-smol-sparsegptq.yaml
```

6.7.4 vLLM Runtime

The resulting safetensors models can be deployed using the vLLM runtime. vLLM is required to perform the tests and evaluation.

6.7.5 Results

First, perform the evaluation for each model:

```
kenning test \
  --cfg llm-smol-awq.yaml \
  --measurements build/awq.json
```

and if you want to evaluate sparsegpt tests:

```
kenning test \
  --cfg llm-smol-sparsegptq.yaml \
  --measurements build/sparsegptq.json
```

Then, generate the comparison report:

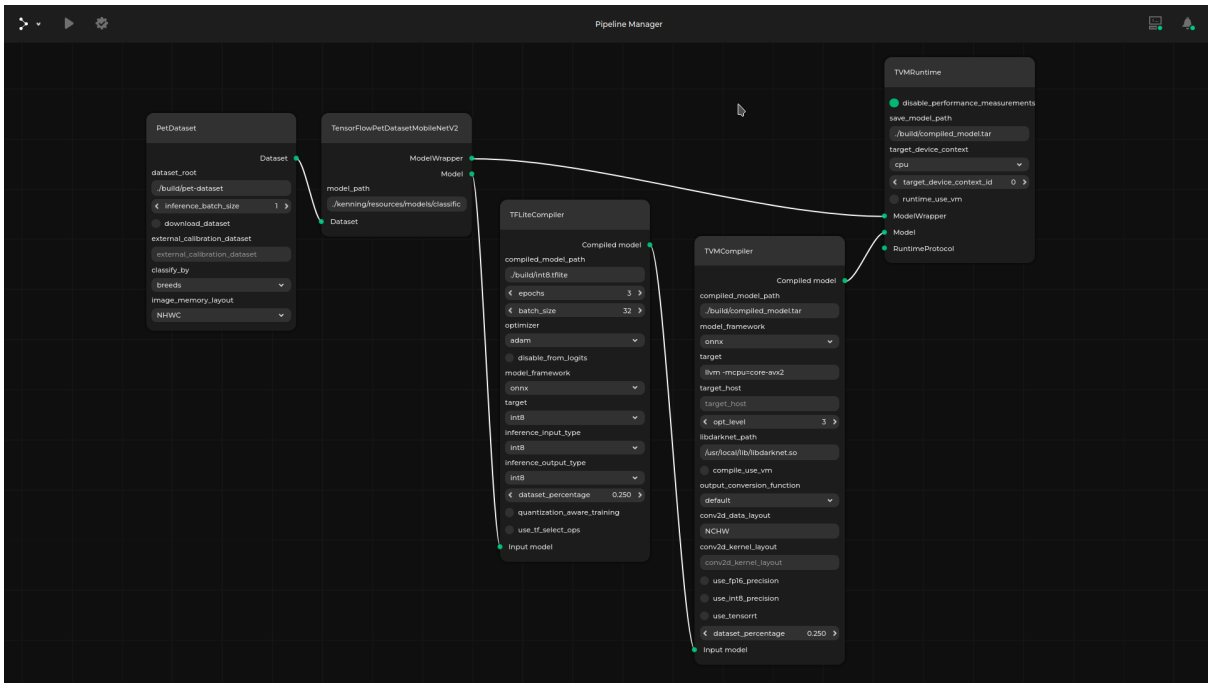
```
kenning report \
  --measurements \
    build/awq.json \
    build/sparsegptq.json \
  --report-path build/llm-comparison.md \
  --to-html
```

6.8 Visualizing Kenning data flows with Pipeline Manager

Pipeline Manager is a GUI tool that helps visualize and edit data flows.

This chapter describes how to set up Pipeline Manager and use it with Kenning graphs.

Pipeline Manager is application-agnostic and does not assume any properties of the application it is working with. Kenning, however, implements a Pipeline Manager client which provides tools for creating complex Kenning pipelines and flows, while also allowing for running and saving these configurations directly from Pipeline Manager's editor.



6.8.1 Installing Pipeline Manager

Kenning requires extra dependencies in order to run the Pipeline Manager integration. To install them, run:

```
uv pip install "kenning[pipeline_manager] @ git+https://github.com/antmicro/
↪kenning.git"
```

6.8.2 Running Pipeline Manager with Kenning

Start the Pipeline Manager client with:

```
kenning visual-editor --file-path measurements.json --workspace-dir ./workspace
```

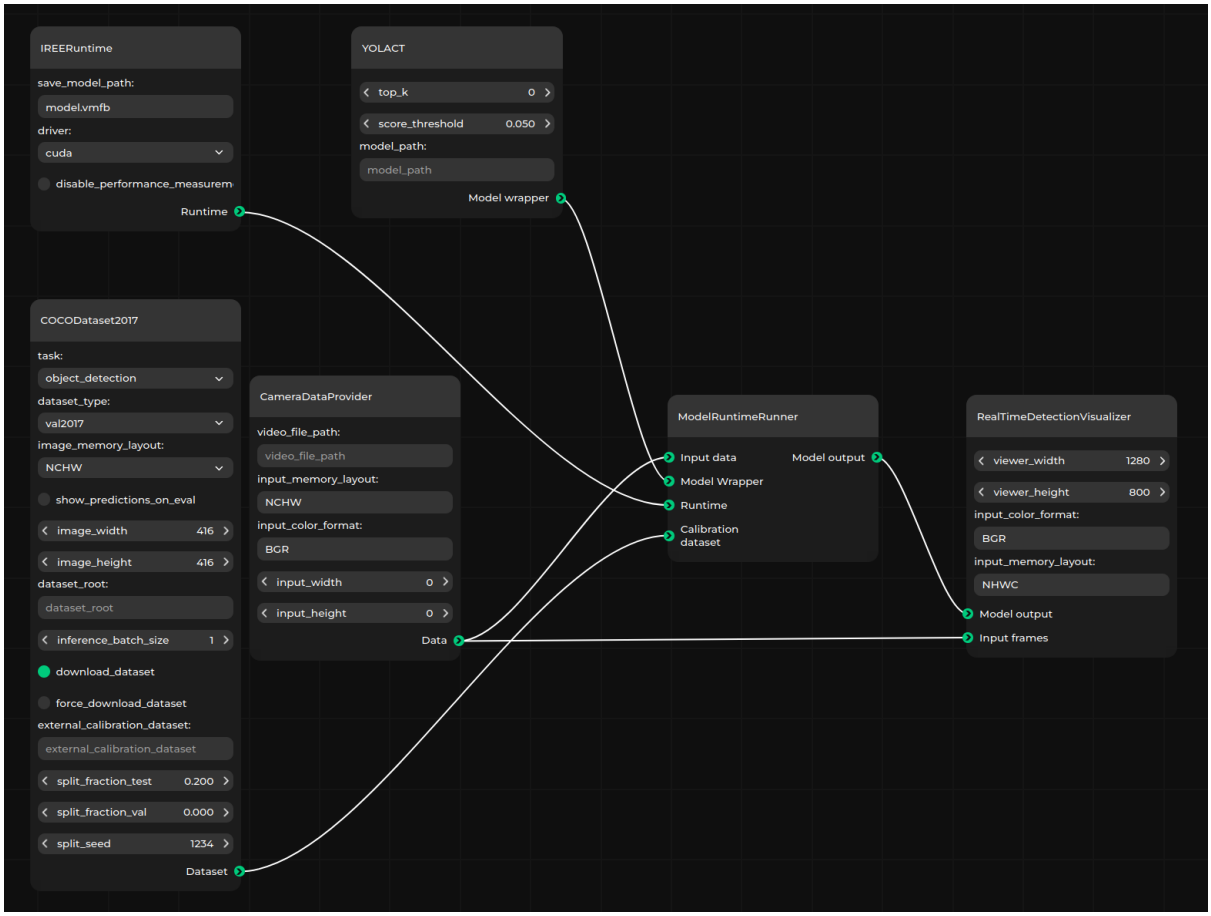
The `--file-path` option specifies where the results of model benchmarking or the runtime data will be stored.

For runtime data, the following arguments are available:

- `--spec-type` - type of Kenning scenario to be run, can be either `pipeline` (for *optimization and deployment pipeline*) or `flow` (for creating *runtime scenarios*). `pipeline` is the default type.
- `--host` - Pipeline Manager server address, default: `127.0.0.1`
- `--port` - Pipeline Manager server port, default: `9000`
- `--verbosity` - log verbosity

6.8.3 Using Pipeline Manager

In its default configuration, the web application is available under `http://127.0.0.1:5000/`.



Below, you can find an example Pipeline Manager workflow:

- `Load File` - option available from the drop-down menu on the top left, loads a JSON configuration describing a Kenning scenario.

For instance, `scripts/jsonconfigs/sample-tflite-pipeline.json` available in Kenning is a basic configuration for an *Kenning example use case for benchmarking using a native framework*.

- Graph editing - adding or removing nodes, editing connections, node options, etc.
- `Validate` - validates and returns the information whether the scenario is valid.

  For example, it will return an error when two optimizers in a chain are incompatible with each other.
- `Run` - creates and runs the optimization pipeline or *Kenning runtime flow*.
- `Save file` - saves current JSON scenario to a specified path.

More information about how to work with Pipeline Manager is available in the Pipeline Manager documentation.

6.9 Structured pruning for PyTorch models

Structured pruning is of the methods for reducing model size, which removes the least contributing neurons, filters and/or convolution kernels. In this example, we present scenarios for structured pruning of `PyTorchPetDatasetMobileNetV2` using Neural Network Intelligence.

6.9.1 Setup

Install required dependencies:

```
uv pip install "kenning[torch,nni,reports] @ git+https://github.com/antmicro/
↪kenning.git"
```

6.9.2 Experiments

In order to compare with an original model, you need to execute a scenario:

Listing 6.4: mobilenetv2-pytorch.json

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.pytorch_pet_dataset.
↪PyTorchPetDatasetMobileNetV2",
    "parameters": {
      "model_path": "kenning:///models/classification/pytorch_pet_dataset_
↪mobilenetv2_full_model.pth",
      "model_name": "torch-native"
    }
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
    "parameters": {
      "dataset_root": "./build/PetDataset",
      "image_memory_layout": "NCHW"
    }
  },
```

(continues on next page)

(continued from previous page)

```
  "optimizers": [],
  "runtime": {
    "type": "kenning.runtimes.pytorch.PyTorchRuntime",
    "parameters": {
      "save_model_path": "kenning:///models/classification/pytorch_pet_dataset_
↪mobilenetv2_full_model.pth"
    }
  }
}
```

To run it, use this command:

```
kenning test \
  --json-cfg mobilenetv2-pytorch.json \
  --measurements build/torch.json
```

Kenning supports activation-based pruners. You can choose a specific pruner with the `pruner_type` parameter:

- `apoz` - `ActivationAPoZRankPruner` based on Average Percentage of Zeros in activations,
- `mean_rank` - `ActivationMeanRankPruner` based on a metric that calculates the smallest mean value of activations.

These activations are collected during dataset inference. The number of samples collected for statistics can be modified with `training_steps`. Moreover, pruning has two modes:

- `dependency_aware` - makes pruner aware of dependencies for channels and groups.
- `normal` - dependencies are ignored.

You can also choose which `activation` the pruner will use - `relu`, `relu6` or `gelu`. Additional configuration can be specified in `config_list` which follows a format defined in the NNI specification. When `exclude_last_layer` is positive, the optimizer will be configured to exclude the last layer from the pruning process, to prevent changing the size of the output. Apart from that, `confidence` defines the coefficient for sparsity inference and batch size of the dummy input for the process. When a GPU is available, it is used by default, but as pruning can be memory-consuming, the `pruning_on_cuda` option enables manual GPU usage configuration during the process.

Other arguments affect fine-tuning of the pruned model, e.g. `criterion` and `optimizer` accept paths to classes, respectively calculating a criterion and optimizing a neural network. The number of `finetuning_epochs`, the `finetuning_batch_size` and `finetuning_learning_rate` can be modified.

Listing 6.5: pruning-mobilenetv2-pytorch.json

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.pytorch_pet_dataset.
↪PyTorchPetDatasetMobileNetV2",
    "parameters": {
      "model_path": "kenning:///models/classification/pytorch_pet_dataset_
```

(continues on next page)

(continued from previous page)

```
↪mobilenetv2_full_model.pth",
      "model_name": "nni-pruning-0_05"
    }
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
    "parameters": {
      "dataset_root": "./build/PetDataset",
      "image_memory_layout": "NCHW"
    }
  },
  "optimizers": [
    {
      "type": "kenning.optimizers.nni_pruning.NNIPruningOptimizer",
      "parameters": {
        "pruner_type": "mean_rank",
        "config_list": [
          {
            "total_sparsity": 0.05,
            "op_types": [
              "Conv2d",
              "Linear"
            ]
          }
        ],
        "training_steps": 16,
        "activation": "relu",
        "compiled_model_path": "build/nni-pruning-0_05.pth",
        "mode": "dependency_aware",
        "finetuning_epochs": 3,
        "finetuning_batch_size": 64,
        "confidence": 8,
        "criterion": "torch.nn.CrossEntropyLoss",
        "optimizer": "torch.optim.Adam",
        "pruning_on_cuda": true,
        "finetuning_learning_rate": 0.00005,
        "exclude_last_layer": true
      }
    }
  ],
  "runtime": {
    "type": "kenning.runtimes.pytorch.PyTorchRuntime",
    "parameters": {
      "save_model_path": "./build/nni-pruning-0_05.pth"
    }
  }
}
```

Run the above scenario with:

```
kenning optimize test \
  --json-cfg pruning-mobilenetv2-pytorch.json \
  --measurements build/nni-pruning.json
```

To ensure better quality of performance measurements, we suggest running optimization and tests separately, like below:

```
kenning optimize --json-cfg pruning-mobilenetv2-pytorch.json
kenning test \
  --json-cfg pruning-mobilenetv2-pytorch.json \
  --measurements build/nni-pruning.json
```

For more size reduction, you can use larger sparsity with adjusted parameters, like below:

```
{
  "type": "kenning.optimizers.nni_pruning.NNIPruningOptimizer",
  "parameters": {
    "pruner_type": "mean_rank",
    "config_list": [
      {
        "total_sparsity": 0.15,
        "op_types": [
          "Conv2d",
          "Linear"
        ]
      }
    ],
    "training_steps": 92,
    "activation": "relu",
    "compiled_model_path": "build/nni-pruning-0_15.pth",
    "mode": "dependency_aware",
    "finetuning_epochs": 10,
    "finetuning_batch_size": 64,
    "confidence": 8,
    "criterion": "torch.nn.CrossEntropyLoss",
    "optimizer": "torch.optim.Adam",
    "pruning_on_cuda": true,
    "finetuning_learning_rate": 1e-04,
    "exclude_last_layer": true
  }
}
```

6.9.3 Results

Models can be compared with the generated report:

```
kenning report \
  --measurements \
    build/torch.json \
    build/nni-pruning.json \
```

(continues on next page)

(continued from previous page)

```
  --report-path build/nni-pruning.md \
  --to-html
```

Pruned models have significantly fewer parameters, which results in decreased GPU and VRAM usage without increasing inference time. Summary of a few examples with different sparsity:

Sparsity	Accuracy	Number of parameters	Fine-tuning epochs	Size reduction
—	0.9632653061	4,130,853	—	0.00%
0.05	0.9299319728	3,660,421	3	11.27%
0.15	0.8102040816	2,813,380	10	31.61%

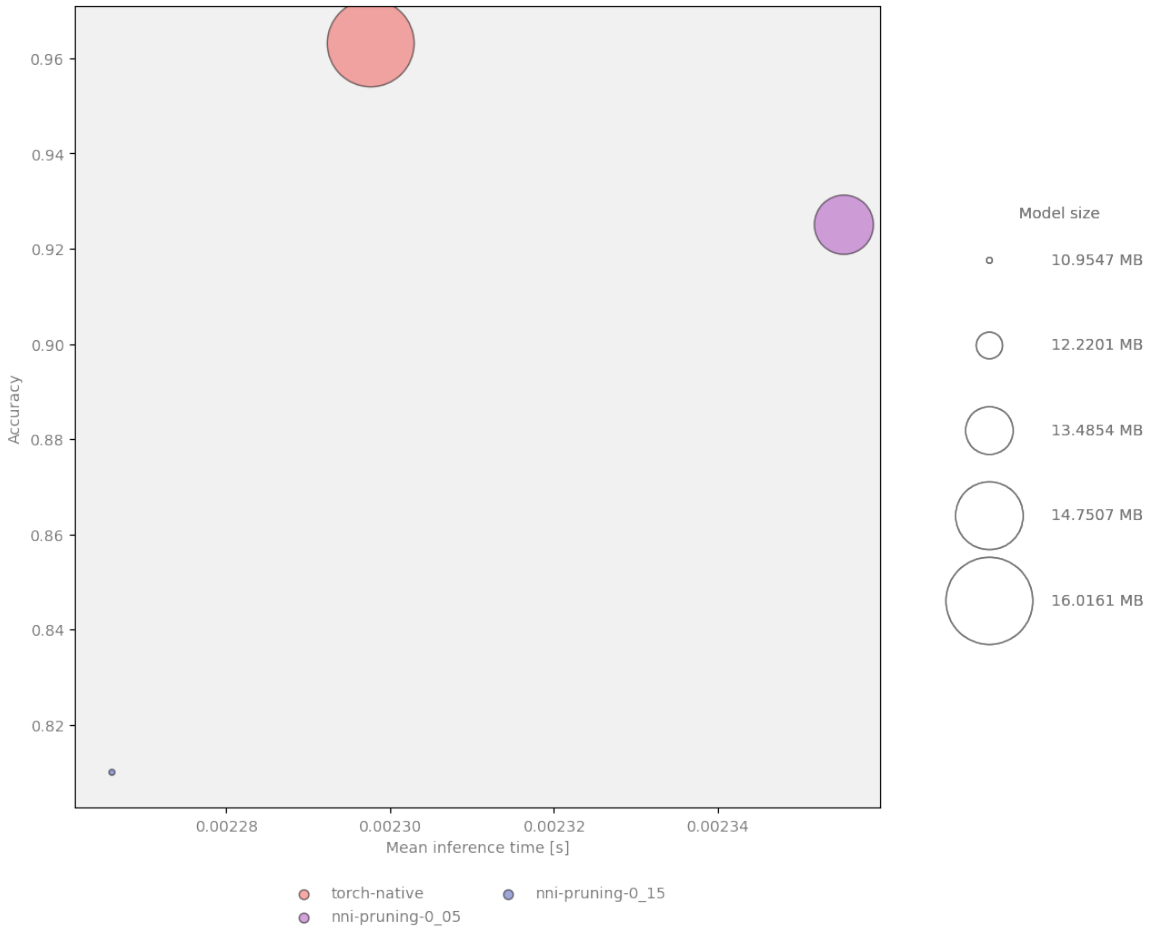


Figure 6.1: Model size, speed and quality comparison for NNI pruning

6.10 Model quantization and compilation using LiteRT and TVM

Let's consider a simple scenario where we want to optimize the inference time and memory usage of a classification model executed on a x86 CPU.

To do this, we are going to use the `PetDataset` Dataset and the `TensorFlowPetDatasetMobileNetV2` ModelWrapper.

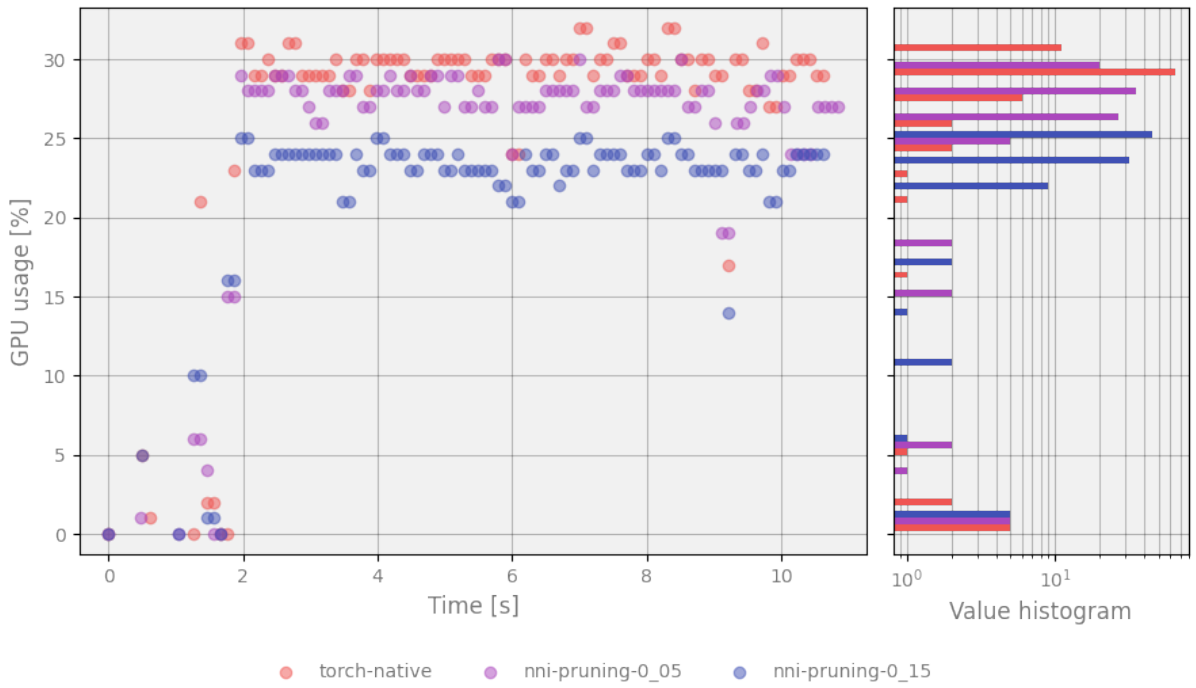


Figure 6.2: Plot represents changes of GPU usage over time

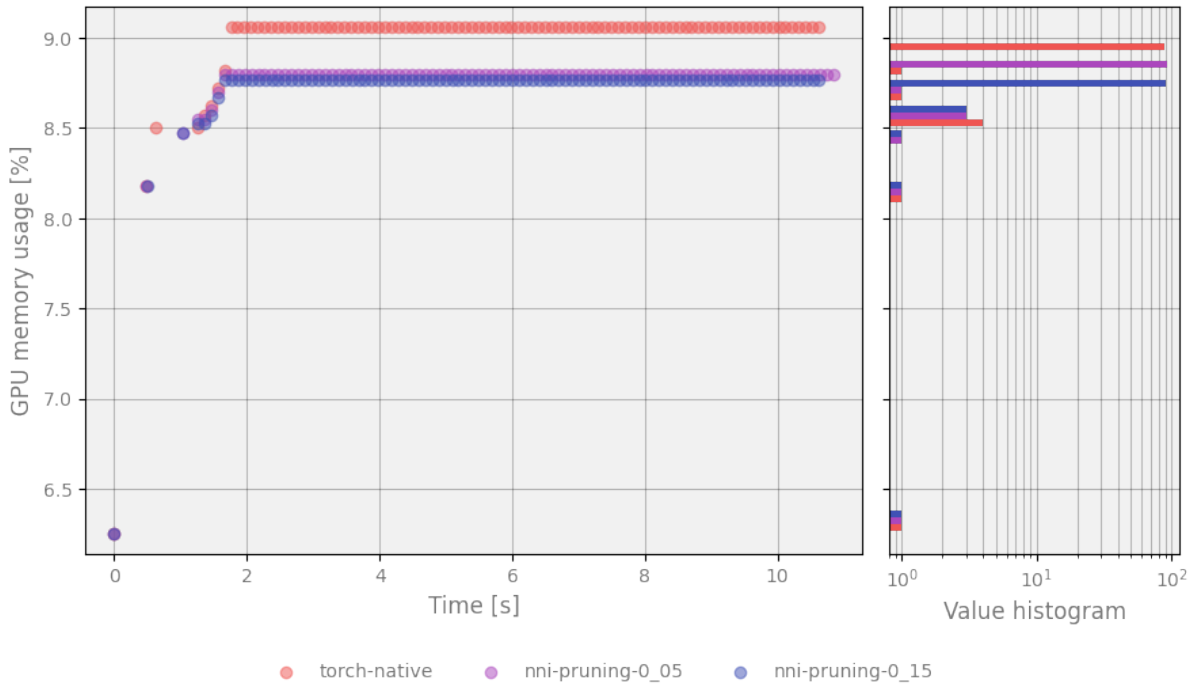


Figure 6.3: Plot represents changes of GPU RAM usage over time

6.10.1 Prepare environment

First of all, we need to install all necessary dependencies:

```
uv pip install "kenning[tensorflow,litert,tvm,reports] @ git+https://github.com/
↪antmicro/kenning.git"
```

6.10.2 Train the model (optional)

We will skip the training process, we will use tensorflow_pet_dataset_mobilenetv2.h5. In Kenning, available models and resources can be downloaded using URIs with the `kenning://` scheme, in this case `kenning:///models/classification/tensorflow_pet_dataset_mobilenetv2.h5`.

The training of the above model can be performed using the following command:

```
kenning train \
    --modelwrapper-cls kenning.modelwrappers.classification.tensorflow_pet_
↪dataset.TensorFlowPetDatasetMobileNetV2 \
    --dataset-cls kenning.datasets.pet_dataset.PetDataset \
    --logdir build/logs \
    --dataset-root build/pet-dataset \
    --model-path build/trained-model.h5 \
    --batch-size 32 \
    --learning-rate 0.0001 \
    --num-epochs 50
```

6.10.3 Benchmarking a model using a native framework

First, we want to check how the trained model performs using the native framework on CPU.

For this, we will use the `kenning test` tool. The tool is configured with JSON files (scenarios).

In our case, the JSON file (named `native.json`) will look like this:

Listing 6.6: mobilenetv2-tensorflow-native.json

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
    "parameters": {
      "model_name": "native",
      "model_path": "kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5",
      "batch_size": 32,
      "learning_rate": 0.0001,
      "num_epochs": 50,
      "logdir": "build/logs"
    }
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
```

(continues on next page)

(continued from previous page)

```
    "parameters": {
      "dataset_root": "./build/PetDataset"
    }
  }
}
```

This JSON provides a configuration for running the model natively and evaluating it against a defined Dataset.

For every class in the JSON file above, two keys are required: `type` which is a module path of our class and `parameters` which is used to provide arguments used to create instances of our classes.

In `model_wrapper`, we specify the model used for evaluation - here it is MobileNetV2 trained on PetDataset. The `model_path` is the path to the saved model. The `TensorFlowPetDatasetMobileNetV2` model wrapper provides methods for loading the model, preprocessing the inputs, postprocessing the outputs and running inference using the native framework (TensorFlow in this case).

The dataset provided for evaluation is PetDataset - here we specify that we want to download the dataset to the `./build/pet-dataset` directory (`dataset_root`). The `PetDataset` class can download the dataset (if necessary), load it, read the inputs and outputs from files, process them, and implement evaluation methods for the model.

With the config above saved in the `native.json` file, run the `kenning test` scenario:

```
kenning test \
  --json-cfg mobilenetv2-tensorflow-native.json \
  --measurements build/native.json
```

This tool runs inference based on the given configuration, evaluates the model and stores quality and performance metrics in JSON format, saved to the `build/native.json` file. All other JSONs in this example use case can be executed with this command.

To visualize the evaluation and benchmark results, run the `kenning report` tool:

```
kenning report \
  --report-path build/benchmarks/native.md \
  --measurements build/native.json \
  --root-dir build/benchmarks \
  --img-dir build/benchmarks/img \
  --report-types performance classification \
  --report-name 'native'
```

The `kenning report` tool takes the output JSON file generated by the `kenning test` tool, and creates a report titled `native`, which is saved in the `build/benchmarks/native.md` directory. As specified in the `--report-types` flag, we create `performance` and `classification` metrics sections in the report (for example, there is also a `detection` report type for object detection tasks).

The `build/benchmarks/img` directory contains images with the `native_*` prefix visualizing the confusion matrix, CPU and memory usage, as well as inference time.

The `build/benchmarks/native.md` file is a Markdown document containing a full report for the

model. The document links to the generated visualizations and provides aggregated information about CPU and memory usage, as well as classification quality metrics, such as accuracy, sensitivity, precision, or G-Mean. Such a file can be included in a larger, Sphinx-based documentation, which allows easy, automated report generation, using e.g. CI, like in the case of the Kenning documentation.

We can also use a simplified command:

```
kenning report \
  --report-path build/benchmarks/native.md \
  --measurements build/native.json
```

The available `--report-types` will be derived from measurements in the `build/native.json` file.

Conveniently, `kenning test` and `kenning report` commands can be reduced to a single `kenning` run:

```
kenning test report \
  --json-cfg mobilenetv2-tensorflow-native.json \
  --measurements build/native.json \
  --report-path build/benchmarks/native.md
```

While native frameworks are great for training and inference, model design, training on GPUs and distributing training across many devices, e.g. in a cloud environment, there is a fairly large variety of inference-focused frameworks for production purposes that focus on getting the most out of hardware in order to get results as fast as possible.

6.10.4 Optimizing a model using TensorFlow Lite

TensorFlow Lite is one of such frameworks. It is a lightweight library for inferring networks on edge - it has a small binary size (which can be reduced further by disabling unused operators) and a highly optimized format of input models, called FlatBuffers.

Before the TensorFlow Lite Interpreter (runtime for the TensorFlow Lite library) can be used, the model first needs to be optimized and compiled to the `.tflite` format.

Let's add a TensorFlow Lite Optimizer that will convert our MobileNetV2 model to a FlatBuffer format, as well as a TensorFlow Lite Runtime that will execute the model:

Listing 6.7: mobilenetv2-tensorflow-tflite-f32.json

```
    "optimizers": [
        {
            "type": "kenning.optimizers.litert.LiteRTCompiler",
            "parameters": {
                "target": "default",
                "compiled_model_path": "./build/fp32.tflite",
                "inference_input_type": "float32",
                "inference_output_type": "float32"
            }
        }
    ],
    "runtime": {
```

(continues on next page)

(continued from previous page)

```
        "type": "kenning.runtimes.litert.LiteRTRuntime",
        "parameters": {
            "save_model_path": "./build/fp32.tflite"
        }
    }
```

The configuration of the already existing blocks does not change, and the dataset will not be downloaded again since the files are already present.

The first new addition in comparison to the previous flow is the presence of the `optimizers` list, which allows us to add one or more objects inheriting from the `kenning.core.optimizer.Optimizer` class. Optimizers read the model from the input file, apply various optimizations, and then save the optimized model to a new file.

In our current scenario, we will use the `LiteRTCompiler` class - it reads the model in a Keras-specific format, optimizes the model and saves it to the `./build/fp32.tflite` file. The parameters of this particular Optimizer are worth noting here (each Optimizer usually has a different set of parameters):

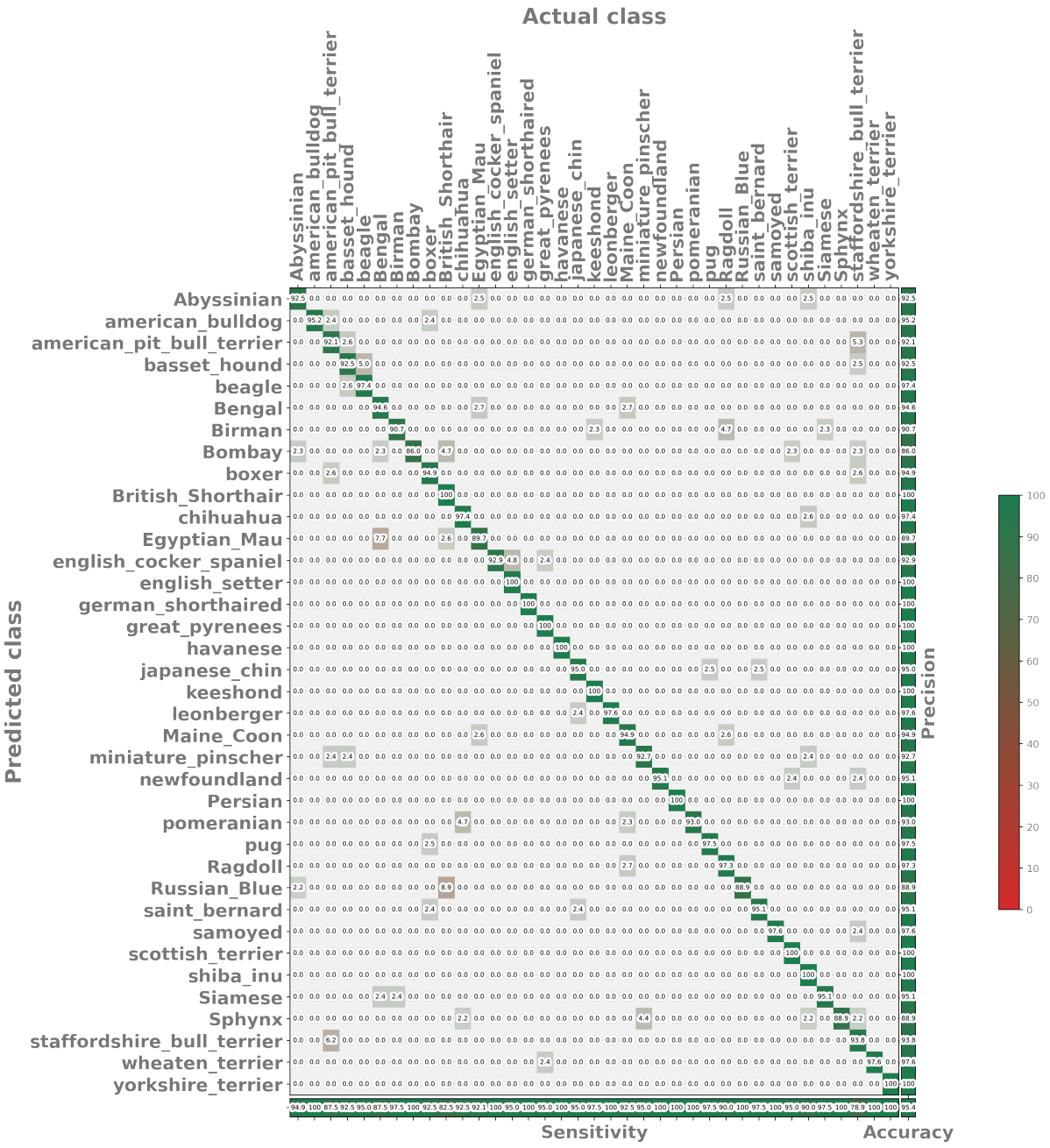
- `target` - indicates what the desired target device (or model type) is, regular CPU is `default`. Another example here could be `edgetpu`, which can compile models for the Google Coral platform.
- `compiled_model_path` - indicates where the model should be saved.
- `inference_input_type` and `inference_output_type` - indicate what the input and output type of the model should be. Usually, all trained models use FP32 weights (32-bit floating point) and activations - using `float32` here keeps the weights unchanged.

Another addition is the `runtime` block which provides a class inheriting from the `kenning.core.runtime.Runtime` class that is able to load the final model and run inference on target hardware. Usually, each `Optimizer` has a corresponding `Runtime` capable of running its results.

To compile the scenario (called `tflite-fp32.json`), run:

```
kenning optimize test report \
  --json-cfg mobilenetv2-tensorflow-tflite-f32.json \
  --measurements build/tflite-fp32.json \
  --report-path build/benchmarks/tflite-fp32.md
```

While it depends on the platform used, you should be able to see a significant improvement in both inference time (model ca. 10-15x faster model compared to the native model) and memory usage (output model ca. 2x smaller). What's worth noting is that we get a significant improvement with no harm to the quality of the model - the outputs stay the same.



6.10.5 Quantizing a model using TensorFlow Lite

To further reduce memory usage, we can quantize the model - it is a process where all weights and activations in a model are calibrated to work with low-precision floating-point or integer reporesentations (i.e. `INT8`), instead of the `FP32` precision. While it may severely harm prediction quality, the quality reduction can be negligible with proper calibration.

The model can be quantized during the compilation process in TensorFlow Lite. With Kenning, it can be achieved with the following simple additions:

Listing 6.8: mobilenetv2-tensorflow-tflite-int8.json

```
"optimizers": [
  {
```

(continues on next page)

(continued from previous page)

```
      "type": "kenning.optimizers.litert.LiteRTCompiler",
      "parameters": {
        "target": "int8",
        "compiled_model_path": "./build/int8.tflite",
        "inference_input_type": "int8",
        "inference_output_type": "int8"
      }
    }
```

The only changes here in comparison to the previous configuration appear in the `LiteRTCompiler` configuration. We change `target`, `inference_input_type` and `inference_output_type` to `int8`. Then, in the background, `LiteRTCompiler` fetches a subset of images from the `PetDataset` object to calibrate the model and the entire model calibration process happens automatically.

Let's run the scenario above (`tflite-int8.json`):

```
kenning optimize test report \
  --json-cfg mobilenetv2-tensorflow-tflite-int8.json \
  --measurements build/tflite-int8.json \
  --report-path build/benchmarks/tflite-int8.md
```

This results in a model over 7 times smaller compared to the native model without significant loss of accuracy (but without speed improvement).

6.10.6 Speeding up inference with Apache TVM

To speed up inference of a quantized model, we can utilize vector extensions in x86 CPUs, more specifically AVX2. To do this, we can use the Apache TVM framework to compile efficient runtimes for various hardware platforms. The scenario looks like this:

Listing 6.9: mobilenetv2-tensorflow-tvm-avx-int8.json

```
  "optimizers": [
    {
      "type": "kenning.optimizers.litert.LiteRTCompiler",
      "parameters": {
        "target": "int8",
        "compiled_model_path": "./build/int8.tflite",
        "inference_input_type": "int8",
        "inference_output_type": "int8"
      }
    },
    {
      "type": "kenning.optimizers.tvm.TVMCompiler",
      "parameters": {
        "target": "llvm -mcpu=core-avx2",
        "opt_level": 3,
        "conv2d_data_layout": "NCHW",
        "compiled_model_path": "./build/int8_tvm.tar"
      }
```

(continues on next page)

(continued from previous page)

```
    }
```

As visible, adding a new framework is just a matter of simply adding and configuring another optimizer and using a `Runtime` corresponding to the final `Optimizer`.

The `TVMCompiler`, with `llvm -mcpu=core-avx2` as the target, optimizes and compiles the model to use vector extensions. The final result is a `.tar` file containing a shared library that implements the entire model.

Let's compile the scenario (`tvm-avx2-int8.json`):

```
kenning optimize test report \
  --json-cfg mobilenetv2-tensorflow-tvm-avx-int8.json \
  --measurements build/tvm-avx2-int8.json \
  --report-path build/benchmarks/tvm-avx2-int8.md
```

This results in a model over 40 times faster compared to the native implementation, with a 3x reduction in size.

This demonstrates how easily we can interconnect various frameworks and get the most out of hardware using Kenning, while performing just minor alterations to the configuration file.

The summary of passes looks as follows:

	Speed boost	Accuracy	Size reduction
native	1	0.9572730984	1
tflite-fp32	15.79405698	0.9572730984	1.965973551
tflite-int8	1.683232669	0.9519662539	7.02033412
tvm-avx2-int8	41.61514549	0.9487005035	3.229375069

6.10.7 Automated model comparison

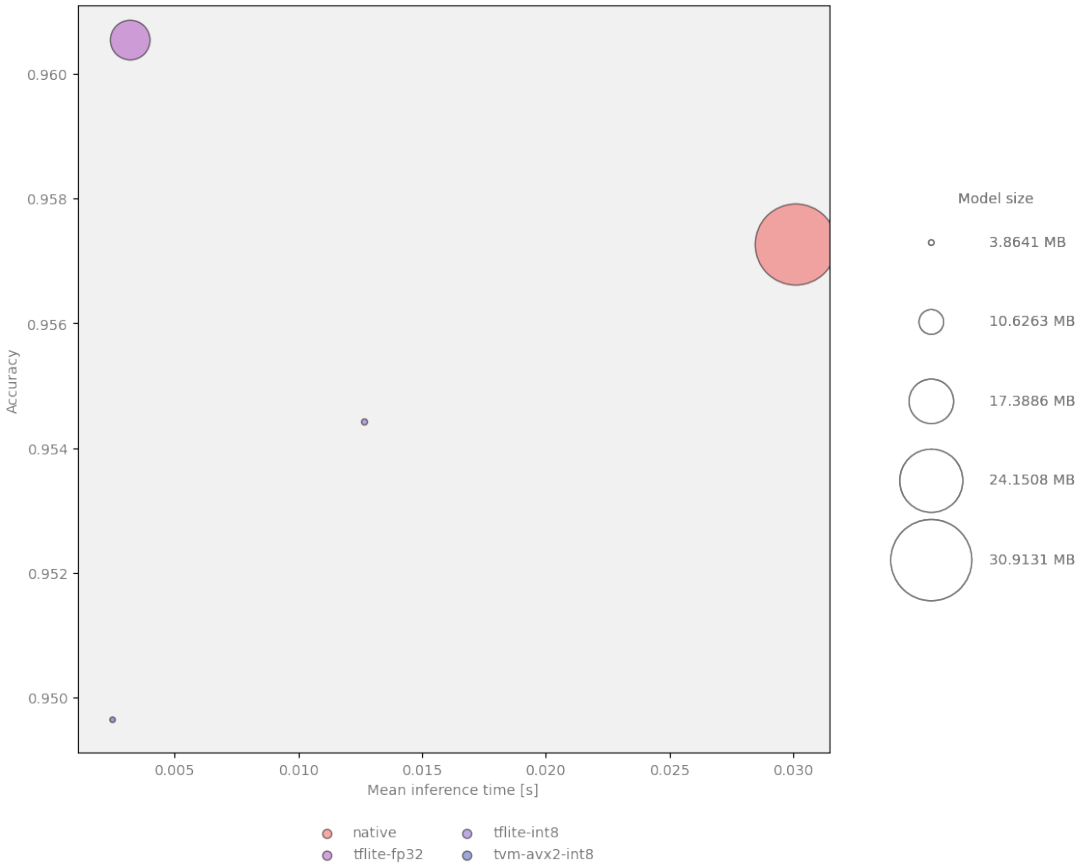
The `kenning report` tool also allows us to compare evaluation results for multiple models. Apart from creating a model summary table, it also creates plots aggregating measurements collected during the evaluation process.

To create a comparison report for the above experiments, run:

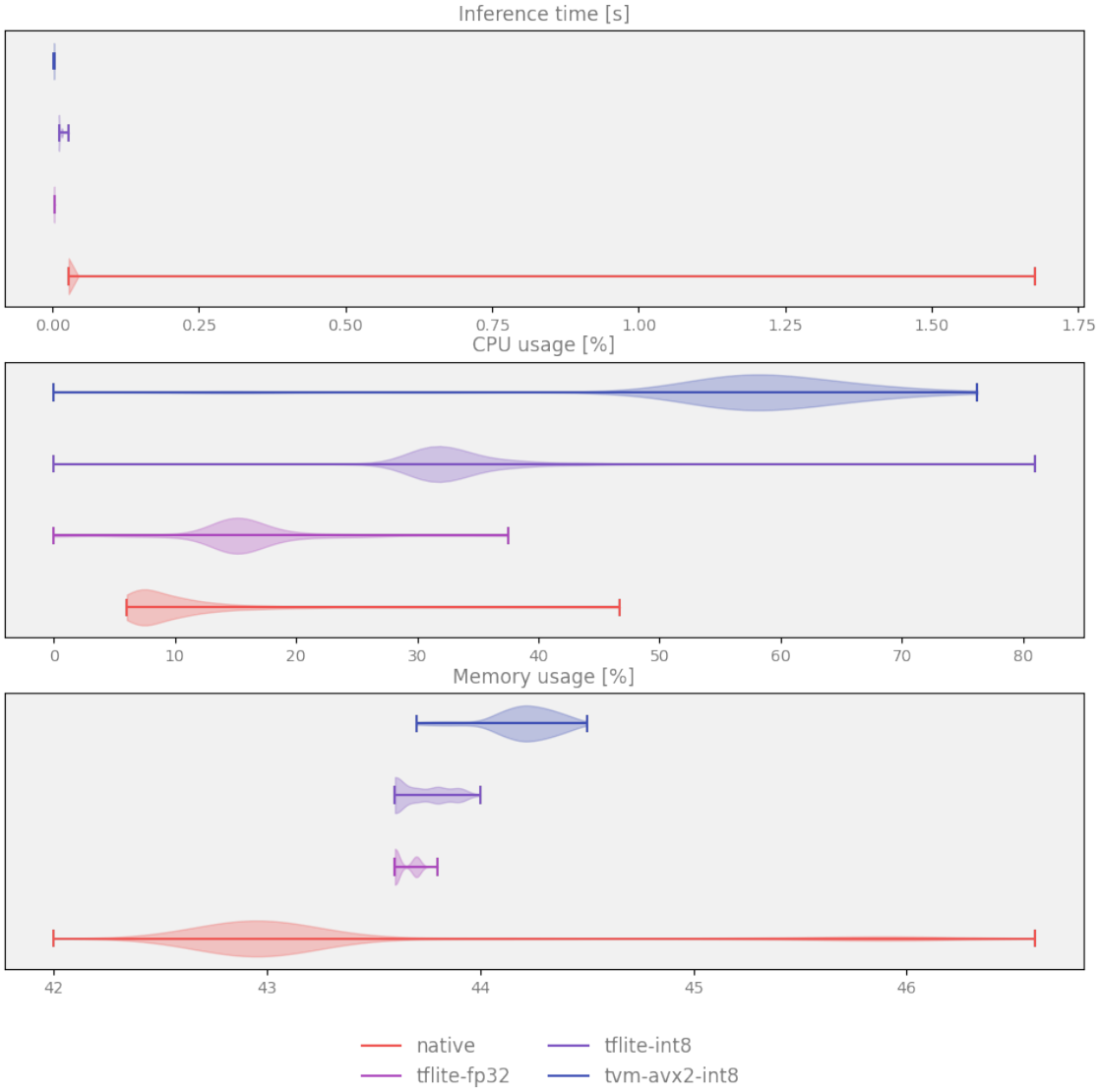
```
kenning report \
    --report-path build/benchmarks/summary.md \
    --measurements \
        build/native.json \
        build/tflite-fp32.json \
        build/tflite-int8.json \
        build/tvm-avx2-int8.json
```

Some examples of comparisons between various models rendered with the script:

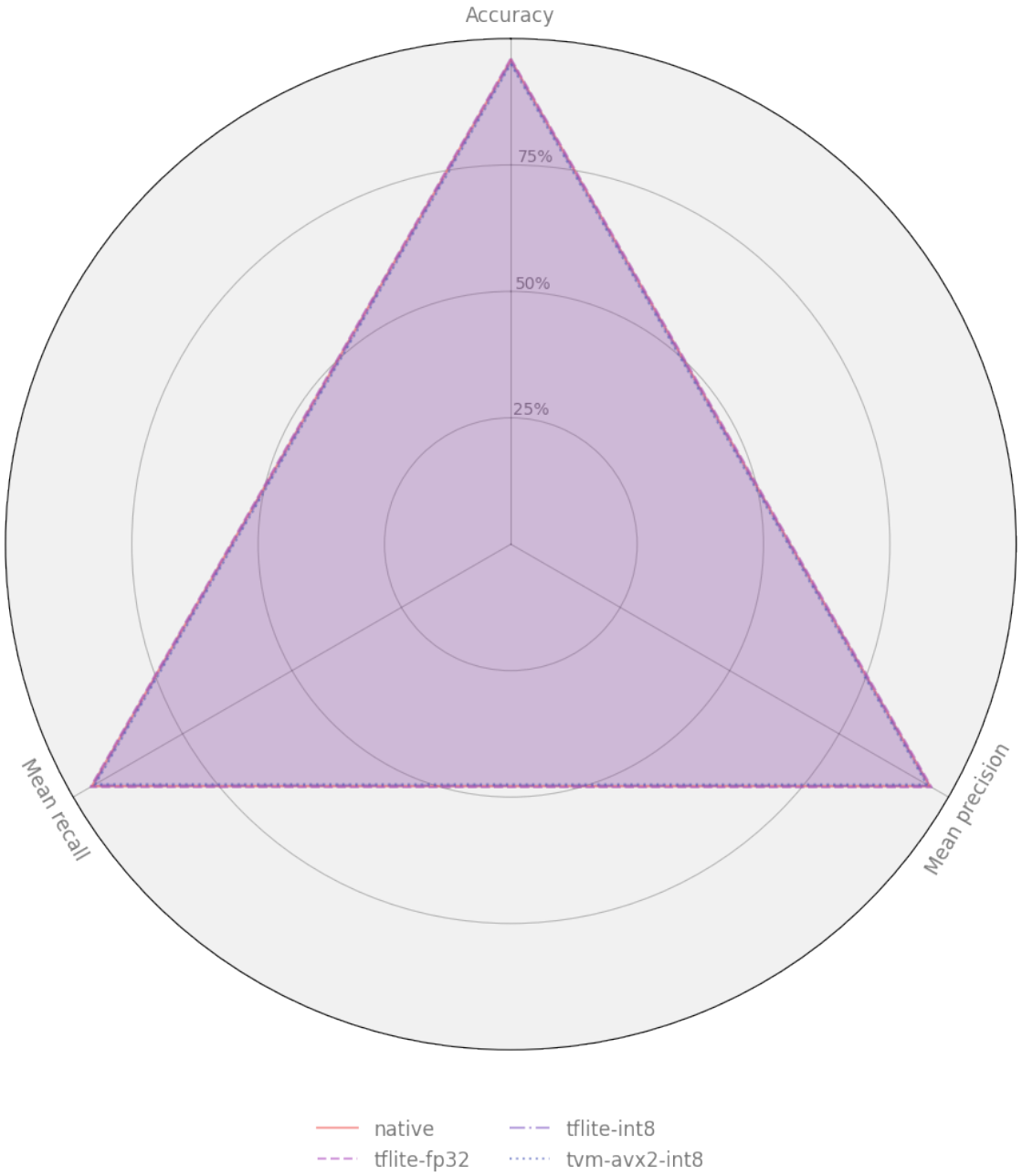
- Accuracy, inference time and model size comparison:



- Resource utilization distribution:



- Comparison of classification metrics:



- And more

6.11 Unstructured Pruning of TensorFlow Models

This section contains a tutorial for unstructured pruning of the TensorFlow classification model.

As is the case with most pruning techniques, this type of unstructured pruning requires fine-tuning after "removing" certain connections through training. In unstructured pruning, the weights for certain connections are set to zero. The weights and layers are not removed, but the matrices holding weights become sparse. It can be used to improve model compression (for storage purposes) and for use with dedicated hardware and libraries that can take advantage of sparse computing.

6.11.1 Setup

A few additional dependencies are required to prune TensorFlow models, they can be installed using the following command:

```
uv pip install "kenning[tensorflow,reports] @ git+https://github.com/antmicro/
↪kenning.git"
```

6.11.2 Experiments

First, we need to know the performance of the original model which can be achieved by running the following pipeline:

Listing 6.10: mobilenetv2-tensorflow.json

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
    "parameters": {
      "model_path": "kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5"
    }
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
    "parameters": {
      "dataset_root": "./build/PetDataset"
    }
  }
}
```

To test it, run:

```
kenning test \
  --json-cfg mobilenetv2-tensorflow.json \
  --measurements build/tf.json
```

`TensorflowPruningOptimizer` has two main parameters for adjusting the pruning process:

- `target_sparsity` - defines sparsity of weights after pruning,
- `prune_dense` - if `true`, only dense layers will be pruned instead of the entire model.

You can also adjust how often a model will be pruned (with `pruning_frequency`), and when it cannot be pruned (`pruning_end`).

You can also chose an `optimizer` (one of `adam`, `RMSprop` or `SGD`) and specify if network's output is normalized with `disable_from_logits`.

In this example, we decided to fine-tune the model for three epochs with batch size equal to 128.

Listing 6.11: pruning-mobilenetv2-tensorflow.json

```
{
  "model_wrapper": {
    "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
    "parameters": {
      "model_path": "kenning:///models/classification/tensorflow_pet_dataset_
↪mobilenetv2.h5"
    }
  },
  "dataset": {
    "type": "kenning.datasets.pet_dataset.PetDataset",
    "parameters": {
      "dataset_root": "./build/PetDataset"
    }
  },
  "optimizers": [
    {
      "type": "kenning.optimizers.tensorflow_pruning.TensorFlowPruningOptimizer",
      "parameters": {
        "compiled_model_path": "./build/tf-pruning-0_15.h5",
        "target_sparsity": 0.15,
        "batch_size": 128,
        "epochs": 3,
        "pruning_frequency": 20,
        "pruning_end": 120,
        "save_to_zip": true
      }
    }
  ]
}
```

To prune the model, run:

```
kenning optimize \
  --json-cfg pruning-mobilenetv2-tensorflow.json
kenning test \
  --json-cfg pruning-mobilenetv2-tensorflow.json \
  --measurements build/tf-pruning.json
```

Despite the fact, that the Kenning CLI is capable of running commands in sequence (like `kenning optimize test [FLAGS]`), we suggest separating them for more measurement precision.

In order to compare a model before and after pruning, you can generate a report using the following command:

```
kenning report \
  --measurements \
    build/tf.json \
    build/tf-pruning.json \
```

(continues on next page)

(continued from previous page)

```
  --report-path build/tf-pruning.md \
  --to-html
```

For greater size reduction, we can use larger sparsity with adjusted parameters, like below:

```
{
  "type": "kenning.optimizers.tensorflow_pruning.TensorFlowPruningOptimizer",
  "parameters": {
    "compiled_model_path": "./build/tf-pruning-0_3.h5",
    "target_sparsity": 0.3,
    "batch_size": 128,
    "epochs": 5,
    "pruning_frequency": 40,
    "pruning_end": 82,
    "save_to_zip": true
  }
}
```

The default TensorFlow version of the pretrained MobileNetV2 (`kenning:///models/classification/tensorflow_pet_dataset_mobilenetv2.h5`) is exported with an optimizer, resulting in size increase. To ensure better measurement quality, we will present the data with the default MobileNetV2 stripped from any unnecessary information.

Summary of pruned models:

Sparsity	Accuracy	Compressed size	Size reduction
—	0.9605442177	15,508,373	0.00%
0.15	0.9190476190	14,062,345	9.32%
0.3	0.8367346939	12,349,076	20.37%

6.11.3 (Optional) Clustering

Kenning provides a few other methods for reducing the size of the model. One of such techniques is clustering. It groups weights into K groups of similar values. Then, it computes K centroids based on those weights and use them as values for new weights. In weight matrices, we store indices to corresponding centroid instead of values. The indices can be stored as integers with a very small number of bits necessary to represent them, reducing the model size significantly.

You can do this by adding `kenning.optimizers.tensorflow_clustering.TensorFlowClusteringOptimizer`:

Listing 6.12: pruning-clustering-mobilenetv2-tensorflow.json

```
  "optimizers": [
    {
      "type": "kenning.optimizers.tensorflow_pruning.TensorFlowPruningOptimizer",
      "parameters": {
        "compiled_model_path": "./build/tf-pruning-0_15.h5",
        "target_sparsity": 0.15,
```

(continues on next page)

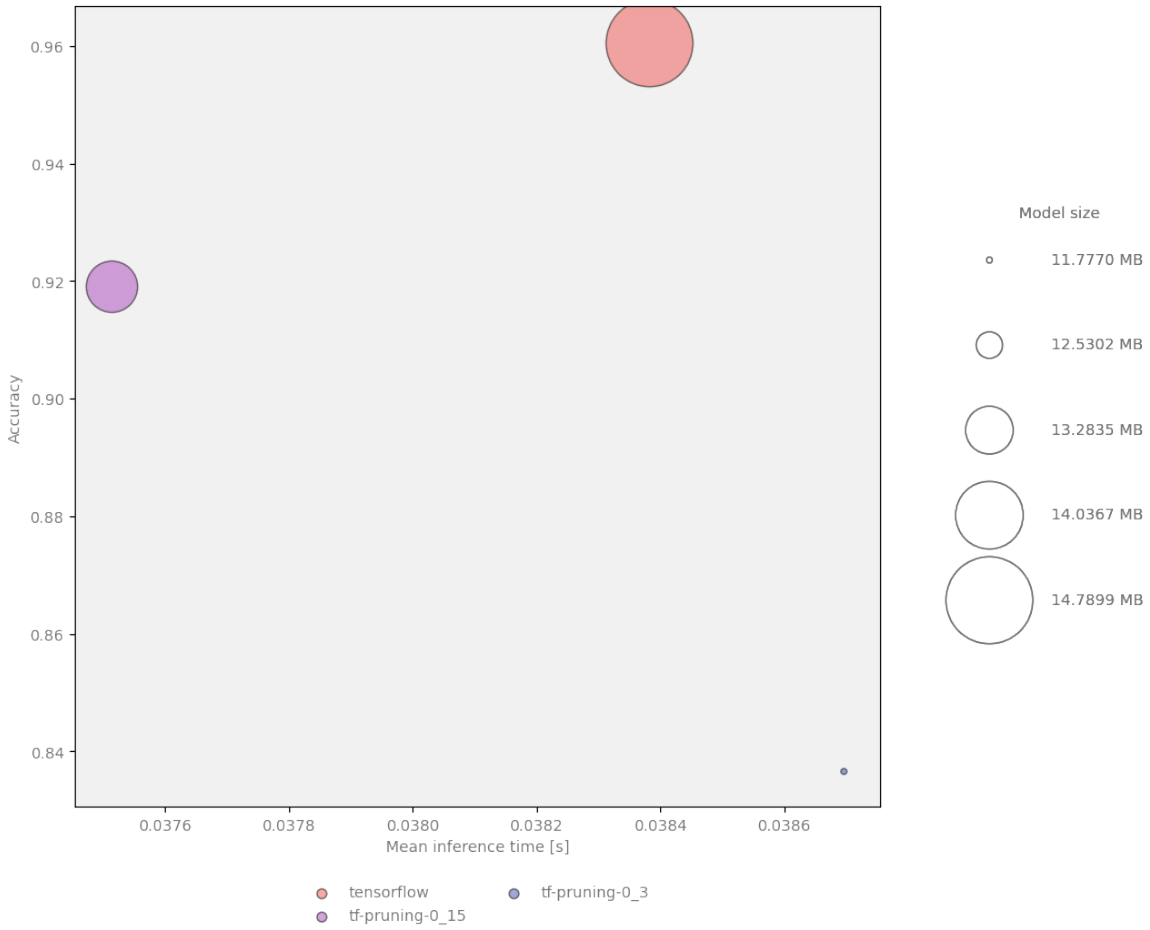


Figure 6.4: Model size, speed and quality comparison for TensorFlow pruning

(continued from previous page)

```
        "batch_size": 128,
        "epochs": 5,
        "pruning_frequency": 10
      }
    },
    {
      "type": "kenning.optimizers.tensorflow_clustering.
↪TensorFlowClusteringOptimizer",
      "parameters": {
        "compiled_model_path": "./build/tf-pruning-clustering.h5",
        "cluster_dense": false,
        "preserve_sparsity": true,
        "clusters_number": 60
      }
    }
  ]
```

To run it, use:

```
kenning optimize test report \
  --json-cfg pruning-clustering-mobilenetv2-tensorflow.json \
  --measurements build/tf-all.json \
  --report-path build/tf-pruning-clustering.md \
  --to-html
```

Clustering allows for substantial model size reduction without significant decrease in quality. Here is a comparison of a model with and without clustering:

Sparsity	Number of clusters	Accuracy	Compressed size	Size reduction
—	—	0.9605442177	15,508,373	0.00%
0.15	—	0.9190476190	14,062,345	9.32%
0.15	60	0.9006802721	4,451,142	71.30%

6.12 Optimizing and comparing an instance segmentation model

This example demonstrates how to optimize an instance segmentation model and compare different optimizers on CPU and GPU.

Base model used for demonstration is going to be YOLACT, short for "You Only Look At CoefficientTs" which is a fully convolutional model for real-time instance segmentation. We will also need a dataset for evaluation purposes - in this case it is going to be `OpenImagesDatasetV6`.

The model will first be optimized for a given target using Kenning optimizers (TVM, ONNX), and finally compiled into a deployable, runtime-specific artifact. Model will be deployed on CPU and GPU using following Kenning compilers:

- ONNXCompiler - wrapper for optimizing and converting models to format compliant for ONNX Runtime.
- TVMCompiler - wrapper for TVM deep neural network compiler.

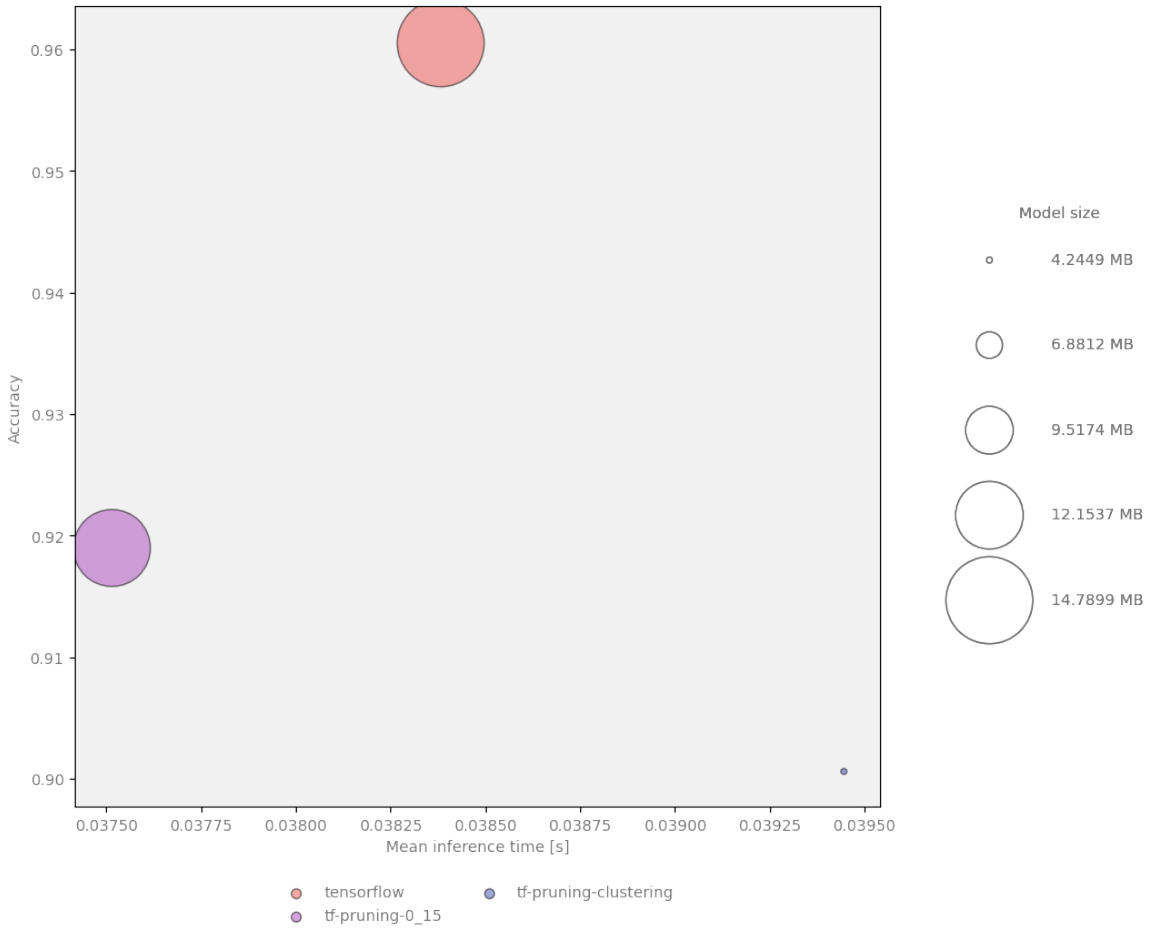


Figure 6.5: Model size, speed and quality comparison for TensorFlow pruning and clustering

6.12.1 Dependencies

To run this scenario, you will need:

1. Hardware:
    - A CUDA-enabled NVIDIA GPU
2. Software:
    - UV - to quickly install Python dependencies and manage virtual environments

6.12.2 Experiments on GPU

In this section, the model will first be optimized for GPU execution using TVM and ONNX Runtime, and then benchmarked to compare the resulting performance.

Info

This example requires a CUDA-enabled GPU.

Depending on the example, you may also need:

- CUDA Toolkit (13.0 recommended), including NVCC and NVRTC, to compile CUDA code.
- cuDNN (9 recommended) for operations that use cuDNN.
- NVML to generate GPU reports.

Check the example to see which of these components are needed.

Installation

First, let's install the dependencies needed to run this scenario on GPU, and we'll get to CPU afterwards.

Let's install Kenning:

```
git clone https://github.com/antmicro/kenning.git && cd kenning
```

and the required libraries:

```
uv pip install \
  --group tvm-cuda \
  ".[object_detection,onnxruntime_gpu,reports]"
```

(GPU) Optimization

Optimization is the process of converting the trained model into a form better suited for fast, efficient inference on a given target. With `TVMCompiler`, the model is compiled through TVM's compiler stack into low-level, target-specific code (e.g. CUDA kernels for GPU, LLVM-generated code for CPU), applying optimizations such as operator fusion and memory layout tuning. With `ONNXCompiler`, the model is instead converted into an optimized ONNX graph, which `ONNXRuntime` can then execute directly using its own set of hardware-specific execution providers (e.g. `CUDAExecutionProvider`, `CPUExecutionProvider`).

(GPU) TVM Optimization

To optimize the model on GPU using TVM, the yolact-tvm-gpu-detection.yml file has been prepared, which looks as follows:

Listing 6.13: yolact-tvm-gpu-detection.yml

```yaml
model_wrapper:
  type: kenning.modelwrappers.instance_segmentation.yolact.YOLACT
  parameters:
    model_path: kenning:///models/instance_segmentation/yolact.onnx
dataset:
  type: kenning.datasets.open_images_dataset.OpenImagesDatasetV6
  parameters:
    dataset_root: ./build/OpenImagesDatasetV6
    inference_batch_size: 1
    task: instance_segmentation
    image_width: 550
    image_height: 550
optimizers:
- type: kenning.optimizers.tvm.TVMCompiler
  parameters:
    model_framework: onnx
    target: "cuda -libs=cudnn,cublas"
    opt_level: 3
    compile_use_vm: false
    output_conversion_function: default
    target_host: null
    compiled_model_path: ./build/compiled-model.tar
runtime:
  type: kenning.runtimes.tvm.TVMRuntime
  parameters:
    save_model_path: ./build/compiled-model.tar
    target_device_context: cuda
    runtime_use_vm: false
```

In this scenario:

- `model_path` points to a location of the YOLACT model in ONNX format. It can be either local file or a remote URL. `kenning://` is a special schema for Kenning's demonstration models.
- `dataset` tells to use Open Images dataset. The model will be downloaded to `./build/OpenImagesDatasetV6`. The `task` field allows to specify whether the dataset is used for instance segmentation or object detection.
- `optimizers` contains only one element - `TVMCompiler`. In there we specify input model framework (`onnx`), and tell to use `cuda` target with `opt_level` equal to 3 (applying all possible optimizations not directly affecting model's output).
- `runtime` tells Kenning to use `TVMRuntime` for model execution, on GPU target.

To learn more about config files see Optimization Pipelines.

Setting `target` to `cuda -libs=cudnn,cublas` tells `TVMCompiler` to compile the model for

CUDA execution, using cuDNN and cuBLAS kernels for the supported operations. Setting `target_device_context` to `cuda` tells `TVMRuntime` to run inference on the GPU instead of the CPU.

To run the optimization and test the model, execute:

```
kenning optimize test \
    --cfg yolact-tvm-gpu-detection.yml \
    --measurements ./build/yolact-gpu-tvm.json \
    --verbosity INFO
```

(GPU) ONNX Optimization

Now it's time for ONNX. Compared to TVM, the difference is that we change the optimizer to `ONNXCompiler` and the runtime to `ONNXRuntime`. The yolact-onnx-gpu-detection.yml file has been prepared for this:

Listing 6.14: yolact-onnx-gpu-detection.yml

```
platform:
  type: CUDAPlatform
  parameters:
    name: jetson_thor_devkit
model_wrapper:
  type: kenning.modelwrappers.instance_segmentation.yolact.YOLACT
  parameters:
    model_path: kenning:///models/instance_segmentation/yolact.onnx
dataset:
  type: kenning.datasets.open_images_dataset.OpenImagesDatasetV6
  parameters:
    dataset_root: ./build/OpenImagesDatasetV6
    inference_batch_size: 1
    task: instance_segmentation
    image_width: 550
    image_height: 550
optimizers:
- type: kenning.optimizers.onnx.ONNXCompiler
  parameters:
      compiled_model_path: ./build/compiled_model.onnx
runtime:
  type: kenning.runtimes.onnx.ONNXRuntime
  parameters:
    save_model_path: ./build/compiled_model.onnx
    execution_providers:
    - CUDAExecutionProvider
    - CPUExecutionProvider
report:
  type: MarkdownReport
  parameters:
    measurements: ./build/yolact-onnx-gpu-detection-cfg.json
    report_path: ./build/yolact-onnx-gpu-detection/report.md
```

(continues on next page)

(continued from previous page)

```
    report_name: "Sample ONNX model on GPU inference report"
    report_types:
      - performance
      - classification
    to_html: true
```

`execution_providers` in `ONNXRuntime` holds a list of possible layer executors, starting from the most preferred one. For GPU execution, `CUDAExecutionProvider` should be listed first, with `CPUExecutionProvider` as a fallback for operations without CUDA support.

Run the scenario as follows:

```
kenning optimize test \
    --cfg yolact-onnx-gpu-detection.yml \
    --measurements ./build/yolact-gpu-onnx.json \
    --verbosity INFO
```

Comparison of GPU runtimes

To create a comparison report comparing performance and model quality for the above optimizers, run:

```
kenning report \
    --report-path build/yolact-report/report.md \
    --report-name "YOLACT detection report" \
    --root-dir build/yolact-report \
    --img-dir build/yolact-report/imgs \
    --report-types performance detection \
    --measurements build/yolact-gpu-tvm.json build/yolact-gpu-onnx.json
```

6.12.3 Experiments on CPU

Most of the steps for CPU will look nearly identical to those for GPU - the CPU section that follows mainly highlights the small modifications needed to switch from GPU to CPU execution.

Installation

Since we currently have `TVM and ONNX Runtime` packages installed for GPU, we need to **uninstall** them and install the CPU-compatible versions instead. We can install CPU-only replacements with:

```
uv pip install ".[object_detection, tvm, onnxruntime, reports]"
```

(CPU) Optimization

(CPU) TVM Optimization

The yolact-tvm-cpu-detection.yml file has been prepared to optimize the model with TVM on CPU, and looks as follows:

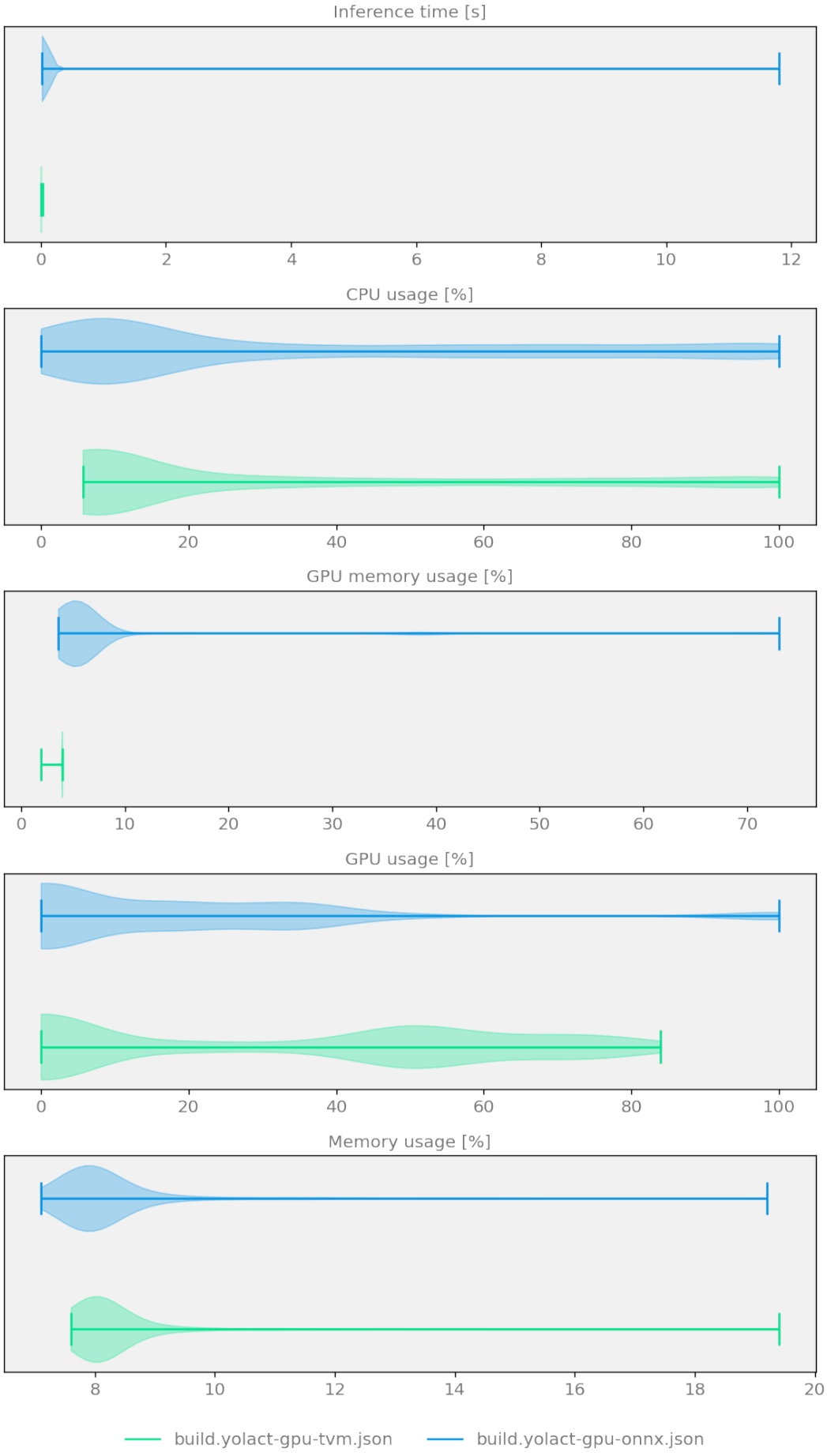


Figure 6.6: Model size, speed and quality comparison for two YOLACT Optimizers running on CUDA GPU

Listing 6.15: yolact-tvm-cpu-detection.yml

```
model_wrapper:
  type: kenning.modelwrappers.instance_segmentation.yolact.YOLACT
  parameters:
    model_path: kenning:///models/instance_segmentation/yolact.onnx
dataset:
  type: kenning.datasets.open_images_dataset.OpenImagesDatasetV6
  parameters:
    dataset_root: ./build/OpenImagesDatasetV6
    inference_batch_size: 1
    task: instance_segmentation
    image_width: 550
    image_height: 550
optimizers:
- type: kenning.optimizers.tvm.TVMCompiler
  parameters:
    model_framework: onnx
    target: llvm
    opt_level: 3
    compile_use_vm: false
    compiled_model_path: ./build/compiled-model.tar
runtime:
  type: kenning.runtimes.tvm.TVMRuntime
  parameters:
    save_model_path: ./build/compiled-model.tar
    target_device_context: cpu
    runtime_use_vm: false
```

Compared to the GPU scenario, the only differences are the `target` and `target_device_context` - here set to `llvm` and `cpu` respectively, instead of `cuda -libs=cudnn,cublas` and `cuda`.

To optimize and test the defined scenario, run:

```
kenning optimize test \
    --cfg yolact-tvm-cpu-detection.yml \
    --measurements ./build/yolact-tvm.json \
    --verbosity INFO
```

(CPU) ONNX Optimization

Similarly to the GPU case, switching to a different runtime is a matter of changing several lines in the scenario, as shown below:

Listing 6.16: yolact-onnx-cpu-detection.yml

```
platform:
  type: LocalPlatform
model_wrapper:
  type: kenning.modelwrappers.instance_segmentation.yolact.YOLACT
  parameters:
```

(continues on next page)

(continued from previous page)

```
    model_path: kenning:///models/instance_segmentation/yolact.onnx
dataset:
  type: kenning.datasets.open_images_dataset.OpenImagesDatasetV6
  parameters:
    dataset_root: ./build/OpenImagesDatasetV6
    inference_batch_size: 1
    task: instance_segmentation
    image_width: 550
    image_height: 550
optimizers:
- type: kenning.optimizers.onnx.ONNXCompiler
  parameters:
    compiled_model_path: ./build/compiled_model.onnx
runtime:
  type: kenning.runtimes.onnx.ONNXRuntime
  parameters:
    save_model_path: ./build/compiled_model.onnx
    execution_providers:
    - CPUExecutionProvider
```

This scenario can be executed with:

```
kenning optimize test \
    --cfg yolact-onnx-cpu-detection.yml \
    --measurements ./build/yolact-onnx.json \
    --verbosity INFO
```

Comparison of CPU runtimes

To create a comparison report comparing performance and model quality for the above optimizers, run:

```
kenning report \
    --report-path build/yolact-report/report.md \
    --report-name "YOLACT detection report" \
    --root-dir build/yolact-report \
    --img-dir build/yolact-report/imgs \
    --report-types performance detection \
    --measurements build/yolact-tvm.json build/yolact-onnx.json
```

6.12.4 Summary

In this example, we optimized and deployed the YOLACT instance segmentation model on both GPU and CPU, using two different Kenning optimizers - `TVMCompiler` and `ONNXCompiler` - paired with their corresponding runtimes, `TVMRuntime` and `ONNXRuntime`.

The overall workflow was the same across both targets and both optimizers: prepare a scenario configuration, run `kenning optimize test` to optimize and benchmark the model, and generate a comparison report with `kenning report`. Switching between GPU and CPU, or between TVM and ONNX, required only small, targeted changes to the scenario - primarily the

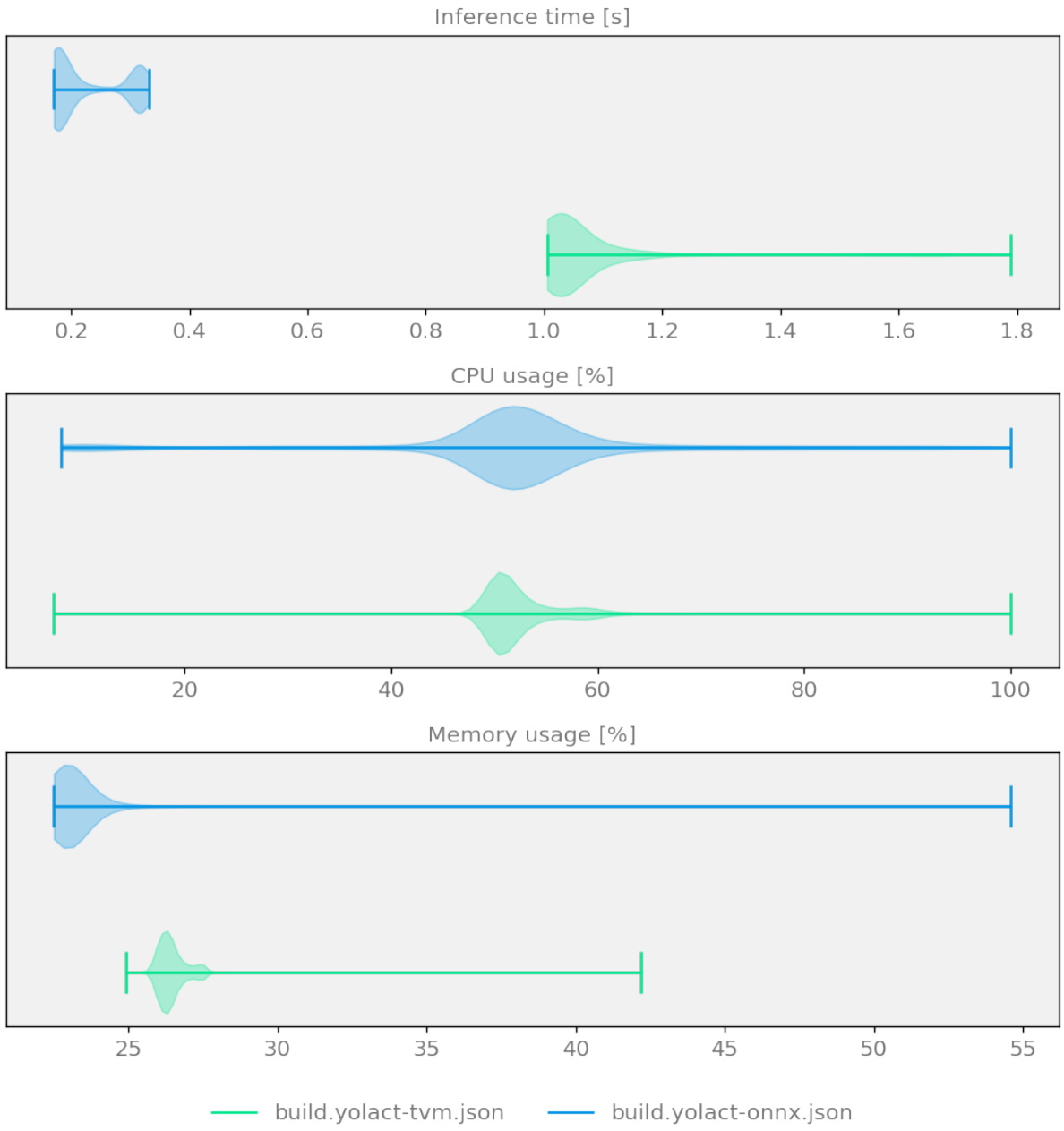


Figure 6.7: Sample comparison plot demonstrating model size, speed and quality for two YOLACT Optimizers

`target/target_device_context` fields for TVM, and the `execution_providers` list for ONNX Runtime.

The resulting reports allow for a direct comparison of model size, inference speed and detection quality across all four configurations (TVM/ONNX × GPU/CPU), making it easier to choose the best-performing setup for a given deployment target.

6.13 Transfer Learning and Optimization for Instance Segmentation Model - YOLACT

This example demonstrates a complete workflow in Kenning - from transfer learning of YOLACT (a model that detects object locations and draws their outlines/masks) for office objects, model optimization (quantization and compilation using TVM), to deployment on a GPU platform running 3 models simultaneously in real time:

- YOLACT (Instance Segmentation)
- MMPose (Human Pose Estimation)
- DINOv2 (Depth Estimation)

The ROS2 GUI Node project will be used to demonstrate its operation.

6.13.1 Dependencies

To run this scenario, you will need:

1. Hardware:
    - A camera for streaming frames
    - A CUDA-enabled NVIDIA GPU for inference acceleration
2. Software:
    - Docker - to use a prepared environment (for ROS2)
    - UV - to quickly install Python dependencies and manage virtual environments
    - repo tool - to clone all necessary repositories
    - nvidia-container-toolkit - to provide access to the GPU in the Docker container

Info

This example requires a CUDA-enabled GPU.

Depending on the example, you may also need:

- CUDA Toolkit (13.0 recommended), including NVCC and NVRTC, to compile CUDA code.
- cuDNN (9 recommended) for operations that use cuDNN.
- NVML to generate GPU reports.

Check the example to see which of these components are needed.

6.13.2 Installation

For transfer learning and model optimization, we can directly clone the Kenning repository from:

```
git clone https://github.com/antmicro/kenning.git && cd kenning
```

and perform the next steps (*2*, *3*, *4*) inside it. However, since we ultimately want to run `ros2-gui-demo`, we will use the `repo` tool for this, which will fetch other necessary repositories and dependencies alongside Kenning (including `GUI Node`, scripts for building the ROS 2 image, etc.):

```
mkdir kenning-transfer-learning && cd kenning-transfer-learning
```

```
repo init -u https://github.com/antmicro/ros2-gui-node.git -m examples/kenning-
↪instance-segmentation/manifest.xml
repo sync -j`nproc`
mkdir build
```

Note

Before executing `repo` command you may need to set up git credential by typing into terminal:

```
git config --global user.email "<e-mail address>"
git config --global user.name "Name Surname"
```

6.13.3 Configuration Files

Everything in Kenning revolves around configuration files, which store configurations for various scenarios, such as model optimization. In the `scripts/configs/` directory, we can find the pytorch-open-images-yolact-tvm-gpu.yaml file, which contains the YOLACT model configuration for report generation, optimization, training, and benchmarking. The individual sections are taken into account when invoking the relevant scenarios using the `kenning [scenario] [params]` command, e.g., `kenning train --cfg path/file.yaml`. In this case, only `model_wrapper` and `dataset` will be taken into account:

```
model_wrapper:
  type: YOLACTOpenImages
  parameters:
    # Output of fine-tuning; also used as input for the optimize stage
    model_path: ./build/yolact_finetuned.pth
    # Starting checkpoint (backbone frozen, only heads are trained)
    pretrained_weights_path: kenning:///models/instance_segmentation/yolact_
↪resnet50.pth
    freeze_backbone: true
    learning_rate: 0.0004
    batch_size: 32
    num_epochs: 1
    logdir: ./build/logs
```

(continues on next page)

(continued from previous page)

```
dataset:
  type: OpenImagesDatasetV6
  parameters:
    dataset_root: ./build/OpenImagesDatasetV6
    inference_batch_size: 1
    task: instance_segmentation
    classes: coco
    download_annotations_type: train
    image_width: 550
    image_height: 550
```

More information about this can be found here: *Defining optimization pipelines in Kenning*. Unfortunately, there is currently no single place describing all block arguments, so you need to search for them manually in the corresponding class definition file.

6.13.4 Transfer Learning

Transfer learning involves loading a pretrained model (weights) that was trained for task `A` and, after minor modifications, adapting it so that it can be used for task `B`. In practice, this means usually adding or modifying the prediction head (e.g., changing the number of recognized classes), freezing the rest of the model so it isn't trained, and then training this model for a few epochs. The next step is reducing the learning rate to avoid damaging the previously learned weights, unfreezing them, and training for a few epochs so the model adapts to the new head.

Selecting Classes for Recognition

The YOLACT model was originally trained on the COCO dataset (part of the architecture, namely `ResNet-50`, was previously trained on ImageNet). It is therefore natural that we want to select objects (and a dataset) that roughly resemble those from the original training - general real-world objects, rather than, for example, computer games.

For this purpose, we will use the OpenImages dataset implemented in Kenning and select a subset of classes from it - objects that can be found in an office.

> **Note**
>
> OpenImages dataset labels are encoded (they do not contain direct names like `Tree`), and the file with all classes and their encodings can be found here: OpenImages Class Descriptions.

These classes will be:

```
/m/02jvh9,Mug
/m/02p0tk3,Human body
/m/0k1tl,Pen
/m/020lf,Computer mouse
/m/01m2v,Computer keyboard
/m/02522,Computer monitor
/m/050k8,Mobile phone
/m/04dr76w,Bottle
/m/02dl1y,Hat
```

(continues on next page)

(continued from previous page)

```
/m/0242l,Coin
/m/080hkjn,Handbag
```

Let's save these classes to the `transfer_learning.csv` file, preferably in the Kenning root directory.

Adapting the Prediction Head

Now it's time to fine-tune YOLACT so that it recognizes new objects. Let's open the pytorch-open-images-yolact-tvm-gpu.yaml file and modify the `dataset` section - let's change `classes: coco` to `classes: "transfer_learning.csv"`:

```
dataset:
  type: OpenImagesDatasetV6
  parameters:
    dataset_root: ./build/OpenImagesDatasetV6
    inference_batch_size: 1
    task: instance_segmentation
    classes: coco
    download_annotations_type: train
    image_width: 550
    image_height: 550
```

Let's change `num_epochs` to `10`. This will train the head for 10 epochs:

```
model_wrapper:
  type: YOLACTOpenImages
  parameters:
    # Output of fine-tuning; also used as input for the optimize stage
    model_path: ./build/yolact_finetuned.pth
    # Starting checkpoint (backbone frozen, only heads are trained)
    pretrained_weights_path: kenning:///models/instance_segmentation/yolact_
↪resnet50.pth
    freeze_backbone: true
    learning_rate: 0.0004
    batch_size: 32
    num_epochs: 1
    logdir: ./build/logs
```

Next, let's run the container with installed dependencies, including `ROS2` and `TVM` (for simplicity, to avoid creating a virtual environment, since we will be using this container later anyway), using:

```
# Building the image, this may take a while
./src/gui_node/environments/build-docker.sh gpu

# Allow non-network local connections to X11 so that the GUI can be started from␣
↪the Docker container
xhost +local:
```

(continues on next page)

(continued from previous page)

```
# Run the container
./src/gui_node/environments/run-docker.sh gpu
```

and let's install Kenning and the required libraries with:

```
uv pip install --project . -e ".[object_detection,pose_estimation,onnxruntime_gpu,
↪torch]" --group tvm-cuda
```

or when not using ROS2, create a virtual environment:

```
uv venv
source .venv/bin/activate
```

Note

See the Kenning installation section for information about the Python versions currently supported.

and run:

```
uv pip install --project . -e ".[object_detection, torch]" --group tvm-cuda
```

Then, let's run the training process for the head itself (it will be modified automatically based on the number of dataset classes):

```
kenning train --cfg scripts/configs/pytorch-open-images-yolact-tvm-gpu.yaml
```

Depending on the available hardware, it may take a while. Our model will be saved to `./build/yolact_finetuned.pth`.

Fine-tuning the Rest of the Model

The next step is to unfreeze the remaining part of the model (changing `freeze_backbone: false`), increase the number of epochs (`num_epochs: 40`), and decrease the learning rate (`learning_rate: 0.00005`). Additionally, the recently trained model (`pretrained_weights_path: ./build/yolact_finetuned.pth`) should be used as a starting point:

```
model_wrapper:
  type: YOLACTOpenImages
  parameters:
    # Output of fine-tuning; also used as input for the optimize stage
    model_path: ./build/yolact_finetuned.pth
    # Starting checkpoint (backbone frozen, only heads are trained)
    pretrained_weights_path: kenning:///models/instance_segmentation/yolact_
↪resnet50.pth
    freeze_backbone: true
    learning_rate: 0.0004
    batch_size: 32
```

(continues on next page)

(continued from previous page)

```
    num_epochs: 1
    logdir: ./build/logs
```

Let's train the model again, this time in its entirety:

```
kenning train --cfg scripts/configs/pytorch-open-images-yolact-tvm-gpu.yaml
```

The resulting model will be located in `./build/final_yolact.pth`.

6.13.5 Optimization of the Trained Model

We can speed up our model almost 5-fold by applying int8 quantization and compiling with TVM (using appropriate flags):

- Quantization consists of reducing the precision of the model's weights and activations (e.g., from `FP32` floating-point format to 8-bit integer format `INT8`). This translates into faster matrix operations, lower RAM/VRAM usage, and reduced energy consumption. However, quantization is a lossy optimization, meaning that a slight drop in model accuracy should be expected.
- TVM compilation involves converting the network into an Intermediate Representation (IR), where computational graph optimizations are performed, including operation fusion (combining consecutive layers into one, which reduces memory transfers). The compiler then generates machine code optimized for a specific hardware architecture (e.g., `x86`, `ARM`, `RISC-V`, `CUDA`), utilizing its specific instructions (e.g., `AVX-512`, `NEON`, or `Tensor Cores`) to maximize inference performance.

In Kenning, we can do this incredibly easily. Simply change `target: cuda` to `target: cuda -arch=sm_86`, where `sm_86` specifies the architecture (Compute Capability) of our graphics card - in our case, the `RTX 3090`. You can check your GPU's architecture using the following command:

```
nvidia-smi --query-gpu=compute_cap --format=csv
```

> **Note**
>
> Convert the returned value (e.g., `8.6`) into the `sm_XX` format by removing the decimal point and adding the `sm_` prefix (for `8.6`, this becomes `sm_86`).

Quantization (in this example, Post-Training Quantization) is enabled using `use_int8_precision: true`. We apply PTQ with calibration, where `dataset_percentage: 0.001` feeds a small slice of the dataset to the compiler. This sample is required to collect activation statistics (min/max value ranges) and calculate accurate quantization scaling factors. The final `optimizers` section for optimization is:

```
optimizers:
  - type: ONNXCompiler
    parameters:
      compiled_model_path: ./build/yolact.onnx

  - type: TVMCompiler
    parameters:
```

(continues on next page)

(continued from previous page)

```
      model_framework: onnx
      # This can speed things up by almost 4x.
      target: cuda -arch=sm_86
      opt_level: 3
      use_int8_precision: true
      # Small calibration slice to keep quantization fast and avoid out-of-memory␣
↪error
      dataset_percentage: 0.001
      compiled_model_path: ./build/yolact_tvm_int8.tar
```

Now, let's run the optimization scenario using the command:

```
kenning optimize --cfg scripts/configs/pytorch-open-images-yolact-tvm-gpu.yaml
```

Our optimized model is located in `./build/yolact_tvm_int8.tar`.

6.13.6 Real-Time Usage Example

ROS2 GUI Node is a project created for visualizing data from ROS 2. ROS 2 itself is a robotics middleware based on a publish-subscribe (pub/sub) pattern and a node-based architecture. Conceptually, it functions much like a microservices framework, enabling the development of efficient and modular applications for edge devices. In this architecture, every module - from the camera, through individual AI models, to the graphical user interface (GUI) - runs as a separate, independent node.

Let's replace the entire contents of the kenning-instance-segmentation.yaml, located in the `src/gui_node/examples/kenning-multimodel-demo` directory, with the following configuration:

```
- type: kenning.dataproviders.ros2_camera_node_data_provider.
↪ROS2CameraNodeDataProvider
  parameters:
    topic_name: camera_frame
    output_memory_layout: NCHW
    output_width: 550
    output_height: 550
  outputs:
    frame: cam_frame
    frame_original: cam_frame_original

- type: kenning.runners.modelruntime_runner.ModelRuntimeRunner
  parameters:
    model_wrapper:
      type: kenning.modelwrappers.instance_segmentation.pytorch_yolact.
↪YOLACTOpenImages
      parameters:
        model_path: build/final_yolact.pth
        max_detections: 100
        score_threshold: 0.2
    runtime:
      type: kenning.runtimes.tvm.TVMRuntime
      parameters:
```

(continues on next page)

(continued from previous page)

```
        save_model_path: build/yolact_tvm_int8.tar
        target_device_context: cuda
  inputs:
    input: cam_frame
  outputs:
    segmentation_output: predictions

- type: kenning.outputcollectors.ros2_yolact_outputcollector.
↪ROS2YolactOutputCollector
  parameters:
    topic_name: instance_segmentation_kenning
    input_color_format: BGR
    input_memory_layout: NCHW
  inputs:
    frame_original: cam_frame_original
    output: predictions
```

In short, we changed the runtime to TVM, updated the model wrapper to point to the appropriate class, modified file paths (including the one for the optimized model), and added a dataset section from which the model will retrieve class names and map them to the model's output (integers).

Let's build the nodes:

```
source /opt/ros/$ROS_DISTRO/setup.bash

colcon build --base-paths src --cmake-args -DBUILD_KENNING_MULTIMODEL_DEMO=y -
↪DPython3_EXECUTABLE=/opt/venv/bin/python3
```

And finally, let's run the demo:

```
source install/setup.sh

ros2 launch gui_node kenning-multimodel-demo.py use_gui:=True
```

6.13.7 Summary

In this example, we used Kenning for YOLACT transfer learning and squeezed as much performance out of it as possible. Finally, we ran it alongside MMPose and DINOv2. Thanks to Kenning, optimization and transfer learning are incredibly simple and convenient, without any loss in performance.

CHAPTER

SEVEN

KENNING ENVIRONMENT VARIABLES

This section contains information about Kenning environment variables and how they influence the behavior of the program.

7.1 KENNING_CACHE_DIR

`KENNING_CACHE_DIR` defines directory for resources used by Kenning, like downloaded datasets. If it is not defined, `$HOME/.kenning` will be used.

7.2 KENNING_MAX_CACHE_SIZE

`KENNING_MAX_CACHE_SIZE` specifies the maximum amount of space used by *cache*. The default value is 50GB.

7.3 KENNING_USE_DEFAULT_EXCEPTHOOK

If `KENNING_USE_DEFAULT_EXCEPTHOOK` environmental variable is defined, `sys.excepthook` will not be overridden.

This function is overridden in order to enable mechanism for deducing which optional dependencies are missing. For instance, if `jinja2` is not available, Kenning will suggest installing `kenning[reports]` which contains missing package with defined version restriction and other requirements.

Kenning CLI does not require custom `sys.excepthook`, so it will not be affected by `USE_DEFAULT_EXCEPTHOOK` variable.

7.4 KENNING_DOCS_VENV

`KENNING_DOCS_VENV` defines path to the virtual environment used for tests marked as `docs`. If is not defined, system's default environment will be used.

7.5 KENNING_ENABLE_ALL_LOGS

If `KENNING_ENABLE_ALL_LOGS` environmental variable is defined, logs from other libraries will be enabled. Passes verbosity of Kenning logs to other libraries.

This is useful for debugging purposes, but it may cause a lot of noise in the logs.

7.6 KENNING_DISABLE_IO_VALIDATION

By default Kenning, before and after each inference, validates received data with defined specification. Similar check is done before evaluation.

These validations can be disabled defining `KENNING_DISABLE_IO_VALIDATION` variable.

CHAPTER

EIGHT

KENNING MEASUREMENTS

Kenning measurements are a set of information describing the compilation and evaluation process happening in Kenning.

They contain such information as:

- classes used to construct the optimization/runtime pipeline, along with their parameters,
- the JSON scenario used in the run,
- the command used to run the scenario,
- versions of the Python modules used,
- performance measurements, such as CPU usage, GPU usage,
- quality measurements, such as predictions, ground truth, confusion matrix

All information is stored in JSON format.

8.1 Performance metrics

While quality measurements are problem-specific (collected in the `evaluate` method of the *Dataset* class), performance metrics are common across devices and applications.

Kenning can fetch performance metrics from CPU, NVIDIA Jeston and NVIDIA GPU.

For CPU, Kenning collects process-specific measurements, based on Kenning PID. For NVIDIA Jetson, Kenning collects process-specific CPU and shared RAM/VRAM usage measurements and overall usage for GPU, engines or power draw. NVIDIA GPU allows to collect process-specific usage measurements with `nvidia-ml-py` library. But when Kenning is run inside Docker container, `--pid host` option is required, because `nvidia-ml-py` reports host machine processes PID, which differs from container processes PID.

Metrics are collected with a certain prefix `<prefix>`, indicating the scope of computations. There are:

- `<prefix>_timestamp` - gives a timestamp for measurement collection in seconds.
- `<prefix>_cpus_percent` - gives per-core CPU utilization in % in a form of a list of lists. They are % of per-CPU usages for every timestamp.
- `<prefix>_mem_percent` - gives overall memory usage in %.
- `<prefix>_gpu_utilization` - gives overall GPU utilization in % (only works on platforms with NVIDIA GPUs and NVIDIA Jetson embedded devices).

- `<prefix>_gpu_mem_utilization` - gives GPU memory utilization in % (only works on platforms with NVIDIA GPUs and NVIDIA Jetson embedded devices).

CHAPTER

NINE

CHOOSING OPTIMAL OPTIMIZATION PIPELINE

The `kenning.scenarios.optimization_runner` script allows to optimize over multiple pipelines and to choose the best performing based on a specified criteria

The script can be run as follows:

```
kenning fine-tune-optimizers --json-cfg <CONFIG_JSON>.json --output <OUTPUT_PATH>.
↪json
```

With the arguments:

- `<CONFIG_JSON>.json` - describes the configuration, which pipelines would be executed and optimization settings
- `<OUTPUT_PATH>.json` - base path to the output files with measurements.

9.1 Optimization config specification

The example configuration looks like this:

```
{
    "optimization_parameters":
    {
        "strategy": "grid_search",
        "optimizable": ["optimizers", "runtime"],
        "metric": "inferencetime_mean",
        "policy": "min"
    },
    "model_wrapper":
    {
        "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
        "parameters":
        {
            "model_path": "kenning:///models/classification/tensorflow_pet_
↪dataset_mobilenetv2.h5"
        }
    },
    "dataset":
    {
        "type": "kenning.datasets.pet_dataset.PetDataset",
```

(continues on next page)

(continued from previous page)

```
        "parameters":
        {
            "dataset_root": "./build/pet-dataset"
        }
    },
    "optimizers":
    [
        {
            "type": "kenning.optimizers.litert.LiteRTCompiler",
            "parameters":
            {
                "target": ["default"],
                "compiled_model_path": ["./build/compiled_model.tflite"]
            }
        },
        {
            "type": "kenning.optimizers.tvm.TVMCompiler",
            "parameters":
            {
                "target": ["llvm"],
                "compiled_model_path": ["./build/compiled_model.tar"],
                "opt_level": [3],
                "conv2d_data_layout": ["NHWC", "NCHW"]
            }
        }
    ],
    "runtime":
    [
        {
            "type": "kenning.runtimes.tvm.TVMRuntime",
            "parameters":
            {
                "save_model_path": ["./build/compiled_model.tar"]
            }
        },
        {
            "type": "kenning.runtimes.litert.LiteRTRuntime",
            "parameters":
            {
                "save_model_path": ["./build/compiled_model.tflite"]
            }
        }
    ]
}
```

The highlighted part describes the settings of the optimization run.

- `strategy` - Describes how the pipelines are chosen for optimization. Currently only available option is `grid_search`, which generates cartesian product of all blocks that should be optimized and runs all compatible pipelines

- `optimizable` - Which Kenning block should be variable in the pipeline. The `kenning.scenarios.optimization_runner` script will choose the Kenning block out of multiple provided in the later part of configuration based on `strategy`.
- `metric` - one of the metrics from *Kenning measurements* over which the optimization should be performed
- `policy` - `min` or `max`, depending on whether the metric should be maximized or minimized

The rest of the JSON configuration describes the pipeline in a format similar to *standard JSON scenarios*. The only difference is within the blocks chosen in the `optimizable` field, which should be list of blocks instead of singular definition. Every block in the list has defined list of arguments for all of it's parameters, this allows to choose the best argument for a particular block option

- For `optimizers` block, since there can be more than one in the pipeline, all possible combinations are tested.
- For the rest of the blocks one of all possible blocks is chosen for each run.

Kenning checks automatically whether a selected combination of blocks (model, optimizations and runtime) is compatible.

9.2 Output details

The `kenning.scenarios.optimization_runner` outputs multiple JSON files:

- `<OUTPUT PATH>.json` - Most optimal pipeline with it's aggregated metrics
- `<OUTPUT PATH>_all_results.json` - All pipeline configs and their respective metrics gathered into single file
- `<OUTPUT PATH>_<RUN ID>.json` - Full benchmark output for every optimization run.

The example `<OUTPUT PATH>.json` JSON file can look like this:

```
{
    "pipeline": {
        "model_wrapper": {
            "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
            "parameters": {
                "model_path": "kenning:///models/classification/tensorflow_pet_
↪dataset_mobilenetv2.h5"
            }
        },
        "dataset": {
            "type": "kenning.datasets.pet_dataset.PetDataset",
            "parameters": {
                "dataset_root": "./build/pet-dataset"
            }
        },
        "optimizers": [
            {
                "type": "kenning.optimizers.tvm.TVMCompiler",
```

(continues on next page)

(continued from previous page)


```
                "parameters": {
                    "target": "llvm",
                    "compiled_model_path": "build/7_compiled_model.tar",
                    "opt_level": 3,
                    "conv2d_data_layout": "NCHW"
                }
            }
        ],
        "runtime": {
            "type": "kenning.runtimes.litert.LiteRTRuntime",
            "parameters": {
                "save_model_path": "build/7_compiled_model.tflite"
            }
        }
    },
    "metrics": {
        "inferencetime_mean": 0.0031105340278004203,
        "inferencetime_std": 0.0003073369739186006,
        "inferencetime_median": 0.003066142999159638,
        "session_utilization_mem_percent_mean": 26.089648437500003,
        "session_utilization_mem_percent_std": 0.2059426531921169,
        "session_utilization_mem_percent_median": 26.1,
        "session_utilization_cpus_percent_avg_mean": 16.55806884765625,
        "session_utilization_cpus_percent_avg_std": 4.068584532658253,
        "session_utilization_cpus_percent_avg_median": 15.85625,
        "accuracy": 0.957273098380732,
        "mean_precision": 0.9574413909275928,
        "mean_sensitivity": 0.9571990915922841,
        "g_mean": 0.9566448082813757,
        "top_5_accuracy": 0.9961899578173902
    }
}
```


- The `pipeline` contains the definition of the best optimization pipeline.
- The `metrics` contains list of all aggregated metrics that can be considered in `metric` field `optimization_parameters`.

CHAPTER

TEN

SAMPLE AUTOGENERATED REPORT

This section contains a CI sample report for running inference on a compiled model.

The CI is set up as follows:

- *Environment*: Github Actions
- *Task*: dog and cat breeds classification based on the Oxford-IIIT Pet Dataset,
- *Training framework*: TensorFlow,
- *Compiler framework*: TVM,
- *Target*: CPU (using LLVM target on TVM).

10.1 Pet Dataset classification using TVM-compiled TensorFlow model

10.1.1 General information for build.local-cpu-tvm-tensorflow-classification.json

Model framework:

- keras ver. 2.21.0

Used commands

This section was generated using:

```
python -m kenning.__init__ \
    optimize \
    test \
    --modelwrapper-cls \
        kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2 \
    --dataset-cls \
        kenning.datasets.pet_dataset.PetDataset \
    --measurements \
        ./build/local-cpu-tvm-tensorflow-classification.json \
    --compiler-cls \
        kenning.optimizers.tvm.TVMCompiler \
    --runtime-cls \
        kenning.runtimes.tvm.TVMRuntime \
    --model-path \
        kenning:///models/classification/tensorflow_pet_dataset_mobilenetv2.h5 \
    --model-framework \
```

(continues on next page)

(continued from previous page)

```
        keras \
    --target \
        llvm \
    --compiled-model-path \
        ./build/compiled-model.tar \
    --opt-level \
        3 \
    --save-model-path \
        ./build/compiled-model.tar \
    --target-device-context \
        cpu \
    --dataset-root \
        ./build/PetDataset/ \
    --inference-batch-size \
        1 \
    --verbosity \
        INFO

python -m kenning.__init__ \
    report \
    --report-path \
        docs/source/generated/local-cpu-tvm-tensorflow-classification.md \
    --report-name \
        Pet Dataset classification using TVM-compiled TensorFlow model \
    --root-dir \
        docs/source/ \
    --img-dir \
        docs/source/generated/img \
    --report-types \
        performance \
        classification \
    --measurements \
        build/local-cpu-tvm-tensorflow-classification.json \
    --smaller-header
```

Input scenario

```
{
    "dataset": {
        "type": "kenning.datasets.pet_dataset.PetDataset",
        "parameters": {
            "classify_by": "breeds",
            "image_memory_layout": "NHWC",
            "dataset_root": "build/PetDataset",
            "inference_batch_size": 1,
            "download_dataset": true,
            "force_download_dataset": false,
            "external_calibration_dataset": null,
            "split_fraction_test": 0.2,
```

(continues on next page)

(continued from previous page)

```
            "split_fraction_val": null,
            "split_seed": 1234,
            "reduce_dataset": 1.0
        }
    },
    "dataconverter": {
        "type": "kenning.dataconverters.modelwrapper_dataconverter.
↪ModelWrapperDataConverter",
        "parameters": {}
    },
    "optimizers": [
        {
            "type": "kenning.optimizers.tvm.TVMCompiler",
            "parameters": {
                "model_framework": "keras",
                "target": "llvm",
                "target_attrs": "",
                "target_microtvm_board": null,
                "target_host": null,
                "zephyr_header_template": null,
                "zephyr_llext_source_template": null,
                "opt_level": 3,
                "libdarknet_path": "/usr/local/lib/libdarknet.so",
                "compile_use_vm": false,
                "output_conversion_function": "default",
                "conv2d_data_layout": "",
                "conv2d_kernel_layout": "",
                "use_fp16_precision": false,
                "use_int8_precision": false,
                "int8_calibrate_chunk_by": -1,
                "use_tensorrt": false,
                "dataset_percentage": 0.25,
                "module_name": null,
                "compiled_model_path": "./build/compiled-model.tar",
                "location": "host"
            }
        }
    ],
    "platform": {
        "type": "kenning.platforms.local.LocalPlatform",
        "parameters": {
            "name": null,
            "platforms_definitions": [
                "kenning:///platforms/platforms.yml",
                "/home/runner/work/kenning/kenning/kenning/resources/platforms/
↪platforms.yml"
            ]
        }
    },
```

(continues on next page)

(continued from previous page)


```
    "model_wrapper": {
        "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
        "parameters": {
            "batch_size": null,
            "learning_rate": null,
            "num_epochs": null,
            "logdir": null,
            "model_path": "kenning:///models/classification/tensorflow_pet_
↪dataset_mobilenetv2.h5",
            "model_name": null
        }
    },
    "runtime": {
        "type": "kenning.runtimes.tvm.TVMRuntime",
        "parameters": {
            "save_model_path": "./build/compiled-model.tar",
            "target_device_context": "cpu",
            "target_device_context_id": 0,
            "runtime_use_vm": false,
            "llext_binary_path": null,
            "batch_size": 1,
            "disable_performance_measurements": false
        }
    }
}
```


10.2 Inference performance metrics for build.local-cpu-tvm-tensorflow-classification.json

10.2.1 Inference time

Table 10.1: Inference time metrics

Statistic	Time [s]
First inference duration	**0.046396**
Mean	**0.044527**
Median	**0.044175**
Standard deviation	**0.001272**
Minimum	**0.042730**
Maximum	**0.058008**

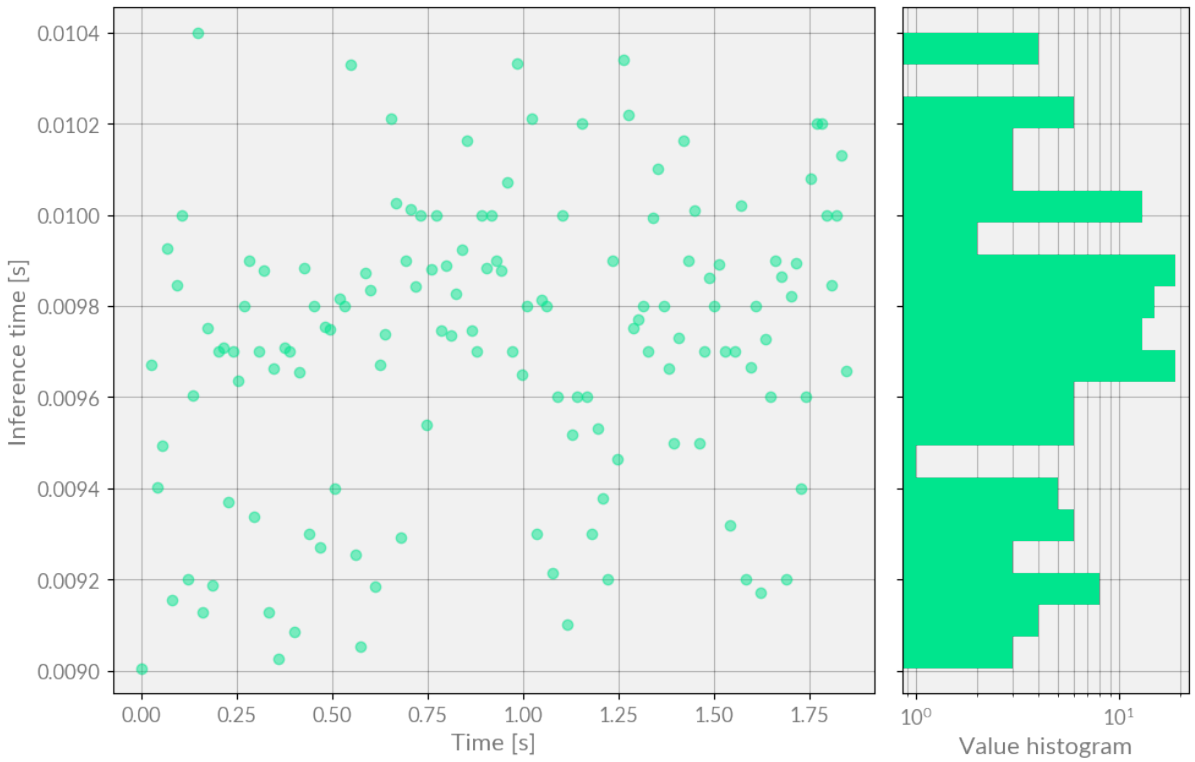


Figure 10.1: Inference time

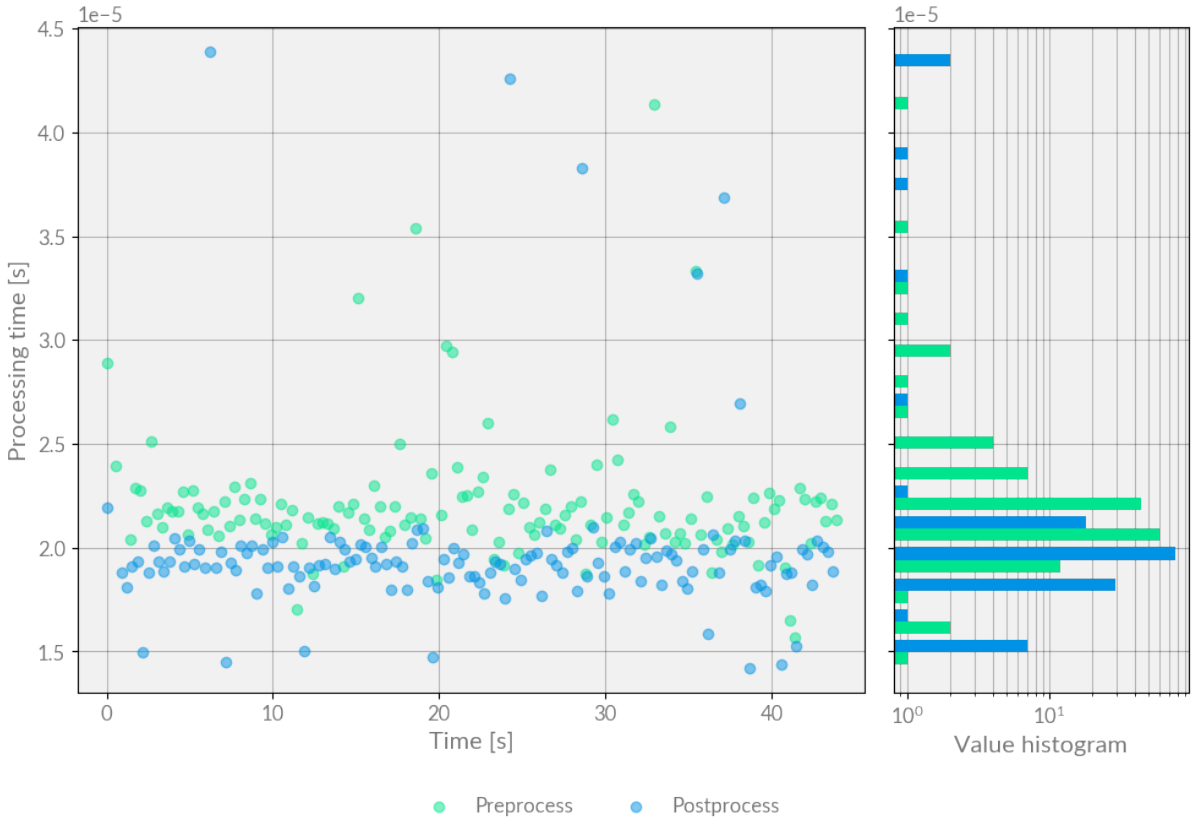


Figure 10.2: Preprocessing and postprocessing time

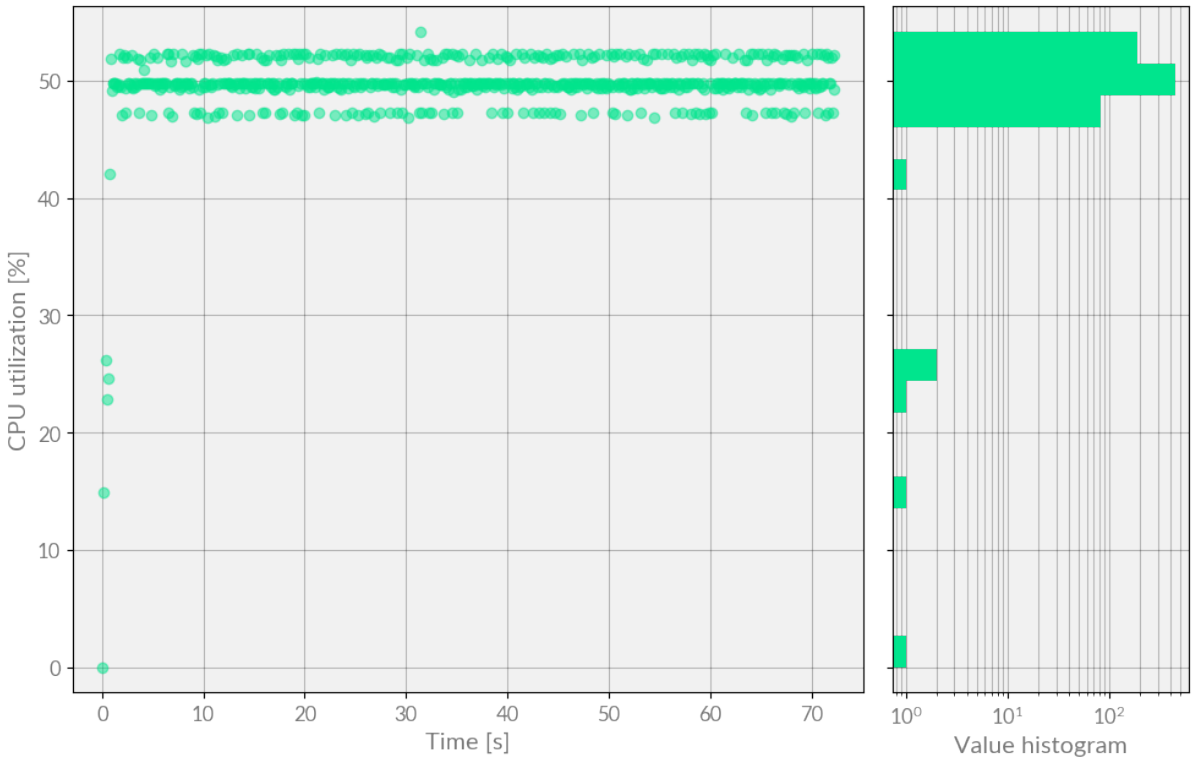


Figure 10.3: Average CPU usage by process during benchmark

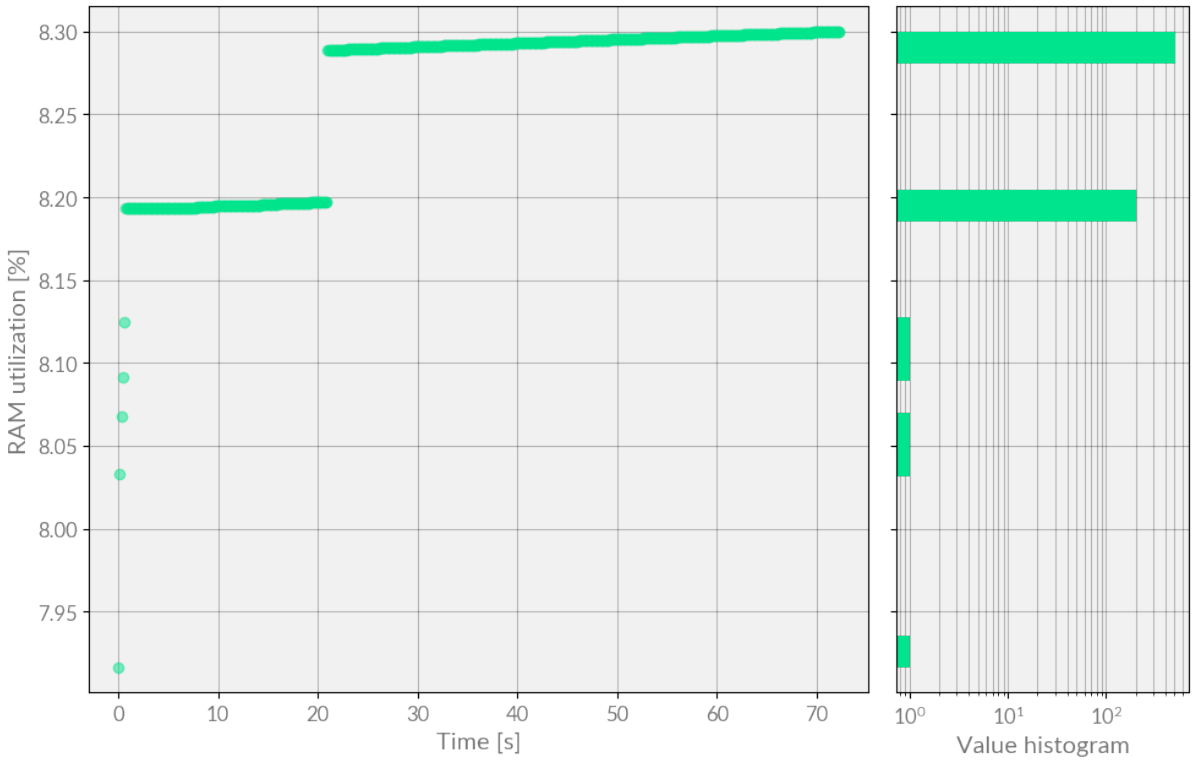


Figure 10.4: RAM usage by process during benchmark

10.2.2 Average CPU usage by process

10.2.3 RAM usage by process

Table 10.2: RAM usage by process metrics

Statistic	Usage [%]
Mean	**8.264**
Standard deviation	**0.050**
Median	**8.292**

10.3 Inference quality metrics for build.local-cpu-tvm-tensorflow-classification.json

Table 10.3: Inference quality metrics

Statistic	Value
Accuracy	**0.957143**
Mean precision	**0.958628**
Mean sensitivity	**0.956698**
G-mean	**0.955802**
Top-5 accuracy	**0.995918**

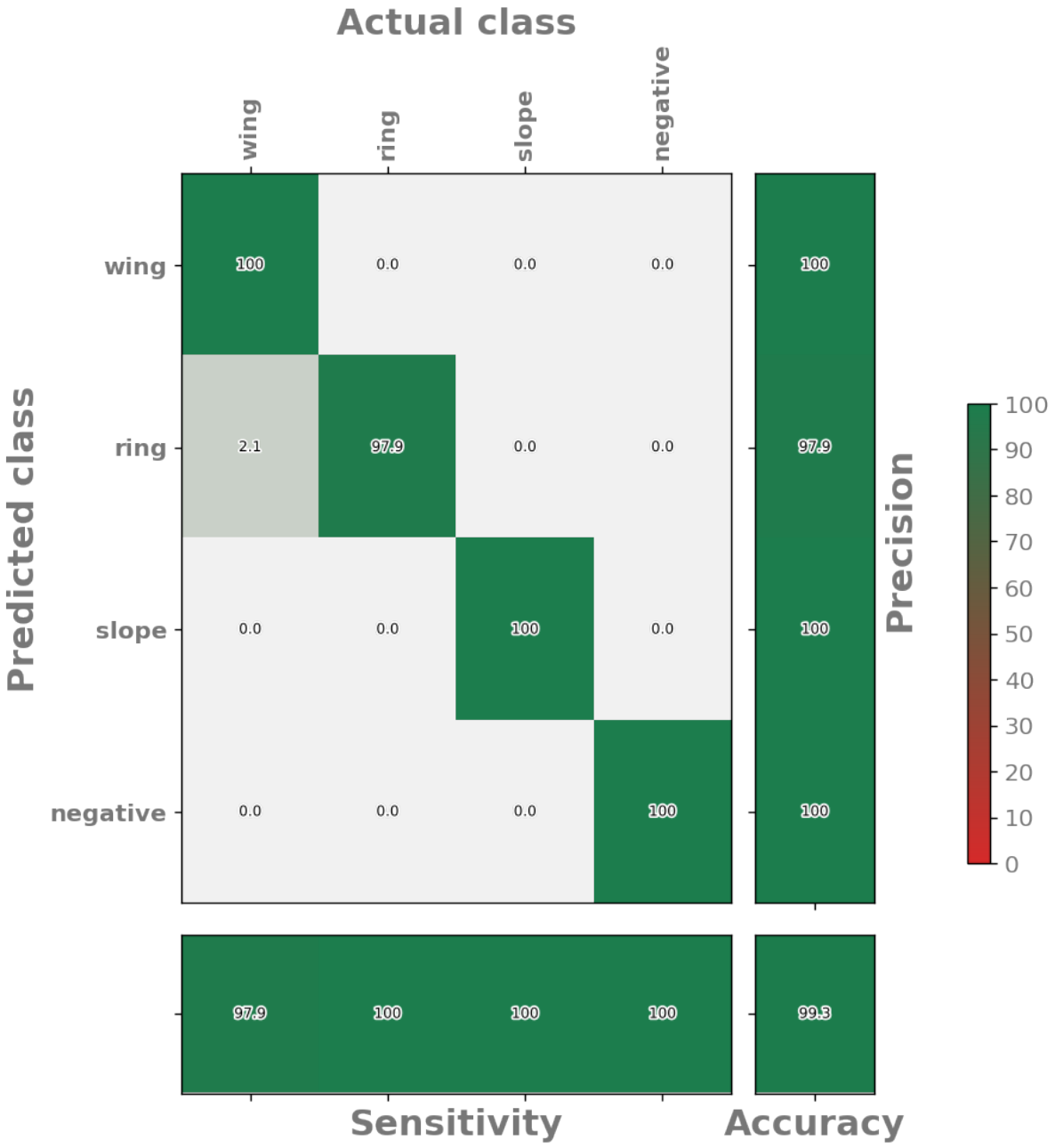


Figure 10.5: Confusion matrix

CHAPTER

ELEVEN

SAMPLE AUTOML REPORT

This section contains a sample AutoML report generated during CI.

The CI is set up as follows:

- *Environment*: Github Actions
- *Task*: Anomaly Detection
- *AutoML framework*: Auto-PyTorch
- *Compiler framework*: `LiteRT`,
- *Platform*: MAX32690 Evaluation Kit

11.1 AutoML statistics

- *Optimized metric*: **f1**
- *The number of generated models*: **65**
- *The number of trained and evaluated models*: **47**
- *The number of successful training processes*: **58**
- *The number of models that caused a crash*: **0**
- *The number of models that failed due to the timeout*: **0**
- *The number of models that failed due to the too large size*: **7**
- *The number of models that failed due to incompatibility*: **0**

11.1.1 Training overview

11.1.2 Summary of generated models

Table 11.1: Summary of generated models' parameters

Model ID	Number of layers	Optimized model size [KB]	Total parameters	Trainable parameters
3	7	15.78515625	2815	2814
4	10	49.8828125	11623	11622
5	17	33.10546875	7498	7497
6	21	33.703125	7613	7612
7	27	61.05078125	14094	14093
8	14	50.42578125	11841	11840

continues on next page

Table 11.1 – continued from previous page

Model ID	Number of layers	Optimized model size [KB]	Total parameters	Trainable parameters
9	21	35.796875	7834	7833
10	17	37.453125	8732	8731
11	23	54.33984375	11691	11690
12	21	30.58984375	7656	7655
13	17	33.16015625	7799	7798
14	12	37.78125	8227	8226
15	10	42.41796875	10264	10263
16	13	45.55859375	10455	10454
17	8	19.86328125	4364	4363
18	27	47.53125	10720	10719
19	21	35.453125	8204	8203
20	9	17.77734375	3190	3189
21	13	32.6171875	7850	7849
22	12	29.12109375	5864	5863
23	15	34.7578125	8009	8008
24	25	39.62890625	8756	8755
25	7	40.13671875	9288	9287
26	11	34.09765625	6843	6842
27	12	57.0625	14867	14866
28	13	40.23046875	7844	7843
29	21	60.14453125	14766	14765
30	19	39.8125	9778	9777
31	19	53.26953125	12119	12118
32	10	27.86328125	5434	5433
33	12	36.3515625	8413	8412
34	17	51.25390625	12285	12284
35	17	47.73828125	11113	11112
36	23	31.2109375	6730	6729
37	11	29.9453125	6124	6123
38	25	38.5625	8307	8306
39	13	61.4765625	14976	14975
40	21	38.625	9280	9279
41	10	38.50390625	10000	9999
42	9	30.87890625	7144	7143
43	17	46.85546875	11825	11824
44	19	16.26953125	2494	2493
45	15	16.72265625	3006	3005
46	19	42.20703125	9575	9574
47	15	22.12109375	4414	4413
48	23	31.94921875	6487	6486
49	15	46.23046875	10910	10909
50	12	40.47265625	8755	8754
51	15	37.50390625	10665	10664
52	17	44.21875	10603	10602
53	9	37.53515625	9222	9221
54	13	37.64453125	8871	8870
55	25	31.11328125	6490	6489

continues on next page

Table 11.1 – continued from previous page

Model ID	Number of layers	Optimized model size [KB]	Total parameters	Trainable parameters
56	19	53.01171875	13504	13503

11.2 Classification comparison

11.2.1 Comparison of inference time, F1 score and model size

Table 11.2: Comparison of model inference time, accuracy and size

Model name	Mean Inference time [s]	Size [MB]	F1 score
automl_conf_0	0.001215	0.045	0.470588
automl_conf_1	0.001049	0.041	0.533333
automl_conf_2	0.001048	0.041	0.533333
automl_conf_3	0.001020	0.035	0.533333
automl_conf_4	0.000938	0.037	0.533333

11.2.2 Detailed metrics comparison

Table 11.3: Summary of classification metrics for models

Model name	Accuracy	Mean precision	Mean sensitivity	G-mean	ROC AUC	F1 score
automl_conf_0	0.964000	0.713923	0.739669	0.699764	0.739669	0.470588
automl_conf_1	**0.972000**	**0.777484**	**0.743802**	**0.702710**	**0.743802**	**0.533333**
automl_conf_2	**0.972000**	**0.777484**	**0.743802**	**0.702710**	**0.743802**	**0.533333**
automl_conf_3	**0.972000**	**0.777484**	**0.743802**	**0.702710**	**0.743802**	**0.533333**
automl_conf_4	**0.972000**	**0.777484**	**0.743802**	**0.702710**	**0.743802**	**0.533333**

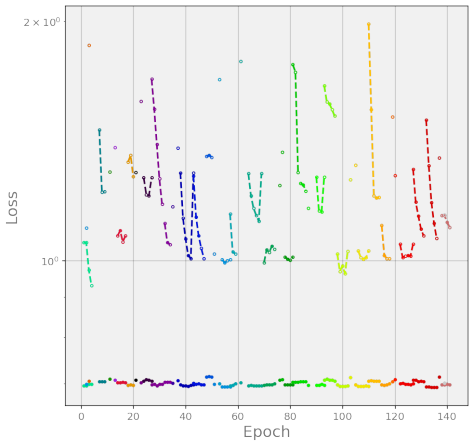


Figure 11.1: Loss value during AutoML training process

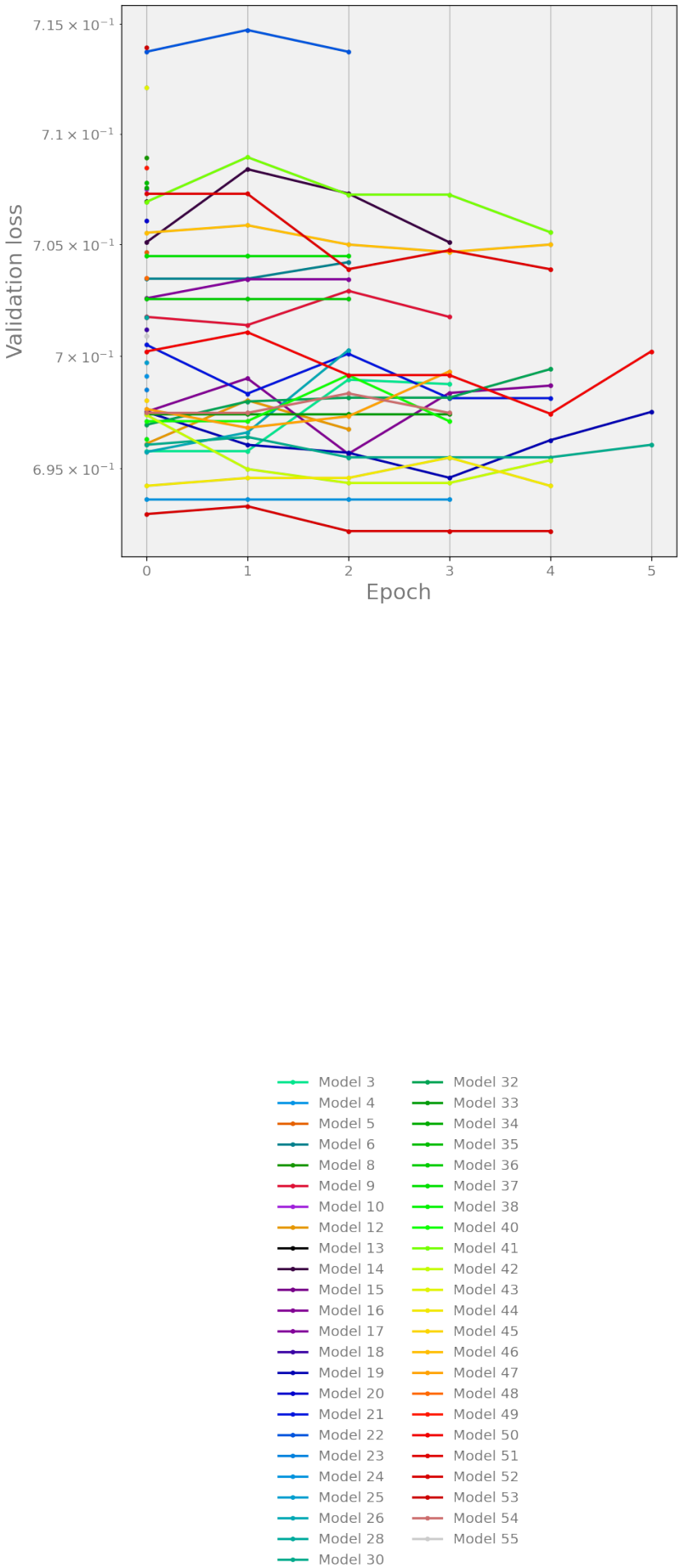


Figure 11.2: Comparison of loss value across models

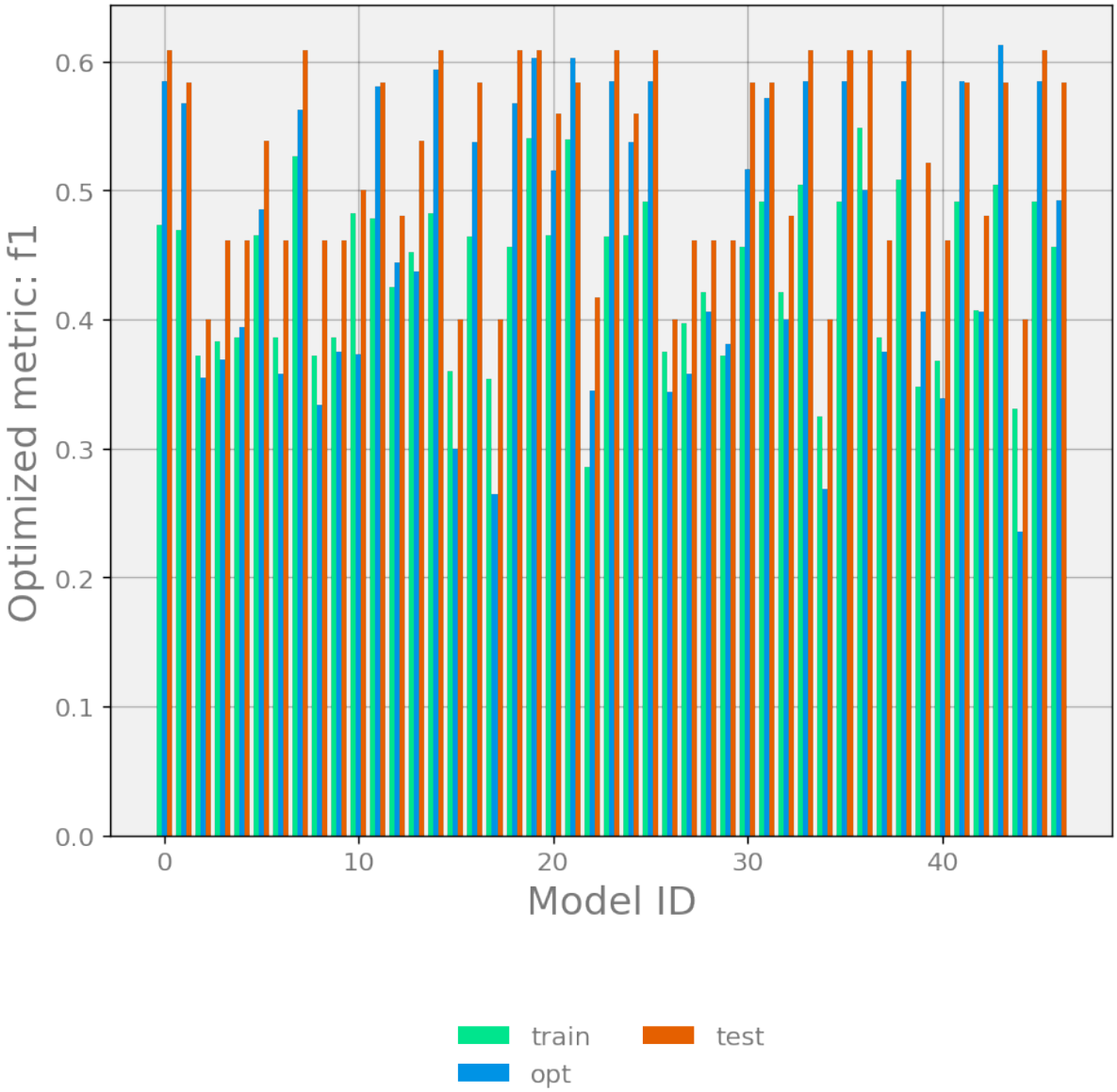


Figure 11.3: Metrics of models trained by AutoML flow

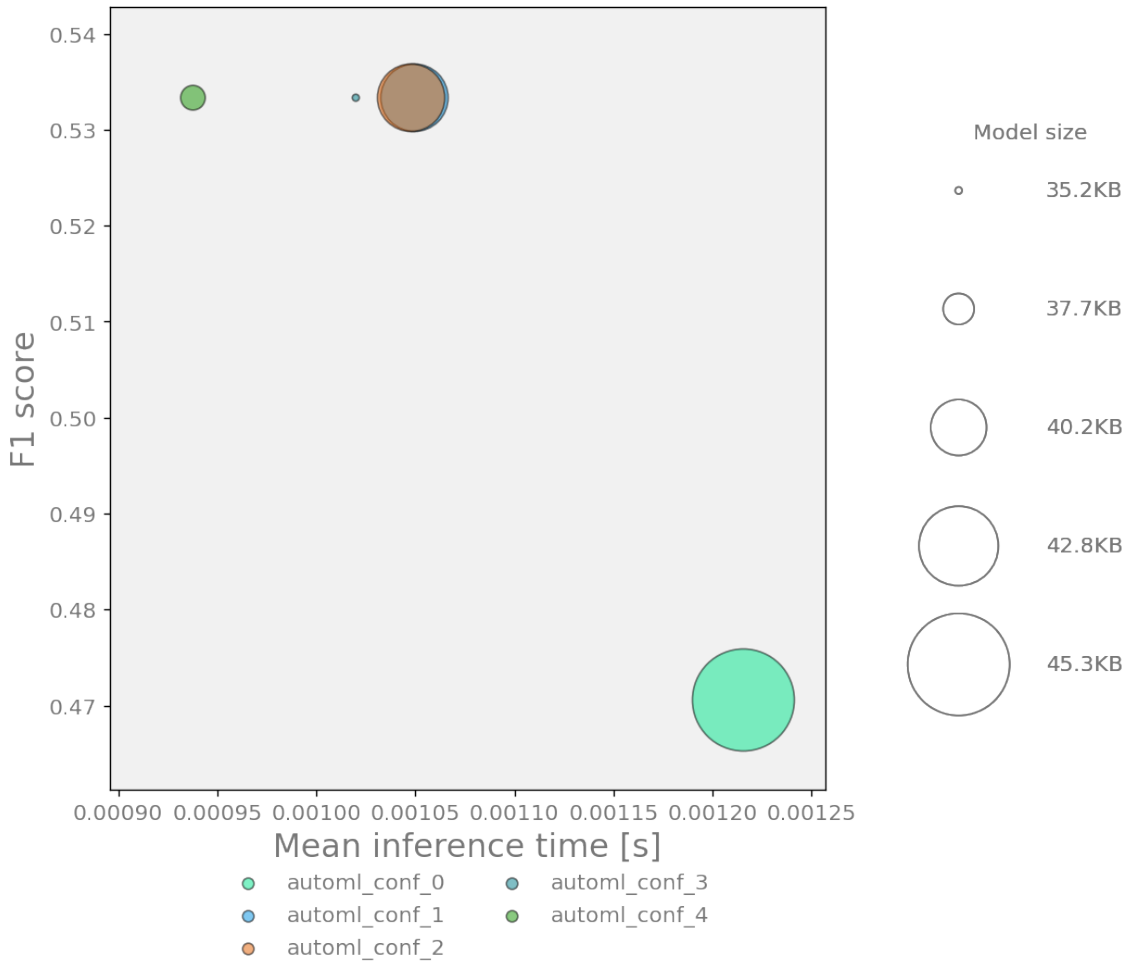


Figure 11.4: Model size, speed and quality summary. The F1 score of the model is presented on Y axis. The inference time of the model is presented on X axis. The size of the model is represented by the size of its point.

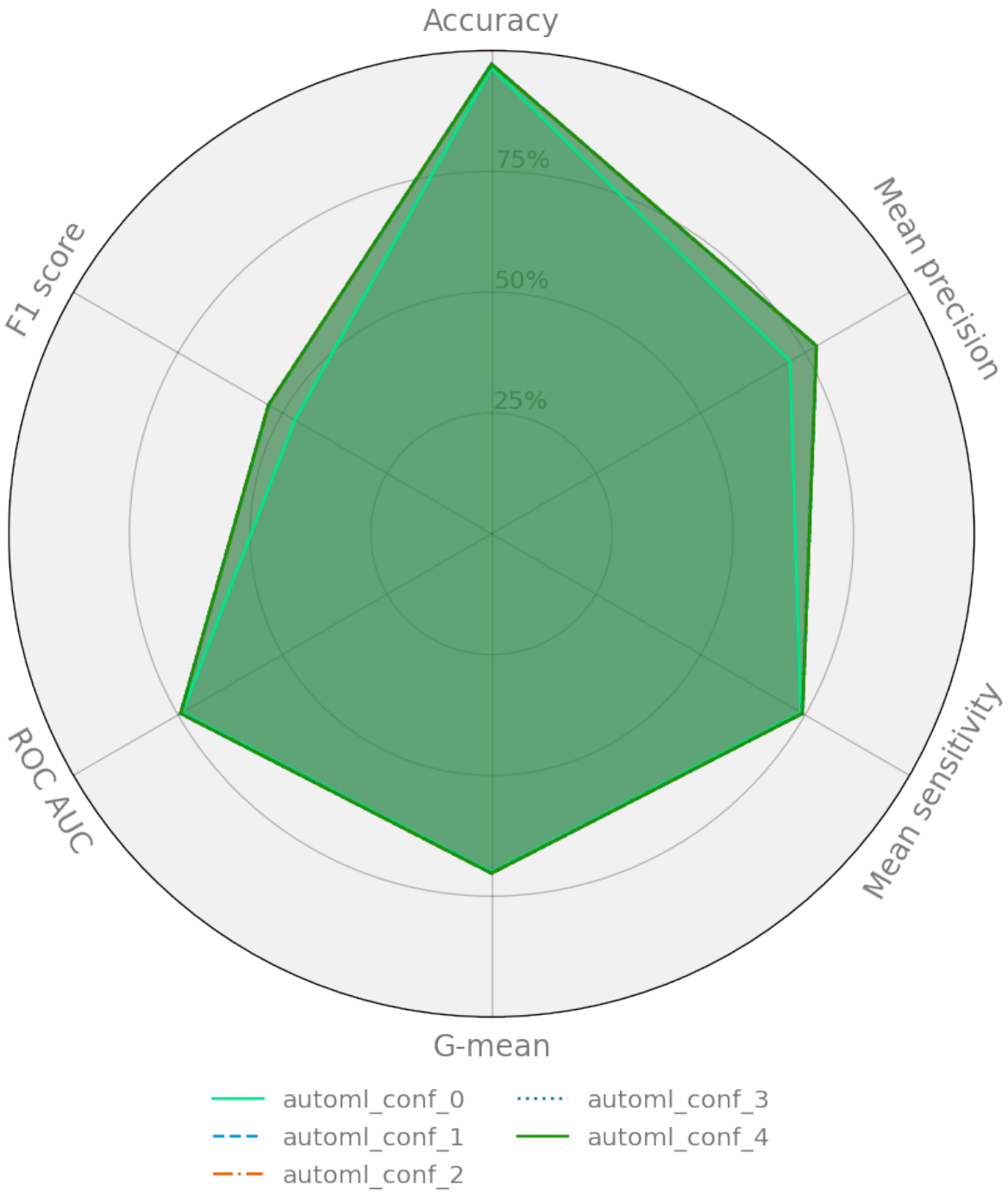


Figure 11.5: Radar chart representing the accuracy, precision and recall for models

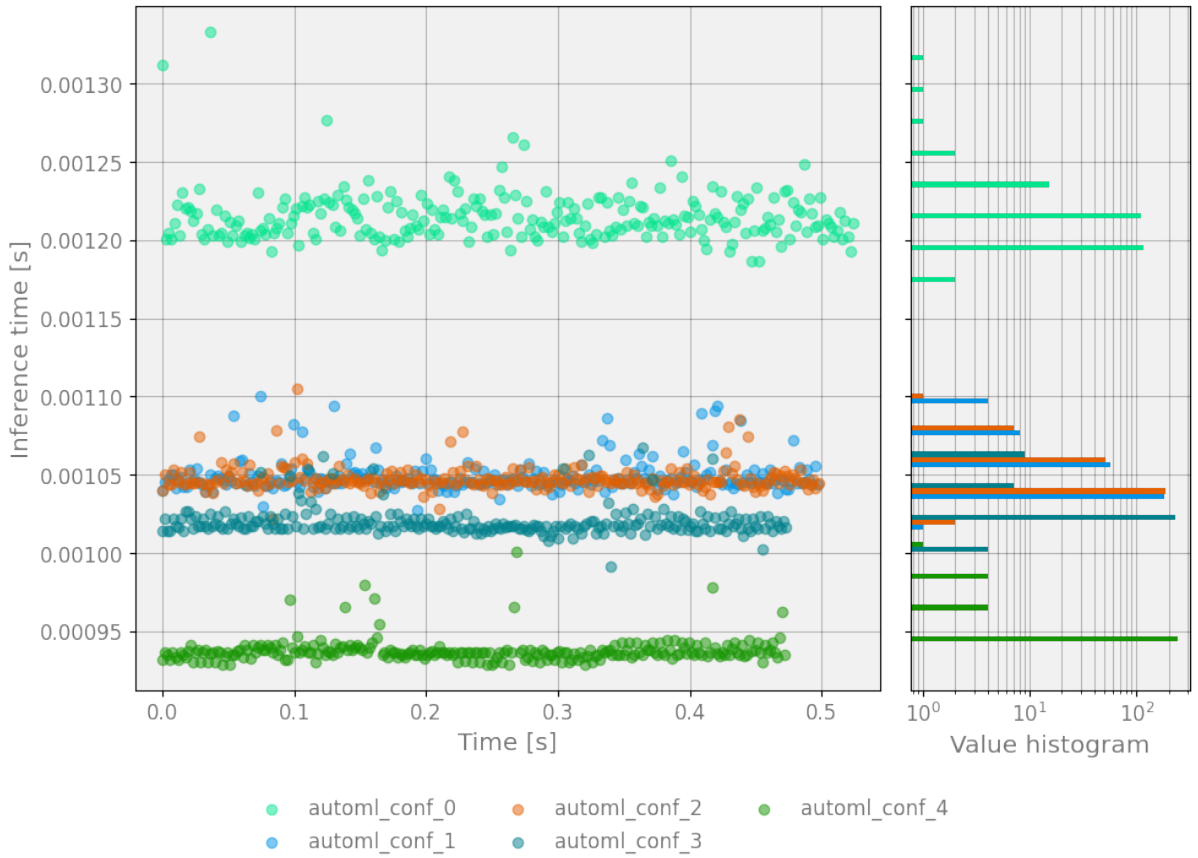


Figure 11.6: Plot represents changes of inference time over time for all models.

11.3 Inference comparison

11.3.1 Performance metrics

Table 11.4: Summary of inference time metrics for models

Model name	Median [s]	Minimum [s]	Standard deviation [s]	Maximum [s]	Mean [s]
automl_conf_0	0.001212	0.001187	0.000016	0.001333	0.001215
automl_conf_1	0.001046	0.001028	0.000010	0.001100	0.001049
automl_conf_2	0.001046	0.001022	0.000008	0.001105	0.001048
automl_conf_3	0.001017	0.000991	0.000009	0.001067	0.001020
automl_conf_4	0.000936	0.000928	0.000008	0.001001	0.000938

11.3.2 Mean comparison plots

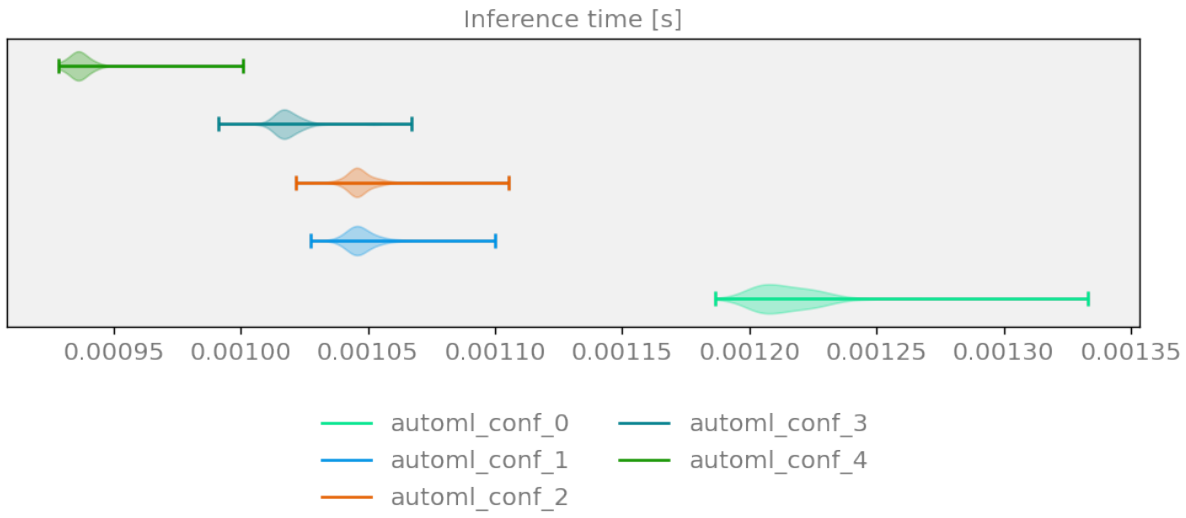


Figure 11.7: Violin chart representing distribution of values for performance metrics for models

Table 11.5: Performance metric for models

Model name	Inference time [s]
automl_conf_0	0.001215
automl_conf_1	0.001049
automl_conf_2	0.001048
automl_conf_3	0.001020
automl_conf_4	0.000938

CHAPTER

TWELVE

SAMPLE RISC-V ZEPHYR TRACING REPORT

This section contains a sample complete report with Zephyr tracing section on RISC-V platform, generated by Kenning using Zephelin.

Testing configuration:

- *Environment*: Github Actions
- *Task*: Gesture recognition on Magic Wand dataset
- *Tracing tool*: Zephelin
- *Compiler framework*: `TVM`
- *Platform*: SiFive HiFive Unmatched

12.1 Sample RISC-V Zephyr Tracing Report

12.1.1 General information for results.json

Model framework:

- torch ver. 2.13.0+cu130

Used commands

This section was generated using:

```
python -m kenning.__init__ \
    optimize \
    test \
    report \
    --cfg \
        /home/runner/work/kenning/kenning/scripts/configs/zephyr-tvm-magic-wand-
↪hifive-unmatched-report.yml \
    --measurements \
        ./results.json \
    --report-path \
        /home/runner/work/kenning/kenning/docs/source/generated/sample-riscv-
↪zephyr-tracing-report.md \
    --report-types \
        classification \
        performance \
        renode_stats \
```

(continues on next page)

(continued from previous page)

```
        zephyr_traces \
        model \
    --root-dir \
        /home/runner/work/kenning/kenning/docs/source/ \
    --img-dir \
        /home/runner/work/kenning/kenning/docs/source/generated/img/ \
    --report-name \
        Sample RISC-V Zephyr Tracing Report \
    --verbosity \
        INFO \
    --to-html
```

Input scenario

```
{
    "dataset": {
        "type": "kenning.datasets.magic_wand_dataset.MagicWandDataset",
        "parameters": {
            "window_size": 128,
            "window_shift": 128,
            "noise_level": 20,
            "dataset_root": "build/data",
            "inference_batch_size": 1,
            "download_dataset": true,
            "force_download_dataset": false,
            "external_calibration_dataset": null,
            "split_fraction_test": 0.2,
            "split_fraction_val": null,
            "split_seed": 1234,
            "reduce_dataset": 1.0
        }
    },
    "dataconverter": {
        "type": "kenning.dataconverters.modelwrapper_dataconverter.
↪ModelWrapperDataConverter",
        "parameters": {}
    },
    "optimizers": [
        {
            "type": "kenning.optimizers.tvm.TVMCompiler",
            "parameters": {
                "model_framework": "any",
                "target": null,
                "target_attrs": "",
                "target_microtvm_board": "hifive_unmatched/fu740/u74",
                "target_host": null,
                "zephyr_header_template": "gh://antmicro:kenning-zephyr-runtime/
↪lib/kenning_inference_lib/runtimes/tvm/generated/model_impl.h.template;
↪branch=main",
```

(continues on next page)

(continued from previous page)


```
                "zephyr_llext_source_template": null,
                "opt_level": 3,
                "libdarknet_path": "/usr/local/lib/libdarknet.so",
                "compile_use_vm": false,
                "output_conversion_function": "default",
                "conv2d_data_layout": "",
                "conv2d_kernel_layout": "",
                "use_fp16_precision": false,
                "use_int8_precision": false,
                "int8_calibrate_chunk_by": -1,
                "use_tensorrt": false,
                "dataset_percentage": 0.25,
                "module_name": null,
                "compiled_model_path": "./build/compiled-model-magic-wand.graph_
↪data",
                "location": "host"
            }
        }
    ],
    "platform": {
        "type": "kenning.platforms.zephyr.ZephyrPlatform",
        "parameters": {
            "zephyr_build_path": "build/hifive_unmatched",
            "llext_binary_path": null,
            "sensors": null,
            "sensors_frequency": null,
            "enable_zephelin_gdb": false,
            "enable_zephelin": true,
            "zephyr_base": null,
            "uart_port": "/tmp/renode_uart_zthc9uoq/uart",
            "uart_baudrate": 115200,
            "uart_log_port": "/tmp/renode_uart_zthc9uoq/uart_log",
            "uart_log_baudrate": 115200,
            "auto_flash": false,
            "openocd_path": "openocd",
            "sensor": null,
            "number_of_batches": 16,
            "simulated": true,
            "runtime_binary_path": null,
            "platform_resc_path": null,
            "resc_dependencies": [],
            "post_start_commands": [
                "logLevel 3 sysbus.uart1"
            ],
            "disable_opcode_counters": false,
            "disable_profiler": false,
            "profiler_dump_path": "/tmp/renode_profiler_k38wa81h.dump",
            "profiler_interval_step": 10.0,
            "runtime_init_log_msg": "Inference server started",
```


(continues on next page)

(continued from previous page)

```
            "runtime_init_timeout": 30,
            "gdb_port": 3333,
            "renode_log_lines_single_read_limit": 5000,
            "name": "hifive_unmatched/fu740/u74",
            "platforms_definitions": [
                "kenning:///platforms/platforms.yml",
                "/home/runner/work/kenning/kenning/kenning/resources/platforms/
↪platforms.yml"
            ]
        }
    },
    "protocol": {
        "type": "kenning.protocols.uart.UARTProtocol",
        "parameters": {
            "port": "/tmp/renode_uart_zthc9uoq/uart",
            "baudrate": 115200,
            "error_recovery": true,
            "timeout": 30
        }
    },
    "model_wrapper": {
        "type": "kenning.modelwrappers.classification.pytorch_magic_wand.
↪PyTorchMagicWandModelWrapper",
        "parameters": {
            "batch_size": null,
            "learning_rate": null,
            "num_epochs": null,
            "window_size": 128,
            "logdir": null,
            "export_dict": false,
            "model_path": "kenning:///models/classification/magic_wand.pth",
            "model_name": null
        }
    },
    "runtime": {
        "type": "kenning.runtimes.tvm.TVMRuntime",
        "parameters": {
            "save_model_path": "./build/compiled-model-magic-wand.graph_data",
            "target_device_context": "cpu",
            "target_device_context_id": 0,
            "runtime_use_vm": false,
            "llext_binary_path": null,
            "batch_size": 1,
            "disable_performance_measurements": false
        }
    },
    "runtime_builder": {
        "type": "kenning.runtimebuilders.zephyr.ZephyrRuntimeBuilder",
        "parameters": {
```

(continues on next page)

(continued from previous page)


```
            "board": "hifive_unmatched/fu740/u74",
            "application_dir": "/home/runner/work/kenning/kenning/zephyr-
↪workspace/kenning-zephyr-runtime/app",
            "build_dir": "/home/runner/work/kenning/kenning/zephyr-workspace/
↪kenning-zephyr-runtime/build",
            "venv_dir": "/home/runner/work/kenning/kenning/zephyr-workspace/
↪kenning-zephyr-runtime/.west-venv",
            "extra_targets": [
                "board-repl"
            ],
            "extra_build_args": [
                "-DCONFIG_KENNING_TVM_MODEL_PRE_GEN=y"
            ],
            "use_llext": false,
            "run_west_update": false,
            "workspace": "/home/runner/work/kenning/kenning/zephyr-workspace/
↪kenning-zephyr-runtime",
            "output_path": "build/hifive_unmatched",
            "model_framework": "tvm"
        }
    }
}
```


12.1.2 Inference quality metrics for results.json

Table 12.1: Inference quality metrics

Statistic	Value
Accuracy	**0.992857**
Mean precision	**0.994681**
Mean sensitivity	**0.994681**
G-mean	**0.994638**

12.1.3 Inference performance metrics for results.json

Inference time

Table 12.2: Inference time metrics

Statistic	Time [s]
First inference duration	**0.009222**
Mean	**0.009712**
Median	**0.009747**
Standard deviation	**0.000315**
Minimum	**0.009006**
Maximum	**0.010400**

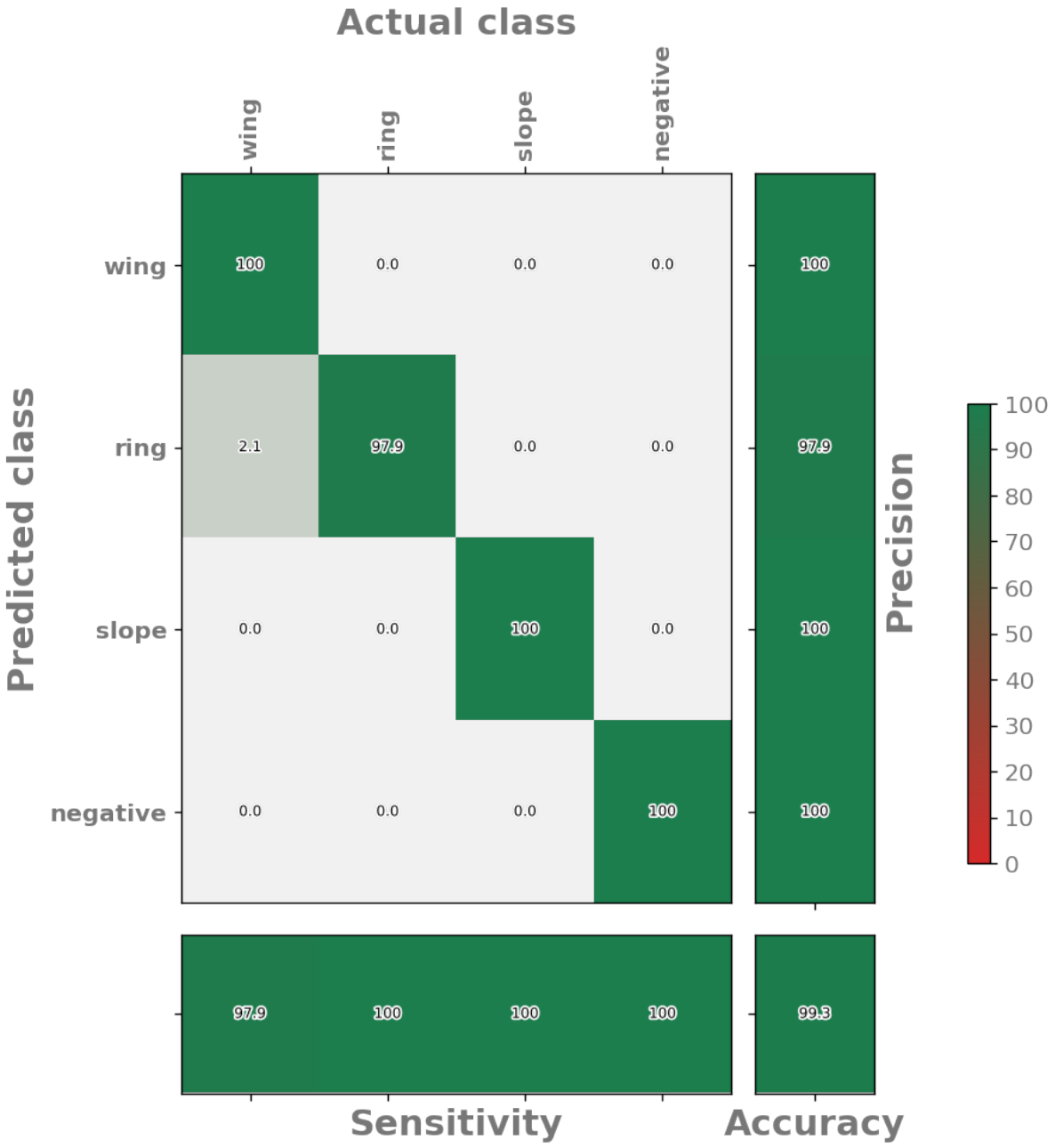


Figure 12.1: Confusion matrix

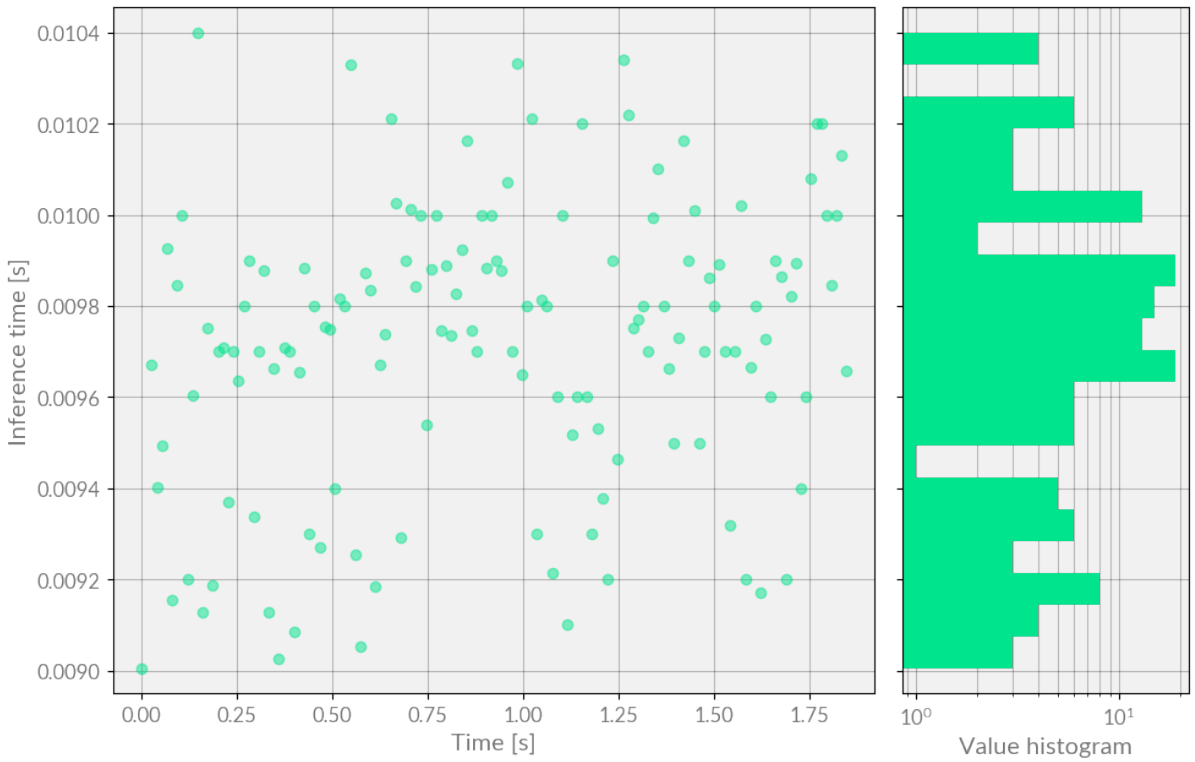


Figure 12.2: Inference time

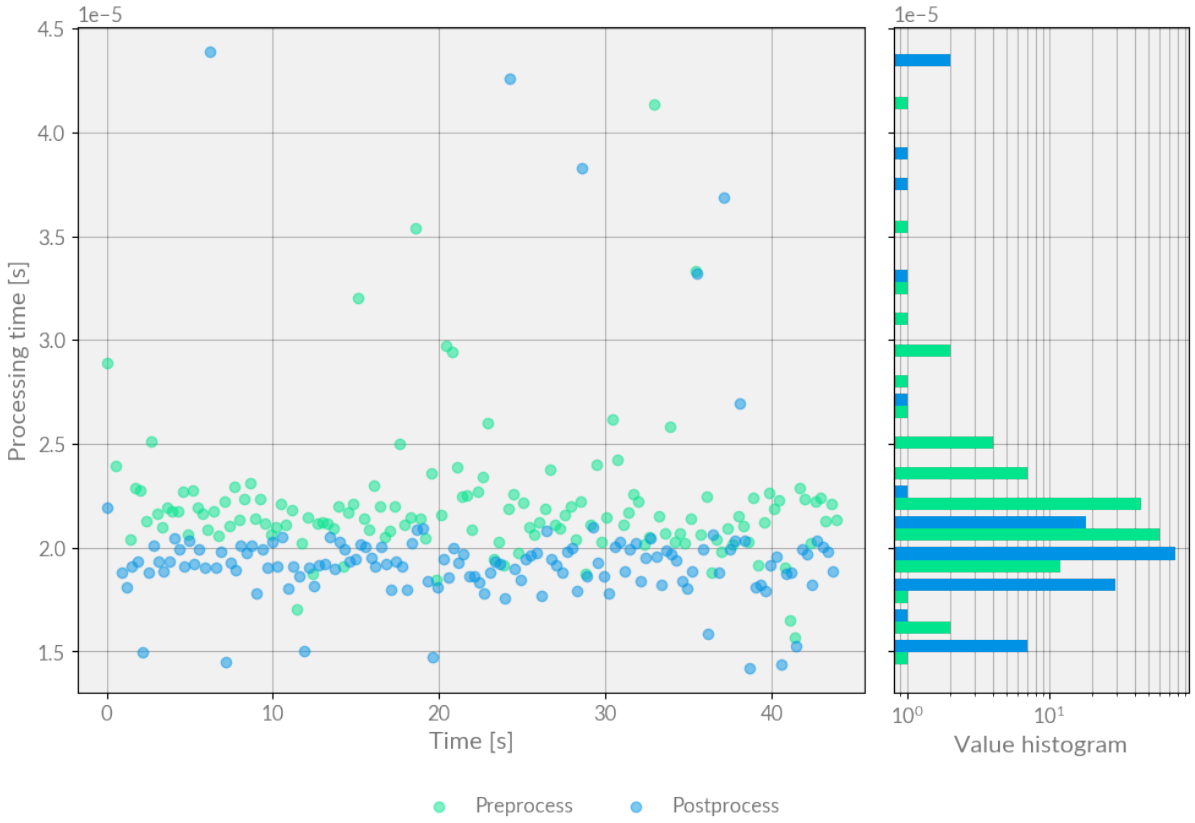


Figure 12.3: Preprocessing and postprocessing time

12.1.4 Renode performance measurements for results.json

Count of instructions used during inference

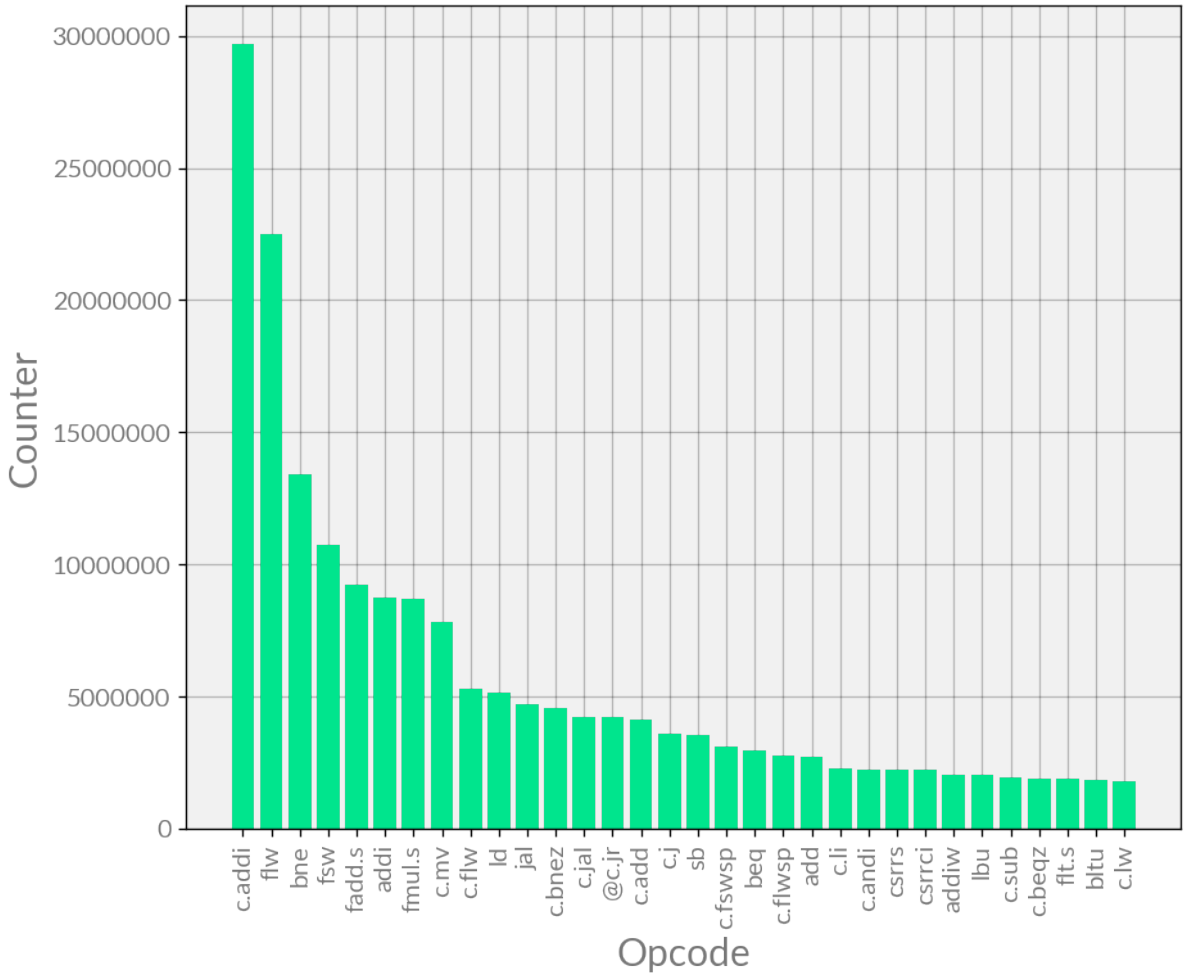


Figure 12.4: Histogram of used instructions during inference

Executed instructions counters

Peripheral access counters

Exceptions counters

Instructions stats

- *Instructions counters per inference pass*: **1420285**
- *Top 10 instructions and counters per inference pass*:
  - *c.addi*: **212017**
  - *flw*: **160863**
  - *bne*: **95863**
  - *fsw*: **76555**
  - *fadd.s*: **66052**

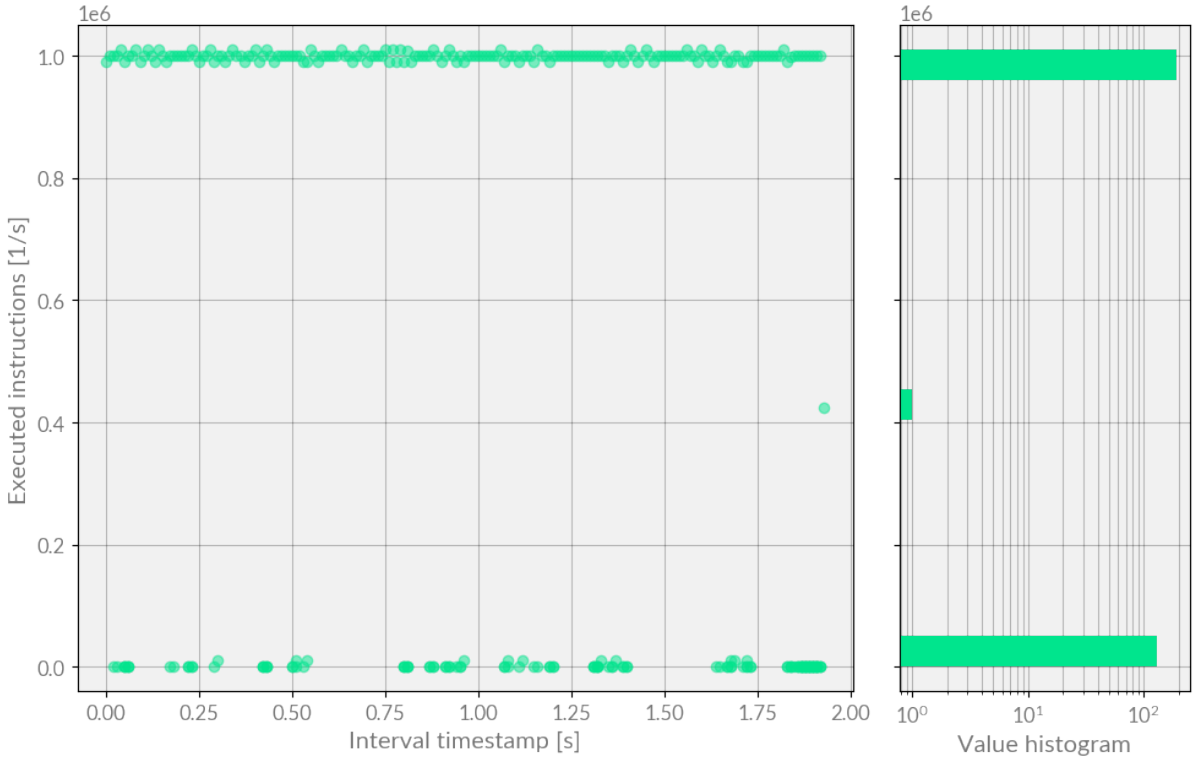


Figure 12.5: Count of executed instructions per second for cpu0 during benchmark

- *addi*: **62444**
- *fmul.s*: **62272**
- *c.mv*: **55975**
- *c.flw*: **37731**
- *ld*: **36721**

Memory allocation stats

- *Total allocated*: **3091632**
- *Total freed*: **3033712**
- *Peak allocated*: **72256**
- *Compiled model size*: **22486.0**

Host memory refers to memory of the CPU controlling the accelerator, while device memory is the memory of the accelerator.

Zephyr traces

(open in a new tab)

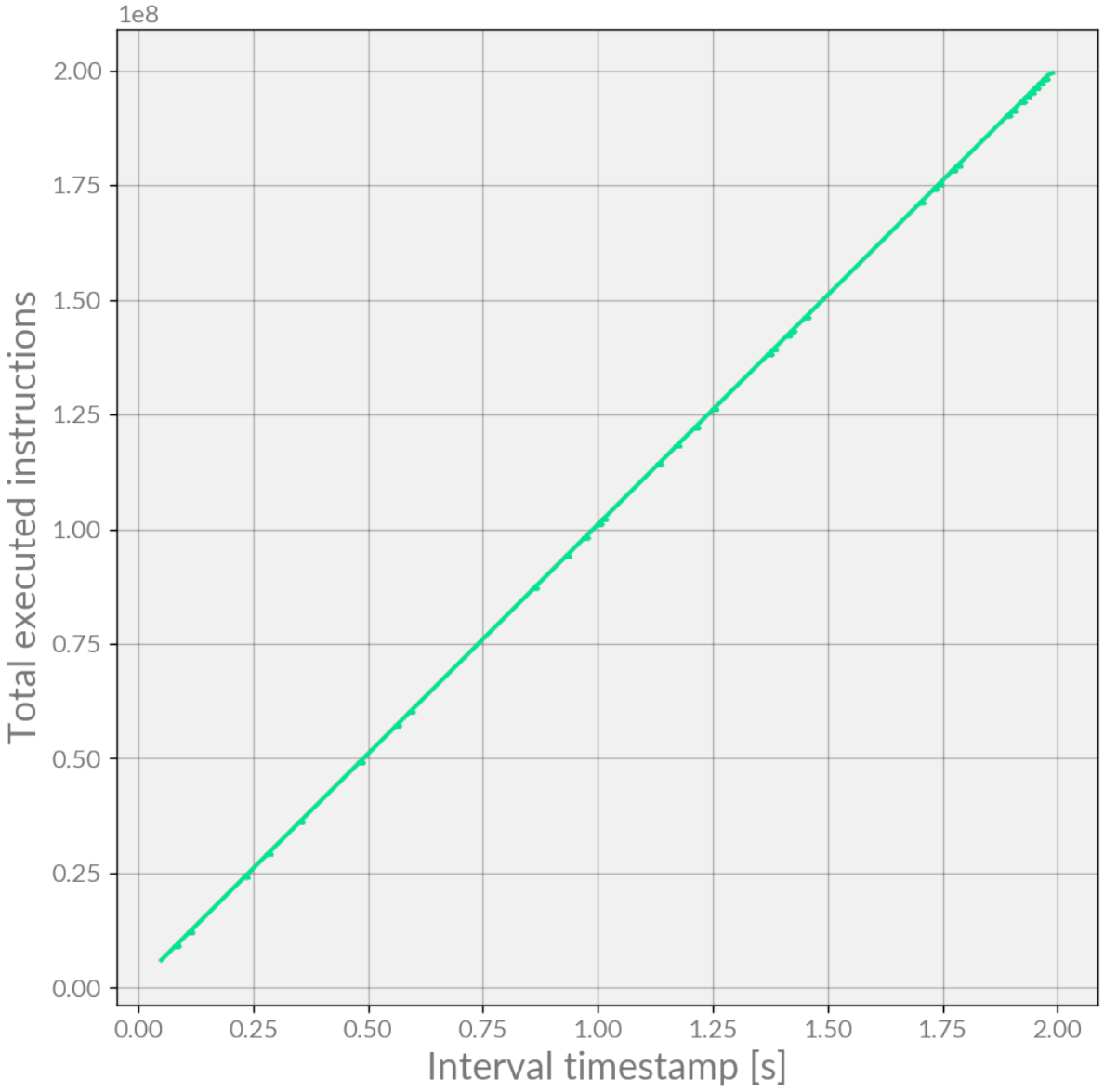


Figure 12.6: Cumulative count of executed instructions for cpu0 during benchmark

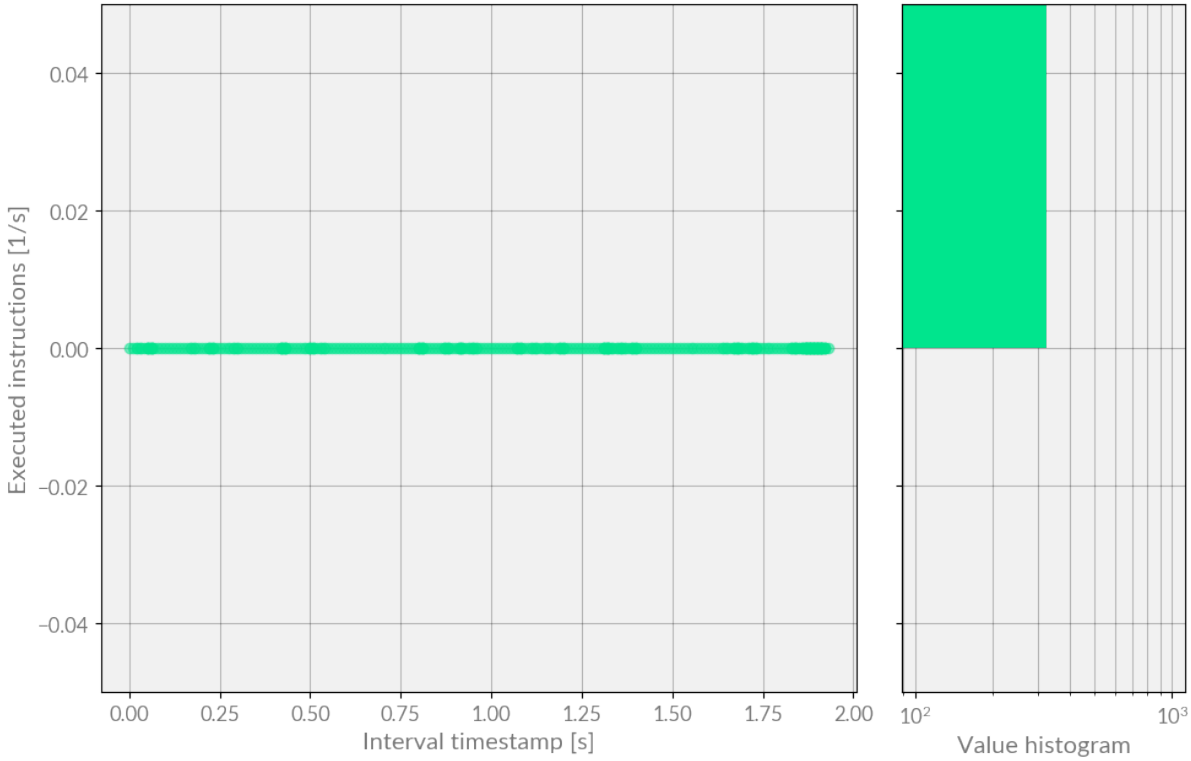
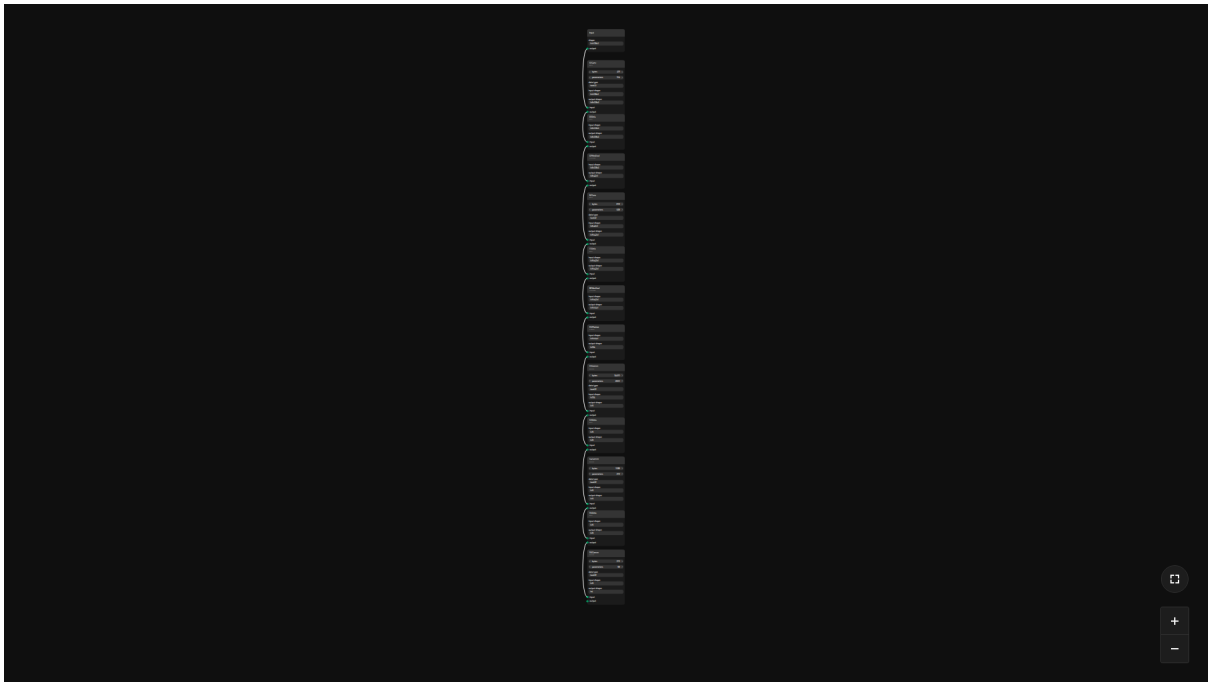


Figure 12.7: Count of executed instructions per second for cpu1 during benchmark

12.1.5 Input model specification for results.json

Model visualization



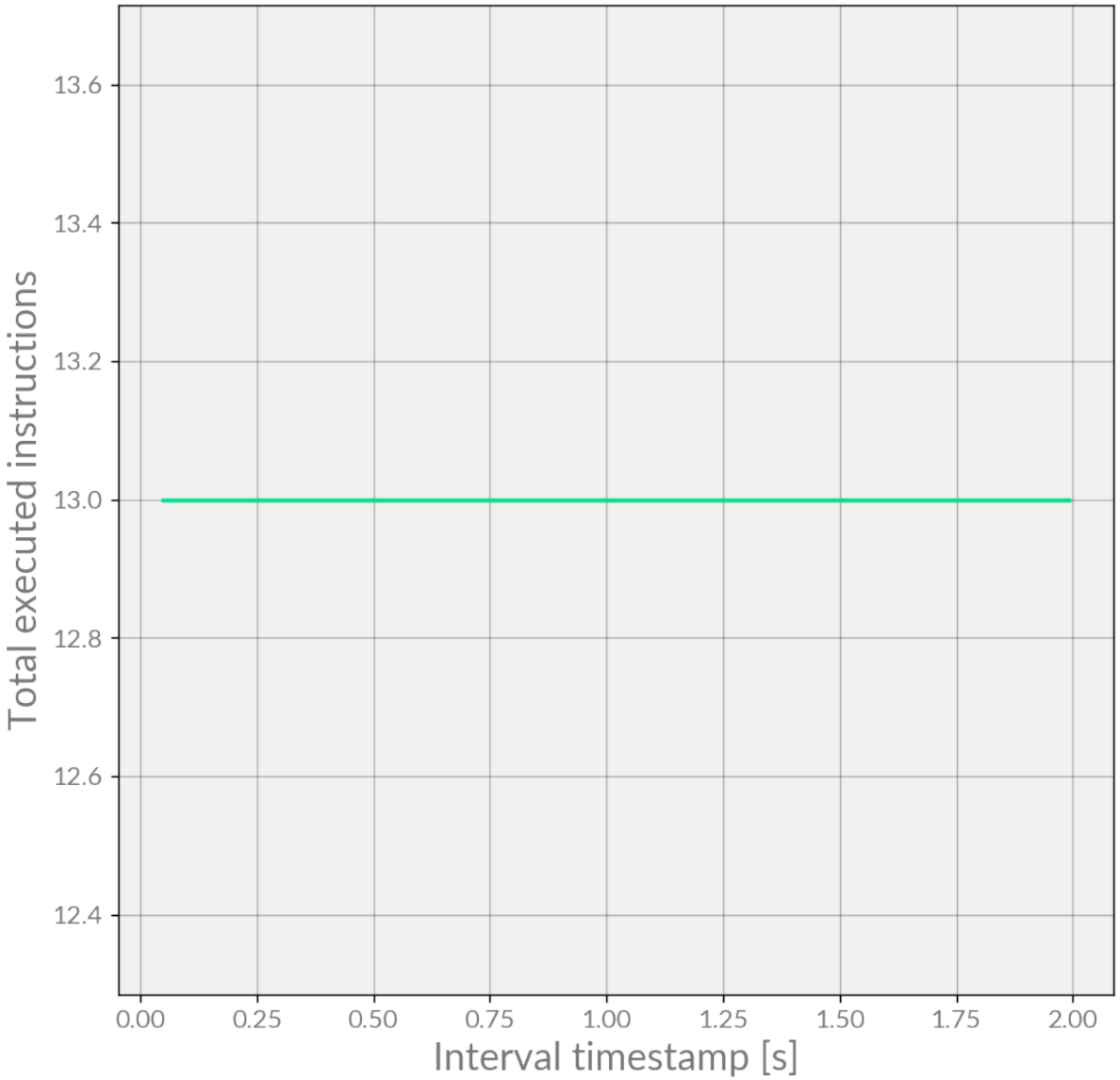


Figure 12.8: Cumulative count of executed instructions for cpu1 during benchmark

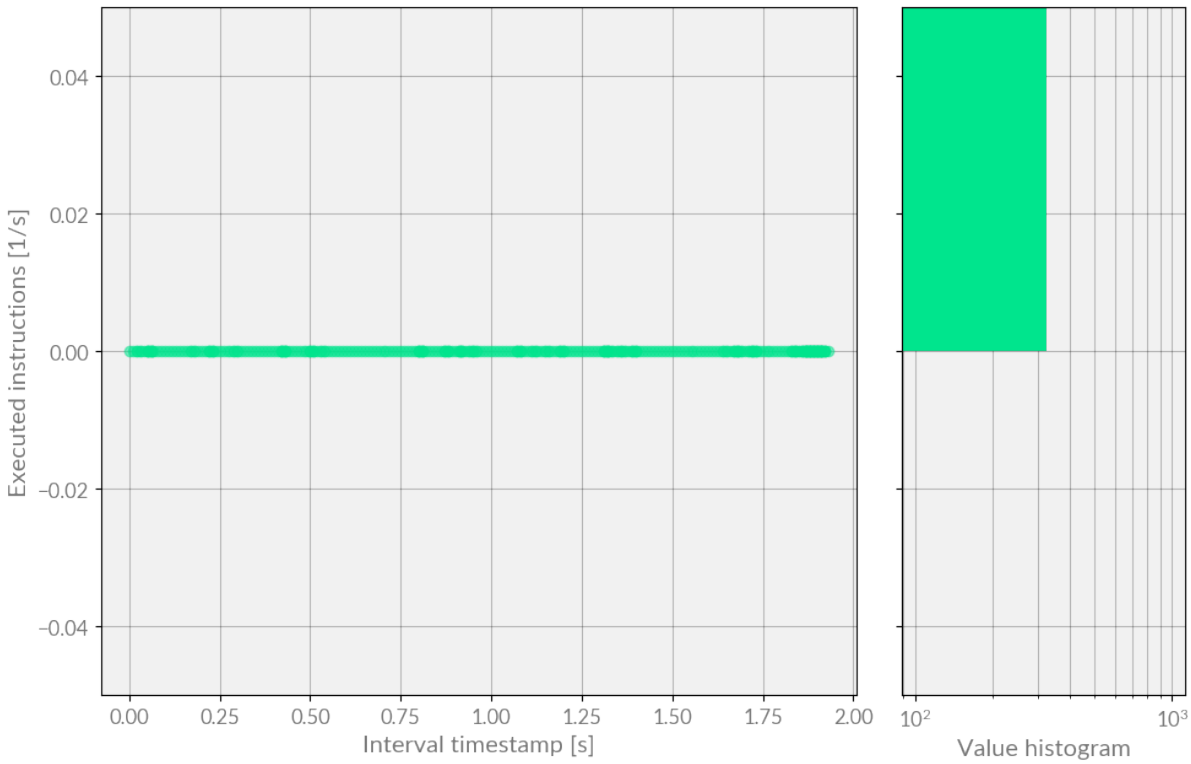


Figure 12.9: Count of executed instructions per second for cpu2 during benchmark

Basic model information

Table 12.3: Basic model information

Model name	Layer count	Number of parameters	ONNX-converted model size (in bytes)	Model Framework (version)
Main	**14**	**4572**	**18288**	**torch ()**

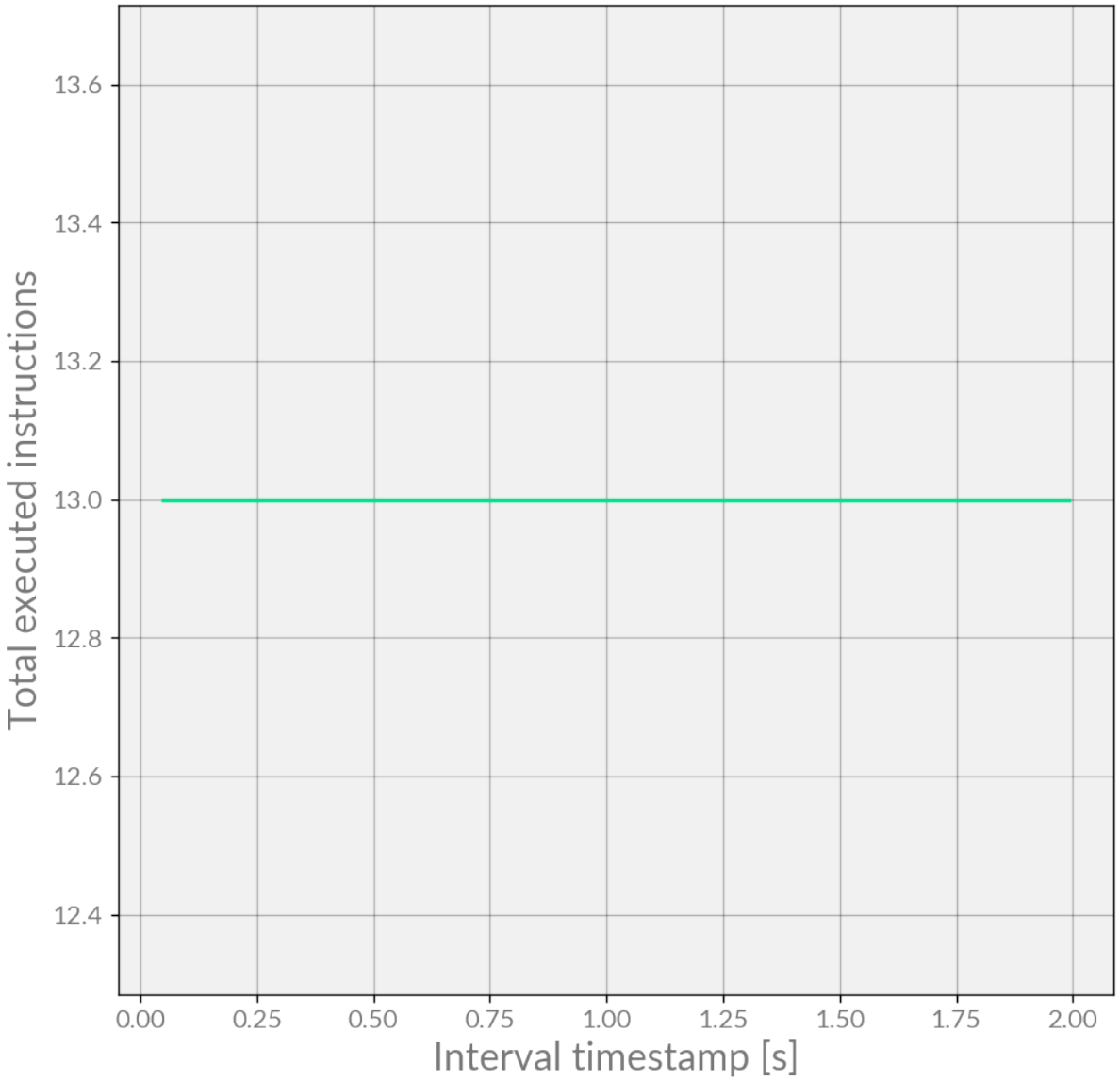


Figure 12.10: Cumulative count of executed instructions for cpu2 during benchmark

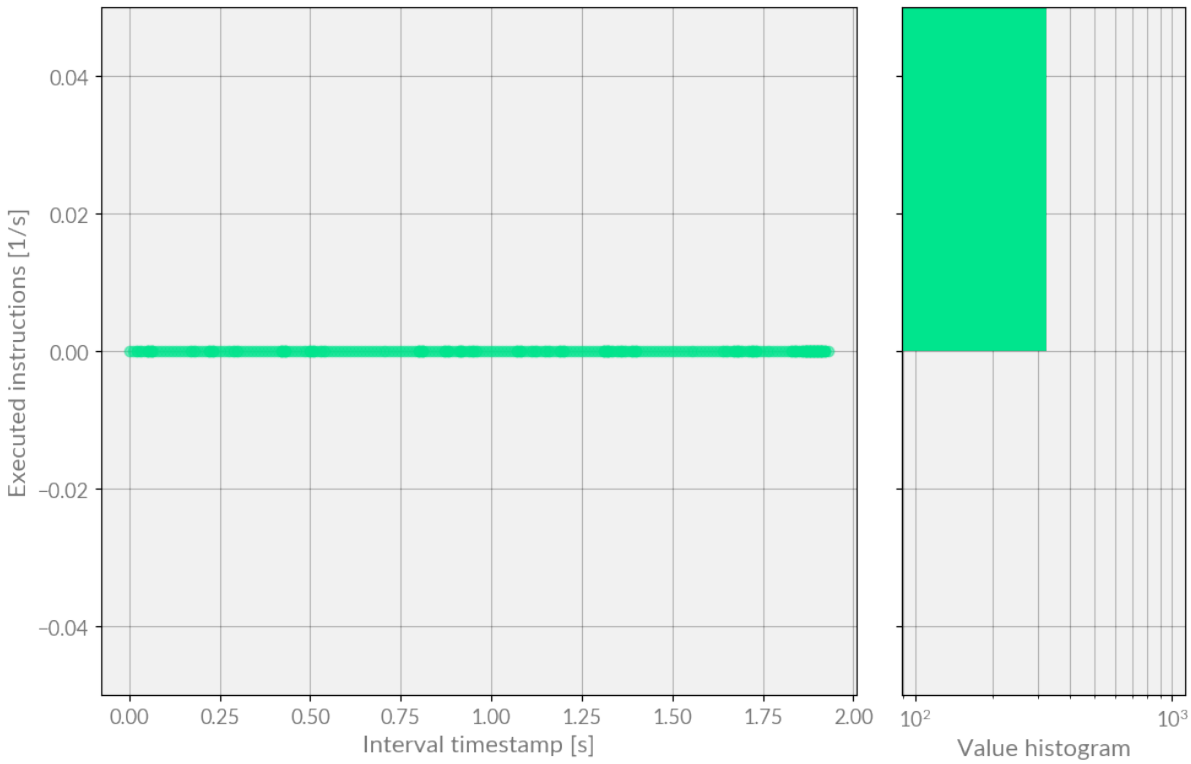


Figure 12.11: Count of executed instructions per second for cpu3 during benchmark

Layers

Table 12.4: Main model layers

Layer #	Name	Operation type	Parameters types	Parameters count	Parameters bytes
1	input_1	Input	FLOAT	0	0
2	/0/Conv	Conv	FLOAT	104	416
3	/2/Relu	Relu	N/A	0	0
4	/3/MaxPool	MaxPool	N/A	0	0
5	/5/Conv	Conv	FLOAT	528	2112
6	/7/Relu	Relu	N/A	0	0
7	/8/MaxPool	MaxPool	N/A	0	0
8	/10/Flatten	Flatten	N/A	0	0
9	/11/Gemm	Gemm	FLOAT	3600	14400
10	/12/Relu	Relu	N/A	0	0
11	/14/Gemm	Gemm	FLOAT	272	1088
12	/15/Relu	Relu	N/A	0	0
13	/16/Gemm	Gemm	FLOAT	68	272
14	out_layer	Output	FLOAT	0	0

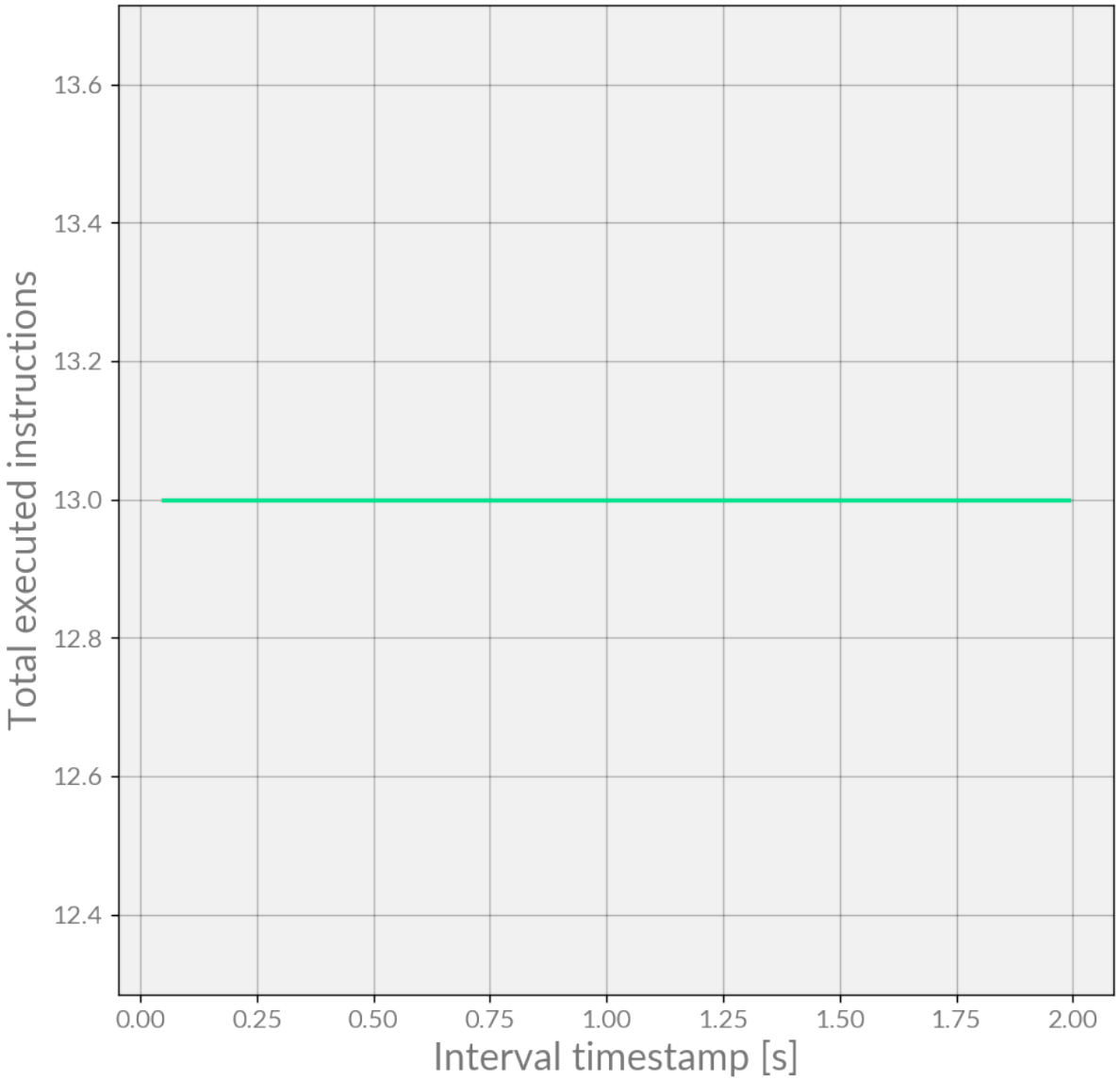


Figure 12.12: Cumulative count of executed instructions for cpu3 during benchmark

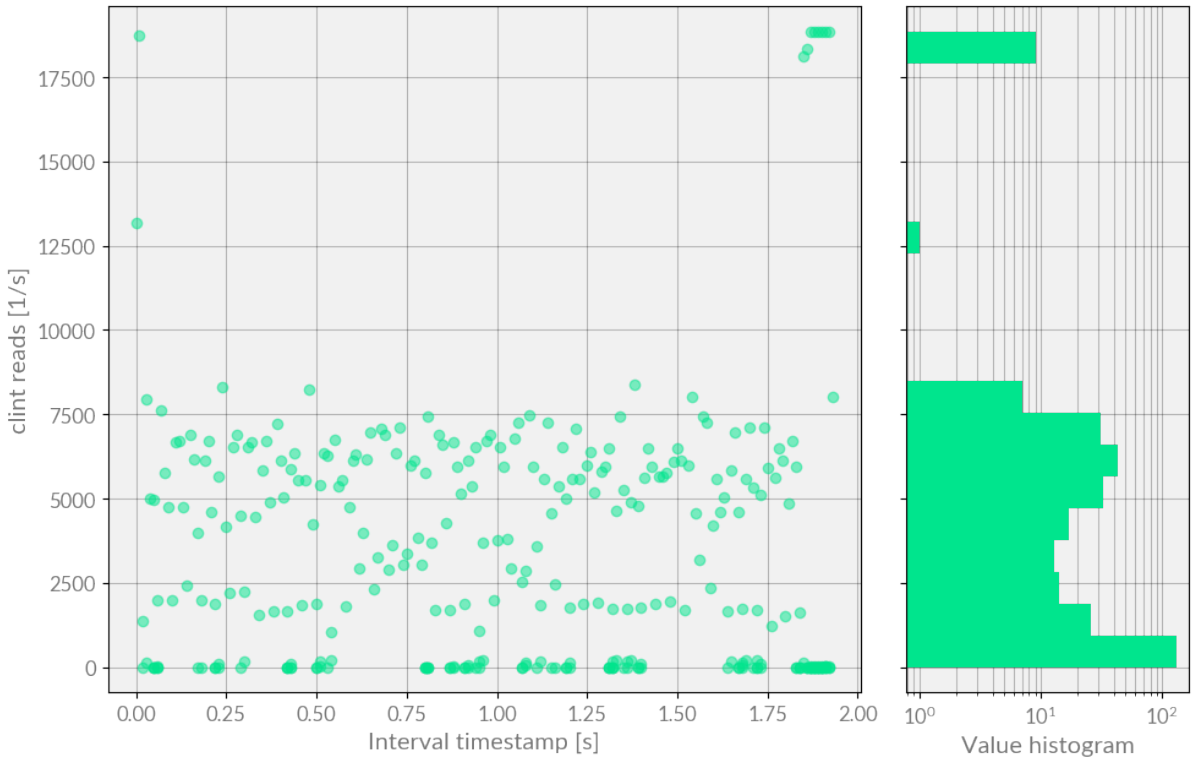


Figure 12.13: Count of clint reads per second during benchmark

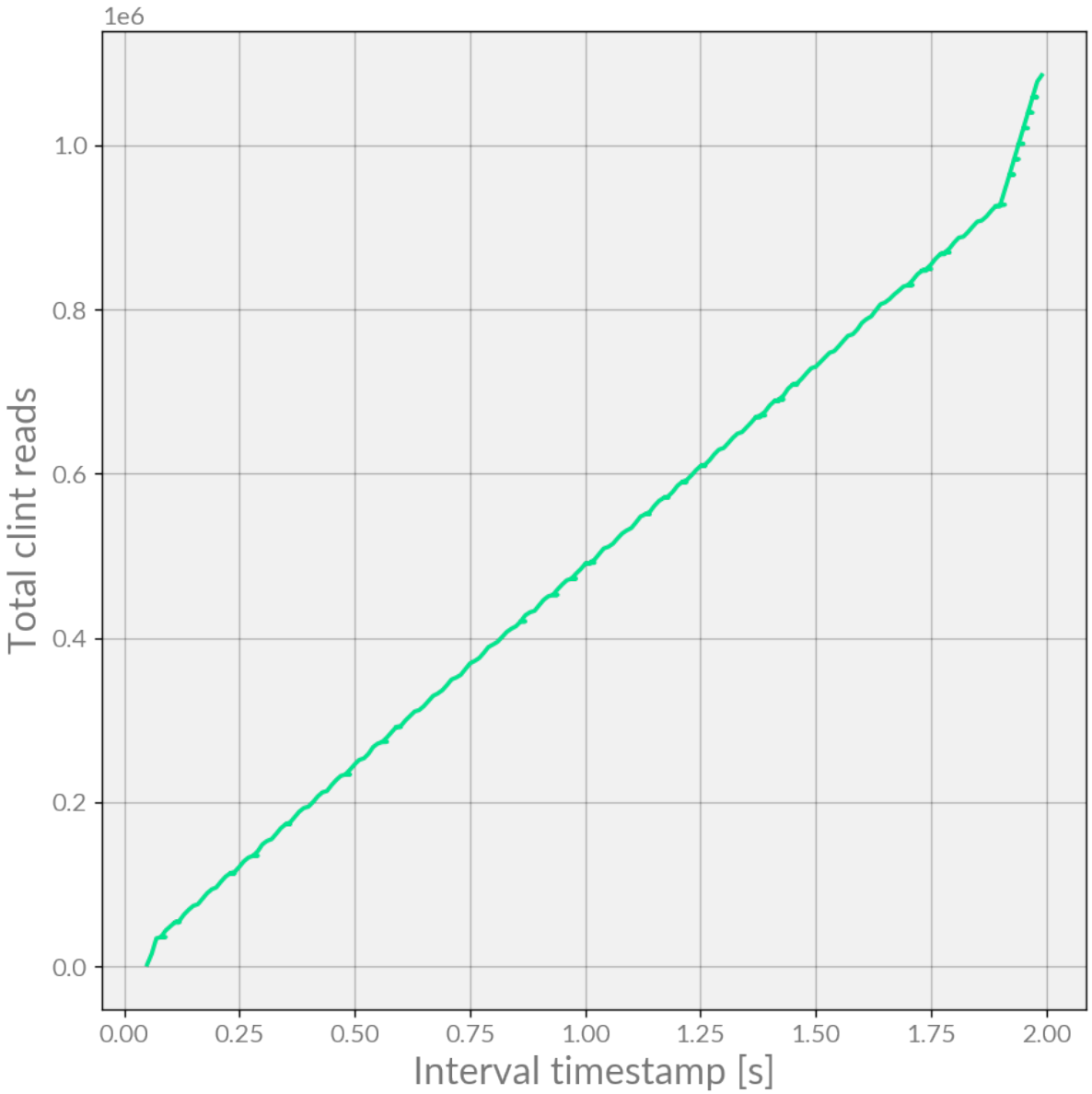


Figure 12.14: Cumulative count of clint reads during benchmark

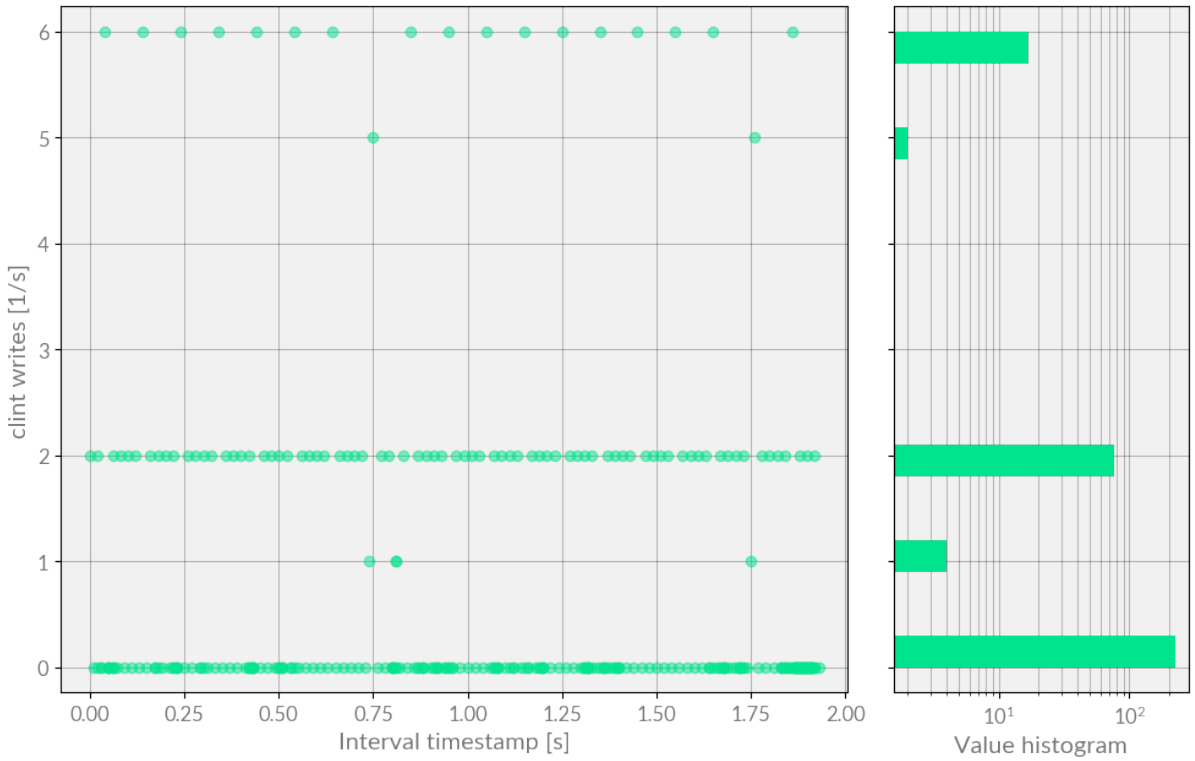


Figure 12.15: Count of clint writes per second during benchmark

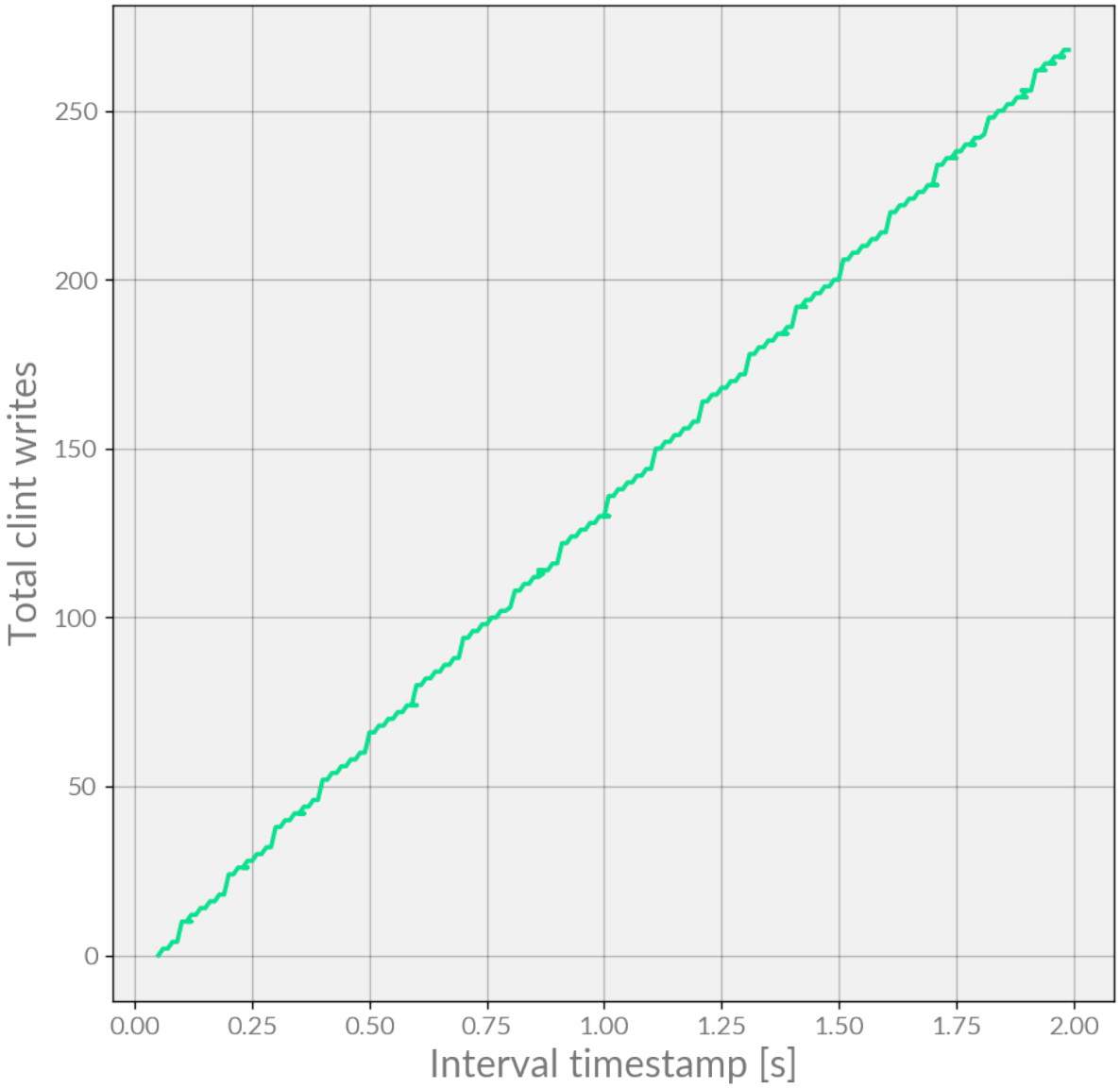


Figure 12.16: Cumulative count of clint writes during benchmark

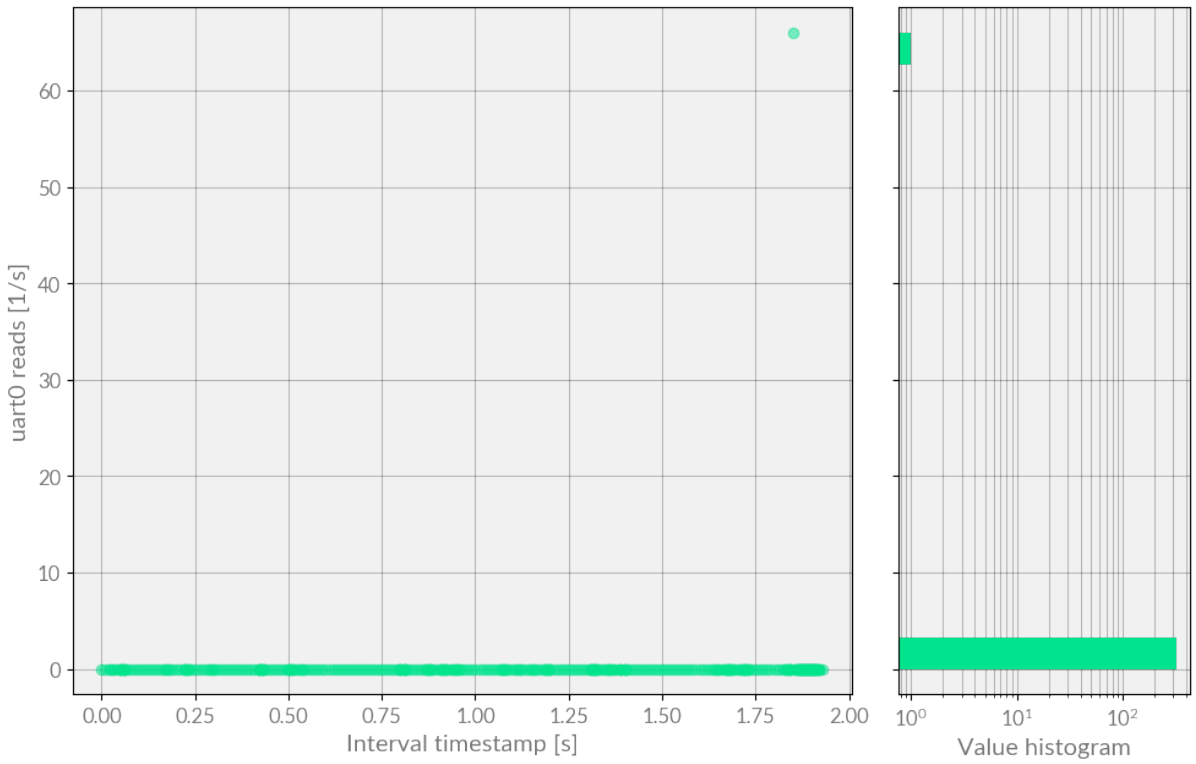


Figure 12.17: Count of uart0 reads per second during benchmark

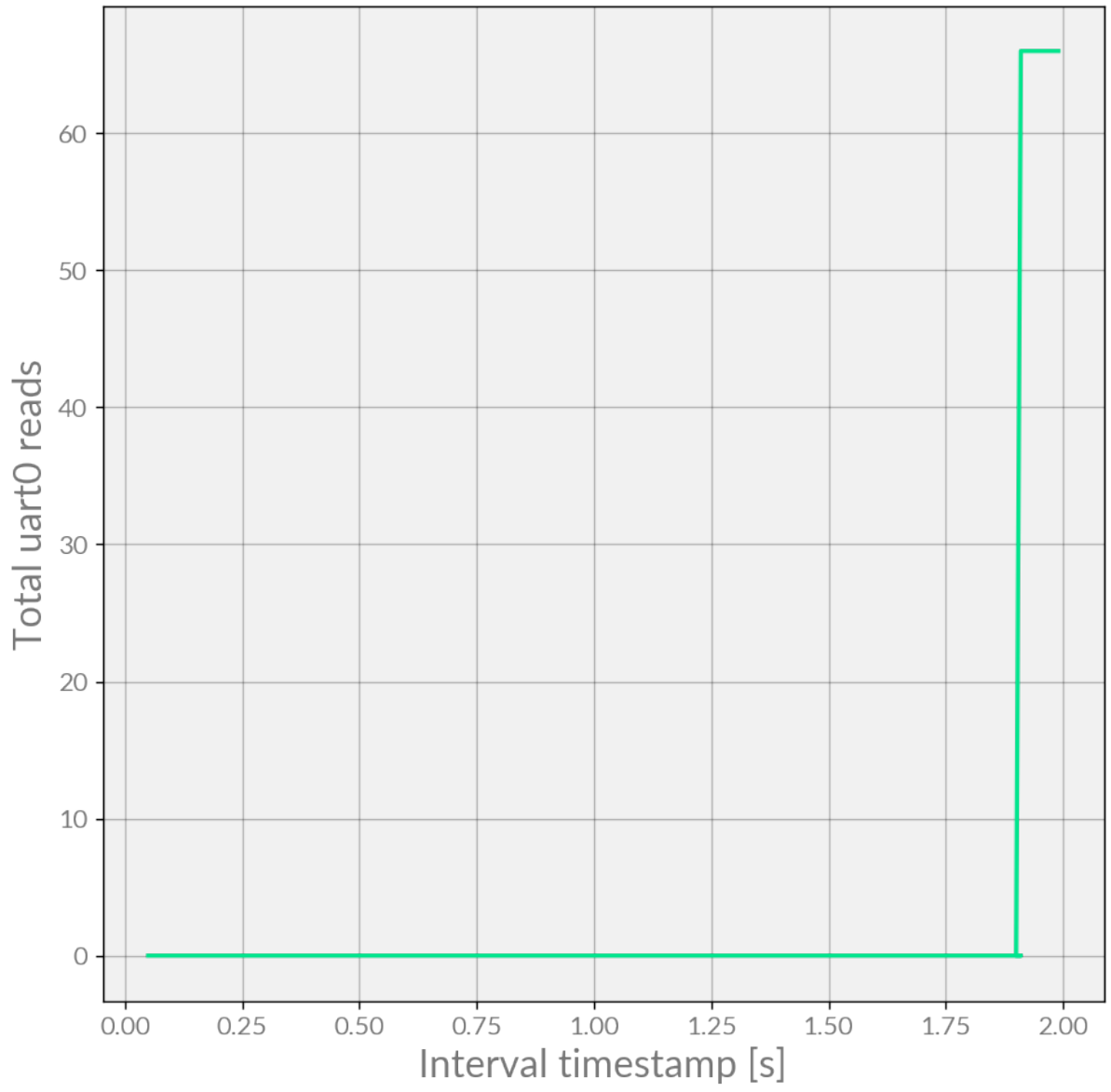


Figure 12.18: Cumulative count of uart0 reads during benchmark

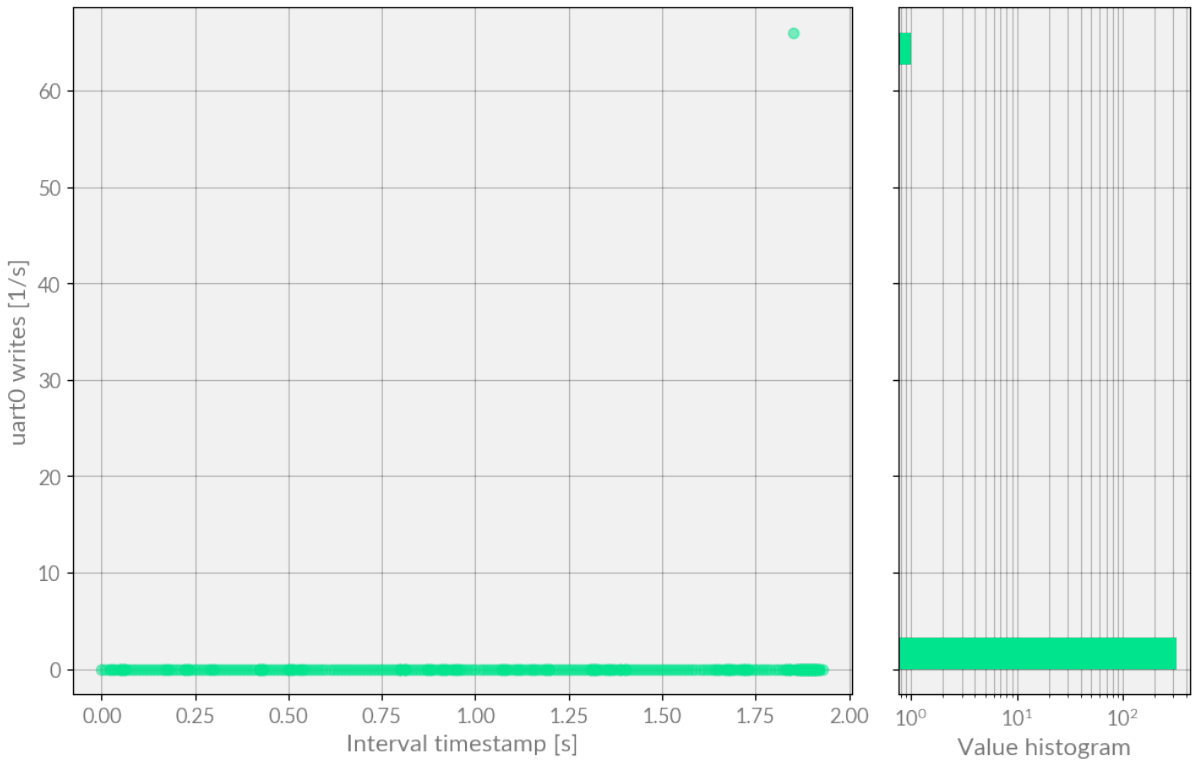


Figure 12.19: Count of uart0 writes per second during benchmark

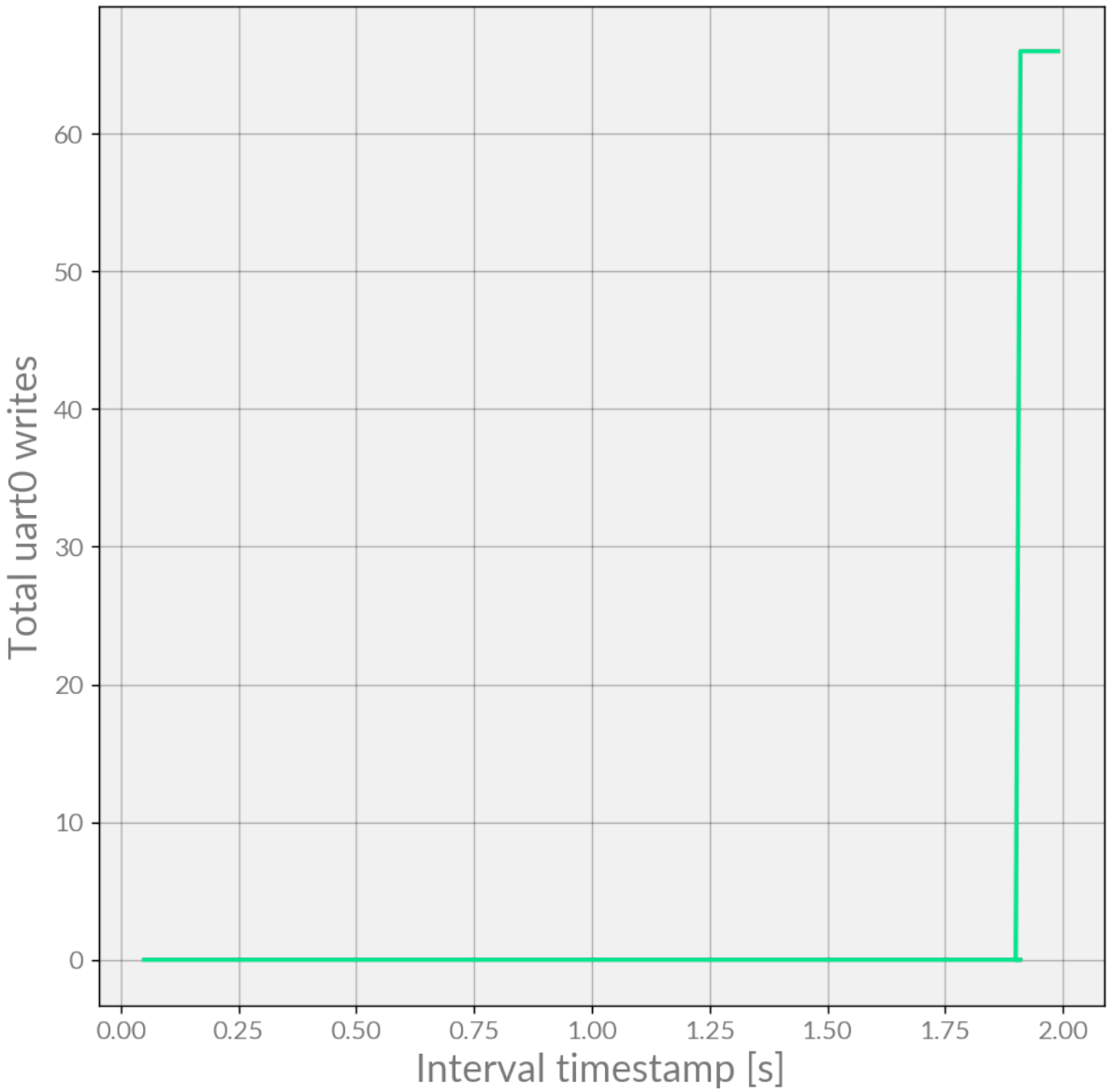


Figure 12.20: Cumulative count of uart0 writes during benchmark

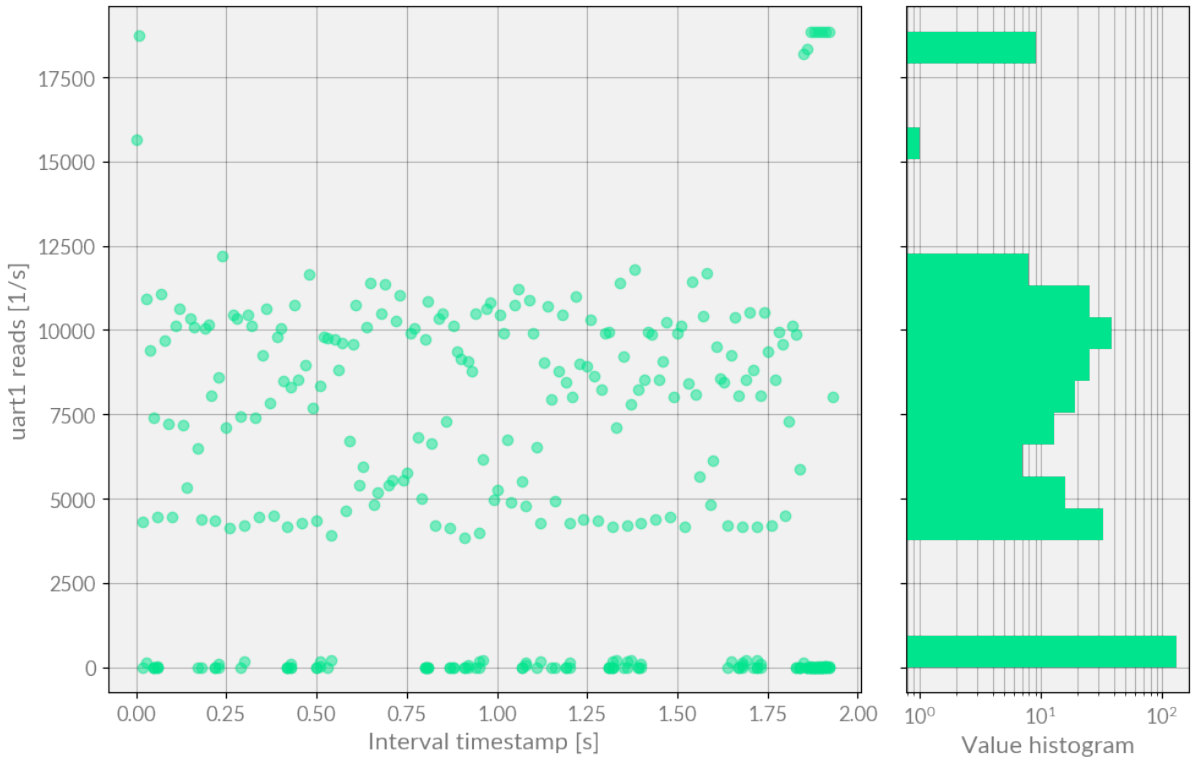


Figure 12.21: Count of uart1 reads per second during benchmark

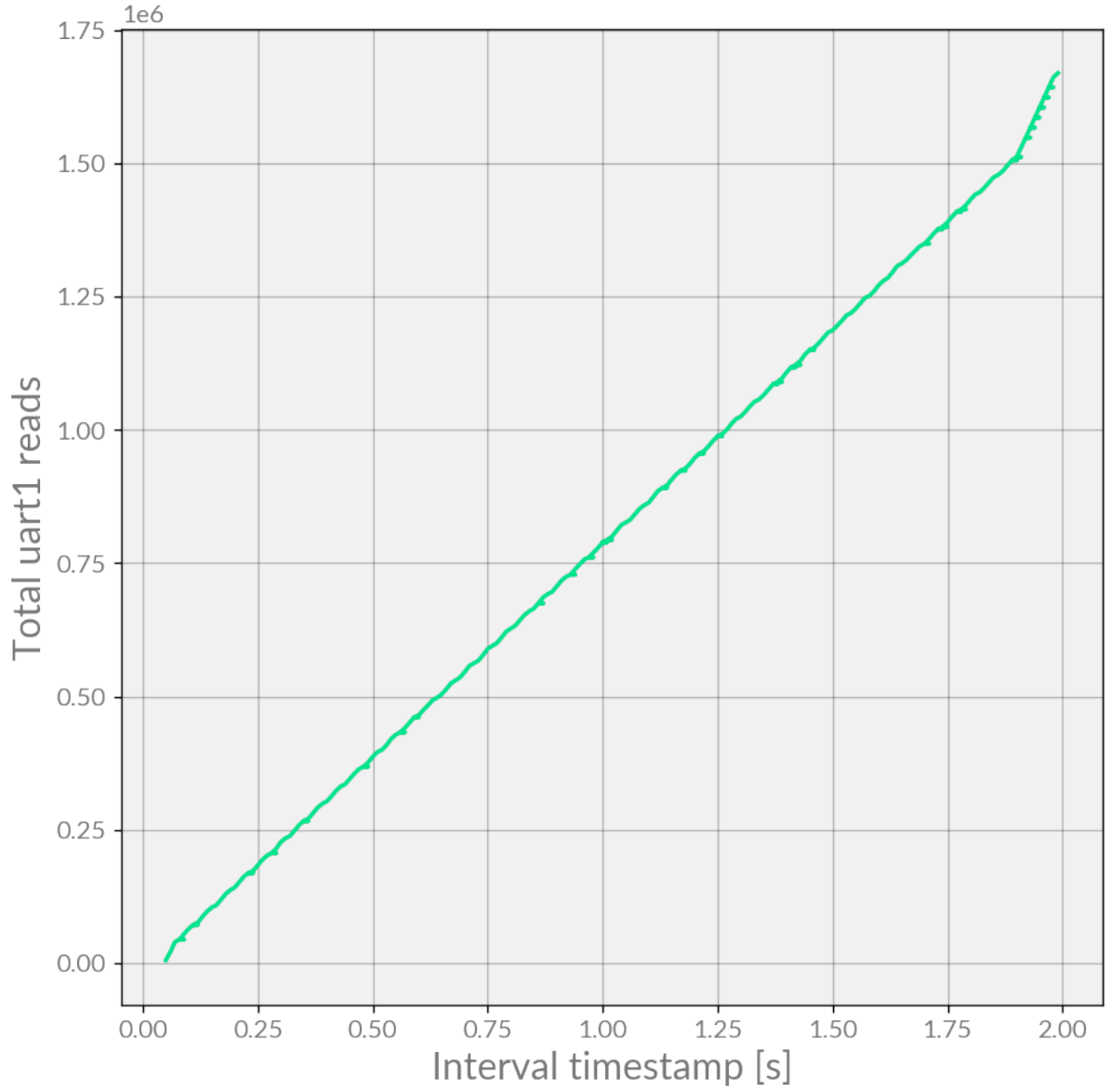


Figure 12.22: Cumulative count of uart1 reads during benchmark

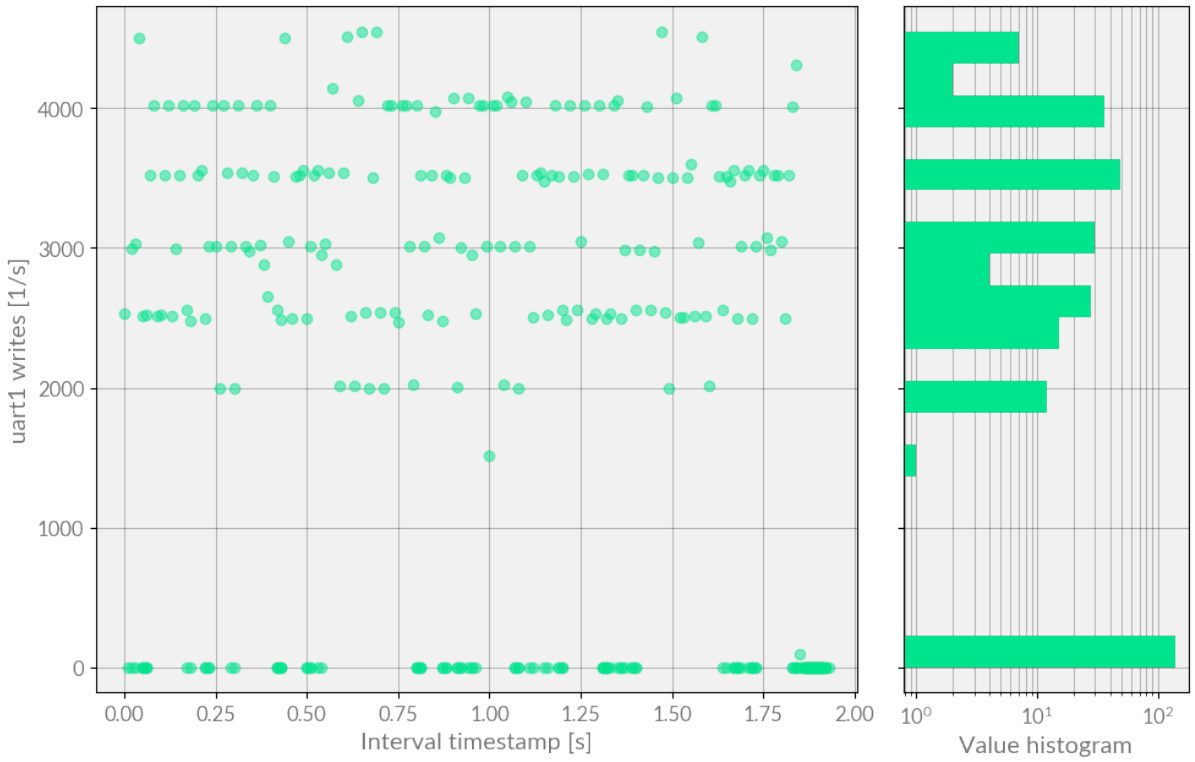


Figure 12.23: Count of uart1 writes per second during benchmark

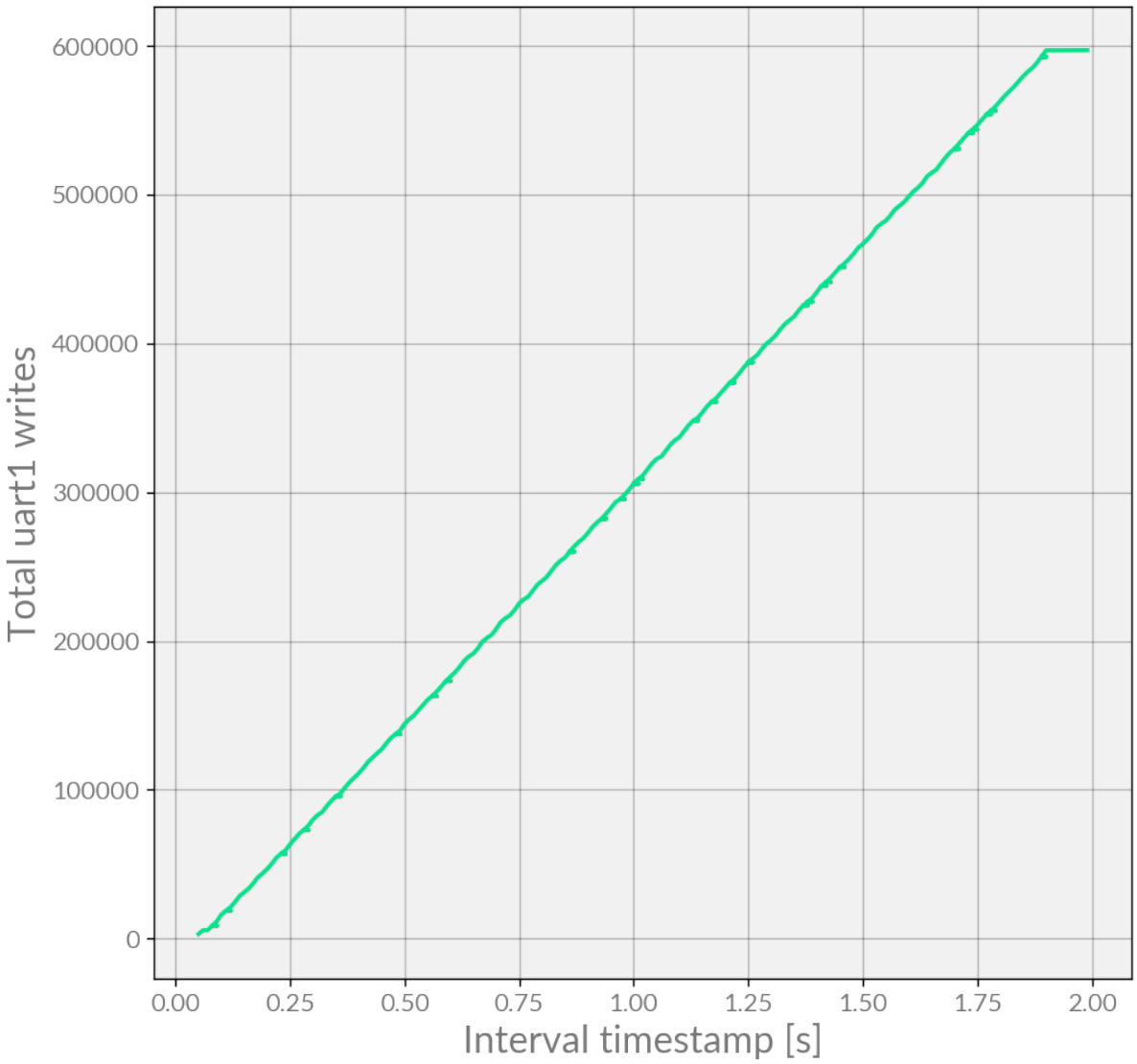


Figure 12.24: Cumulative count of uart1 writes during benchmark

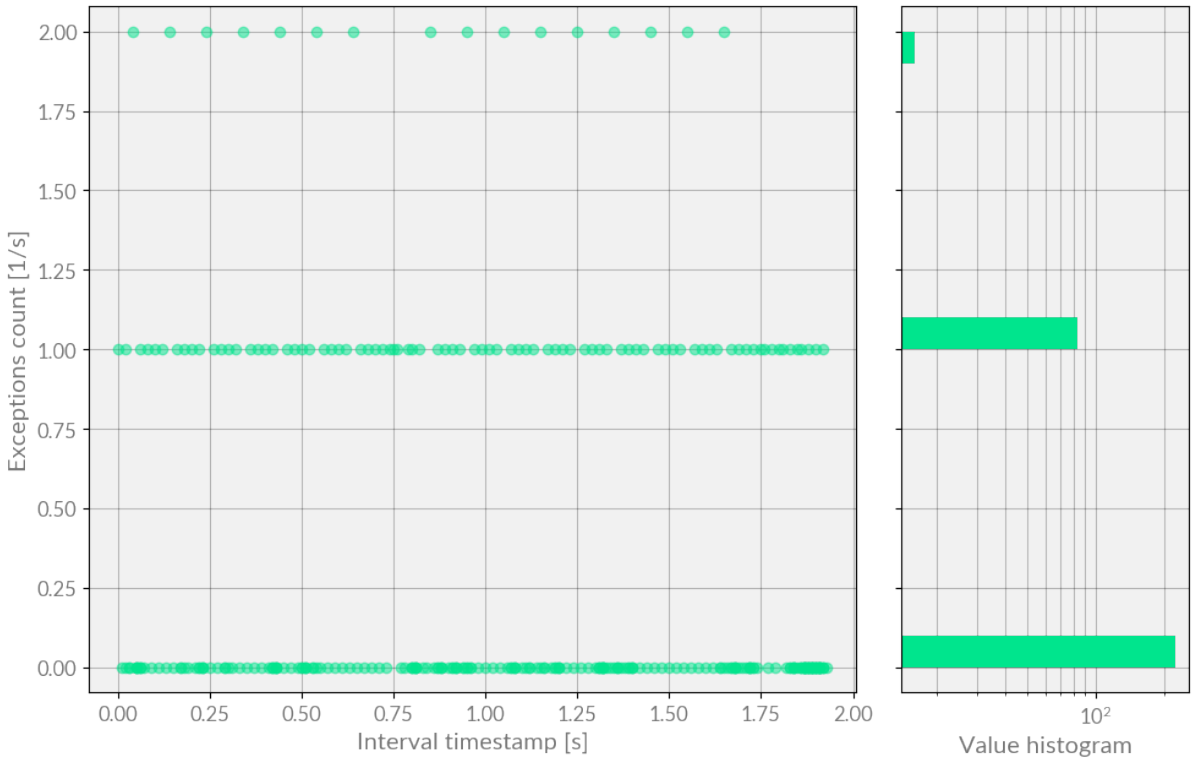


Figure 12.25: Count of raised exceptions per second during benchmark

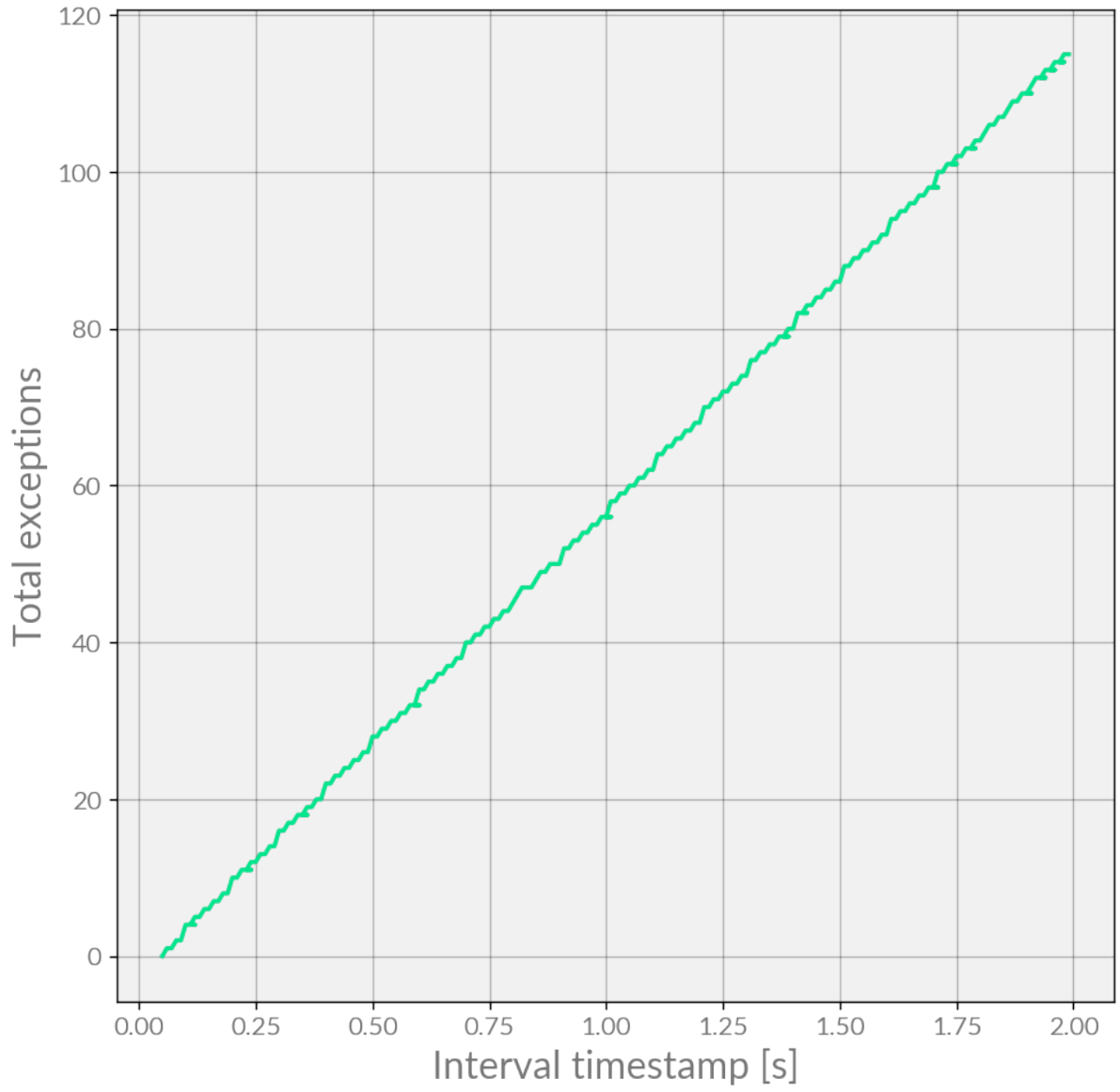


Figure 12.26: Cumulative count of raised exceptions during benchmark

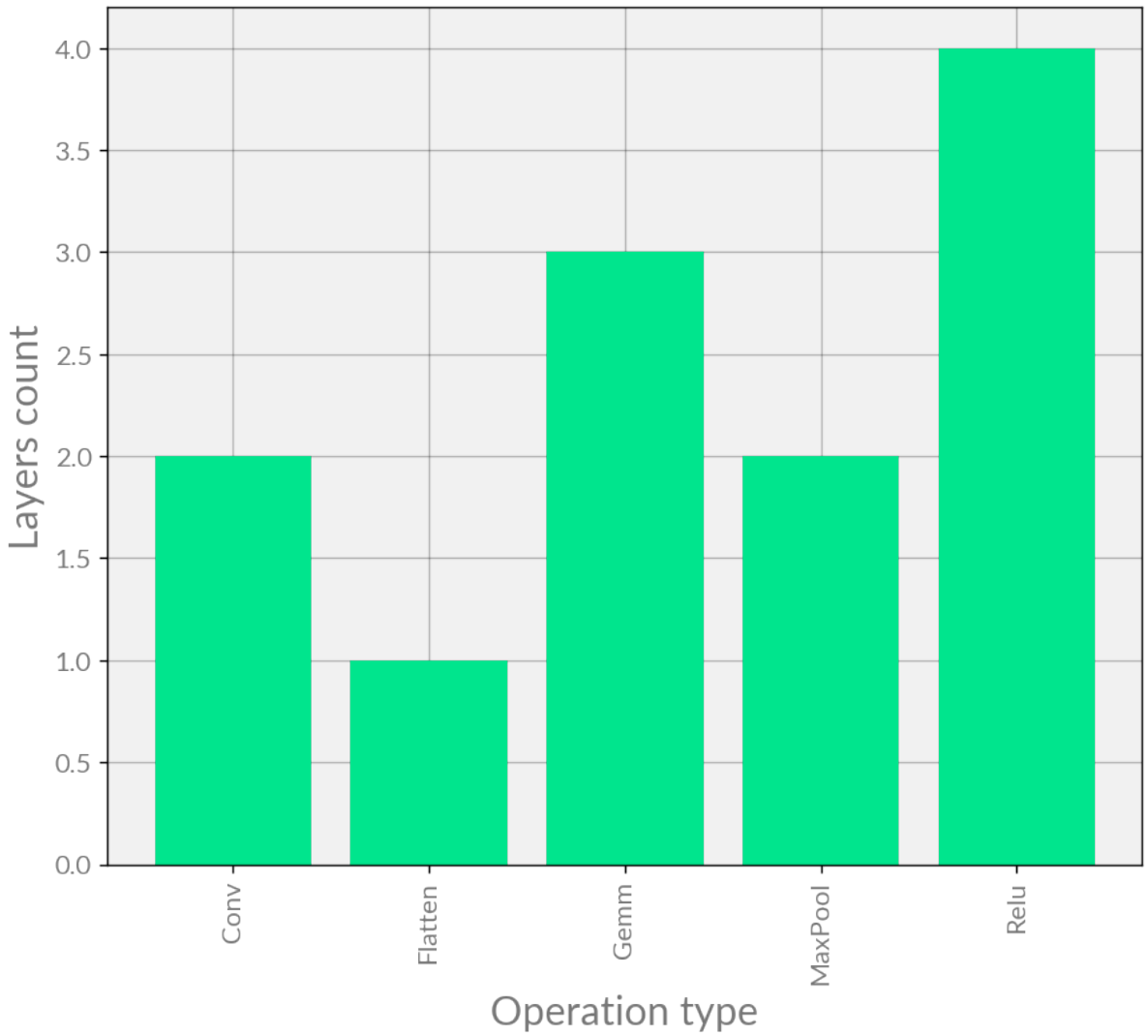


Figure 12.27: Layer operation type counts

CHAPTER

THIRTEEN

CREATING APPLICATIONS WITH KENNING

The *KenningFlow* allows you to run an arbitrary sequence of processing blocks that provide data, execute models using existing Kenning classes and wrappers, and processes results. You can use it to quickly create applications with Kenning after optimizing the model and using it in actual use cases.

Kenning for runtime uses both existing classes, such as *ModelWrapper*, *Runtime*, and dedicated *Runner*-based classes. The latter family of classes are actual functional blocks used in *KenningFlow* that can be used for:

- Obtaining data from sources - *DataProvider*, e.g. iterating files in the filesystem, grabbing frames from a camera or downloading data from a remote source,
- Processing and delivering data - *OutputCollector*, e.g. sending results to the client application, visualizing model results in GUI, or storing results in a summary file,
- Running and processing various models,
- Applying other actions, such as additional data analysis, preprocessing, packing, and more.

A *KenningFlow* scenario definition can be saved in a JSON file and then run using the `kenning.scenarios.json_flow_runner` script.

13.1 JSON structure

JSON configuration consist of a list of dictionaries describing each *Runner*-based instance.

A sample *Runner* specification looks as follows:

```
{
  "type": "kenning.dataproviders.camera_dataprovider.CameraDataProvider",
  "parameters": {
    "video_file_path": "/dev/video0",
    "input_memory_layout": "NCHW",
    "input_width": 608,
    "input_height": 608
  },
  "outputs": {
    "frame": "cam_frame"
  }
}
```

Each *Runner* dictionary consists of:

- `type` - *Runner* class. E.g. CameraDataProvider,
- `parameters` - parameters passed to class constructor. In this case, we specify a path to a video device (`/dev/video0`), expected memory format (`NCHW`), image size (`608x608`)
- `inputs` - (*optional*) *Runner* instance inputs. In the example above, there are none,
- `outputs` - (*optional*) *Runner* instance outputs. In the example above, it is a single output - camera frame defined as the variable `cam_frame` in the flow.

13.1.1 Runner IO

The input and output specification in *Runner* classes is the same as described in *Model and I/O metadata*.

13.1.2 IO compatibility

IO compatibility is checked during flow JSON parsing.

The *Runner* input is considered to be compatible with associated outputs if:

- in case of `numpy.ndarray`: `dtype` and `ndim` are equal and each dimension has either the same length or input dimension is set as `-1`, which represents any length. In the input spec, there can also be multiple valid shapes. If so, they are placed in an array, i.e. `[(1, -1, -1, 3), (1, 3, -1, -1)]`,
- in other cases: `type` fields are either equal or the input `type` field is `Any`.

13.1.3 IO non-standard types

If the input or output is not a `numpy.ndarray`, then its type is described by the `type` field, which is a string. In the case of a detection output from an IO specification (described above) it is a `List[DetectObject]`. This is interpreted as a list of `DetectObjects`. The `DetectObject` is a named tuple describing detection output (class names, rectangle positions, score).

13.1.4 IO names and mapping

The inputs and outputs present in JSON are mappings from *Runner*'s local IO names to flow global variable names, i.e. one *Runner* can define its outputs as `{"output_name": "data"}` and another runner can use it as its input with `{"input_name": "data"}`. These global variables must be unique and the variable defined as input needs to be defined in a previous block as output to prevent cycles in a flow's structure. *Runner* IO names are specific to runner type and model (for `ModelRuntimeRunner`).

> **Note**
>
> IO names can be obtained using the `get_io_specification` method.

13.1.5 Runtime example

In order to create a *KenningFlow* presenting YOLOv4 model performance, create a file `flow_scenario_detection.json` and include the following configuration in it:

```
[
  {
    "type": "kenning.dataproviders.camera_dataprovider.CameraDataProvider",
    "parameters": {
      "video_file_path": "/dev/video0",
      "input_memory_layout": "NCHW",
      "input_width": 608,
      "input_height": 608
    },
    "outputs": {
      "frame": "cam_frame"
    }
  },
  {
    "type": "kenning.runners.modelruntime_runner.ModelRuntimeRunner",
    "parameters": {
      "model_wrapper": {
        "type": "kenning.modelwrappers.object_detection.yolov4.ONNXYOLOV4",
        "parameters": {
          "model_path": "kenning:///models/detection/yolov4.onnx"
        }
      },
      "runtime": {
        "type": "kenning.runtimes.onnx.ONNXRuntime",
        "parameters":
        {
          "save_model_path": "kenning:///models/detection/yolov4.onnx",
          "execution_providers": ["CUDAExecutionProvider"]
        }
      }
    },
    "inputs": {
      "input": "cam_frame"
    },
    "outputs": {
      "detection_output": "predictions"
    }
  },
  {
    "type": "kenning.outputcollectors.real_time_visualizers.
↪RealTimeDetectionVisualizer",
    "parameters": {
      "viewer_width": 512,
      "viewer_height": 512,
      "input_memory_layout": "NCHW",
      "input_color_format": "BGR"
    },
    "inputs": {
      "frame": "cam_frame",
      "detection_data": "predictions"
```

(continues on next page)

(continued from previous page)

```
    }
  }
]
```

This JSON creates a *KenningFlow* that consists of three runners - CameraDataProvider, ModelRuntimeRunner and RealTimeDetectionVisualizer:

- The first one captures frames from a camera and passes it as a `cam_frame` variable.
- The next one passes `cam_frame` to a detection model (in this case YOLOv4) and returns predicted detection objects as `predictions`.
- The last one gets both outputs (`cam_frame` and `predictions`) and shows a detection visualization using DearPyGui.

13.2 KenningFlow execution

Now, you can execute KenningFlow using the above configuration.

With the config saved in the `flow_scenario_detection.json` file, run the `kenning.scenarios.json_flow_runner` as follows:

```
kenning flow --json-cfg flow_scenario_detection.json
```

This module runs KenningFlow defined in given JSON file. With provided config it should read image from the camera and visualize output of detection model YOLOv4.

13.3 Implemented Runners

Available implementations of *Runner* can be found in the *Runner documentation*.

To create custom runners, check *Implementing new Runners for KenningFlow*.

CHAPTER

FOURTEEN

KENNING ZEPHYR RUNTIME

This chapter describes Kenning Zephyr Runtime - a set of tools for deploying and evaluating models optimized with Kenning on Zephyr RTOS.

Kenning Zephyr Runtime is a C library that consists of:

- `kenning_inference_lib` provides an implementation of *Kenning Protocol* (for communication with Kenning) and *Kenning Runtime*-like unified API allowing easy preparation and execution of ML models with the following frameworks:
  - TFLite Micro
  - microTVM
  - IREE
  - ExecuTorch
  - AI8X for Analog Devices MAX78xxx platforms
- `app` is a server-like application using `kenning_inference_lib` to evaluate models with Kenning running on host using *Kenning Protocol* to send and receive models, outputs and runtime statistics
- `demo_app` is an example application, showcasing how `kenning_inference_lib` can be used without Kenning as a library to run model inference on production regardless of underlying framework

Note

To check support for various runtimes on various boards, see Renode Zephyr Dashboard.

It provides coverage details of Kenning Zephyr Runtime across boards supported by Zephyr RTOS on an example of `demo_app`.

Each demo is prefixed with `KENNING` (e.g. `KENNING MICROTVM`).

Overview of Kenning Zephyr Runtime use-cases:

- Running Kenning evaluation examples - see *instructions on preparing the environment* and then *running a simple `app` example*.
- Adding support for a new runtime (ML framework) - see *information on* `runtime_wrapper` and `loaders`.

- Adding support for Kenning evaluation on a new board - see *instructions on configuring Kenning Zephyr Runtime for a new board*.
- Using Kenning Zephyr Runtime as a library for model inference - see `model` *API*.

14.1 Usage

14.1.1 Preparing the environment

To use Kenning Zephyr Runtime, first clone the repository:

```
mkdir -p zephyr-workspace && cd zephyr-workspace
git clone https://github.com/antmicro/kenning-zephyr-runtime.git
cd kenning-zephyr-runtime
```

Install the following dependencies:

- Zephyr dependencies
- `jq`
- `curl`
- `west`
- `CMake`

Note

On Debian-based distributions, it can be done with:

```
apt update

apt install -y --no-install-recommends ccache curl device-tree-compiler dfu-
↪util file \
  g++-multilib gcc gcc-multilib git jq libmagic1 libsdl2-dev make ninja-build \
  python3-dev python3-pip python3-setuptools python3-tk python3-wheel python3-
↪venv \
  mono-complete wget xxd xz-utils patch
```

After that, Python packages can be installed and `west` workspace can be configured:

```
./scripts/prepare_zephyr_env.sh
source .venv/bin/activate
```

This will:

- Create a Python virtual environment
- Install Python dependencies of Kenning and Kenning Zephyr Runtime
- Create a Zephyr workspace and download all modules
- Install Zephyr SDK, with toolchains for: x86, RISCV-64, ARM and ARM64 (Aarch64)

The script will also detect if uv tool is present in your system and if so, use it instead of pip. Using uv with Kenning Zephyr Runtime is encouraged for fast dependency installation.

Alternatively, you can do this manually (for example if you want to install a different set of toolchains):

```
uv venv .venv --python=3.11
source .venv/bin/activate
uv pip install pip setuptools west --upgrade
west init -l .
west update
uv pip install -r requirements.txt -r ../zephyr/scripts/requirements-base.txt
west zephyr-export
```

Install latest Zephyr SDK with:

```
west sdk install --toolchains x86_64-zephyr-elf arm-zephyr-eabi riscv64-zephyr-
↪elf aarch64-zephyr-elf
```

Note

By default, the Zephyr workspace will be created in a directory containing Kenning Zephyr Runtime repository. Therefore modules will be installed in `../tvm`, `../zephyr` etc. For that reason it is advised to clone the repository into a dedicated folder (as shown above).

One of the Python dependencies installed will be Kenning. You can also install Kenning manually, for a different selection of additional dependencies. For example:

```
uv pip install "kenning[iree,tvm,tensorflow,reports,renode,uart,litert,torch,
↪zephyr] @ git+https://github.com/antmicro/kenning.git"
```

Eventually, run:

```
./scripts/prepare_modules.sh
```

Some C/C++ dependencies of Kenning Zephyr Runtime are not packaged as Zephyr modules out-of-the-box. This script will add necessary files to Zephyr and to the modules themselves. Those files are stored in `modules` directory in Kenning Zephyr Runtime.

14.1.2 Model evaluation with Kenning (using app)

Basic evaluation scenario

As an example we will generate a report for a simple CNN model trained on a Magic Wand dataset (recognizing movements based on accelerometer data), ran with TFLite Runtime on `stm32f746g_disco` board.

First, build `app` on `stm32f746g_disco`, using west. Kenning Zephyr Runtime comes with `<runtime>.conf` configuration files that set up inference with a specified `<runtime>`. `litert.conf` config file can be used to run TFLite Micro - this extra configuration file can be added with `-DEXTRA_CONF_FILE` option:

```
west build -p always -b stm32f746g_disco app -- -DEXTRA_CONF_FILE=litert.conf
```

Option `-p always` will force west to run a clean build. Drop the option to run incremental builds.

Built application can be found at `build/zephyr/zephyr.elf` (`build` is the default build directory).

> **Warning**
>
> Due to a version conflict, Python dependencies for building with ExecuTorch runtime (`executorch.conf` configuration file) cannot be installed automatically. Therefore command `uv pip install executorch>=0.7.0` has to be ran before building.

We could run the application on a physical `stm32f746g_disco` board, which would require running `west flash` command. However in this example we will use the Renode to simulate the board. For that, we need to generate a `.repl` file (a platform definition file for Renode - for details, please refer to Renode documentation):

```
west build -t board-repl
```

This step will need to be repeated every time a clean build is ran or a different board is used.

Before running the simulation we need to install Renode, which can be done with a script:

```
source ./scripts/prepare_renode.sh
```

This will:

- Download the latest Renode release (if missing)
- Export environmental variables, that will allow Kenning to find Renode (for that reason the script has to be ran with `source` and needs to be re-ran every time a new shell is used)

Finally, we can run Kenning with one of the example *scenarios* provided with Kenning Zephyr Runtime:

```
kenning optimize test report \
    --cfg kenning-scenarios/magic-wand-inference/tflite/renode-stm32f746g.yml \
    --measurements results.json \
    --report-path reports/stm32-renode-tflite-magic-wand/report.md \
    --to-html
```

This example scenario looks as follows:

```
# platform definition, telling where the model will be deployed
platform:
  type: ZephyrPlatform
  parameters:
    name: stm32f746g_disco
    simulated: true
    zephyr_build_path: ./build

# dataset to use for evaluation of the model (dataset will be downloaded␣
↪automatically)
dataset:
  type: MagicWandDataset
  parameters:
```

(continues on next page)

(continued from previous page)

```
    dataset_root: ./output/MagicWandDataset

# model that will be executed on device, in here we use trained Keras model
# for gesture recognition
model_wrapper:
  type: MagicWandModelWrapper
  parameters:
    model_path: kenning:///models/classification/magic_wand.h5

# list of optimizations to apply on a model before deployment
# the final runtime is derived from the last Optimizer
optimizers:
- type: LiteRTCompiler
  parameters:
    compiled_model_path: ./output/magic-wand.tflite
    inference_input_type: float32
    inference_output_type: float32
```

When running this, Kenning will:

- Optimize the model with `LiteRTCompiler`, creating `./output/magic-wand.tflite` model.
- Automatically run Renode simulation of the built app, finding ELF in `zephyr_build_path` directory (`simulated: true` option means that Kenning will run Renode simulation automatically).
- Connect over (virtual) UART and send model and data for evaluation to the simulated board.
- Receive evaluation results and save them under `results.json` (as specified with the `--measurements` CLI option).
- Generate a report and save it under `reports/stm32-renode-tflite-magic-wand/`

To run evaluation on a physical board (not one simulated in Renode) we would need to set `simulated` to `false` and provide a path to the correct `UART` port with `uart_port` (example).

It is also possible to run a standalone Renode simulation manually and have Kenning connect to the existing simulation.

To do that you should run:

```
python ./scripts/run_renode.py
```

This script will run a Renode simulation of the application currently at `build/zephyr/zephyr.elf` and mount the UART port at `/tmp/uart`. You can then run Kenning separately with `platform` parameters: `simulation: false` and `uart_port: /tmp/uart`. You can run multiple inference sessions without restarting the simulation (just as you would with a physical board).

Automatic building of Kenning Zephyr Runtime

Kenning itself can build Kenning Zephyr Runtime evaluation app and generate the `.repl` file, allowing you to skip manual steps. To do that, you need to add `runtime_builder` to the scenario, as shown below:

```
platform:
  type: ZephyrPlatform
  parameters:
    name: stm32f746g_disco
    simulated: true

# new component responsible for automated build of Zephyr app
runtime_builder:
  type: ZephyrRuntimeBuilder
  parameters:
    workspace: .
    run_west_update: false
    output_path: ./output
    extra_targets: [board-repl]

dataset:
  type: MagicWandDataset
  parameters:
    dataset_root: ./output/MagicWandDataset
model_wrapper:
  type: MagicWandModelWrapper
  parameters:
    model_path: kenning:///models/classification/magic_wand.h5
optimizers:
- type: LiteRTCompiler
  parameters:
    compiled_model_path: ./output/magic-wand.tflite
    inference_input_type: float32
    inference_output_type: float32
```

This example scenario can be ran without first building the app:

```
kenning optimize test report \
    --cfg kenning-scenarios/magic-wand-inference/tflite/renode-auto-stm32f746g.
↪yml \
    --measurements results.json \
    --report-path reports/stm32-renode-tflite-magic-wand/report.md \
    --to-html
```

Tracing with Zephelin

Kenning and Kenning Zephyr Runtime are integrated with Zephelin - Antmicro's tool for ML-aware tracing in Zephyr RTOS.

You can use that tool to generate a report with detailed information about your model's time and memory performance, broken down into individual layers. Zephelin allows for collecting traces either through an UART port, USB port or with GDB debugger. Zephelin supports TVM and TFLite.

First, install Zephelin's Python dependencies:

```
uv pip install -r ../zephelin/requirements.txt
```

Then, you need to build Kenning Zephyr Runtime with Zephelin included, by adding extra configuration files.

For tracing with UART:

```
west build -p -b stm32f746g_disco app -- -DEXTRA_CONF_FILE="tvm.conf;$(realpath ./
↪zpl.conf);zpl_uart.conf"
```

For tracing with GDB:

```
west build -p -b stm32f746g_disco app -- -DEXTRA_CONF_FILE="tvm.conf;$(realpath ./
↪zpl.conf);zpl_gdb.conf"
```

For GDB tracing, you also need to install GDB with support for multiple architectures. On Linux Debian it can be done with:

```
apt install -y gdb-multiarch
```

Finally, you can run optimization, testing and report generation, using a scenario with platform parameter `enable_zephelin` set to `true` (or `enable_zephelin_gdb` for GDB tracing):

```
platform:
  type: ZephyrPlatform
  parameters:
    name: stm32f746g_disco
    simulated: true
    # this line enables trace collection
    enable_zephelin_gdb: true
    zephyr_build_path: ./build/

model_wrapper:
  type: MagicWandModelWrapper
  parameters:
    model_path: kenning:///models/classification/magic_wand.h5

dataset:
  type: MagicWandDataset
  parameters:
    dataset_root: ./output/MagicWandDataset

optimizers:
  - type: TVMCompiler
    parameters:
      compiled_model_path: ./build/compiled-model-magic-wand.graph_data
```

Since we will be running Kenning Zephyr Runtime in a Renode simulation, we need to prepare a `.repl` file:

```
west build -t board-repl
```

To include traces in the report, you must specify it with `--report-types` option, as shown below:

```
kenning optimize test report \
    --cfg kenning-scenarios/magic-wand-inference/tvm/renode-zephelin-gdb-
↪stm32f746g.yml \
    --measurements build/zephelin-zephyr-stm32-tvm-magic-wand.json --verbosity␣
↪INFO \
    --report-path build/zephelin-zephyr-stm32-tvm.md \
    --report-types zephyr_traces renode_stats performance classification \
    --to-html
```

A dedicated report section (`zephyr_traces`) will be generated, with an interactive window allowing you to analyze the traces with Zephelin Trace Viewer. A *sample Zephyr tracing report* has been included in this documentation.

Note

For more details on trace collection in Zephyr, check Zephelin documentation.

Using Linkable Loadable Extensions for switching entire AI runtimes in running app

In this example, we will build Kenning Zephyr Runtime and microTVM separately. Then Kenning will send built microTVM to Kenning Zephyr Runtime over Kenning Protocol and it will be dynamically linked with Zephyr's LLEXT subsystem as en extension.

First, build Kenning Zephyr Runtime with LLEXT:

```
west build -p always -b stm32f746g_disco app -- -DEXTRA_CONF_FILE=llext.conf
```

This will build app with no default runtime, and necessary logic to load necessary implementation of runtime from UART-delivered extension.

If running with Renode, generate the `.repl` file:

```
west build -t board-repl
```

Then, build the extension:

```
west build app -t llext-tvm -- -DEXTRA_CONF_FILE="llext.conf;llext_tvm.conf"
```

It will be placed in `build/llext` directory.

Evaluation can be ran with the following scenario:

```
platform:
  type: ZephyrPlatform
  parameters:
    name: stm32f746g_disco
    simulated: true
    zephyr_build_path: ./build
    # this option tells Kenning to upload following LLEXT to device before␣
↪running evaluation
    llext_binary_path: ./build/llext/tvm.llext
dataset:
```

(continues on next page)

(continued from previous page)

```
  type: MagicWandDataset
  parameters:
    dataset_root: ./output/MagicWandDataset
model_wrapper:
  type: MagicWandModelWrapper
  parameters:
    model_path: kenning:///models/classification/magic_wand.h5
optimizers:
- type: TVMCompiler
  parameters:
    compiled_model_path: ./output/microtvm-magic-wand.graph_data
```

You can see, that `llext_binary_path` was parameter added to `platform`.

```
kenning optimize test report \
    --cfg kenning-scenarios/magic-wand-inference/tvm/renode-llext-stm32f746g.yml \
    --measurements results.json --verbosity INFO \
    --report-path reports/stm32-renode-tvm-llext-magic-wand/report.md \
    --to-html \
    --verbosity INFO
```

LLEXT is also compatible with *runtime building by Kenning*.

UART configuration - adding support for a new board

By default all Zephyr boards have 1 UART port dedicated to Zephyr console. Kenning Zephyr Runtime will print welcome message and logs to that console - when running Renode simulation Kenning will wait for the welcome message on that UART, before sending any requests (to make sure Kenning Zephyr Runtime booted successfully).

Another UART port is needed for two-way communication with Kenning over Kenning Protocol (sending model data, inputs and outputs). To inform the app which UART port to use for that purpose we assign `kcomms` alias to the port, using Zephyr devicetree. Overlay files adding the alias for various boards can be found in `app/boards`, for example the overlay file for `stm32f746g_disco`.

To run Kenning evaluation with `app` on any given board, such a file has to be created. Overlay files should be placed at `app/boards/<board_name>.overlay` (where `<board_name>` is the name of the board in Zephyr, except if the board name contains any `/` characters, they should be changed to `_`) for west to find and use them.

On some boards only one UART port is available, then `kcomms` can be set to the same port as Zephyr console. However in such a case printing logs and the welcome message to the console has to be disabled, by adding to the configuration file (`app/prj.conf` or another config file added with `-DEXTRA_CONF_FILE=litert.conf`):

```
CONFIG_LOG_BACKEND_UART=n
CONFIG_BOOT_BANNER=n
CONFIG_PRINTK=n
```

Kenning will detect, that `kcomms` is set to the same port as Zephyr console and will wait for a set amount of time before sending any requests, instead of waiting for the welcome message.

> **Note**
>
> Kenning Zephyr Runtime also sends logs through Kenning Protocol, aside from printing them. Therefore after printing logs to the UART console is disabled, Kenning will still receive and display them.

Some boards may also require additional configuration. Those should be placed at `app/boards/<board_name>.conf`.

14.1.3 Building and running the `demo_app`

The `demo_app` is an example on how to use `kenning_inference_lib` for model inference. In can also be used for easy debugging of various runtimes.

Building

The example below shows how to build `demo_app` with IREE runtime in embedded_elf mode and run it with Renode emulator.

In here we select `iree_embedded_elf.conf` config file to use IREE in embedded_elf mode - in Zephyr applications the default application config is in `prj.conf` file, but extra configuration files can be specified with `-DEXTRA_CONF_FILE` option.

```
west build -p always -b b_u585i_iot02a demo_app -- -DEXTRA_CONF_FILE=iree_
↪embedded_elf.conf
```

Option `-p always` will force west to run a clean build. Drop the option to run incremental builds.

Built application can be found at `build/zephyr/zephyr.elf` (`build` is the default build directory).

To run a Renode simulation, we also need to generate a `.repl` file:

```
west build -t board-repl
```

Running

Before running the simulation, we need to install Renode, which can be done with a script:

```
source ./scripts/prepare_renode.sh
```

This step has to be repeated every time a new shell is used, since the script exports environmental variables.

Finally, we can use a Python script to run the demo:

```
python ./scripts/run_renode.py
```

The script will automatically capture output from the Zephyr UART console and print it to the terminal. Output should contain classification results for 10 inferences, along with some inference statistics.

Example:

```
*** Booting Zephyr OS build cb93f97bd632 ***
I: model output: [wing: 213.957581, ring: 80.423096, slope: 113.229347, negative:␣
↪158.669266]
I: model output: [wing: 162.148727, ring: 140.959763, slope: 149.957077,␣
↪negative: 236.156738]
I: model output: [wing: 188.821167, ring: 250.954254, slope: 465.087280,␣
↪negative: 329.155548]
I: model output: [wing: 338.350281, ring: 124.087746, slope: 176.398376,␣
↪negative: 253.115128]
I: model output: [wing: -4.008125, ring: 17.447971, slope: -7.546308, negative:␣
↪11.472966]
I: model output: [wing: 92.145882, ring: 120.856918, slope: 199.117325, negative:␣
↪148.276291]
I: model output: [wing: 48.781986, ring: -10.816504, slope: 2.117259, negative: 8.
↪108253]
I: model output: [wing: 409.882965, ring: 152.557022, slope: 218.346588,␣
↪negative: 307.647247]
I: model output: [wing: 131.864807, ring: 56.820183, slope: 77.920113, negative:␣
↪98.029968]
I: model output: [wing: 111.868919, ring: 157.771622, slope: 303.319855,␣
↪negative: 198.856461]
I: inference session statistics:
I:      total inference time: 5644 ms
I:      inference time per batch: 564 ms
I:      device_bytes_allocated: 256032
I:      device_bytes_freed: 224144
I:      device_bytes_peak: 54288
I:      host_bytes_allocated: 32384
I:      host_bytes_freed: 30848
I:      host_bytes_peak: 17024
I:      total_allocated: 473962
I:      total_freed: 369068
I:      peak_allocated: 132518
```

Debugging

To run Renode simulation of the app with a debug server, use `--debug` option:

```
python ./scripts/run_renode.py --debug
```

A GDB server will be started on port `3333`. Connection can be made in `gdb` by running:

```
target remote :3333
```

Debug symbols can be loaded from the built binary: `build/zephyr/zephyr.elf`.

Please refer to GDB documentation for further instructions on how to debug with GDB.

> **Note**
>
> Please remember to use version of GDB compatible with the architecture of the selected

board.

14.1.4 Building with artificially increased memory size for simulation

In some cases we would like to evaluate a model that won't fit in the board memory together with Kenning Zephyr Runtime, for example when target application is smaller than Kenning Zephyr Runtime, mostly for evaluation purposes. For such cases, there is a west target called `increase-memory`.

To do that, run normal build first. For example:

```
west build -p always -b 96b_nitrogen demo_app -- -DEXTRA_CONF_FILE=tvm.conf
```

It should fail, due to insufficient memory. Then you can run:

```
west build -t increase-memory -- -DCONFIG_KENNING_INCREASE_MEMORY_SIZE=2048
```

This will generate board overlay with increased memory and save it in `<application>/boards/<board_name>_increased_memory.overlay`. Example overlay looks like this:

```
&sram0 {
    reg = <0x20000000 0x200000>;
};
```

The new overlay file will remain there even after clean builds, so this step does not need to be repeated. To use the overlay, run the build with `-CONFIG_KENNING_INCREASE_MEMORY=y` option:

```
west build -p always -b 96b_nitrogen demo_app -- -DEXTRA_CONF_FILE=tvm.conf -
↪DCONFIG_KENNING_INCREASE_MEMORY=y
```

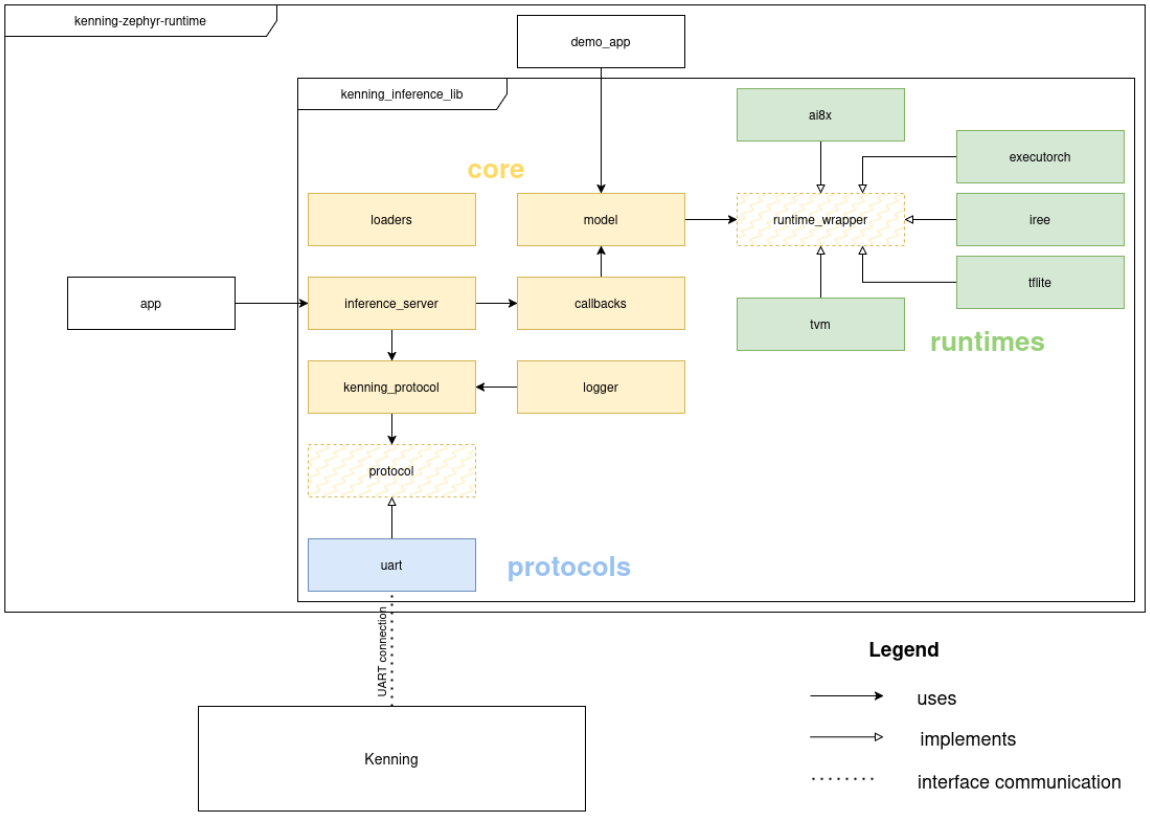
From here you may proceed normally. This option is available for both `app` and `demo_app`.

Note

Increased memory will only work in a Renode simulation.

14.2 General code structure

Relationships between Kenning and various elements of Kenning Zephyr Runtime, as well as internal structure of `kenning_inference_lib` is displayed on the diagram below.



14.2.1 Runtimes

`runtime_wrapper` serves as a common interface for various ML frameworks (runtimes).

`runtime_wrapper.h` is a header file, that contains the following function declarations:

- `runtime_init` - Initialize the runtime (framework).
- `runtime_init_weights` - Load an ML model.
- `runtime_init_input` - Load input into the model.
- `runtime_run_model` - Run inference.
- `runtime_run_model_bench` - Run inference with a time benchmark (results can be downloaded with `runtime_get_statistics`).
- `runtime_get_model_output` - Get output from the model (after inference).
- `runtime_get_statistics` - Get statistics (since the beginning of the current inference session, so since `runtime_init` was last called) - that includes inference time and possibly other framework-dependent statistics.
- `runtime_deinit` - De-initialize the runtime.

To add a new framework (runtime) to Kenning Zephyr Runtime, a source file needs to be created that implements all of these functions (example), as well as creates loaders for model and model input (for details see the *section about loaders*)

Implementation is selected at build time, by linking the correct files with CMake.

Kenning Zephyr Runtime also supports dynamic linking of runtimes through Zephyr's LLEXT subsystem.

14.2.2 The model API and loaders

The `model` is an abstraction over `runtime_wrapper` and handles tasks such as data validation. It also stores model metadata (such as shapes of the model's inputs and outputs), which it makes available to `runtime_wrapper` implementations.

It declares and implements functions such as `model_load_input`, `model_run` or `model_load_struct` (for loading model metadata). For all function declarations see model.h.

All `model` functions, that take data (model, input or model metadata) as input have 2 versions:

- Loading data from an array (`model_load_struct`, `model_load_weights`, `model_load_input`)
- Loading data using `loaders` (`model_load_struct_from_loader`, `model_load_weights_from_loader`, `model_load_input_from_loader`)

`loaders` are a mechanism for loading data without large temporary buffers, therefore saving both execution time and memory.

For example, assume we have a model with input size of 2kB, which we receive byte-by-byte over a serial interface. To load the input with `model_load_input` (which takes a pointer to an array and array size as arguments) we would need to create a 2kB temporary buffer to store the data arriving over the interface and only call `model_load_input` after all of the data has been received.

To save that memory we can use `loaders` - structs stored in a global table (`g_ldr_tables`), that contain:

- pointer to the destination buffer (the final place, where the data is supposed to be loaded - in case of model input it would be an array in the `runtime_wrapper` implementation of the selected framework)
- number of bytes written
- maximum number of bytes, that can be written
- state of the loader
- pointer to a function for loading the data
- pointer to a function for resetting the loader

Functions for loading data and resetting have default implementations and a macro is defined in `loaders.h` to create an instance of a loader with default functions at the given address and with the given size: `MSG_LOADER_BUF(_addr, _max_size)`. However a loader with custom implementations of these functions can also be created, to define a custom behaviour.

Every implementation of `runtime_wrapper.h` creates it's own loaders in `runtime_init` for model and model input. Pointers to these loaders should be placed in `g_ldr_tables` at indexes `[1][LOADER_TYPE_MODEL]` for model and `[1][LOADER_TYPE_DATA]` for input.

Going back to our example - to avoid using a 2kB temporary buffer we can receive the data in batches, saving it into a smaller buffer and repeatedly calling `g_ldr_ tables[1][LOADER_TYPE_DATA]->save` for each batch, then finally calling `model_load_input_from_loader`. An example of that use case can be seen in the `receive_message_payload` function in `kenning_protocol.c`.

14.2.3 Communication stack

The communication stack consists of 2 abstraction layers:

- `protocol` - lower layer, provides functions `protocol_init`, `protocol_read_data` and `protocol_write_data`, allows for sending raw bytes over an interface
- `kenning_protocol` - implementation of *Kenning Protocol*, using the data link provided by `protocol`, provides functions `protocol_transmit` and `protocol_listen` (requests are not supported). `protocol_listen` uses `loaders` to receive message payload, it takes as a callback function that returns the correct callback based on message type.

Header file `protocol.h` only serves as an interface and declares the functions. Function are defined by implementations. Currently 1 implementation is provided (`uart.c`).

14.2.4 Inference server and callbacks

`inference_server` provides functions: `init_server`, `wait_for_protocol_event` and `handle_protocol_event`.

`wait_for_protocol_event` receives a request from Kenning by calling `protocol_listen`, while `handle_protocol_event` calls the correct callback function from `callbacks` (selected based on message type) and sends response to the request based on on values returned by the callback.

Callback functions from `callbacks` translate requests from Kenning to proper actions by calling functions from `model`. For example `output_callback` calls `model_get_output`.

14.2.5 Logger

Kenning Zephyr Runtime uses built-in logging framework from Zephyr RTOS. However it defines it's own logging backend to send log messages back to Kenning over *Kenning Protocol*.

CHAPTER

FIFTEEN

DEVELOPING KENNING BLOCKS

This chapter describes the development process of Kenning components.

To run below examples it is required to install Kenning with dependencies as follows:

```
uv pip install "kenning[tensorflow] @ git+https://github.com/antmicro/kenning.git"
```

15.1 Model and I/O metadata

Since not all model formats supported by Kenning provide information about inputs and outputs or other data required for compilation or runtime purposes, each model processed by Kenning comes with a JSON file describing an I/O specification (and other useful metadata).

The JSON file with metadata for file `<model-name>.<ext>` is saved as `<model-name>.<ext>.json` in the same directory.

The example metadata file looks as follows (for ResNet50 for the ImageNet classification problem):

```
{
    "input": [
        {
            "name": "input_0",
            "shape": [1, 224, 224, 3],
            "dtype": "int8",
            "order": 0,
            "scale": 1.0774157047271729,
            "zero_point": -13,
            "prequantized_dtype": "float32"
        }
    ],
    "output": [
        {
            "name": "output_0",
            "shape": [1, 1000],
            "dtype": "int8",
            "order": 0,
            "scale": 0.00390625,
            "zero_point": -128,
            "prequantized_dtype": "float32"
        }
```

(continues on next page)

(continued from previous page)

```
    ]
}
```

A sample metadata JSON file may look as follows (for the YOLOv4 detection model):

```
{
    "input": [
        {
            "name": "input",
            "shape": [1, 3, 608, 608],
            "dtype": "float32"
        }
    ],
    "output": [
        {
            "name": "output",
            "shape": [1, 255, 76, 76],
            "dtype": "float32"
        },
        {
            "name": "output.3",
            "shape": [1, 255, 38, 38],
            "dtype": "float32"
        },
        {
            "name": "output.7",
            "shape": [1, 255, 19, 19],
            "dtype": "float32"
        }
    ],
    "processed_output": [
        {
            "name": "detection_output",
            "type": "List",
            "dtype": {
                "type": "List",
                "dtype": "kenning.datasets.helpers.detection_and_segmentation.
↪DetectObject"
            }
        }
    ],
    "model_version": 4
}
```

In general, the metadata file consist of four fields:

- `input` is a specification of input data received by model wrapper (before preprocessing),
- `processed_input` is a specification of data passed to wrapped model (after preprocessing),
- `output` is a specification of data returned by wrapped model (before postprocessing),

- `processed_output` is a specification of output data returned by model wrapper (after post-processing).

If `processed_input` or `processed_output` is not specified we assume that there is no processing and it is the same as `input` or `output` respectively. Each array consist of dictionaries describing model inputs and outputs.

Additionally, any non-conflicting miscellaneous keywords may be included into the metadata JSON, but those will not be validated by Kenning.

Parameters common to all fields (except for the miscellaneous):

- `name` - input/output name,
- `shape` - input/output tensor shape, if `processed_input` is present, `input` can have list of valid shapes,
- `dtype` - input/output type,

Parameters specific to `processed_input` and `output`:

- `order` - some of the runtimes/optimizers allow accessing inputs and outputs by id. This field describes the id of the current input/output,
- `scale` - scale parameter for the quantization purposes. Present only if the input/output requires quantization/dequantization,
- `zero_point` - zero point parameter for the quantization purposes. Present only if the input/output requires quantization/dequantization,
- `prequantized_dtype` - input/output data type before quantization.
- `class_name` - list of class names from the dataset. It is used by output collectors to present the data in a human-readable way.

Parameters specific to `input`:

- `mean` - mean used to normalize inputs before model training,
- `std` - standard deviation used to normalize inputs before model training.

Parameters specific to `output` and `processed_output`:

- `class_name` - list of class names from the dataset. It is used by output collectors to present the data in a human-readable way.

Parameters specific to all fields which data is not compatible with `np.ndarray`:

- `type` - can be either `List`, `Dict` or `Any`.
- `dtype` - if `type` is `List`, it can specify type of elements, should be represented as whole path which can be imported (i.e. `kenning.datasets.helpers.detection_and_segmentation.SegmObject` for segmentation results). It can also receive dictionary with specification of list items.
- `fields` - if `type` is `Dict`, it should be a dictionary with names as keys and values as dictionary matching `input` and `output` specification. Example from `PyTorchCOCOMaskRCNN`:

```
{
    "type": "Dict",
    "fields": {
```

(continues on next page)

(continued from previous page)

```
        "boxes": {"shape": [-1, 4], "dtype": "float32"},
        "labels": {"shape": [-1,], "dtype": "int64"},
        "scores": {"shape": [-1,], "dtype": "float32"},
        "masks": {
            "shape": [-1, 1, 416, 416],
            "dtype": "float32",
        },
    },
}
```

The model metadata is used by all classes in Kenning in order to understand the format of the inputs and outputs.

Warning

The *Optimizer* objects can affect the format of inputs and outputs, e.g. quantize the network. It is crucial to update the I/O specification when a block modifies it.

15.2 Implementing a new Kenning component

Firstly, check *Kenning API* for available building blocks for Kenning and their documentation. Adding a new block to Kenning is a matter of creating a new class inheriting from one of `kenning.core` classes, providing configuration parameters, and implementing methods - at least the unimplemented ones.

For example purposes, let's create a sample *Optimizer*-based class, which will convert an input model to the TensorFlow Lite format.

First, let's create minimal code with an empty class:

```
from kenning.core.optimizer import Optimizer


class TensorFlowLiteCompiler(Optimizer):
    pass
```

15.2.1 Defining arguments for core classes

Kenning classes can be created in three ways:

- Using constructor in Python,
- Using argparse from command-line (see *Using Kenning via command line arguments*),
- Using JSON-based dictionaries from JSON scenarios (see *Defining optimization pipelines in Kenning*)

To support all three methods, the newly implemented class requires creating a dictionary called `arguments_structure` that holds all configurable parameters of the class, along with their description, type and additional information.

This structure is used to create:

- an `argparse` group, to configure class parameters from terminal level (via the *ArgumentsHandler*'s `form_argparse` method). Later, a class can be created with the `from_argparse` method.
- a JSON schema to configure the class from a JSON file (via the *ArgumentsHandler*'s `form_parameterschema` method). Later, a class can be created with the `from_parameterschema` method.

`arguments_structure` is a dictionary in the following form:

```
arguments_structure = {
    "argument_name": {
        "description": "Help for the argument",
        "type": str,
        "required": True,
    }
}
```

The `argument_name` is a name used in:

- the Python constructor,
- an Argparse argument (in a form of `--argument-name`),
- a JSON argument.

The fields describing the argument are as follows:

- `argparse_name` - if there is a need for a different flag in argparse, it can be provided here,
- `description` - description of the node, displayed for a parsing error for JSON, or in help in case of command-line access,
- `type` - type of argument, i.e.:
  - `Path` from `pathlib` module,
  - `ResourceURI` - `Path` based supporting non-local resources, e.g. from GitHub (`gh://`), Kenning (`kenning://`) or HTTPS (`https://`),
  - `str`,
  - `float`,
  - `int`,
  - `bool`,
  - `list` or `list[type]` or `list[type1 | type2 | ...]`,
- `default` - default value for the argument (used for AutoML - the value should be in the defined ranges/enum),
- `required` - boolean, tells if argument is required or not,
- `enum` - a list of possible values for the argument (used for AutoML),
- `nullable` - tells if argument can be empty (`None`),
- `AutoML` - specify whether argument is used for AutoML flow, supported only in model wrappers,

- `item_range` - tuple of lower and upper bound of numerical argument (`type` is either `int` or `float`) or elements of list argument (`type` is list) (used only for AutoML),
- `list_range` - tuple of lower and upper bound of list argument length (used only for AutoML),
- `overridable` - tells if argument set in JSON can be overridden with argparse (default: `False`).

Let's add parameters to the example class:

```
from typing import Optional, Literal
from pathlib import Path
from kenning.core.model import ModelWrapper
from kenning.core.optimizer import Optimizer
from kenning.core.dataset import Dataset


class TensorFlowLiteCompiler(Optimizer):
    arguments_structure = {
        "inferenceinputtype": {
            "argparse_name": "--inference-input-type",
            "description": "Data type of the input layer",
            "default": "float32",
            "enum": ["float32", "int8", "uint8"],
        },
        "inferenceoutputtype": {
            "argparse_name": "--inference-output-type",
            "description": "Data type of the output layer",
            "default": "float32",
            "enum": ["float32", "int8", "uint8"],
        },
        "quantize_model": {
            "argparse_name": "--quantize-model",
            "description": "Tells if model should be quantized",
            "type": bool,
            "default": False,
        },
        "dataset_percentage": {
            "description": "Tells how much data from dataset (from 0.0 to "
            "1.0) will be used for calibration dataset",
            "type": float,
            "default": 0.25,
        },
    }

    def __init__(
        self,
        dataset: Dataset,
        compiled_model_path: Path,
        location: Literal["host", "target"] = "host",
        inferenceinputtype: str = "float32",
```

(continues on next page)

(continued from previous page)

```
        inferenceoutputtype: str = "float32",
        dataset_percentage: float = 0.25,
        quantize_model: bool = False,
        model_wrapper: Optional[ModelWrapper] = None,
    ):
        self.inferenceinputtype = inferenceinputtype
        self.inferenceoutputtype = inferenceoutputtype
        self.dataset_percentage = dataset_percentage
        self.quantize_model = quantize_model
        super().__init__(
            dataset,
            compiled_model_path,
            location=location,
            model_wrapper=model_wrapper,
        )

    @classmethod
    def from_argparse(cls, dataset, args):
        return cls(
            dataset,
            args.compiled_model_path,
            args.inference_input_type,
            args.inference_output_type,
            args.dataset_percentage,
            args.quantize_model,
        )
```

In addition to defined arguments, there are also default *Optimizer* arguments - the *Dataset* object and path to save the model (`compiled_model_path`).

Also, a `from_argparse` object creator is implemented, since there are additional parameters (`dataset`) to handle. The `from_parameterschema` function is created automatically.

Additionally, if the new class does not inherit from any Kenning core classes, then it should inherit directly from *ArgumentsHandler* class which is responsible for creating `argparse` groups and JSON schema from `arguments_structure`.

The above implementation of arguments is common for all core classes.

15.2.2 Defining supported output and input types

The Kenning classes for consecutive steps are meant to work in a seamless manner, which means providing various ways to pass the model from one class to another.

Usually, each class can accept multiple model input formats and provides at least one output format that can be accepted in other classes (except for `terminal` compilers, such as Apache TVM, that compile models to a runtime library).

The list of supported output formats is represented in a class with an `outputtypes` list:

```
outputtypes = ["tflite"]
```

Likewise, the list of support input formats is also organized in the same class as the `inputtypes` list:

```
inputtypes = ["keras", "tensorflow"]
```

Let's update the code with supported types:

```
from kenning.core.model import ModelWrapper
from kenning.core.optimizer import Optimizer
from kenning.core.dataset import Dataset

from typing import Optional, Literal


import tensorflow as tf
from pathlib import Path


class TensorFlowLiteCompiler(Optimizer):
    outputtypes = ["tflite"]

    inputtypes = ["keras", "tensorflow"]

    arguments_structure = {
        "inferenceinputtype": {
            "argparse_name": "--inference-input-type",
            "description": "Data type of the input layer",
            "default": "float32",
            "enum": ["float32", "int8", "uint8"],
        },
        "inferenceoutputtype": {
            "argparse_name": "--inference-output-type",
            "description": "Data type of the output layer",
            "default": "float32",
            "enum": ["float32", "int8", "uint8"],
        },
        "quantize_model": {
            "argparse_name": "--quantize-model",
            "description": "Tells if model should be quantized",
            "type": bool,
            "default": False,
        },
        "dataset_percentage": {
            "description": "Tells how much data from dataset (from 0.0 to "
            "1.0) will be used for calibration dataset",
            "type": float,
            "default": 0.25,
        },
    }

    def __init__(
```

(continues on next page)

(continued from previous page)

```
        self,
        dataset: Dataset,
        compiled_model_path: Path,
        location: Literal["host", "target"] = "host",
        inferenceinputtype: str = "float32",
        inferenceoutputtype: str = "float32",
        dataset_percentage: float = 0.25,
        quantize_model: bool = False,
        model_wrapper: Optional[ModelWrapper] = None,
    ):
        self.inferenceinputtype = inferenceinputtype
        self.inferenceoutputtype = inferenceoutputtype
        self.dataset_percentage = dataset_percentage
        self.quantize_model = quantize_model
        super().__init__(
            dataset,
            compiled_model_path,
            location=location,
            model_wrapper=model_wrapper,
        )

    @classmethod
    def from_argparse(cls, dataset, args):
        return cls(
            dataset,
            args.compiled_model_path,
            args.inference_input_type,
            args.inference_output_type,
            args.dataset_percentage,
            args.quantize_model,
        )
```

15.2.3 Implementing unimplemented methods

The remaining aspect of developing Kenning classes is implementing unimplemented methods. Unimplemented methods raise the `NotImplementedError`.

In case of the *Optimizer* class, it is the `compile` method and `get_framework_and_version`.

When implementing unimplemented methods, it is crucial to follow type hints both for inputs and outputs - more details can be found in the documentation for the method. Sticking to the type hints ensures compatibility between blocks, which is required for a seamless connection between compilation components.

Let's finish the implementation of the *Optimizer*-based class.

```
from kenning.core.model import ModelWrapper
from kenning.core.optimizer import Optimizer
from kenning.core.dataset import Dataset
from kenning.utils.update_h5_file import update_h5_file
import tensorflow as tf
```

(continues on next page)

(continued from previous page)

```
from pathlib import Path
from typing import Optional, Dict, List, Literal

from kenning.converters import converter_registry


class TensorFlowLiteCompiler(Optimizer):
    outputtypes = ["mytflite"]

    inputtypes = [
        "keras",
        "tensorflow",
    ]

    arguments_structure = {
        "inferenceinputtype": {
            "argparse_name": "--inference-input-type",
            "description": "Data type of the input layer",
            "default": "float32",
            "enum": ["float32", "int8", "uint8"],
        },
        "inferenceoutputtype": {
            "argparse_name": "--inference-output-type",
            "description": "Data type of the output layer",
            "default": "float32",
            "enum": ["float32", "int8", "uint8"],
        },
        "quantize_model": {
            "argparse_name": "--quantize-model",
            "description": "Tells if model should be quantized",
            "type": bool,
            "default": False,
        },
        "dataset_percentage": {
            "description": "Tells how much data from dataset (from 0.0 to "
            "1.0) will be used for calibration dataset",
            "type": float,
            "default": 0.25,
        },
    }

    def __init__(
        self,
        dataset: Dataset,
        compiled_model_path: Path,
        location: Literal["host", "target"] = "host",
        inferenceinputtype: str = "float32",
        inferenceoutputtype: str = "float32",
        dataset_percentage: float = 0.25,
```

(continues on next page)

(continued from previous page)

```
        quantize_model: bool = False,
        model_wrapper: Optional[ModelWrapper] = None,
    ):
        self.inferenceinputtype = inferenceinputtype
        self.inferenceoutputtype = inferenceoutputtype
        self.dataset_percentage = dataset_percentage
        self.quantize_model = quantize_model
        super().__init__(dataset, compiled_model_path, location=location)

    @classmethod
    def from_argparse(cls, dataset, args):
        return cls(
            dataset,
            args.compiled_model_path,
            args.inference_input_type,
            args.inference_output_type,
            args.dataset_percentage,
            args.quantize_model,
        )

    def compile(
        self,
        inputmodelpath: Path,
        io_spec: Optional[Dict[str, List[Dict]]] = None,
    ):

        # load I/O specification for the model
        if io_spec is None:
            io_spec = self.load_io_specification(inputmodelpath)

        # load the model using chosen input type
        if self.inputtype == "keras":
            if inputmodelpath.suffix in (".h5", ".hdf5"):
                # Update to the newest version of Tensorflow.
                inputmodelpath = update_h5_file(inputmodelpath)
            model = tf.keras.models.load_model(str(inputmodelpath))
            keras_model = model

            if self.quantize_model:
                import tensorflow_model_optimization as tfmodt

                def annotate_model(layer):
                    # Mark every dense layer so that it will be quantized.
                    if isinstance(layer, tf.keras.layers.Dense):
                        return (
                            tfmot.quantization.keras.quantize_annotate_layer(
                                layer
                            )
                        )
```

(continues on next page)

(continued from previous page)

```
                    return layer

                # Create a copy with the quantize annotations applied.
                quant_aware_annotate_model = tf.keras.models.clone_model(
                    model, clone_function=annotate_model
                )

                # Perform quantization
                pcqat_model = tfmot.quantization.keras.quantize_apply(
                    quant_aware_annotate_model,
                    tfmot.experimental.combine.
↪Default8BitClusterPreserveQuantizeScheme(
                        preserve_sparsity=True
                    ),
                )

                keras_model = self.train_model(pcqat_model)
            converter = tf.lite.TFLiteConverter.from_keras_model(keras_model)
        else:
            # We need to find the converter somewhere else.
            inputtype = self.get_input_type(inputmodelpath)
            conversion_kwargs = {
                "io_spec": io_spec_processed,
                "target": self.target,
                "inferenceinputtype": self.inferenceinputtype,
                "inferenceoutputtype": self.inferenceoutputtype,
            }

            # Use the converter_registry to find a converter from
            # our input type to tflite
            converter = converter_registry.convert(
                inputmodelpath, inputtype, "tflite", **conversion_kwargs
            )

        # compile and save the model
        tflite_model = converter.convert()
        with open(self.compiled_model_path, "wb") as f:
            f.write(tflite_model)

        # update the I/O specification
        interpreter = tf.lite.Interpreter(model_content=tflite_model)
        signature = interpreter.get_signature_runner()

        def update_io_spec(sig_det, int_det, key):
            for order, spec in enumerate(io_spec[key]):
                old_name = spec["name"]
                new_name = sig_det[old_name]["name"]
                spec["name"] = new_name
                spec["order"] = order
```

(continues on next page)

(continued from previous page)

```
            quantized = any([det["quantization"][0] != 0 for det in int_det])
            new_spec = []
            for det in int_det:
                spec = [
                    spec
                    for spec in io_spec[key]
                    if det["name"] == spec["name"]
                ][0]

                if quantized:
                    scale, zero_point = det["quantization"]
                    spec["scale"] = scale
                    spec["zero_point"] = zero_point
                    spec["prequantized_dtype"] = spec["dtype"]
                    spec["dtype"] = np.dtype(det["dtype"]).name
                new_spec.append(spec)
            io_spec[key] = new_spec

        update_io_spec(
            signature.get_input_details(),
            interpreter.get_input_details(),
            "input",
        )
        update_io_spec(
            signature.get_output_details(),
            interpreter.get_output_details(),
            "output",
        )

        # save updated I/O specification
        self.save_io_specification(inputmodelpath, io_spec)

    def get_framework_and_version(self):
        return "tensorflow", tf.__version__
```

There are several important things regarding the code snippet above:

- The information regarding inputs and outputs can be collected with the `self.load_io_specification` method, present in all classes.
- The information about the input format to use is delivered in the `self.inputtype` field - it is updated automatically by the function consulting the best supported format for previous and current block.
- If the I/O metadata is affected by the current block, it needs to be updated and saved along with the compiled model using the `self.save_io_specification` method.
- When the input type is different than the ones defined in `self.inputtype`, `converter_registry` is used to find a suitable *ModelConverter*. It is possible that intermediate formats will be needed during a conversion.

15.2.4 Using the implemented block

In the Python script, the above class can be used with other classes from the *Kenning API* as is - it is a regular Python code.

To use the implemented block in the JSON scenario (as described in *Defining optimization pipelines in Kenning*, the module implementing the class needs to be available from the current directory, or the path to the module needs to be added to the `PYTHONPATH` variable.

Let's assume that the class was implemented in the `my_optimizer.py` file. The scenario can look as follows:

```
{
    "model_wrapper":
    {
        "type": "kenning.modelwrappers.classification.tensorflow_pet_dataset.
↪TensorFlowPetDatasetMobileNetV2",
        "parameters":
        {
            "model_path": "kenning:///models/classification/tensorflow_pet_
↪dataset_mobilenetv2.h5"
        }
    },
    "dataset":
    {
        "type": "kenning.datasets.pet_dataset.PetDataset",
        "parameters":
        {
            "dataset_root": "./build/pet-dataset"
        }
    },
    "optimizers":
    [
        {
            "type": "my_optimizer.TensorFlowLiteCompiler",
            "parameters":
            {
                "compiled_model_path": "./build/compiled-model.tflite",
                "inference_input_type": "float32",
                "inference_output_type": "float32"
            }
        }
    ],
    "runtime":
    {
        "type": "kenning.runtimes.litert.LiteRTRuntime",
        "parameters":
        {
            "save_model_path": "./build/compiled-model.tflite"
        }
    }
}
```

The emphasized line demonstrates usage of the implemented `TensorFlowLiteCompiler` from the `my_optimizer.py` script.

This sums up the Kenning development process.

15.3 Implementing Kenning runtime blocks

15.3.1 Implementing new Runners for KenningFlow

The process of creating new *Runner* is almost the same as the process of implementing Kenning components described above, with a few additional steps.

First of all, the new component needs to inherit from a Runner class (not necessarily directly). Then you need to implement the following methods:

- `cleanup` - cleans resources after execution is stopped
- `should_close` - (*optional*) returns boolean indicating whether a runner ended processing and requests closing Possible reasons: a signal from terminal, user request in GUI, end of data to process, an error, and more. Default implementation always returns `False`
- `run` - method that gets runner inputs, processes them and returns obtained results.

Inputs for the runner are passed to the `run` method as a dictionary where the key is the input name specified in the KenningFlow JSON and the value is simply the input value.

E.g. for `DetectionVisualizer` defined in JSON as

```
{
    "type": "kenning.outputcollectors.detection_visualizer.DetectionVisualizer",
    "parameters": {
        "output_width": 608,
        "output_height": 608
    },
    "inputs": {
        "frame": "cam_frame",
        "detection_data": "predictions"
    }
}
```

the `run` method access inputs as follows

```
from typing import Any, Dict


def run(self, inputs: Dict[str, Any]) -> Dict[str, Any]:
    input_data = inputs["frame"]
    output_data = inputs["detection_data"]
    self.process_output(input_data, output_data)
```

Note

In the example above, the `run` method does not contain a `return` statement, because this runner does not have any outputs. If you want to create a runner with outputs, this method

should return a similar dictionary containing outputs.

15.4 Adjusting ModelWrapper for AutoML flow

15.4.1 Implementing a ModelWrapper

To create an AutoML model the default modelwrapper-api has to implement automl-model-api interface. The description of the ModelWrapper implementation can be found in model-io-metadata section. But it has to be extended with a few additional steps, like in the AutoPyTorch-based *example with a simple fully-connected neural network*:

- add `kenning.core.automl.AutoMLModel` based class inheritance to the ModelWrapper (line 46),
- implement the model class with parameters that can be tuned by AutoML (lines 14-43),
- point to the model class in the ModelWrapper using full class path (line 63),
- define which ModelWrapper arguments can be tuned (lines 52-60),
  - names have to match with the model class parameters,
  - AutoML-specific configuration is described in *Defining arguments for core classes*,
- implement or override inherited methods, in case of `AutoPyTorchModel`, they are:
  - (required) `get_io_specification_from_dataset` - creates IO spec based on the dataset (lines 138-148),
  - (optional) `model_params_from_context` - generates additional parameters for the model based on the dataset and optional platform (lines 76-86),
  - (optional) `define_forbidden_clauses` - adds forbidden configurations to the search space,
  - (optional) `register_components` - adds custom components to AutoPyTorch.

Listing 15.1: fc_automl.py

```
1  from typing import Any, Dict, List, Optional, Tuple
2
3  import numpy as np
4  import torch
5  from torch import nn
6
7  from kenning.automl.auto_pytorch import AutoPyTorchModel
8  from kenning.core.platform import Platform
9  from kenning.datasets.anomaly_detection_dataset import AnomalyDetectionDataset
10 from kenning.modelwrappers.frameworks.pytorch import PyTorchWrapper
11 from kenning.utils.resource_manager import PathOrURI
12
13
14 class FCNetwork(nn.Sequential):
15     """
16     The simple fully-connected neural network model.
```

(continues on next page)

(continued from previous page)

```
    """

    def __init__(
        self,
        # The input shape is provided by AutoPyTorch
        input_shape: Tuple[int, ...],
        # Remaining arguments should have the same names as parameters
        # from `arguments_structure` and `model_params_from_context` method
        neurons: List[int],
        outputs: int,
    ):
        self.neurons = neurons
        linears = [
            nn.Linear(in_, out)
            for in_, out in zip(
                [input_shape[-1]] + neurons, neurons + [outputs]
            )
        ]
        super().__init__(
            *[
                module
                for linear in linears[:-1]
                for module in (linear, nn.ReLU())
            ],
            linears[-1],
            nn.Softmax(dim=-1),
        )


class PyTorchFullyConnected(PyTorchWrapper, AutoPyTorchModel):
    """
    The example AutoML-compatible fully connected model.
    """

    arguments_structure = {
        # Required: Example parameter which can be adjusted by AutoML flow
        "neurons": {
            "description": "List of neurons in consecutive layers",
            "type": list[int],
            "default": [8, 16, 8],
            "AutoML": True,
            "list_range": (1, 16),
            "item_range": (1, 256),
        },
    }
    # Required: The path to the class representing model
    model_class = f"{FCNetwork.__module__}.{FCNetwork.__name__}"

    def __init__(
```

(continues on next page)

(continued from previous page)

```
        self,
        model_path: PathOrURI,
        dataset: AnomalyDetectionDataset,
        from_file: bool = True,
        model_name: Optional[str] = None,
        neurons: List[int] = [8, 16, 8],
    ):
        super().__init__(model_path, dataset, from_file, model_name)
        self.neurons = neurons

    # Optional: Method generating additional model parameters
    # based on the dataset
    @classmethod
    def model_params_from_context(
        cls,
        dataset: AnomalyDetectionDataset,
        platform: Optional[Platform] = None,
    ):
        return {
            "outputs": len(dataset.get_class_names()),
        }

    def create_model_structure(self):
        self.model = FCNetwork(
            input_shape=(
                -1,
                self.dataset.window_size * self.dataset.num_features,
            ),
            neurons=self.neurons,
            **self.model_params_from_context(self.dataset),
        )

    def prepare_model(self):
        if self.model_prepared:
            return None

        if self.from_file:
            self.load_model(self.model_path)
            self.model_prepared = True
        else:
            self.create_model_structure()

            def weights_init(m):
                if isinstance(m, nn.Linear):
                    torch.nn.init.xavier_uniform_(m.weight)
                    torch.nn.init.zeros_(m.bias)

            self.model.apply(weights_init)
            self.model_prepared = True
```

(continues on next page)

(continued from previous page)

```
            self.save_model(self.model_path)
        self.model.to(self.device)

    def train_model(self):
        raise NotImplementedError

    def preprocess_input(self, X: List[Any]) -> List[Any]:
        import torch

        X = np.asarray(X[0])
        return [torch.from_numpy(X.reshape(X.shape[0], -1)).to(self.device)]

    def postprocess_outputs(self, y: List[Any]) -> List[np.ndarray]:
        output = (
            np.argmax(
                y[0].detach().cpu().numpy(),
                axis=-1,
            )
            .reshape(-1)
            .astype(np.int8)
        )
        return (output,)

    # Required: Method generating IO specification
    # based on the dataset
    def get_io_specification_from_dataset(
        cls,
        dataset,
    ) -> Dict[str, List[Dict]]:
        return cls._get_io_specification(
            dataset.num_features,
            dataset.window_size,
            dataset.batch_size,
        )

    @classmethod
    def _get_io_specification(
        cls,
        num_features,
        window_size,
        batch_size: int = -1,
    ) -> Dict[str, List[Dict]]:
        return {
            "input": [
                {
                    "name": "input_1",
                    "shape": (
                        batch_size,
                        window_size,
```

(continues on next page)

(continued from previous page)

```
                        num_features,
                    ),
                    "dtype": "float32",
                }
            ],
            "processed_input": [
                {
                    "name": "input_1",
                    "shape": (
                        batch_size,
                        window_size * num_features
                        if window_size > 0 and num_features > 0
                        else -1,
                    ),
                    "dtype": "float32",
                }
            ],
            "output": [
                {
                    "name": "distances",
                    "shape": (batch_size, 2),
                    "dtype": "float32",
                }
            ],
            "processed_output": [
                {
                    "name": "anomalies",
                    "shape": (batch_size,),
                    "dtype": "int8",
                }
            ],
        }

    def get_io_specification_from_model(self) -> Dict[str, List[Dict]]:
        return self.get_io_specification_from_dataset(self.dataset)

    @classmethod
    def derive_io_spec_from_json_params(
        cls, json_dict: Dict
    ) -> Dict[str, List[Dict]]:
        return cls._get_io_specification(-1, -1)
```

15.4.2 Using the implemented model

To use the model, its full module path has to be specified in the `use_models` parameter:

Listing 15.2: fc-scenario.yml

```
dataset:
  type: AnomalyDetectionDataset
```

(continues on next page)

(continued from previous page)

```
  parameters:
    dataset_root: ./workspace/CATS
    csv_file: kenning:///datasets/anomaly_detection/cats_nano.csv
    split_fraction_test: 0.1
    split_seed: 12345
    inference_batch_size: 1

automl:
  type: AutoPyTorchML
  parameters:
    time_limit: 2
    use_models:
      # Use model wrapper defined in fc_automl.py
      - fc_automl.PyTorchFullyConnected
    output_directory: ./workspace/automl-results
    n_best_models: 5
    optimize_metric: f1
    min_budget: 1
    max_budget: 2
```

Running the simples AutoML flow, assuming the implementation and the scenario are saved under `fc_automl.py` and `fc-scenario.yml` respectively, boils down to:

```
# Make sure the required dependencies are installed
uv pip install "kenning[torch,anomaly_detection,auto_pytorch] @ git+https://
↪github.com/antmicro/kenning.git"
# Run AutoML flow with new model
PYTHONPATH="$(pwd):$PYTHONPATH" kenning automl --cfg ./fc-scenario.yml
```

CHAPTER

SIXTEEN

KENNING RESOURCES

For managing resources, such as models, compilers, datasets and more Kenning provides *ResourceManager*. It handles:

- Downloading of remote files,
- Resolving custom URL schemes for cleaner and easy to follow file addresses,
- Managing directory with files, including quota checking, old files removal and full cleanup,
- Providing actual path to the resources in the filesystem.

To run below examples it is required to install Kenning as follows:

```
uv pip install "kenning @ git+https://github.com/antmicro/kenning.git"
```

16.1 Accessing resources

To download a resource you can use `ResourceManager.get_resource` method. The parameters of this method are a URI of the resource and optional output path, where the file should be saved. If the output path is not specified, the file will be saved in `$KENNING_CACHE_DIR/uri.path`.

```
import pathlib
from kenning.utils.resource_manager import ResourceManager, ResourceURI

model_path = pathlib.Path("./model.h5")
ResourceManager().get_resource(
    "kenning:///models/classification/magic_wand.h5", model_path
)
```

Another way of accessing those resources is to use *ResourceURI* class. You can simply create an instance of this class and use it similarly to `pathlib.Path`:

```
model_path = ResourceURI("kenning:///models/classification/magic_wand.h5")
```

The provided path can be URL to remote resource, path to local file or URI which scheme is defined in `ResourceManager` conversions dictionary.

16.2 Caching

If the resource was not accessed before, the cached file was modified or the remote resource was updated, then it will be downloaded to local cache directory. The location of cache directory can be set using the environment variable `KENNING_CACHE_DIR`. If it is not set, then the default `~/.kenning` path will be used. You can also customize it using `ResourceManager.set_cache_dir` method.

By default, the maximum size of the cache is set to 50 GB, but it can be changed using the environment variable `KENNING_MAX_CACHE_SIZE` or in the runtime using `ResourceManager.set_max_cache_size` method. Before each download the file size is checked and if the available cache space is not large enough, then the oldest files are deleted until there is enough space. If the file is bigger than maximum size of the cache, then the exception is raised.

16.3 Custom URI schemes

As you can see above, the `ResourceManager` supports non-standard URI scheme `kenning://`. In fact, it can support various schemes defined by the user.

The default available schemes are:

- `http://`, `https://`
- `file://` - provides the absolute path to the file in the filesystem
- `kenning://` - a scheme for downloading Kenning-specific resources from `https://dl.antmicro.com`. For example, `kenning:///models/classification/magic_wand.h5` resolves to `https://dl.antmicro.com/kenning/models/classification/magic_wand.h5`
- `gh://` - used for downloading files from Github repositories, e.g. `gh://antmicro:kenning-bare-metal-iree-runtime/sim/config/platforms/springbok.repl;branch=main` resolves to `https://raw.githubusercontent.com/antmicro/kenning-bare-metal-iree-runtime/main/sim/config/platforms/springbok.repl`
- `hf://` - used for downloading files from Hugging Face model hub, e.g. `hf://company/model-id` resolves to `https://huggingface.co/company/model-id`. Using this scheme may require logging in to `huggingface_hub` using `huggingface-cli`.

This allows using links such as:

- `https://dl.antmicro.com/kenning/models/classification/magic_wand.h5`
- `kenning:///models/classification/magic_wand.h5`
- `gh://antmicro:kenning-bare-metal-iree-runtime/sim/config/platforms/springbok.repl;branch=main`
- `file:///some/directory/magic_wand.h5`
- `hf://mistralai/Mistral-7B-Instruct-v0.1`

If the provided path does not have any scheme, the it will be interpreted as a path. For example:

```
ResourceURI("/some/path/magic_wand.h5")
```

Would behave the same as `Path('/some/path/magic_wand.h5')`.

Before the URI is passed to format string or callable converter it is parsed into parts: scheme, netloc, path, params, query and fragment. Additionally the netloc and path are parsed into lists

by splitting using either `'.'` or `'/'` separator and the params and query parts are parsed into dictionaries.

The conversion can be:

- `None` - no conversion is done.
- a format string - user can use URI attributes such as: scheme, netloc, path, params, query and fragment enclosed in curly braces. The netloc and path can be also used as lists (i.e. it is possible to use `{path[0]}` or `{netloc[1:]}` inside the format string) and the params and query can be used as dictionary (i.e. you can use `{params[branch]}` for URIs like `gh://antmicro:kenning-bare-metal-iree-runtime/sim/config/platforms/springbok.repl;branch=main`).
- a callable - the provided URI is parsed and passed into it. The callable can have parameters such as `scheme`, `netloc`, `path`, `params`, `query`, `netloc_list`, `path_list`, `params_dict`, `query_dict` and should return string. The parameters with `_list` or `_dicts` suffix are corresponding parts parsed into lists or dicts. Example implementation of such conversion can be seen above as `_gh_converter`.

The above default scheme conversions can be defined as:

```
BASE_URL_SCHEMES = {
    "http": None,
    "https": None,
    "kenning": "https://dl.antmicro.com/kenning/{path}",
    "gh": _gh_converter,
    "file": lambda path: Path(path).expanduser().resolve(),
}
```

The keys in this dictionary are scheme names, and values are conversion methods.

The `_gh_converter` is defined as:

```
from typing import Dict


def _gh_converter(netloc: str, path: str, params_dict: Dict[str, str]) -> str:
    netloc = netloc.split(":")
    return (
        f"https://raw.githubusercontent.com/{netloc[0]}/{netloc[1]}/"
        f"{params_dict['branch']}{path}"
    )
```

Those schemes can be customized by user using the `ResourceManager.add_custom_url_schemes` method which takes a dictionary similar to the above as an input - the key is the name of scheme and the value is the converter (`None`, string or callable). If any scheme already has defined conversion, then it is overwritten.

16.4 CLI commands

You can manage cache using Kenning CLI commands. The available commands are:

- `kenning cache list_files` - list cached files, their size and total cache size. To see full paths add `-v` argument.

- `kenning cache clear` - removes all cached files,
- `kenning cache settings` - prints default cache directory path and default max cache size.

CHAPTER

SEVENTEEN

KENNING PROTOCOLS

Kenning enables workflows spanning multiple machines, usually utilizing client-server architecture, where the server (also referred to as the target platform or the target device) is running tasks (such as inference or model optimization) on the client's (host's) requests, which e.g. delivers models to run or input samples from the dataset.

Protocol was created as a uniform interface for communication between two Kenning instances, serving as an abstraction for the underlying communication logic and technology stack.

There are multiple `Protocol` implementations - below is outlined the inheritance hierarchy.

- *BytesBasedProtocol* *(abstract - requires further implementations)* - implements `kenning.core.protocol.Protocol` abstract methods using a simple protocol with custom data format. Cannot be used on its own (requires implementing the underlying logic of requests and transmissions).
  - *KenningProtocol* *(abstract - requires further implementations)* - implements communication logic and flow control for requests and transmissions relying on `BytesBasedProtocol`. Introduces `send_data` and `receive_data` abstract methods (for sending and receiving raw bytes) that user needs to implement to send prepared packets over selected mean of communication. There are:
    - *NetworkProtocol* - Implements methods `send_data` and `receive_data` using Python sockets (TCP/IP protocol).
    - *UARTProtocol* - Implements methods `send_data` and `receive_data` using the UART standard.
- *ROS2Protocol* - Implementation based on the ROS2 framework.

Note

Things to take into account:

- Some of the implementations are not complete communication solutions and require implementations of their own.
- Not all implementations are compatible with all platforms (for example Kenning Zephyr Runtime can currently only communicate over UART), some implementations do not support all methods (for example `UARTProtocol` only supports client-side API).

17.1 BytesBasedProtocol

BytesBasedProtocol is an abstract class implementing all `kenning.core.protocol.Protocol` abstract methods and introducing custom protocol message format. In addition to initialization and disconnecting, `BytesBasedProtocol` requires implementing following methods:

- *transmit*
- *transmit_blocking*
- *request*
- *request_blocking*
- *listen*
- *listen_blocking*
- *event_active*
- *kill_event*

These methods allow for requesting, sending and receiving payload (`bytes`), along with a set of flags (defined by enum `kenning.protocols.bytes_based_protocol.TransmissionFlag`).

Each request and transmission is identified by a `MessageType`, which serves to differentiate between concurrent communication streams (for example: the client is requesting inference output and at the same time the server is sending logs), as well as to identify the correct server callback. This class does not work as a standalone communication solution - aforementioned abstract methods, as well as `kenning.core.protocols.Protocol` methods *initialize_client*, *initialize_server* and *disconnect* have to be provided by its implementations.

17.1.1 BytesBasedProtocol implementations

- *KenningProtocol*
  - *NetworkProtocol*
  - *UARTProtocol*

17.1.2 Available MessageType values

- `PING` - Message for checking the connection.
- `STATUS` - Message for reporting server status.
- `DATA` - Message contains inference input.
- `MODEL` - Message contains model to load.
- `PROCESS` - Message means the data should be processed.
- `OUTPUT` - Host requests the output from the target.
- `STATS` - Host requests the inference statistics from the target.
- `IO_SPEC` - Message contains io specification to load.
- `OPTIMIZERS` - Message contains optimizers config.
- `OPTIMIZE_MODEL` - Host requests model optimization and receives optimized model.
- `RUNTIME` - Message contains runtime that should be used for inference (i.e. LLEXT binary).

- `UNOPTIMIZED_MODEL` - Message contains an unoptimized model.
- `LOGS` - Log messages sent from the target device (server).

17.1.3 Available TransmissionFlags

- `SUCCESS` - Transmission is informing about a success (for example successful inference).
- `FAIL`- Transmission is informing about a failure.
- `IS_HOST_MESSAGE`- Not set if the transmission was sent by the target device. Set otherwise.
- `IS_KENNING`- Messages sent by Kenning.
- `IS_ZEPHYR`- Messages sent by Kenning Zephyr Runtime.

17.1.4 Protocol specification

Sending requests from the client side:

- *upload_runtime* - Sends request: `MessageType.RUNTIME`, binary data from the file as payload. Expects a transmission with `TransmissionFlag.SUCCESS` and either: `TransmissionFlag.IS_ZEPHYR` or `TransmissionFlag.IS_KENNING` and `ServerAction.UPLOADING_RUNTIME` as payload.
- *upload_io_specification* - Sends request, `MessageType.IO_SPEC`, binary data from the file as payload. Expects a transmission with `TransmissionFlag.SUCCESS` and either: `TransmissionFlag.IS_ZEPHYR` or `TransmissionFlag.IS_KENNING` and `ServerAction.UPLOADING_IOSPEC` as payload.
- *upload_model* - Sends request, `MessageType.MODEL`, binary data from the file as payload. Expects a transmission with `TransmissionFlag.SUCCESS` and either: `TransmissionFlag.IS_ZEPHYR` or `TransmissionFlag.IS_KENNING` and `ServerAction.UPLOADING_MODEL` as payload.
- *upload_input* - Sends request, `MessageType.DATA`, raw bytes as payload. Expects a transmission with `TransmissionFlag.SUCCESS` and either: `TransmissionFlag.IS_ZEPHYR` or `TransmissionFlag.IS_KENNING` and `ServerAction.UPLOADING_INPUT` as payload.
- *request_processing* - Sends request, `MessageType.PROCESS`, no payload. Expects a transmission with `TransmissionFlag.SUCCESS` and either: `TransmissionFlag.IS_ZEPHYR` or `TransmissionFlag.IS_KENNING` and `ServerAction.PROCESSING_INPUT` as payload.
- *download_output* - Sends request `MessageType.OUTPUT`, no payload. Expects a transmission with `TransmissionFlag.SUCCESS` and non-empty payload.
- *download_statistics* - Sends request `MessageType.STATS`, no payload. Expects a transmission with `TransmissionFlag.SUCCESS` and non-empty payload.
- *upload_optimizers* - Sends request `MessageType.OPTIMIZERS`, JSON serialized to string and encoded as payload. Expects a transmission with `TransmissionFlag.SUCCESS` and either: `TransmissionFlag.IS_ZEPHYR` or `TransmissionFlag.IS_KENNING` and `ServerAction.UPLOADING_OPTIMIZERS` as payload.
- *request_optimization* - Sends request `MessageType.UNOPTIMIZED_MODEL` with binary data from the model file as payload. After receiving a response client sends a request `MessageType.OPTIMIZE_MODEL` with no payload. Expects a transmission with compiled model as payload and `TransmissionFlag.SUCCESS` set.

Serving requests on the server side, by MessageType (after the serve method has been called to provide callbacks):

- Request with message type: IO_SPEC, MODEL, DATA, PROCESS, OPTIMIZERS or UNOPTIMIZED_MODEL - Calls appropriate server callback. Sends a transmission with the ServerAction (from the ServerStatus returned by the callback) as payload, as well as the TransmissionFlag.IS_KENNING set. If the success field of the ServerStatus is set to True, TransmissionFlag.SUCCESS is set. Otherwise TransmissionFlag.FAIL is set.
- Request with message type: OPTIMIZE_MODEL, OUTPUT, STATS - Calls appropriate callback. If the callback returned ServerStatus with the success field set to true, sends a transmission with the bytes returned by the callback as payload, as well as TransmissionFlag.IS_KENNING and TransmissionFlag.SUCCESS set. Otherwise sends a transmission with the ServerAction as payload, as well as TransmissionFlag.IS_KENNING and TransmissionFlag.FAIL set.

Logs are sent by the server as unprompted transmissions (MessageType.LOGS).

Sending multiple log messages per transmission is allowed.

Payload format:

<message 1 size (1 byte)><message 1 ASCII string><message 2 ASCII size (1 byte)><message 2 ASCII string>...<message n size (1byte)><message n ASCII string>.

Methods:

class kenning.protocols.bytes_based_protocol.**BytesBasedProtocol**(timeout: *int* = -1)

Bases: Protocol, ABC

Implements abstract methods from the Protocol class, using an underlying mechanism of transmissions and requests (that mechanism needs to be provided by this class'es extensions).

check_status(status: ServerStatus, artifacts: *tuple[bytes, list[TransmissionFlag]]*) → bool

Parses ServerStatus from the contents of a transmission (payload and flags) and compares it against given status.

Parameters

status : *ServerStatus*
Status to compare against.

artifacts : Tuple[bytes, List[TransmissionFlag]]
Payload and bytes from a transmission. Flags should either contain 'IS_ZEPHYR' flag or the 'IS_KENNING' flag.

Returns
True if status is equal, False if status is not as expected, or if some or the 'artifacts' are None.

Return type
bool

Raises
ValueError – Flags do not containing 'IS_ZEPHYR' flag or the 'IS_KENNING' flag.

download_output() → tuple[bool, bytes | None]

Downloads the outputs from the target device.

Requests and downloads the latest inference output from the target device for quality measurements.

Returns

Tuple with download status (True if successful) and downloaded data.

Return type

Tuple[bool, Optional[Any]]

download_statistics(final: *bool* = **False**) → *Measurements*

Downloads inference statistics from the target device.

By default no statistics are gathered.

Parameters

final : bool

If the inference is finished

Returns

Inference statistics on target device.

Return type

Measurements

abstractmethod event_active(message_type: *MessageType* | *None* = **None**) → bool

Checks if an active protocol event (Transmission, Request etc.) of a given message type exists.

Parameters

message_type : Optional[MessageType]

Message type to check, or None (which will check for an event, that accepts all message types).

Returns

True if event exists, False otherwise.

Return type

bool

abstractmethod kill_event(message_type: *MessageType* | *None* = **None**)

Forcibly stops an active event (Transmission, Request etc.).

Parameters

message_type : Optional[MessageType]

Message type to check, or None (which will stop an event, that accepts all message types).

Raises

ValueError – There is no such event.

abstractmethod listen(message_type: *MessageType* | *None* = **None**, transmission_callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, request_callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, limit: *int* = -1, failure_callback: *Callable[[MessageType], None]* | *None* = **None**)

Waits for transmissions and requests from the other side, without blocking the current thread.

Parameters

message_type : Optional[MessageType]
Message type of the requests/transmissions to listen for, or None (to listen to requests/transmissions of any message type).

transmission_callback : Optional[ProtocolSuccessCallback]
Function, that will be called when a transmission is successfully received. Message type, payload and flags from the transmission will be passed to the function. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

request_callback : Optional[ProtocolSuccessCallback]
Function, that will be called when a request is successfully received. Message type, payload and flags from the request will be passed to the function. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

limit : int
Meximum number of requests/transmissions (including failures), that will be received before listening stops.

failure_callback : Optional[ProtocolFailureCallback]
Function, that will be called if a request/transmission is attempted by the other side, but fails. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

abstractmethod listen_blocking(message_type: *MessageType* | *None* = **None**, transmission_callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, request_callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, timeout: *float* | *None* = **None**, failure_callback: *Callable[[MessageType], None]* | *None* = **None**) → tuple[IncomingEventType | None, MessageType | None, bytes | None, list[TransmissionFlag] | None]

Blocks the current thread until a transmission or a request is received.

Parameters

message_type : Optional[MessageType]
Message type of the requests/transmissions to listen for, or None (to listen to requests/transmissions of any message type).

transmission_callback : Optional[ProtocolSuccessCallback]
Function, that will be called when a transmission is successfully re-

ceived. Message type, payload and flags from the transmission will be passed to the function.

request_callback : Optional[ProtocolSuccessCallback]
Function, that will be called when a request is successfully received. Message type, payload and flags from the request will be passed to the function.

timeout : Optional[float]
Maximum blocking time in seconds, or None to block indefinitely. If that time passes the 'deny_callback' will be called.

failure_callback : Optional[ProtocolFailureCallback]
Function, that will be called if a request/transmission is attempted by the other side, but fails. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

Returns
Enum denoting whether a request or a transmission was received, message type of the received tranmsission/request, received payload, received flags. Alternatively: (None, None, None, None) if no request/transmission was received in the specified timeout or if a request/transmission was attempted but failed.

Return type
tuple[Optional[IncomingEventType], Optional[MessageType], Optional[bytes], Optional[List[TransmissionFlag]]]

listen_to_server_logs()
Starts continuously receiving and printing logs sent by the server.

listen_to_trace_data(tracedump_callback: *Callable[[bytes], None]*)
Starts continuously receiving batches of trace data from server. Calls a callback function with each received batch.

Parameters

tracedump_callback : Callable[[bytes], None]
Function, that will be called for each received batch of data.

abstractmethod request(message_type: *MessageType*, callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]*, payload: *bytes | None* = None, flags: *list[TransmissionFlag]* = [], retry: *int* = 1, deny_callback: *Callable[[MessageType], None] | None* = None)
Prompts the other side for a transmission and waits for a response, without blocking the current thread. Bytes and flags can also be sent along with the request.

Parameters

message_type : MessageType
Value denoting what type of transmission is being requested.

callback : ProtocolSuccessCallback
Function, that will be called when the transmission is received. Message type, payload and flags from the transmission will be passed to the function. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

payload : Optional[bytes]
Bytes to send along with the request (or None if the payload is to be empty).

flags : List[TransmissionFlag]
A list of flags to be sent (available flags in the TransmissionFlag enum above - please note that some flags are only allowed for a specific message type).

retry : int
Denotes how many times the request will be re-sent after failing, before calling 'deny_callback'. Negative number denotes infinite retries.

deny_callback : Optional[ProtocolFailureCallback]
Function, that will be called if the request is denied or otherwise fails. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

abstractmethod request_blocking(message_type: *MessageType*, callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, payload: *bytes* | *None* = **None**, flags: *list[TransmissionFlag]* = [], timeout: *float* | *None* = **None**, retry: *int* = 1, deny_callback: *Callable[[MessageType], None]* | *None* = **None**) → tuple[bytes | None, list[TransmissionFlag] | None]

Prompts the other side for a transmission and blocks the current thread until a response is received. Bytes and flags can also be sent along with the request.

Parameters

message_type : MessageType
Value denoting what type of transmission is being requested.

callback : Optional[ProtocolSuccessCallback]
Function, that will be called when the transmission is received. Message type, payload and flags from the transmission will be passed to the function.

payload : Optional[bytes]
Bytes to send along with the request (or None if the payload is to be empty).

flags : List[TransmissionFlag]
A list of flags to be sent (available flags in the TransmissionFlag enum above - please note that some flags are only allowed for a specific message type).

timeout : Optional[float]
Maximum blocking time in seconds, or None to block indefinitely. If that time passes the 'deny_callback' will be called.

retry : int
Denotes how many times the request will be re-sent after failing, before calling 'deny_callback'. Negative number denotes infinite retries.

deny_callback : Optional[ProtocolFailureCallback]
Function, that will be called if the request is denied or otherwise fails.

Returns
Payload and flags received as response to the request, or (None, None) if the request was denied or otherwise failed.

Return type
Tuple[Optional[bytes], Optional[List[TransmissionFlag]]]

request_optimization(model_path: ~pathlib.Path | None, get_time_func: ~typing.Callable[[], float] = <built-in function perf_counter>) → tuple[bool, bytes | None]

Request optimization of model.

Parameters

model_path : Optional[Path]
Path to the model for optimization. If None, empty bytestream will be sent.

get_time_func : Callable[[], float]
Function that returns current timestamp.

Returns
First element is equal to True if optimization finished successfully and the second element contains compiled model.

Return type
Tuple[bool, Optional[bytes]]

request_processing(get_time_func: ~typing.Callable[[], float] = <built-in function perf_counter>) → bool

Requests processing of input data and waits for acknowledgement.

This method triggers inference on target device and waits until the end of inference on target device is reached.

This method measures processing time on the target device from the level of the host.

Target may send its own measurements in the statistics.

Parameters

get_time_func : Callable[[], float]
Function that returns current timestamp.

Returns
True if inference finished successfully.

Return type
bool

serve(upload_input_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_model_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, process_input_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, download_output_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, download_stats_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, upload_iospec_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_optimizers_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_unoptimized_model_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, download_optimized_model_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, upload_runtime_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**)

Waits for requests from the other device (the client) and calls appropriate callbacks. Some callbacks take bytes as arguments, all return ServerStatus and some also return bytes. These responses are then sent back to the client.

Callbacks are guaranteed to be executed in the order requests were sent in.

This method is non-blocking.

Parameters

upload_input_callback : Optional[ServerUploadCallback]
Called, when the client uploads input ('upload_input' method below). Should upload model input into the runtime.

upload_model_callback : Optional[ServerUploadCallback]
Called. when the client uploads optimized model ('upload_model' method below). It should load the model and start inference session.

process_input_callback : Optional[ServerUploadCallback]
Called, when the client requests inference. Should return after inference is completed.

download_output_callback : Optional[ServerDownloadCallback]
Called, when the client requests inference output, should return status of the server and the output.

download_stats_callback : Optional[ServerDownloadCallback]
Called, when the client requests inference stats, should end the inference session and return status of the server and stats.

upload_iospec_callback : Optional[ServerUploadCallback]
Called to upload model input/output specifications (iospec).

upload_optimizers_callback : Optional[ServerUploadCallback]
Called to upload optimizer config (serialized JSON) - 'upload_optimizers' method call by the client.

upload_unoptimized_model_callback : Optional[ServerUploadCallback]
Called to upload an unoptimized ML model, should save it.

download_optimized_model_callback : Optional[ServerDownloadCallback]
Called, when client requests optimization of the model (uploaded with 'upload_unoptimized_model_callback'). Should optimize the model and return it.

upload_runtime_callback : Optional[ServerUploadCallback]
Called, when the client uploads runtime.

start_sending_logs()

Starts sending over the protocol all logs, except for the logs generated by the operation of sending logs itself (otherwise we would have an infinite logging recursion).

stop_sending_logs()

Stops sending logs over the protocol.

abstractmethod transmit(message_type: *MessageType*, payload: *bytes* | *None* = **None**, flags: *list[TransmissionFlag]* = [], failure_callback: *Callable[[MessageType], None]* | *None* = **None**)

Sends bytes and a set of flags to the other side, without blocking the current thread.

Parameters

message_type : MessageType
Value denoting what is being sent. It serves to differentiate one transmission from another (analogous to port number in a TCP protocol).

payload : Optional[bytes]
Bytes to send (or None if the payload is to be empty).

flags : List[TransmissionFlag]
A list of flags to be sent (available flags in the TransmissionFlag enum above - please note that some flags are only allowed for a specific message type).

failure_callback : Optional[ProtocolFailureCallback]
Function, that will be called if the transmission fails to send. Note: This will be executed on a separate thread, so make sure the function is thread-safe.

abstractmethod transmit_blocking(message_type: *MessageType*, payload: *bytes* | *None* = **None**, flags: *list[TransmissionFlag]* = [], timeout: *float* | *None* = **None**, failure_callback: *Callable[[MessageType], None]* | *None* = **None**)

Sends bytes and a set of flags to the other side, blocks the current thread until the transmission is completed.

Parameters

message_type : MessageType
Value denoting what is being sent. It serves to differentiate one transmission from another (analogous to port number in a TCP protocol).

payload : Optional[bytes]
Bytes to send (or None if the payload is to be empty).

flags : List[TransmissionFlag]
A list of flags to be sent (available flags in the TransmissionFlag enum above - please note that some flags are only allowed for a specific message type).

timeout : Optional[float]
Maximum blocking time in seconds, or None to block indefinitely. If

that time passes the 'failure_callback' will be called.

failure_callback : Optional[ProtocolFailureCallback]
Function, that will be called if the transmission fails to send.

upload_input(data: *bytes*) → bool

Uploads input to the target device and waits for acknowledgement.

This method should wait until the target device confirms the data is delivered and preprocessed for inference.

Parameters

data : Any
Input data for inference.

Returns
True if ready for inference.

Return type
bool

upload_io_specification(path: *Path | None*) → bool

Uploads input/output specification to the target device.

This method takes the specification in a json format from the given Path and sends it to the target device.

This method should receive the status of uploading the data to the target.

Parameters

path : Optional[Path]
Path to the json file. If None, empty bytestream will be sent.

Returns
True if data upload finished successfully.

Return type
bool

upload_model(path: *Path | None*) → bool

Uploads the model to the target device.

This method takes the model from given Path and sends it to the target device.

This method should receive the status of uploading the model from the target.

Parameters

path : Optional[Path]
Path to the model. If None, empty bytestream will be sent.

Returns
True if model upload finished successfully.

Return type
bool

upload_optimizers(optimizers_cfg: *dict[str, Any]*) → bool

Upload optimizers config to the target device.

Parameters

optimizers_cfg : Dict[str, Any]
Config JSON of optimizers.

Returns
True if data upload finished successfully.

Return type
bool

upload_runtime(path: *Path* | *None*) → bool

Uploads the runtime to the target device.

This method takes the binary from given Path and sends it to the target device.

This method should receive the status of runtime loading from the target.

Parameters

path : Optional[Path]
Path to the runtime binary. If None, empty bytestream will be sent.

Returns
True if runtime upload finished successfully.

Return type
bool

17.2 KenningProtocol

KenningProtocol is an abstract class implementing the abstract methods of *BytesBasedProtocol* (so methods such as `request`, `transmit_blocking`) using abstract methods *receive_data* and *send_data* (which have to be provided by implementations of this class).

Therefore, it uses a data link that allows for sending/receiving raw bytes and builds a protocol over it that allows for asynchronous, simultaneous communication through multiple channels (each `MessageType` is essentially a separate channel).

17.2.1 KenningProtocol implementations

- *NetworkProtocol*
- *UARTProtocol*

17.2.2 Message structure

Size	6 bits	2 bits	8 bits	16 bits	0 or 32 bits	0-n bits
Offset	0-5	6-7	8-15	16-31	32-63	64
Field	Msg Type	Flow control	Checksum	Flags	Payload size	Payload

17.2.3 Flow control values

- `REQUEST` - message is a request for transmission
- `REQUEST_RETRANSMIT`- message is a request to repeat the last message (because of invalid checksum)
- `ACKNOWLEDGE` - message is an acknowledgement of the last message
- `TRANSMISSION` - message is part of a transmission

17.2.4 Checksum

Bit-wise XOR of all bytes in the message (except for the checksum itself) with `0x4B` byte.

17.2.5 Flags

Flag	Offset in the `Flags` field	Meaning for `REQUEST` and `TRANSMISSION` messages	Meaning for `ACKNOWLEDGE` messages
`SUCCESS`	0	can be set/read as a `TransmissionFlag`	last message was received and accepted
`FAIL`	1	can be set/read as a `TransmissionFlag`	last message was received, but it was rejected (`ACKNOWLEDGE` with a `FAIL` flag is used to deny a request or reject a transmission)
`IS_HOST_MES`	2	can be set/read as a `TransmissionFlag`	-
`HAS_PAYLOAD`	3	message has payload	-
`FIRST`	4	first message in this transmission/request	-
`LAST`	5	last message in this transmission/request (it is possible for a message to be both `FIRST` and `LAST`, if the transmission/request only has a single message	-
`IS_KENNING`	6	can be set/read as a `TransmissionFlag`	-
`IS_ZEPHYR`	7	can be set/read as a `TransmissionFlag`	-
`SERIALIZED` (only for `MessageType.IO_SPEC`)	12	can be set/read as a `TransmissionFlag`	-

As you can see, certain flags are used by `KenningProtocol` for flow-control and protocol logic. Other flags are available to the protocol user (like `BytesBasedProtocol` class), as `TransmissionFlag`. And with some flags, that depends on what message it is. Some flags

are only present for a specific message type (these are carried in the last 4 bits of the `Flags` field).

17.2.6 Payload size

Size of the payload in bytes.

17.2.7 Example communication scenarios

> **Note**
>
> Transmission flags are omitted.

Request (no payload) denied:

- A -> B `MessageType.OUTPUT`, `REQUEST`, (`FIRST`, `LAST`)
- A <- B `MessageType.OUTPUT`, `ACKNOWLEDGE`, (`FAIL`)

Unprompted single-message transmission:

- A -> B `MessageType.LOGS`, `TRANSMISSION`, (`HAS_PAYLOAD`, `FIRST`, `LAST`)

Request (no payload) with a multi-message transmission as a response:

- A -> B `MessageType.STATS`, `REQUEST`, (`FIRST`, `LAST`)
- A <- B `MessageType.STATS`, `TRANSMISSION`, (`HAS_PAYLOAD`, `FIRST`)
- A <- B `MessageType.STATS`, `TRANSMISSION`, (`HAS_PAYLOAD`)
- A <- B `MessageType.STATS`, `TRANSMISSION`, (`HAS_PAYLOAD`)
- A <- B `MessageType.STATS`, `TRANSMISSION`, (`HAS_PAYLOAD`, `LAST`)

Multi-message request with payload with a single-message transmission as a response:

- A -> B `MessageType.IO_SPEC`, `REQUEST`, (`HAS_PAYLOAD`, `FIRST`)
- A -> B `MessageType.IO_SPEC`, `REQUEST`, (`HAS_PAYLOAD`)
- A -> B `MessageType.IO_SPEC`, `REQUEST`, (`HAS_PAYLOAD`, `LAST`)
- A <- B `MessageType.STATUS`, `TRANSMISSION`, (`HAS_PAYLOAD`, `FIRST`, `LAST`)

17.2.8 Methods

class kenning.protocols.kenning_protocol.**KenningProtocol**(timeout: *int* = -1, error_recovery: *bool* = **False**, max_message_size: *int* = 1024)

Bases: `BytesBasedProtocol`, `ABC`

Class for managing the flow of Kenning Protocol.

event_active(message_type: *MessageType* | *None* = **None**) → bool

Checks if an active protocol event (Transmission, Request etc.) of a given message type exists.

Parameters

message_type : Optional[MessageType]
Message type to check, or None (which will check for an event, that accepts all message types).

Returns
True if event exists, False otherwise.

Return type
bool

finish_event(event: *ProtocolEvent*, log_failure: *bool* = **True**)

Removes an event from the 'current_protocol_events' dict and logs it's success or failure.

Parameters

event : ProtocolEvent
Event to finish

log_failure : bool
If true, in case of event failure an error will be logged (used to suppress errors, when the event is being re-tried).

kill_event(message_type: *MessageType* | *None* = **None**)

Forcibly stops an active event (Transmission, Request etc.).

Parameters

message_type : Optional[MessageType]
Message type to check, or None (which will stop an event, that accepts all message types).

Raises
ValueError – There is no such event.

listen(message_type: *MessageType* | *None* = **None**, transmission_callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, request_callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, limit: *int* = -1, failure_callback: *Callable[[MessageType], None]* | *None* = **None**)

Waits for transmissions and requests from the other side, without blocking the current thread.

Parameters

message_type : Optional[MessageType]
Message type of the requests/transmissions to listen for, or None (to listen to requests/transmissions of any message type).

transmission_callback : Optional[ProtocolSuccessCallback]
Function, that will be called when a transmission is successfully received. Message type, payload and flags from the transmission will be passed to the function. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

request_callback : Optional[ProtocolSuccessCallback]
Function, that will be called when a request is successfully received.

Message type, payload and flags from the request will be passed to the function. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

limit : int
Meximum number of requests/transmissions (including failures), that will be received before listening stops.

failure_callback : Optional[ProtocolFailureCallback]
Function, that will be called if a request/transmission is attempted by the other side, but fails. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

listen_blocking(message_type: *MessageType* | *None* = **None**, transmission_callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, request_callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, timeout: *float* | *None* = **None**, failure_callback: *Callable[[MessageType], None]* | *None* = **None**) → tuple[IncomingEventType | None, MessageType | None, bytes | None, list[TransmissionFlag] | None]

Blocks the current thread until a transmission or a request is received.

Parameters

message_type : Optional[MessageType]
Message type of the requests/transmissions to listen for, or None (to listen to requests/transmissions of any message type).

transmission_callback : Optional[ProtocolSuccessCallback]
Function, that will be called when a transmission is successfully received. Message type, payload and flags from the transmission will be passed to the function.

request_callback : Optional[ProtocolSuccessCallback]
Function, that will be called when a request is successfully received. Message type, payload and flags from the request will be passed to the function.

timeout : Optional[float]
Maximum blocking time in seconds, or None to block indefinitely. If that time passes the 'deny_callback' will be called.

failure_callback : Optional[ProtocolFailureCallback]
Function, that will be called if a request/transmission is attempted by the other side, but fails. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

Returns
Enum denoting whether a request or a transmission was received, message type of the received tranmsission/request, received payload, received flags. Alternatively: (None, None, None, None) if no request/transmission was received in the specified timeout or if a request/transmission was attempted but failed.

Return type
tuple[Optional[IncomingEventType], Optional[MessageType], Op-

tional[bytes], Optional[List[TransmissionFlag]]]

abstractmethod receive_data(timeout: *float* | *None*) → bytes | None

Receives data from the target device.

Parameters

timeout : Optional[float]
Maximum time waiting for data in seconds (None - infinite timeout).

Returns
Received bytes, or None if no bytes were received.

Return type
Optional[bytes]

receive_message(timeout: *float* | *None* = **None**) → Message | None

Waits for incoming data from the other side of connection.

This method should wait for the input data to arrive and return the appropriate status code along with received data.

Parameters

timeout : Optional[float]
Receive timeout in seconds. If timeout > 0, this specifies the maximum wait time, in seconds. If timeout <= 0, the call won't block, and will report the currently ready file objects. If timeout is None, the call will block until a monitored file object becomes ready.

Returns
Received message, or None if message was not received.

Return type
Optional[Message]

receiver()

Method sitting in a loop, receiving messages and passing them to the relevant ProtocolEvent class object in the 'current_protocol_events' dict, based on message type.

This method is meant to be running constantly on a separate thread, if the protocol is active.

request(message_type: *MessageType*, callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]*, payload: *bytes* | *None* = **None**, flags: *list[TransmissionFlag]* = [], retry: *int* = 1, deny_callback: *Callable[[MessageType], None]* | *None* = **None**)

Prompts the other side for a transmission and waits for a response, without blocking the current thread. Bytes and flags can also be sent along with the request.

Parameters

message_type : MessageType
Value denoting what type of transmission is being requested.

callback : ProtocolSuccessCallback
Function, that will be called when the transmission is received. Message type, payload and flags from the transmission will be passed to

the function. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

payload : Optional[bytes]
Bytes to send along with the request (or None if the payload is to be empty).

flags : List[TransmissionFlag]
A list of flags to be sent (available flags in the TransmissionFlag enum above - please note that some flags are only allowed for a specific message type).

retry : int
Denotes how many times the request will be re-sent after failing, before calling 'deny_callback'. Negative number denotes infinite retries.

deny_callback : Optional[ProtocolFailureCallback]
Function, that will be called if the request is denied or otherwise fails. Note: This will be executed on a separate thread, so make sure the callback function is thread-safe.

request_blocking(message_type: *MessageType*, callback: *Callable[[MessageType, bytes, list[TransmissionFlag]], None]* | *None* = **None**, payload: *bytes* | *None* = **None**, flags: *list[TransmissionFlag]* = [], timeout: *float* | *None* = **None**, retry: *int* = 1, deny_callback: *Callable[[MessageType], None]* | *None* = **None**) → tuple[bytes | None, list[TransmissionFlag] | None]

Prompts the other side for a transmission and blocks the current thread until a response is received. Bytes and flags can also be sent along with the request.

Parameters

message_type : MessageType
Value denoting what type of transmission is being requested.

callback : Optional[ProtocolSuccessCallback]
Function, that will be called when the transmission is received. Message type, payload and flags from the transmission will be passed to the function.

payload : Optional[bytes]
Bytes to send along with the request (or None if the payload is to be empty).

flags : List[TransmissionFlag]
A list of flags to be sent (available flags in the TransmissionFlag enum above - please note that some flags are only allowed for a specific message type).

timeout : Optional[float]
Maximum blocking time in seconds, or None to block indefinitely. If that time passes the 'deny_callback' will be called.

retry : int
Denotes how many times the request will be re-sent after failing, before calling 'deny_callback'. Negative number denotes infinite retries.

deny_callback : Optional[ProtocolFailureCallback]
Function, that will be called if the request is denied or otherwise fails.

Returns
Payload and flags received as response to the request, or (None, None) if the request was denied or otherwise failed.

Return type
Tuple[Optional[bytes], Optional[List[TransmissionFlag]]]

run_event(event: *ProtocolEvent*, success_callback: *Callable[[ProtocolEvent | IncomingEvent], None]*, deny_callback: *Callable[[ProtocolEvent], None]*)

Adds a protocol event to the 'current_protocol_events' dict, so that it will be serviced by the receiver thread, starts it in non-blocking mode and passes callbacks.

Parameters

event : ProtocolEvent
Event to start.

success_callback : Callable[[ProtocolEvent | IncomingEvent], None]
Function, that will be called if the event succeeds.

deny_callback : Callable[[ProtocolEvent], None]
Function, that will be called if the event fails.

Raises

- **ValueError** – Attempted to start the event, while another event of the same type was already in progress.
- **ProtocolNotStartedError** – Protocol is not active, call 'start' first.

run_event_blocking(event: *ProtocolEvent*, timeout: *float | None*) → tuple[bool, ProtocolEvent]

Adds a protocol event to the 'current_protocol_events' dict, so that it will be serviced by the receiver thread, starts it in blocking mode and blocks the current thread until it completes.

Parameters

event : ProtocolEvent
Event to start.

timeout : Optional[float]
Maximum blocking time in seconds (or None for infinite timeout).

Returns
True if event succeeded, False if not, ProtocolEvent object returned by the event (see the dostring of the relevant ProtocolEvent to see what object will be passed here).

Return type
Tuple[bool, ProtocolEvent]

Raises

- **ValueError** – Attempted to start the event, while another event of the same type was already in progress.

- **ProtocolNotStartedError** – Protocol is not active, call 'start' first.

abstractmethod send_data(data: *bytes*) → bool

Sends data to the target device.

Data can be model to use, input to process, additional configuration.

Parameters

data : bytes
Data to send.

Returns
True if successful.

Return type
bool

send_message(message: *Message*) → bool

Sends message to the target device.

Parameters

message : Message
Message to be sent.

Returns
True if succeeded.

Return type
bool

send_messages(message_type: *MessageType*, messages: *list[Message]*)

Gives a job to the 'transmitter' thread pool, that will send messages and call 'messages_sent' method on the appropriate ProtocolEvent object in the 'self.current_protocol_events' dict (based on message type).

This is meant be be called by the ProtocolEvent objects to send messages and update their state.

Parameters

message_type : MessageType
Message type of the ProtocolEvent, that sends the messages. This is needed, so that we can later find that object in the 'self.current_protocol_events' dict and call 'messages_sent'. We could not use a reference here, because for example if a Listen object in the dict has an IncomingTransmission object inside, and that inner object calls this method, we need to call 'message_sent' on the whole Listen object, not just the inner one.

messages : List[Message]
List of messages to send.

start(callback_runner_sharing_protocol: KenningProtocol | *None* = **None**)

Starts the protocol, creates and runs threads for receiving and transmitting messages, as well as a thread for execution of callback functions in non-blocking protocol events.

KenningProtocol normally creates a thread for executing callback functions (so called ‘callback runner’). Optionally, multiple KenningProtocol instances can share that thread (for example to make sure that their callbacks are not executed concurrently, if they’re not thread-safe). To do that, the *start* method should be called on instance 1 with *callback_runner_sharing_protocol* argument set to None, and then the method should be called on other instances, with instance 1 passed as *callback_runner_sharing_protocol*.

Example:

``` kp1 = KenningProtocol() kp2 = KenningProtocol() kp3 = KenningProtocol()

# These three instances will share the same callback runner: kp1.start() kp2.start(kp1) kp3.start(kp2)

# Instances do stuff here...

# This will not affect kp1 or kp3: kp2.stop()

# This will also not affect kp3: kp1.stop() ```

**Parameters**

**callback_runner_sharing_protocol : Optional[*KenningProtocol*]**
The other protocol instance, that the callback runner is to be shared with. The other instance must already be running.

**Raises**

**ProtocolNotStartedError** – A KenningProtocol instance was passed to share the callback runner with, but it was not running.

**stop**()

Stops the protocol and joins all threads.

**transmit**(message_type: *MessageType*, payload: *bytes | None* = **None**, flags: *list[TransmissionFlag]* = [], failure_callback: *Callable[[MessageType], None] | None* = **None**)

Sends bytes and a set of flags to the other side, without blocking the current thread.

**Parameters**

**message_type : MessageType**
Value denoting what is being sent. It serves to differentiate one transmission from another (analogous to port number in a TCP protocol).

**payload : Optional[bytes]**
Bytes to send (or None if the payload is to be empty).

**flags : List[TransmissionFlag]**
A list of flags to be sent (available flags in the TransmissionFlag enum above - please note that some flags are only allowed for a specific message type).

**failure_callback : Optional[ProtocolFailureCallback]**
Function, that will be called if the transmission fails to send. Note: This will be executed on a separate thread, so make sure the function is thread-safe.
```

transmit_blocking(message_type: *MessageType*, payload: *bytes | None* = **None**, flags: *list[TransmissionFlag]* = [], timeout: *float | None* = **None**, failure_callback: *Callable[[MessageType], None] | None* = **None**)

Sends bytes and a set of flags to the other side, blocks the current thread until the transmission is completed.

Parameters

message_type : MessageType
Value denoting what is being sent. It serves to differentiate one transmission from another (analogous to port number in a TCP protocol).

payload : Optional[bytes]
Bytes to send (or None if the payload is to be empty).

flags : List[TransmissionFlag]
A list of flags to be sent (available flags in the TransmissionFlag enum above - please note that some flags are only allowed for a specific message type).

timeout : Optional[float]
Maximum blocking time in seconds, or None to block indefinitely. If that time passes the 'failure_callback' will be called.

failure_callback : Optional[ProtocolFailureCallback]
Function, that will be called if the transmission fails to send.

17.3 NetworkProtocol

NetworkProtocol implements abstract methods from *KenningProtocol* (*send_data* and *receive_data*), as well as abstract methods from *Protocol*, that neither *KenningProtocol* nor *BytesBasedProtocol* implemented (so *disconnect*, *initialize_client*, and *initialize_server*).

It uses Python sockets.

After calling `initialize_server` it waits for a client to connect and creates a socket. Only one client at a time can be connected.

When a client connects, the server sends a single byte (`0x00`) to the client, as a confirmation and test of the connection.

The client does not respond.

17.3.1 Methods

class kenning.protocols.network.**NetworkProtocol**(host: *str* = 'localhost', port: *int* = 12345, timeout: *int* = -1, error_recovery: *bool* = **False**, packet_size: *int* = 4096, max_message_size: *int* = 1048576, multi_client: *bool* = **False**)

Bases: KenningProtocol

A TCP-based protocol.

Protocol is implemented using BSD sockets and selectors-based pooling.

accept_client(timeout: *float* | *None*) → bool

Waits (blocking the current thread) on the new client and accepts or rejects the connection.

Parameters

timeout : Optional[float]
Maximum time waiting for the client in seconds. None means infinite timeout.

Returns
True if client connected successfully, False otherwise.

Return type
bool

Raises
ProtocolNotStartedError – Server socket was not initialized.

disconnect()

Ends connection with the other side.

initialize_client() → bool

Initializes client side of the protocol.

The client side is supposed to run on host testing the target hardware.

The parameters for the client should be provided in the constructor.

Returns
True if succeeded.

Return type
bool

initialize_server(client_connected_callback: *Callable[[], None]* | *None* = **None**, client_disconnected_callback: *Callable[[], None]* | *None* = **None**) → bool

Initializes server side of the protocol.

The server side is supposed to run on target hardware.

The parameters for the server should be provided in the constructor.

Parameters

client_connected_callback : Optional[Callable[Any, None]]
Called when a client connects to the server. Either IP address or another distinguishing characteristic of the client will be passed to the callback (depending on the underlying protocol).

client_disconnected_callback : Optional[Callable[None, None]]
Called, when the current client disconnects from the server.

Returns
True if succeeded.

Return type
bool

receive_data(timeout: *float* | *None*) → bytes | None

Receives data from the target device.

Parameters

timeout : Optional[float]
Maximum time waiting for data in seconds (None - infinite timeout).

Returns
Received bytes, or None if no bytes were received.

Return type
Optional[bytes]

send_data(data: *bytes*) → bool

Sends data to the target device.

Data can be model to use, input to process, additional configuration.

Parameters

data : bytes
Data to send.

Returns
True if successful.

Return type
bool

17.4 UARTProtocol

UARTProtocol implements abstract methods from *KenningProtocol* (*send_data* and *receive_data*), as well as abstract methods from *Protocol*, that neither *KenningProtocol* nor *BytesBasedProtocol* implemented (so *disconnect* and *initialize_client*).

The *initialize_server* method raises `kenning.core.exceptions.NotSupportedError`, since Kenning Server working on UART is not supported.

When initializing, the client sends a `MessageType.PING` request with `TransmissionFlag.SUCCESS`, as a test of the connection and a signal starting a session.

The server responds with a `MessageType.PING` transmission with `TransmissionFlag.SUCCESS` if it accepts the connection and `TransmissionFlag.FAIL` if it does not accept the connection (for example because a client is already connected).

When disconnecting, the client sends a `MessageType.PING` request with `TransmissionFlag.FAIL`.

The server responds with a `MessageType.PING`, `TransmissionFlag.SUCCESS` transmission.

The `UARTProtocol` class overrides some of the `kenning.core.protocol.Protocol` abstract methods, that are already implemented by `BytesBasedProtocol`, in order to change their behaviour:

- *download_statistics* - It expects the statistics to arrive in a custom format, instead of a serialized JSON. Statistics are parsed by the `kenning.protocols.uart._parse_stats` function.

- *upload_io_specification* - Instead of sending the input/output specification as a JSON serialized to string, it sends it serialized to a custom format that enables easy mapping of the binary data to a packed C struct. The format is defined in the form of a dict (`kenning.interfaces.io_spec_serializer.IOSPEC_STRUCT_FIELDS`).

17.4.1 Methods

17.5 ROS2Protocol

ROS2Protocol is an implementation based on the ROS 2 framework.

It does not support server-side API methods (*initialize_server*, *serve*), target-side model optimization (*upload_optimizers*, *request_optimization*), dynamic runtime changes (*upload_runtime*) or sending/receiving logs (*listen_to_server_logs*, *start_sending_logs*, *stop_sending_logs*).

17.5.1 Methods

class kenning.protocols.ros2.**ROS2Protocol**(process_action_type_str: *str*, process_action_name: *str*, model_service_type_str: *str* = 'std_srvs.srv.Trigger', model_service_name: *str* = 'cvnode_prepare', measurements_service_type_str: *str* = 'std_srvs.srv.Trigger', measurements_service_name: *str* = 'cvnode_measurements', timeout: *int* = -1)

Bases: Protocol

A ROS2-based runtime protocol for communication. It supports only a client side of the runtime protocol.

Protocol is implemented using ROS2 services and action.

deduce_data_converter_from_io_spec(io_specification: *dict* | *Path* | *None* = **None**) → ROS2DataConverter

Function that reads model IO specification and then tries to deduce optimal data converter type.

Parameters

io_specification : Optional[Union[Dict, Path]]
Dictionary with IO specification or Path to file with it.

Returns
Optimal DataConverter type.

Return type
DataConverter

disconnect()

Ends connection with the other side.

download_output() → tuple[bool, GoalHandle | None]

Gets result from processing goal from the target ROS2 node.

Returns
Tuple with status (True if successful) and received processing goal handle result.

Return type
Tuple[bool, Optional[GoalHandle]]

download_statistics(final: *bool* = **False**) → *Measurements*

Downloads inference statistics from the target device.

By default no statistics are gathered.

Parameters

final : bool
If the inference is finished

Returns
Inference statistics on target device.

Return type
Measurements

initialize_client()

Initializes client side of the protocol.

The client side is supposed to run on host testing the target hardware.

The parameters for the client should be provided in the constructor.

Returns
True if succeeded.

Return type
bool

initialize_server(client_connected_callback: *Callable[[Any], None]* | *None* = **None**, client_disconnected_callback: *Callable[[None], None]* | *None* = **None**) → bool

Initializes server side of the protocol.

The server side is supposed to run on target hardware.

The parameters for the server should be provided in the constructor.

Parameters

client_connected_callback : Optional[Callable[Any, None]]
Called when a client connects to the server. Either IP address or another distinguishing characteristic of the client will be passed to the callback (depending on the underlying protocol).

client_disconnected_callback : Optional[Callable[None, None]]
Called, when the current client disconnects from the server.

Returns
True if succeeded.

Return type
bool

listen_to_server_logs()

Starts continuously receiving and printing logs sent by the server.

listen_to_trace_data(tracedump_callback: *Callable[[bytes], None]*)

Starts continuously receiving batches of trace data from server. Calls a callback function with each received batch.

Parameters

tracedump_callback : Callable[[bytes], None]
Function, that will be called for each received batch of data.

request_optimization(model_path: ~pathlib.Path | None, get_time_func: ~typing.Callable[[], float] = <built-in function perf_counter>) → tuple[bool, bytes | None]

Request optimization of model.

Parameters

model_path : Optional[Path]
Path to the model for optimization. If None, empty bytestream will be sent.

get_time_func : Callable[[], float]
Function that returns current timestamp.

Returns
First element is equal to True if optimization finished successfully and the second element contains compiled model.

Return type
Tuple[bool, Optional[bytes]]

request_processing(get_time_func: ~typing.Callable[[], float] = <built-in function perf_counter>) → bool

Requests processing of input data and waits for acknowledgement.

This method triggers inference on target device and waits until the end of inference on target device is reached.

This method measures processing time on the target device from the level of the host.

Target may send its own measurements in the statistics.

Parameters

get_time_func : Callable[[], float]
Function that returns current timestamp.

Returns
True if inference finished successfully.

Return type
bool

serve(upload_input_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_model_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, process_input_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, download_output_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, download_stats_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, upload_iospec_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_optimizers_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_unoptimized_model_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, download_optimized_model_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, upload_runtime_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**)

Waits for requests from the other device (the client) and calls appropriate callbacks. Some callbacks take bytes as arguments, all return ServerStatus and some also return bytes. These responses are then sent back to the client.

Callbacks are guaranteed to be executed in the order requests were sent in.

This method is non-blocking.

Parameters

upload_input_callback : Optional[ServerUploadCallback]
Called, when the client uploads input ('upload_input' method below). Should upload model input into the runtime.

upload_model_callback : Optional[ServerUploadCallback]
Called. when the client uploads optimized model ('upload_model' method below). It should load the model and start inference session.

process_input_callback : Optional[ServerUploadCallback]
Called, when the client requests inference. Should return after inference is completed.

download_output_callback : Optional[ServerDownloadCallback]
Called, when the client requests inference output, should return status of the server and the output.

download_stats_callback : Optional[ServerDownloadCallback]
Called, when the client requests inference stats, should end the inference session and return status of the server and stats.

upload_iospec_callback : Optional[ServerUploadCallback]
Called to upload model input/output specifications (iospec).

upload_optimizers_callback : Optional[ServerUploadCallback]
Called to upload optimizer config (serialized JSON) - 'upload_optimizers' method call by the client.

upload_unoptimized_model_callback : Optional[ServerUploadCallback]
Called to upload an unoptimized ML model, should save it.

download_optimized_model_callback : Optional[ServerDownloadCallback]
Called, when client requests optimization of the model (uploaded with 'upload_unoptimized_model_callback'). Should optimize the model and return it.

upload_runtime_callback : Optional[ServerUploadCallback]
Called, when the client uploads runtime.

start_sending_logs()

Starts sending over the protocol all logs, except for the logs generated by the operation of sending logs itself (otherwise we would have an infinite logging recursion).

stop_sending_logs()

Stops sending logs over the protocol.

upload_input(data: *GoalHandle*) → bool

Initializes processing goal and sends it to the action server.

Parameters

data : GoalHandle
Goal to be sent to the action server containing the input data. Should be a ROS2 action goal compatible with the action server.

Returns
True if the message was sent successfully, False otherwise.

Return type
bool

upload_io_specification(path: *Path* | *None*) → bool

Uploads input/output specification to the target device.

This method takes the specification in a json format from the given Path and sends it to the target device.

This method should receive the status of uploading the data to the target.

Parameters

path : Optional[Path]
Path to the json file. If None, empty bytestream will be sent.

Returns
True if data upload finished successfully.

Return type
bool

upload_model(path: *Path* | *None*) → bool

Uploads the model to the target device.

This method takes the model from given Path and sends it to the target device.

This method should receive the status of uploading the model from the target.

Parameters

path : Optional[Path]
Path to the model. If None, empty bytestream will be sent.

Returns
True if model upload finished successfully.

Return type
bool

upload_optimizers(optimizers_cfg: *dict[str, Any]*) → bool

Upload optimizers config to the target device.

Parameters

optimizers_cfg : Dict[str, Any]
Config JSON of optimizers.

Returns
True if data upload finished successfully.

Return type
bool

upload_runtime(path: *Path | None*) → bool

Uploads the runtime to the target device.

This method takes the binary from given Path and sends it to the target device.

This method should receive the status of runtime loading from the target.

Parameters

path : Optional[Path]
Path to the runtime binary. If None, empty bytestream will be sent.

Returns
True if runtime upload finished successfully.

Return type
bool

CHAPTER

EIGHTEEN

KENNING PLATFORMS

Kenning platforms were created to automatically provide board-specific arguments and parameters required for various stages of Kenning pipelines. It is used to:

- Provide information on platforms' constraints, such as RAM size
- Provide information for optimizing the model using compilers such as TVM or IREE
- Provide information on supported target runtime (Linux-based, bare metal or Zephyr-based)
- Provide information on building the target evaluation application
- Provide information on interfacing with the target platform (e.g. via UART)

18.1 Specification

Platforms' definitions should be written in YAML or JSON file representing a dictionary, where:

- keys are the unique boards' IDs
- values are dictionaries with board-specific parameters.

The example specification of platforms looks as follows:

```
max32690evkit/max32690/m4:
  display_name: MAX32690 Evaluation Kit
  flash_size_kb: 3072
  ram_size_kb: 128
  uart_port_wildcard: /dev/serial/by-id/usb-FTDI_FT231X_USB_UART_*-if00-port0
  uart_baudrate: 115200
  uart_log_baudrate: 115200
  compilation_flags:
  - "-keys=arm_cpu,cpu"
  - "-device=arm_cpu"
  - "-march=armv7e-m"
  - "-mcpu=cortex-m4"
  - "-model=max32690"
  openocd_flash_cmd:
  - source [find interface/cmsis-dap.cfg]
  - source [find target/max32690.cfg]
  - init
  - targets
```

(continues on next page)

(continued from previous page)

```
  - reset init
  - flash write_image erase {binary_path}
  - reset run
  - shutdown
  platform_resc_path: gh://antmicro:kenning-zephyr-runtime/renode/scripts/
↪max32690evkit.resc;branch=main
  default_platform: ZephyrPlatform
  default_optimizer:
    - LiteRTCompiler
    - TVMCompiler

max32690fthr/max32690/m4:
  display_name: MAX32690FTHR
  flash_size_kb: 3072
  ram_size_kb: 128
  uart_baudrate: 115200
  uart_log_baudrate: 115200
  compilation_flags:
  - "-keys=arm_cpu,cpu"
  - "-device=arm_cpu"
  - "-march=armv7e-m"
  - "-mcpu=cortex-m4"
  - "-model=max32690"
  platform_resc_path: gh://antmicro:kenning-zephyr-runtime/renode/scripts/
↪max32690fthr.resc;branch=main
  default_platform: ZephyrPlatform
  default_optimizer:
    - LiteRTCompiler
    - TVMCompiler

max78002evkit/max78002/m4:
  display_name: MAX78002 Evaluation Kit
  ai8x_device: MAX78002
  ai8x_device_id: 87
  ai8x_weights_memory_kb: 2048
  ai8x_data_memory_kb: 1280
  flash_size_kb: 2560
  ram_size_kb: 64
  uart_port_wildcard: /dev/serial/by-id/usb-FTDI_FT231X_USB_UART_*-if00-port0
  uart_baudrate: 115200
  uart_log_baudrate: 115200
  compilation_flags:
  - "-keys=arm_cpu,cpu"
  - "-device=arm_cpu"
  - "-march=armv7e-m"
  - "-mcpu=cortex-m4"
  - "-model=max78002"
  openocd_flash_cmd:
  - source [find interface/cmsis-dap.cfg]
```

(continues on next page)

(continued from previous page)

```
  - source [find target/max78002.cfg]
  - init
  - targets
  - reset init
  - flash write_image erase {binary_path}
  - reset run
  - shutdown
  default_platform: ZephyrPlatform
  default_optimizer:
    - Ai8xCompiler
    - LiteRTCompiler
    - TVMCompiler

rv32-springbok:
  display_name: RISC-V 32-bit Springbok
  runtime_binary_path: kenning:///renode/springbok/iree_runtime
  platform_resc_path: gh://antmicro:kenning-bare-metal-iree-runtime/sim/config/
↪springbok.resc;branch=main
  resc_dependencies:
  - gh://antmicro:kenning-bare-metal-iree-runtime/sim/config/platforms/springbok.
↪repl;branch=main
  - gh://antmicro:iree-rv32-springbok/sim/config/infrastructure/SpringbokRiscV32.
↪cs;branch=repo-as-submodule
  post_start_commands:
  - sysbus.vec_controlblock WriteDoubleWord 0xc 0
  runtime_init_log_msg: Runtime started
  default_platform: BareMetalPlatform
  default_optimizer:
    - IREECompiler

stm32f746g_disco:
  display_name: STM32F746G Discovery
  flash_size_kb: 1024
  ram_size_kb: 256
  uart_baudrate: 115200
  platform_resc_path: gh://antmicro:kenning-zephyr-runtime/renode/scripts/
↪stm32f746g_disco.resc;branch=main
  default_platform: ZephyrPlatform
  default_optimizer:
    - LiteRTCompiler
    - TVMCompiler

hifive_unmatched/fu740/s7:
  display_name: HiFive Unmatched
  uart_baudrate: 115200
  platform_resc_path: gh://antmicro:kenning-zephyr-runtime/renode/scripts/hifive_
↪unmatched.resc;branch=main
  default_platform: ZephyrPlatform
  default_optimizer:
```

(continues on next page)

(continued from previous page)

```
    - LiteRTCompiler
    - TVMCompiler

nvidia_rtx_4090:
  display_name: NVIDIA RTX 4090
  ram_size_kb: 25153536 # 24564MiB (src:nvidia-smi) * 1024kiB/MiB
  default_platform: CUDAPlatform
  compute_capability: sm_89
  opt_level: 3
  contextname: cuda
  use_tvm_vm: False
  conversion_func: default
  target_host: null
  compilation_flags:
  - "-keys=cuda,gpu"
  - "-libs=cudnn,cublas"
  - "-arch=native"
  - "-max_num_threads=1024"
  - "-max_threads_per_block=1024"
  - "-registers_per_block=65536"
  - "-shared_memory_per_block=49152"
  - "-thread_warp_size=32"
  default_optimizer:
    - TVMCompiler
    - IREECompiler

jetson_thor_devkit:
  display_name: NVIDIA Jetson Thor
  ram_size_kb: 134217728 # 128GB (src:nvidia) * 1024^2 kb/GB
  default_platform: CUDAPlatform
  compute_capability: sm_110
  opt_level: 3
  contextname: cuda
  use_tvm_vm: False
  conversion_func: default
  target_host: aarch64
  compilation_flags:
    - "-keys=cuda,gpu"
    - "-libs=cudnn,cublas"
    - "-arch=native"
    - "-max_num_threads=1024"
    - "-max_threads_per_block=1024"
    - "-registers_per_block=65536"
    - "-shared_memory_per_block=49152"
    - "-thread_warp_size=32"
  default_optimizer:
    - TVMCompiler

coralnpu:
```

(continues on next page)

(continued from previous page)

```
  display_name: CoralNPU
  default_platform: CoralNPUPlatform
  compilation_flags:
    - "--iree-hal-target-device=local"
    - "--iree-hal-local-target-device-backends=llvm-cpu"
    - "--iree-llvmcpu-target-cpu=host"
    - "--iree-hal-target-device=coralnpu"
  default_optimizer:
    - IREECompiler
```

Looking at MAX32690 platform there are fields for:

- TVM compilation (`compilation_flags`),
- flashing to HW (`openocd_flash_cmd`),
- simulation (`platform_resc_path`),
- defining model constrains (`flash_size_kb`, `ram_size_kb`),
- automatically finding its UART port (`uart_port_wildcard`) and specifying its baud rate (`uart_baudrate`, `uart_log_baudrate`),
- choosing the platform type (`default_platform`).

Other parameters with descriptions can be found e.g. in:

- `kenning.platforms.bare_metal.BareMetalPlatform` encapsulating platforms running bare metal runtime
- `kenning.platforms.zephyr.ZephyrPlatform` encapsulating platforms running Kenning Zephyr Runtime

18.2 Generating definitions for boards supported by Zephyr RTOS

Kenning provides a `generate-platforms` subcommand that allows to generate definitions of platforms based on given sources. Currently, the only supported sources of information are Zephyr RTOS device trees.

In order to generate definitions, fully set-up Zephyr repository with SDK is required. For detailed information on how to set up the SDK, follow Zephyr documentation.

Based on Docker image for Kenning Zephyr Runtime, it boils down to:

```
docker run --rm -it -v $(pwd):$(pwd) -w $(pwd) ghcr.io/antmicro/kenning-zephyr-
↪runtime:latest /bin/bash
# Get the latest release of Kenning Zephyr Runtime
git clone https://github.com/antmicro/kenning-zephyr-runtime.git
cd kenning-zephyr-runtime/
# Install zephyr SDK and other dependencies
python3 -m venv .venv --system-site-packages
source .venv/bin/activate
pip install pip setuptools --upgrade
python -m west init -l .
python -m west update
```

(continues on next page)

(continued from previous page)

```
python -m west zephyr-export
pip install -r requirements.txt -r ../zephyr/scripts/requirements-base.txt
python -m west sdk install --toolchains x86_64-zephyr-elf aarch64-zephyr-elf arm-
↪zephyr-eabi riscv64-zephyr-elf
pip3 install 'kenning[zephyr] @ git+https://github.com/antmicro/kenning.git'
```

Note

Using existing Zephyr repository can be achieved by exporting `ZEPHYR_BASE` variable or providing `--zephyr-base` flag to the script. It can potentially require additional dependencies from `kenning[zephyr]`.

Platform generation can be triggered with a following command:

```
kenning generate-platforms zephyr \
  --zephyr-base ../zephyr \
  --platforms ./platforms.yml
```

The script:

- generates flat device trees using Zephyr CMake rules,
- parses received device trees with dts2repl,
- tries to find baudrate based on chosen console output,
- tries to find compilation flags for ARM CPUs based on their names (from `compatible` parameter) and architectures returned by GCC for ARM (`arm-zephyr-eabi-gcc` and `aarch64-zephyr-elf-gcc` from Zephyr SDK),
- calculates size of the flash based on chosen flash registers,
- calculates size of the RAM based on chose sram and Zephyr's memory-regions,
- finds URL to the board image.

To use newly generated platforms, provide its path to the `--platform-definitions` flag in `kenning` command or directly in the scenario:

```
platform:
  type: ZephyrPlatform
  parameters:
    # Chooses the file with platform definition
    platforms_definitions: [./platforms.yml]
    # Chooses MAX32690 Evaluation Kit
    name: max32690evkit/max32690/m4
```

CHAPTER

NINETEEN

INTEGRATION WITH ROS 2

This example demonstrates the integration of Kenning with ROS 2 nodes and communication infrastructure. Kenning can be used together with ROS 2 for:

- Evaluation of ROS 2 nodes and subsystems in terms of performance and quality
- Running AI models supported by Kenning and exposing topics/services for accessing them from ROS 2 nodes
- Delegating evaluation of models to remote target devices using ROS 2 communication

19.1 Dependencies

To run this scenario, you will need:

1. Software:
    - ROS 2 Jazzy environment
    - OpenCV for image processing
    - Apache TVM for model optimization and runtime (**when built with the proper argument, see below**)
    - CUDNN and CUDA libraries for NVIDIA GPU support (**if you want to use a GPU**)
    - Docker to use a prepared environment (**optional**)
    - nvidia-container-toolkit to provide access to the GPU in the Docker container (**optional**)
2. Hardware:
    - A CUDA-enabled NVIDIA GPU for inference acceleration (**optional**)

19.2 Installation

To simplify the installation, a Docker image (**Ubuntu 24.04, Python 3.12**) containing all the dependencies required to run the environment has been prepared. You can either pull the prebuilt image (**GPU, CUDA only**) or build it from scratch yourself. Currently, three platforms are supported:

- x86_64 / arm64 (CPU)
- x86_64 / arm64 (GPU, CUDA)
- NVIDIA Jetson

NOTE

See README.md for more information about supported platforms. The resulting image comes with UV ready to use inside the container (venv is activated), for example `uv pip install torch` can be used. TVM is not built from source by default, use `--build-tvm` when building the image, or install a prebuilt wheel manually afterwards, see README.md for details.

19.2.1 Pulling built image

The built image can be pulled with (**GPU, CUDA only**):

```
docker pull ghcr.io/antmicro/ros2-gui-node:kenning-ros2-demo-gpu
```

19.2.2 Building the image from scratch

Or you can build it from scratch by first cloning the repository:

```
git clone https://github.com/antmicro/ros2-gui-node
cd ros2-gui-node
git submodule update --init --recursive
```

Then run the bash script for building the image:

```
sudo ./environments/build-docker.sh <cpu | gpu | jetson> [--build-tvm]
```

NOTE

For more details on how to use this script and what it does, refer to: README.md

19.3 Running the container

The pulled or built image can be run with the following command (you need to pass the appropriate platform argument):

```
sudo ./environments/run-docker.sh [cpu | gpu | jetson]
```

NOTE

For more details on how to use this script and what it does, refer to: README.md

19.4 Running Kenning together with ROS 2

The easiest option to execute Kenning process in ROS 2 project is to use ROS 2 launch files providing `kenning` as an executable to run, with `ros` as a subcommand:

```
from launch_ros.actions import Node

# ...
kenning_node = Node(
    name="kenning_node",
    executable="kenning",
    arguments=["ros", "flow", "--verbosity", "DEBUG"],
```

(continues on next page)

(continued from previous page)

```
    parameters=[
        {
            "config_file": "./examples/kenning-instance-segmentation/kenning-
↪instance-segmentation.yaml"
        }
    ],
)
```

You can pass standard command line arguments like verbosity level using **arguments** parameters in Node. You can set different verbosity level for Kenning logger and ROS 2 logger. If you want to see all logs for Kenning and ROS 2, set arguments to:

```
arguments = ["--verbosity", "DEBUG", "--ros-args", "--log-level", "DEBUG"]
```

19.5 Setting Kenning parameters

You can use parameters section of Node to set all Kenning-related parameters. To set Kenning pipeline you need to set appropriate arguments in **Node**:

```
arguments=["ros","optimize","test" ...
```

is equivalent to running Kenning command with:

```
kenning optimize test ...
```

To use scenario config file, **config_file** parameter is used to provide path to the standard Kenning's scenario file:

```
"config_file":"./examples/kenning-instance-segmentation/kenning-instance-
↪segmentation.yaml"
```

But you can also provide every standard command line argument supported by Kenning, using ROS 2 parameters, for example:

```
arguments = (["ros", "optimize", "test", "--verbosity", "DEBUG"],)
parameters = (
    [
        {
            "config_file": "./scripts/configs/tensorflow-pet-dataset-mobilenet.yml
↪",
            "measurements": "./workspace/data.json",
            "report_path": "./report/report.md",
            "report_name": "Mobilenet Pet Dataset Test",
        }
    ],
)
```

is equivalent to running the command:

```
kenning optimize test --cfg ./scripts/configs/tensorflow-pet-dataset-mobilenet.
↪yml --measurements ./workspace/data.json --report-path ./report/report.md --
↪report-name "Mobilenet Pet Dataset Test"
```

19.6 Summary

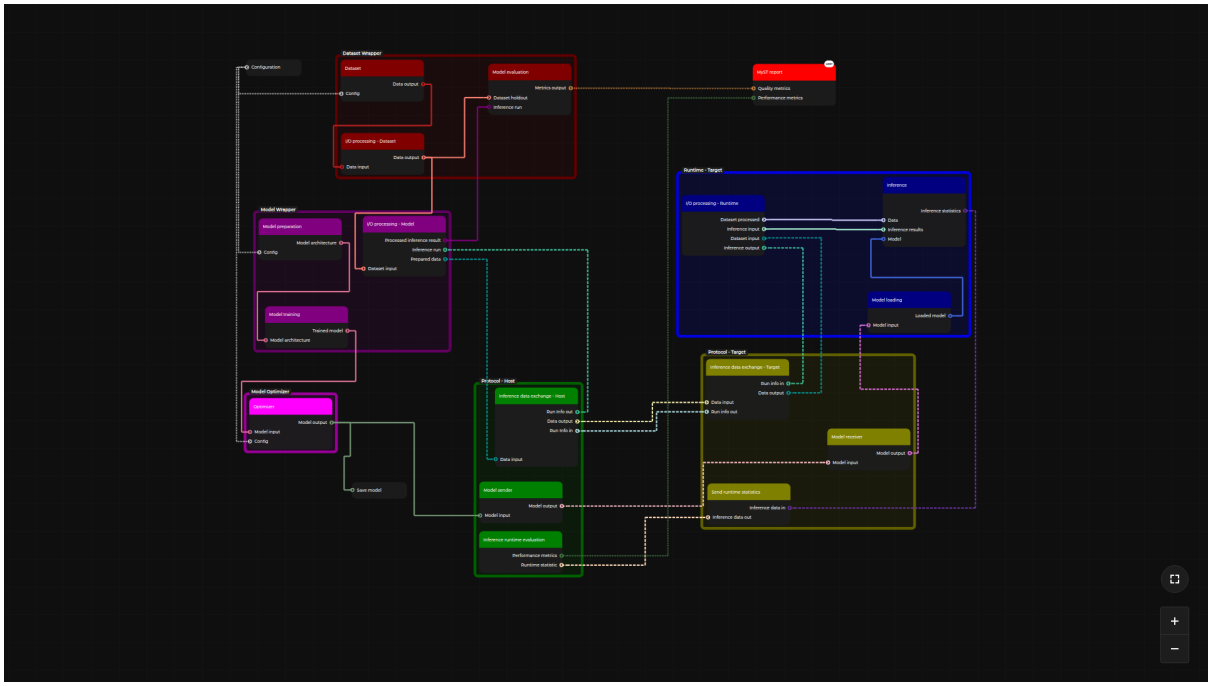
In this example, we have prepared an environment to work with ROS 2 and Kenning. We highly recommend reading the README.md before building the image.

CHAPTER

TWENTY

KENNING API

20.1 Deployment API overview



Kenning provides:

- a *Dataset* class - performs dataset download, preparation, input preprocessing, output postprocessing and model evaluation,
- a *ModelWrapper* class - trains the model, prepares the model, performs model-specific input preprocessing and output postprocessing, runs inference on host using a native framework,
- a *Optimizer* class - optimizes and compiles the model,
- a *Runtime* class - loads the model, performs inference on compiled model, runs target-specific processing of inputs and outputs, and runs performance benchmarks,
- a *Protocol* class - implements the communication protocol between the host and the target,
- a *DataConverter* class - performs conversion of data between different formats used by surrounding blocks,
- a *DataProvider* class - implements providing data for inference from such sources as camera, TCP connection, or others,

- a *OutputCollector* class - implements parsing and utilizing data coming from inference (such as displaying visualizations or sending results via TCP),
- a *ModelConverter* class - converts model from one format to another.

20.1.1 Model processing

The orange blocks and arrows in `class-flow` represent a model's life cycle:

- the model is designed, trained, evaluated and improved - the training is implemented in the *ModelWrapper*.

> **Note**
>
> This is an optional step - an already trained model can also be wrapped and used.

- the model is passed to the *Optimizer* where it is optimized for given hardware and later compiled,
- during inference testing, the model is sent to the target using *Protocol*,
- the model is loaded on target side and used for inference using *Runtime*.

Once the development of the model is complete, the optimized and compiled model can be used directly on target device using *Runtime*.

20.1.2 I/O data flow

The data flow is represented in the `class-flow` with green blocks. The input data flow is depicted using green arrows, and the output data flow is depicted using grey arrows.

Firstly, the input and output data is loaded from dataset files and processed. Later, since every model has its specific input preprocessing and output postprocessing routines, the data is passed to the *ModelWrapper* methods in order to apply modifications. During inference testing, the data is sent to and from the target using *Protocol*.

Lastly, since *Runtime*s also have their specific representations of data, proper I/O processing is applied.

20.1.3 Data flow reporting

Report rendering requires performance metrics and quality metrics. The flow for this is presented with grey lines and blocks in `class-flow`.

On target side, performance metrics are computed and sent back to the host using the *Protocol*, and later passed to report rendering. After the output data goes through processing in the *Runtime* and *ModelWrapper*, it is compared to the ground truth in the *Dataset* during model evaluation. In the end, the results of model evaluation are passed to report rendering.

The final report is generated as an RST file containing figures, as can be observed in the *Sample autogenerated report*.

20.2 KenningFlow

`kenning.core.flow.KenningFlow` class allows for creation and execution of arbitrary flows built of runners. It is responsible for validating all runners provided in a config file and their IO compatibility.

class kenning.core.flow.**KenningFlow**(runners: *list[*Runner*]*)

Bases: `object`

Allows for creation of custom flows using Kenning core classes.

KenningFlow class creates and executes customized flows consisting of the runners implemented based on kenning.core classes, such as DatasetProvider, ModelRunner, OutputCollector. Designed flows may be formed into non-linear, graph-like structures.

The flow may be defined either directly via dictionaries or in a predefined JSON format.

The JSON format must follow well defined structure. Each runner should consist of following entries:

type - Type of a Kenning class to use for this module parameters - Inner parameters of chosen class inputs - Optional, set of pairs (local name, global name) outputs - Optional, set of pairs (local name, global name)

All global names (inputs and outputs) must be unique. All local names are predefined for each class. All variables used as input to a runner must be defined as a output of a runner that is placed before that runner.

classmethod form_parameterschema() → dict

Creates schema for the KenningFlow class.

Returns
Schema for the class.

Return type
Dict

classmethod from_json(runners_specifications: *list[dict[str, Any]]*) → *KenningFlow*

Constructor wrapper that takes the parameters from json dict.

This function checks if the given dictionary is valid according to the json schema defined in `form_parameterschema`. If it is then it parses json and invokes the constructor.

Parameters

runners_specifications : List[Dict[str, Any]]
List of runners that creates the flow.

Returns
Object of class KenningFlow.

Return type
KenningFlow

Raises

- **jsonschema.ValidationError** – Raised for invalid JSON description

- **Exception** – Raised for undefined and redefined variables, depending on context

run()

Main process function. Repeatedly runs constructed graph in a loop.

run_single_step()

Runs flow one time.

20.3 Runner

`kenning.core.runner.Runner` - based classes are responsible for executing various operation in KenningFlow (i.e. data providing, model execution, data visualization).

The available runner implementations are:

- *DataProvider* - base class for data providing,
- ModelRuntimeRunner - for running model inference,
- *OutputCollector* - for processing model output.

class kenning.core.runner.**Runner**(inputs_sources: *dict[str, tuple[int, str]]*, inputs_specs: *dict[str, dict]*, outputs: *dict[str, str]*)

Bases: `IOInterface`, `ArgumentsHandler`, `ABC`

Represents an operation block in Kenning Flow.

cleanup()

Method that cleans resources after Runner is no longer needed.

abstractmethod **run**(inputs: *dict[str, Any]*) → dict[str, Any]

Method used to run this Runner.

Parameters

inputs : Dict[str, Any]
Inputs provided to this block.

Returns
Output of this block.

Return type
Dict[str, Any]

should_close() → bool

Method that checks if Runner got some exit indication (exception etc.) and the flow should close.

Returns
True if there was some exit indication.

Return type
bool

20.4 Dataset

`kennning.core.dataset.Dataset` - based classes are responsible for:

- dataset preparation, including download routines (use the `--download-dataset` flag to download the dataset data),
- input preprocessing into a format expected by most models for a given task,
- output postprocessing for the evaluation process,
- model evaluation based on its predictions,
- sample subdivision into training and validation datasets.

The Dataset objects are used by:

- *ModelWrapper* - for training purposes and model evaluation,
- *Optimizer* - can be used e.g. for extracting a calibration dataset for quantization purposes,
- *Runtime* - for model evaluation on target hardware.

The available dataset implementations are included in the `kenning.datasets` submodule. Example implementations:

- PetDataset for classification,
- OpenImagesDatasetV6 for object detection,
- RandomizedClassificationDataset.

class kenning.core.dataset.**Dataset**(root: *Path*, batch_size: *int* = 1, download_dataset: *bool* = **True**, force_download_dataset: *bool* = **False**, external_calibration_dataset: *Path* | *None* = **None**, split_fraction_test: *float* = 0.2, split_fraction_val: *float* | *None* = **None**, split_seed: *int* = 1234, dataset_percentage: *float* = 1, shuffle_data: *bool* = **True**, prepare_dataset: *bool* = **True**)

Bases: `ArgumentsHandler`, `ABC`

Wraps the datasets for training, evaluation and optimization.

This class provides an API for datasets used by models, compilers (i.e. for calibration) and benchmarking scripts.

Each Dataset object should implement methods for:

- processing inputs and outputs from dataset files,
- downloading the dataset,
- evaluating the model based on dataset's inputs and outputs.

The Dataset object provides routines for iterating over dataset samples with configured batch size, splitting the dataset into subsets and extracting loaded data from dataset files for training purposes.

dataX

List of input data (or data representing input data, i.e. file paths).

Type
List[Any]

dataY

List of output data (or data representing output data).

Type
List[Any]

batch_size

The batch size for the dataset.

Type
int

_dataindex

ID of the next data to be delivered for inference.

Type
int

dataXtrain

dataX subset representing a training set. Available after executing train_test_split_representations, otherwise empty.

Type
List[Any]

dataYtrain

dataY subset representing a training set. Available after executing train_test_split_representations, otherwise empty.

Type
List[Any]

dataXtest

dataX subset representing a testing set. Available after executing train_test_split_representations, otherwise empty.

Type
List[Any]

dataYtest

dataY subset representing a testing set. Available after executing train_test_split_representations, otherwise empty.

Type
List[Any]

dataXval

Optional dataX subset representing a validation set. Available after executing train_test_split_representations, otherwise empty.

Type
List[Any]

dataYval

Optional dataY subset representing a validation set. Available after executing train_test_split_representations, otherwise empty.

Type
List[Any]

begin_evaluate() → *Measurements*

Runs before model evaluation begins. Can be used to save one-off metadata in measurements or control internal evaluation state.

This function is run after train_test_split_representations, so it's safe to assume the split-related fields have already been assigned, but there is no guarantee that all of the test data will be used for evaluation, as the inference loop can finish earlier.

Returns
The measurements object containing metadata or any other values that need to be saved just once.

Return type
Measurements

calibration_dataset_generator(percentage: *float* = 0.25, seed: *int* | *None* = None) → Generator[list[Any], None, None]

Creates generator for the calibration data.

Parameters

percentage : float
The fraction of data to use for calibration.

seed : Optional[int]
The seed for random state, by default seed used for the dataset split.

Yields
List[Any] – List with batch input samples for calibration

abstractmethod download_dataset_fun()

Downloads the dataset to the root directory defined in the constructor.

end_evaluate() → *Measurements*

Runs after model evaluation has finished. Can be used to save one-off metadata in measurements or control internal evaluation state.

Returns
The measurements object containing metadata or any other values that need to be saved just once.

Return type
Measurements

abstractmethod evaluate(predictions: *list*, truth: *list*) → *Measurements*

Evaluates the model based on the predictions.

The method should compute various quality metrics fitting for the problem the model solves - i.e. for classification it may be accuracy, precision, G-mean, for detection it may be IoU and mAP.

The evaluation results should be returned in a form of Measurements object.

Parameters

predictions : List
The list of predictions from the model.

truth : List
The ground truth for given batch.

Returns
The dictionary containing the evaluation results.

Return type
Measurements

classmethod from_argparse(args: *Namespace*) → *Dataset*

Constructor wrapper that takes the parameters from argparse args.

This method takes the arguments created in `form_argparse` and uses them to create the object.

Parameters

args : Namespace
Arguments from ArgumentParser object.

Returns
Object of class Dataset.

Return type
Dataset

classmethod from_json(json_dict: *dict*) → *Dataset*

Constructor wrapper that takes the parameters from json dict.

This function checks if the given dictionary is valid according to the `arguments_structure` defined. If it is then it invokes the constructor.

Parameters

json_dict : Dict
Arguments for the constructor.

Returns
Object of class Dataset.

Return type
Dataset

abstractmethod get_class_names() → list[str]

Returns list of class names in order of their IDs.

Returns
List of class names.

Return type
List[str]

get_data() → tuple[list, list]

Returns the tuple of all inputs and outputs for the dataset.

> **Warning**
>
> It loads all entries with prepare_input_samples and prepare_output_samples to the memory - for large datasets it may result in filling the whole memory.

Returns
The list of data samples.

Return type
Tuple[List, List]

get_data_unloaded() → tuple[list, list]

Returns the input and output representations before loading.

The representations can be opened using prepare_input_samples and prepare_output_samples.

Returns
The list of data samples representations.

Return type
Tuple[List, List]

abstractmethod get_input_mean_std() → tuple[Any, Any]

Returns mean and std values for input tensors.

The mean and std values returned here should be computed using `compute_input_mean_std` method.

Returns
Tuple of two variables describing mean and standardization values for a given train dataset.

Return type
Tuple[Any, Any]

iter_test() → DatasetIterator

Iterates over test data obtained from split.

Returns
Iterator over data samples.

Return type
DatasetIterator

iter_train() → DatasetIterator

Iterates over train data obtained from split.

Returns
Iterator over data samples.

Return type
DatasetIterator

iter_val() → DatasetIterator

Iterates over validation data obtained from split.

Returns
Iterator over data samples.

Return type
DatasetIterator

abstractmethod prepare()

Prepares dataX and dataY attributes based on the dataset contents.

This can i.e. store file paths in dataX and classes in dataY that will be later loaded using prepare_input_samples and prepare_output_samples.

prepare_external_calibration_dataset(percentage: *float* = 0.25, seed: *int* = 12345) → list[Path]

Prepares the data for external calibration dataset.

This method is supposed to scan external_calibration_dataset directory and prepares the list of entries that are suitable for the prepare_input_samples method.

This method is called by the `calibration_dataset_generator` method to get the data for calibration when external_calibration_dataset is provided.

By default, this method scans for all files in the directory and returns the list of those files.

Parameters

percentage : float
Percentage of dataset to be used.

seed : int
Random state seed.

Returns
List of objects that are usable by the `prepare_input_samples` method.

Return type
List[Path]

prepare_input_samples(samples: *list*) → list

Prepares input samples, i.e. load images from files, converts them.

By default the method returns data as is - without any conversions. Since the input samples can be large, it does not make sense to load all data to the memory - this method handles loading data for a given data batch.

Parameters

samples : List
List of input samples to be processed.

Returns
Preprocessed input samples.

Return type
List

prepare_output_samples(samples: *list*) → list

Prepares output samples.

By default the method returns data as is. It can be used i.e. to create the one-hot output vector with class association based on a given sample.

Parameters

samples : List
List of output samples to be processed.

Returns
Preprocessed output samples.

Return type
List

save_dataset_checksum()
Writes dataset checksum to file.

set_batch_size(batch_size: *int*)
Sets the batch size of the data in the iterator batches.

Parameters

batch_size : int
Number of input samples per batch.

test_subset_len() → int | None
Returns the length of a single batch from the training set.

Returns
The number of samples in a single batch from the testing set or None if the dataset has not been split

Return type
Optional[int]

train_subset_len() → int | None
Returns the length of a single batch from the training set.

Returns
The number of samples in a single batch from the training set or None if the dataset has not been split

Return type
Optional[int]

train_test_split_representations(test_fraction: *float* | *None* = **None**, val_fraction: *float* | *None* = **None**, seed: *int* | *None* = **None**, stratify: *bool* = **True**, append_index: *bool* = **False**) → tuple[list, ...]
Splits the data representations into train dataset and test dataset.

Parameters

test_fraction : Optional[float]
The fraction of data to leave for model testing.

val_fraction : Optional[float]
The fraction of data to leave for model validation.

seed : Optional[int]
The seed for random state.

stratify : bool
Whether to stratify the split.

append_index : bool
Whether to return the indices of the split. If True, the returned tuple will have indices appended at the end. For example, if the split is (X_train, X_test, y_train, y_test), the returned tuple will be (X_train, X_test, y_train, y_test, train_indices, test_indices).

Returns
Split data into train, test and optionally validation subsets.

Return type
Tuple[List, ...]

val_subset_len() → int | None

Returns the length of a single batch from the training set.

Returns
The number of samples in a single batch from the validation set or None if the dataset has not been split

Return type
Optional[int]

verify_dataset_checksum() → bool

Checks whether dataset is already downloaded in its directory.

Returns
True if dataset is downloaded.

Return type
bool

20.5 ModelWrapper

`kenning.core.model.ModelWrapper` base class requires implementing methods for:

- model preparation,
- model saving and loading,
- model saving to the ONNX format,
- model-specific preprocessing of inputs and postprocessing of outputs, if necessary,
- model inference,
- providing metadata (framework name and version),
- model training,
- input format specification,
- conversion of model inputs and outputs to bytes for the `kenning.core.protocol.Protocol` objects.

The ModelWrapper provides methods for running inference in a loop for data from a dataset and measuring the model's quality and inference performance.

The kenning.modelwrappers.frameworks submodule contains framework-wise implementations of the ModelWrapper class - they implement all methods common for given frameworks regardless of the model used.

For the Pet Dataset wrapper object, there is an example classifier implemented in TensorFlow 2.x called TensorFlowPetDatasetMobileNetV2 <https://github.com/antmicro/kenning/blob/main/kenning/modelwrappers/classification/tensorflow_pet_dataset.py>_.

Model wrapper examples:

- PyTorchWrapper and TensorFlowWrapper implement common methods for all PyTorch and TensorFlow framework models,
- PyTorchPetDatasetMobileNetV2 wraps the MobileNetV2 model for Pet classification implemented in PyTorch,
- TensorFlowDatasetMobileNetV2 wraps the MobileNetV2 model for Pet classification implemented in TensorFlow,
- TVMDarknetCOCOYOLOV3 wraps the YOLOv3 model for COCO object detection implemented in Darknet (without training and inference methods).

class kenning.core.model.**ModelWrapper**(model_path: *Path* | ResourceURI, dataset: Dataset | *None*, from_file: *bool* = **True**, model_name: *str* | *None* = **None**)

Bases: IOInterface, ArgumentsHandler, ABC

Wraps the given model.

abstractmethod convert_input_to_bytes(inputdata: *Any*) → bytes

Converts the input returned by the preprocess_input method to bytes.

Parameters

inputdata : Any
The preprocessed inputs.

Returns
Input data as byte stream.

Return type
bytes

abstractmethod convert_output_from_bytes(outputdata: *bytes*) → list[Any]

Converts bytes array to the model output format.

The converted output should be compatible with postprocess_outputs method.

Parameters

outputdata : bytes
Output data in raw bytes.

Returns
List of output data from a model. The converted data should be compatible with the postprocess_outputs method.

Return type
List[Any]

abstractmethod classmethod derive_io_spec_from_json_params(json_dict: *dict*) → dict[str, list[dict]]

Creates IO specification by deriving parameters from parsed JSON dictionary. The resulting IO specification may differ from the results of *get_io_specification*, information that couldn't be retrieved from JSON parameters are absent from final IO spec or are filled with general value (example: '-1' for unknown dimension shape).

Parameters

json_dict : Dict
JSON dictionary formed by parsing the input JSON with ModelWrapper's parameterschema.

Returns
Dictionary that conveys input and output layers specification.

Return type
Dict[str, List[Dict]]

classmethod from_argparse(dataset: Dataset | *None*, args: *Namespace*, from_file: *bool* = **True**) → *ModelWrapper*

Constructor wrapper that takes the parameters from argparse args.

Parameters

dataset : Optional[*Dataset*]
The dataset object to feed to the model.

args : Namespace
Arguments from ArgumentParser object.

from_file : bool
Determines if the model should be loaded from model_path.

Returns
Object of class ModelWrapper.

Return type
ModelWrapper

classmethod from_json(json_dict: *dict*, dataset: Dataset | *None* = **None**, from_file: *bool* = **True**) → *ModelWrapper*

Constructor wrapper that takes the parameters from json dict.

This function checks if the given dictionary is valid according to the `arguments_structure` defined. If it is then it invokes the constructor.

Parameters

json_dict : Dict
Arguments for the constructor.

dataset : Optional[*Dataset*]
The dataset object to feed to the model.

from_file : bool
Determines if the model should be loaded from model_path.

Returns
Object of class ModelWrapper.

Return type
ModelWrapper

abstractmethod classmethod get_framework() → str

Returns name of the framework.

Returns
Framework name.

Return type
str

abstractmethod classmethod get_framework_version() → str

Returns the framework version.

Returns
Framework version.

Return type
str

get_io_specification() → dict[str, list[dict]]

Returns a saved dictionary with *input* and *output* keys that map to input and output specifications.

A single specification is a list of dictionaries with names, shapes and dtypes for each layer. The order of the dictionaries is assumed to be expected by the *ModelWrapper*.

It is later used in optimization and compilation steps.

Returns
Dictionary that conveys input and output layers specification.

Return type
Dict[str, List[Dict]]

abstractmethod get_io_specification_from_model() → dict[str, list[dict]]

Returns a new instance of dictionary with *input* and *output* keys that map to input and output specifications.

A single specification is a list of dictionaries with names, shapes and dtypes for each layer. The order of the dictionaries is assumed to be expected by the *ModelWrapper*.

It is later used in optimization and compilation steps.

It is used by *get_io_specification* function to get the specification and save it for later use.

Returns
Dictionary that conveys input and output layers specification.

Return type
Dict[str, List[Dict]]

get_model_size() → float

Returns the model size.

By default, the size of file with model is returned.

Returns
The size of the optimized model in KB.

Return type
float

abstractmethod classmethod get_output_formats() → list[str]

Returns list of names of possible output formats.

Returns
List of possible output format names.

Return type
List[str]

get_path() → Path | *ResourceURI*

Returns path to the model in a form of a Path or ResourceURI object.

Returns
Path or URI to the model.

Return type
PathOrURI

abstractmethod load_model(model_path*: Path* | ResourceURI)

Loads the model from file.

Parameters

model_path : PathOrURI
Path or URI to the model file.

classmethod parse_io_specification_from_json(json_dict)

Return dictionary with 'input' and 'output' keys that will map to input and output specification of an object created by the argument json schema.

A single specification is a list of dictionaries with names, shapes and dtypes for each layer.

Since no object initialization is done for this method, some IO specification may be incomplete, this method fills in -1 in case the information is missing from the JSON dictionary.

Parameters

json_dict : Dict
Parameters for object constructor in JSON format.

Returns
Dictionary that conveys input and output layers specification.

Return type
Dict[str, List[Dict]]

postprocess_outputs(y*: list[Any]*) → list[Any]

Processes the outputs for a given model.

By default no action is taken, and the outputs are passed unmodified.

Parameters

y : List[Any]
The list of output data from the model.

Returns
The post processed outputs from the model that need to be in format requested by the Dataset object.

Return type
List[Any]

abstractmethod prepare_model()

Downloads the model (if required) and loads it to the device.

Should be used whenever the model is actually required.

The prepare_model method should check model_prepared field to determine if the model is not already loaded.

It should also set model_prepared field to True once the model is prepared.

preprocess_input(X: *list[Any]*) → list[Any]

Preprocesses the inputs for a given model before inference.

By default no action is taken, and the inputs are passed unmodified.

Parameters

X : List[Any]
The input data from the Dataset object.

Returns
The preprocessed inputs that are ready to be fed to the model.

Return type
List[Any]

read_platform(platform: Platform)

Reads Platform data to configure model.

Platform-based entities come with lots of information on hardware architecture that can be used by the ModelWrapper class.

By default no data is read.

It is important to take into account that different Platform-based classes come with a different sets of attributes.

Parameters

platform : *Platform*
object with platform details

abstractmethod run_inference(X: *list[Any]*) → list[Any]

Runs inference for a given preprocessed input.

Parameters

X : List[Any]
The preprocessed inputs for the model.

Returns
The results of the inference.

Return type
List[Any]

run_test_inference() → *Measurements*

Runs the inference on test split of the dataset.

Returns
The inference results.

Return type
Measurements

abstractmethod save_model(model_path: *Path* | ResourceURI)

Saves the model to file.

Parameters

model_path : PathOrURI
Path or URI to the model file.

abstractmethod save_to_onnx(model_path: *Path* | ResourceURI)

Saves the model in the ONNX format.

Parameters

model_path : PathOrURI
Path or URI to the model file.

abstractmethod train_model()

Trains the model with a given dataset.

This method should implement training routine for a given dataset and save a working model to a given path in a form of a single file.

The training should be performed with given batch size, learning rate, and number of epochs.

The model needs to be saved explicitly.

20.6 Optimizer

`kenning.core.optimizer.Optimizer` objects wrap the deep learning compilation process. They can perform the model optimization (operation fusion, quantization) as well. Kenning supports executing optimizations also on the target device. To do so you can use `location` parameter which specifies where given *Optimizer* would be executed (either `'host'` or `'target'`).

Example model optimizers:

- LiteRTCompiler - wraps TensorFlow Lite compilation,
- TVMCompiler - wraps TVM compilation.

class kenning.core.optimizer.**Optimizer**(dataset: Dataset | *None*, compiled_model_path: *Path* | ResourceURI, location: *'host'* | *'target'* = `'host'`, model_wrapper: ModelWrapper | *None* = **None**)

Bases: ArgumentsHandler, ABC

Compiles the given model to a different format or runtime.

abstractmethod compile(input_model_path: *Path* | ResourceURI, io_spec: *dict[str, list[dict]]* | *None* = **None**, **kwargs: *dict*)

Compiles the given model to a target format.

The function compiles the model and saves it to the output file.

The model can be compiled to a binary, a different framework or a different programming language.

If *io_spec* is passed, then the function uses it during the compilation, otherwise *load_io_specification* is used to fetch the specification saved in *input_model_path* + *.json*.

The compiled model is saved to compiled_model_path and the specification is saved to compiled_model_path + .json

Parameters

input_model_path : PathOrURI
Path to the input model.

io_spec : Optional[Dict[str, List[Dict]]]
Dictionary that has *input* and *output* keys that contain list of dictionaries mapping (property name) -> (property value) for the layers.

****kwargs : Dict**
Additional keyword arguments used mainly by converters.

classmethod from_argparse(dataset: Dataset | *None*, args: *Namespace*) → *Optimizer*

Constructor wrapper that takes the parameters from argparse args.

Parameters

dataset : Optional[*Dataset*]
The dataset object that is optionally used for optimization.

args : Namespace
Arguments from ArgumentParser object.

Returns
Object of class Optimizer.

Return type
Optimizer

classmethod from_json(json_dict: *dict*, dataset: Dataset | *None* = **None**, model_wrapper: ModelWrapper | *None* = **None**) → *Optimizer*

Constructor wrapper that takes the parameters from json dict.

This function checks if the given dictionary is valid according to the `arguments_structure` defined. If it is then it invokes the constructor.

Parameters

json_dict : Dict
Arguments for the constructor.

dataset : Optional[*Dataset*]
The dataset object that is optionally used for optimization.

model_wrapper : Optional[*ModelWrapper*]
ModelWrapper for the optimized model (optional).

Returns
Object of class Optimizer.

Return type
Optimizer

abstractmethod get_framework() → str
Returns name of the framework.

Returns
Framework name.

Return type
str

abstractmethod classmethod get_framework_version() → str
Returns version of the framework.

Returns
Framework version.

Return type
str

classmethod get_input_formats() → list[str]
Returns list of names of possible input formats.

Returns
Names of possible input formats.

Return type
List[str]

get_input_type(model_path: *Path* | ResourceURI) → str
Return input model type. If input type is set to "any", then it is derived from model file extension.

Parameters

model_path : PathOrURI
Path to the input model.

Returns
Input model type.

Return type
str

Raises
Exception – Raised if input model type cannot be determined.

get_model_class() → Any | None
Function that tries to deduce model class of model wrapper.

Returns
Model class of the model wrapper or None when model class is not available.

Return type
Optional[Any]

get_optimized_model_size() → float

Returns the optimized model size.

By default, the size of file with optimized model is returned.

Returns
The size of the optimized model in KB.

Return type
float

classmethod get_output_formats() → list[str]

Returns list of names of possible output formats.

Returns
List of possible output formats.

Return type
List[str]

static get_spec_path(model_path: *Path* | ResourceURI) → Path | *ResourceURI*

Returns input/output specification path for the model saved in *model_path*. It concatenates *model_path* and *.json*.

Parameters

model_path : PathOrURI
Path where the model is saved.

Returns
Path to the input/output specification of a given model.

Return type
PathOrURI

init()

Initializes optimizer, should be called before compilation.

load_io_specification(model_path: *Path* | ResourceURI) → dict[str, list[dict]] | None

Returns saved input and output specification of a model saved in *model_path* if there is one. Otherwise returns None.

Parameters

model_path : PathOrURI
Path to the model which specification the function should read.

Returns
Specification of a model saved in *model_path* if there is one. None otherwise.

Return type
Optional[Dict[str, List[Dict]]]

read_platform(platform: Platform)

Reads Platform data to configure optimization/compilation.

Platform-based entities come with lots of information on hardware architecture that can be used by the Optimizer class.

By default no data is read.

It is important to take into account that different Platform-based classes come with a different sets of attributes.

Parameters

platform : *Platform*
object with platform details

run_compatibility_checks(platform: Platform, runtime: Runtime | *None*, input_model_path: *Path* | ResourceURI) → bool

Runs preliminary compatibility checks.

Performs most basic initial checks of this optimizer in the final pipeline.

Parameters

platform : *Platform*
object with platform details

runtime : Optional[*Runtime*]
runtime that will be used, provided only if optimizer is the final optimizer.

input_model_path : PathOrURI
path to model that would be used during pipeline execution.

Returns
Information whether compatibility checks were successful. True by default.

Return type
bool

save_io_specification(input_model_path: *Path* | ResourceURI, io_spec: *dict[str, list[dict]]* | *None* = None)

Internal function that saves input/output model specification which is used during both inference and compilation. If *io_spec* is None, the function uses specification of an input model stored in *input_model_path* + *.json*. If there is no specification stored in this path the function does not do anything.

The input/output specification is a list of dictionaries mapping properties names to their values. Legal properties names are *dtype*, *prequantized_dtype*, *shape*, *name*, *scale*, *zero_point*.

The order of the layers has to be preserved.

Parameters

input_model_path : PathOrURI
Path to the input model.

io_spec : Optional[Dict[str, List[Dict]]]
Specification of the input/ouput layers.

set_compiled_model_path(compiled_model_path: *Path*)

Sets path for compiled model.

compiled_model_path
[PathOrURI] Path to be set.

set_input_type(inputtype: *str*)

Sets input type of the model for the compiler.

inputtype
[str] Path to be set.

zpl_prepare_cmd_flags() → list[str]

Provides optimizer specific flags for Zephelin prepare command.

Returns
List of flags specific for current optimizer.

Return type
List[str]

20.7 Runtime

The `kenning.core.runtime.Runtime` class provides interfaces for methods for running compiled models locally or remotely on a target device. Runtimes are usually compiler-specific (frameworks for deep learning compilers provide runtime libraries to run compiled models on particular hardware).

The client (host) side of the `Runtime` class utilizes the methods from *Dataset*, *ModelWrapper* and *Protocol* classes to run inference on a target device. The server (target) side of the `Runtime` class requires method implementation for:

- loading a model delivered by the client,
- preparing inputs delivered by the client,
- running inference,
- preparing outputs for delivery to the client,
- (optionally) sending inference statistics.

Runtime examples:

- LiteRTRuntime for models compiled with TensorFlow Lite,
- TVMRuntime for models compiled with TVM.

class kenning.core.runtime.**Runtime**(model_path: *Path* | ResourceURI = PosixPath('.'), disable_performance_measurements: *bool* = **False**, batch_size: *int* = 1)

Bases: `ArgumentsHandler`, `ABC`

Runtime object provides an API for testing inference on target devices.

abstractmethod extract_output() → list[Any]

Extracts and postprocesses the output of the model.

Returns
Postprocessed and reordered outputs of the model.

Return type
List[Any]

classmethod from_argparse(args: *Namespace*) → *Runtime*

Constructor wrapper that takes the parameters from argparse args.

Parameters

args : Namespace
Arguments from ArgumentParser object.

Returns
Object of class Runtime.

Return type
Runtime

classmethod from_json(json_dict: *dict*) → *Runtime*

Constructor wrapper that takes the parameters from json dict.

This function checks if the given dictionary is valid according to the `arguments_structure` defined. If it is then it invokes the constructor.

Parameters

json_dict : Dict
Arguments for the constructor.

Returns
Object of class Runtime.

Return type
Runtime

static get_available_ram(platform: Platform) → float | None

Gets size of the RAM (or other type of memory where the model will be stored) for the given platform.

Parameters

platform : *Platform*
The platform representation.

Returns
The size of RAM.

Return type
Optional[float]

classmethod get_input_formats() → list[str]

Returns list of names of possible input formats names.

Returns
List of possible input format names.

Return type
List[str]

static get_io_spec_path(model_path: *Path* | ResourceURI) → Path

Gets path to a input/output specification file which is *model_path* and *.json* concatenated.

Parameters

model_path : PathOrURI
URI to the compiled model.

Returns
Returns path to the specification.

Return type
Path

get_time() → float

Gets the current timestamp.

Returns
Current timestamp.

Return type
float

infer(X: *list[ndarray]*, model_wrapper: ModelWrapper, postprocess: *bool* = **True**) → list[Any]

Runs inference on single batch locally using a given runtime.

Parameters

X : List[np.ndarray]
Batch of data provided for inference.

model_wrapper : *ModelWrapper*
Model that is executed on target hardware.

postprocess : bool
Indicates if model output should be postprocessed.

Returns
Obtained values.

Return type
List[Any]

inference_session_end()

Calling this function indicates that the inference session has ended.

This method should be called once all the inference data is sent to the server by the client.

This will stop performance tracking.

inference_session_start()

Calling this function indicates that the client is connected.

This method should be called once the client has connected to a server.

This will enable performance tracking.

abstractmethod load_input(input_data: *list[Any]*) → bool

Loads and converts delivered data to the accelerator for inference.

This method is called when the input is received from the client. It is supposed to prepare input before running inference.

Parameters

input_data : List[Any]
Input data in bytes delivered by the client, preprocessed.

Returns
True if succeeded.

Return type
bool

load_input_from_bytes(input_data: *bytes*) → bool

The method accepts *input_data* in bytes and loads it according to the input specification.

It creates *np.ndarray* for every input layer using the metadata in *self.input_spec* and quantizes the data if needed.

Some compilers can change the order of the layers. If that's the case the method also reorders the layers to match the specification of the model.

Parameters

input_data : bytes
Input data in bytes delivered by the client.

Returns
Output of the *load_input* method indicating if the operation succeeded.

Return type
bool

Raises
AttributeError – Raised if output specification is not loaded.

prepare_io_specification(input_data: *bytes | None*) → bool

Receives the io_specification from the client in bytes and saves it for later use.

`input_data` stores the io_specification representation in bytes. If `input_data` is None, the io_specification is extracted from another source (i.e. from existing file). If it can not be found in this path, io_specification is not loaded.

When no specification file is found, the function returns False as some Runtimes may not need io_specification to run the inference.

Parameters

input_data : Optional[bytes]
The io_specification` or None, if it should be loaded from another source.

Returns
True if succeeded.

Return type
bool

prepare_local()

Runs initialization for the local inference.

Raises ModelNotLoadedError if model loading fails.

abstractmethod prepare_model(input_data: *bytes* | *None*)

Receives the model to infer from the client in bytes.

The method should load bytes with the model, optionally save to file and allocate the model on target device for inference.

`input_data` stores the model representation in bytes. If `input_data` is None, the model is extracted from another source (i.e. from existing file or directory).

Parameters

input_data : Optional[bytes]
Model data or None, if the model should be loaded from another source.

Raises
ModelNotLoadedError – Raised if model is not loaded.

preprocess_model_to_upload(path: *Path* | ResourceURI) → Path | *ResourceURI*

The method preprocesses the model to be uploaded to the client and returns a new path to it.

The method is used to prepare the model to be sent to the client. It can be used to change the model representation, for example, to compress it.

Parameters

path : PathOrURI
Path to the model to preprocess.

Returns
Path to the preprocessed model.

Return type
PathOrURI

read_io_specification(io_spec: *dict*)

Saves input/output specification so that it can be used during the inference.

input_spec and *output_spec* are lists, where every element is a dictionary mapping (property name) -> (property value) for the layers.

The standard property names are: *name*, *dtype* and *shape*.

If the model is quantized it also has *scale*, *zero_point* and *prequantized_dtype* properties.

If the layers of the model are reorder it also has *order* property.

Parameters

io_spec : Dict
Specification of the input/output layers.

Raises

ModuleIOSpecificationFormatError – Raised if preprocessed input data has more than one available shape.

read_platform(platform: Platform)

Reads Platform data to configure runtime.

Platform-based entities come with lots of information on hardware architecture that can be used by the Runtime class.

By default no data is read.

It is important to take into account that different Platform-based classes come with a different sets of attributes.

Parameters

platform : *Platform*
object with platform details

abstractmethod run()

Runs inference on prepared input.

The input should be introduced in runtime's model representation, or it should be delivered using a variable that was assigned in load_input method.

Raises

ModelNotLoadedError – Raised if model is not loaded.

run_compatibility_checks(platform: Platform, total_app_size: *float*) → bool

Run initial compatibility checks.

Performs most basic initial checks of this runtime in the final pipeline.

Defined optionally, successful by default.

Parameters

platform : *Platform*
object with platform details

total_app_size : float
total size of running application - model and app.

Returns
Information whether compatibility checks were successful.

Return type
bool

upload_output(input_data: *bytes*) → bytes

Returns the output to the client, in bytes.

The method converts the direct output from the model to bytes and returns them.

The wrapper later sends the data to the client.

Parameters

input_data : bytes
Not used here.

Returns
Data to send to the client.

Return type
bytes

upload_stats(input_data: *bytes*) → bytes

Returns statistics of inference passes to the client.

Default implementation converts collected metrics in MeasurementsCollector to JSON format and returns them for sending.

Parameters

input_data : bytes
Not used here.

Returns
Statistics to be sent to the client.

Return type
bytes

20.8 Platform

The `kenning.core.platform.Platform` classes represents the targeted environment providing additional context for optimization, compilation and evaluation of the model.

Platform examples:

- LocalPlatform
- BareMetalPlatform
- ZephyrPlatform

class kenning.core.platform.**Platform**(name: *str* | *None* = **None**, platforms_definitions: *list[*ResourceURI*]* | *None* = **None**)

Bases: `ArgumentsHandler`, `ABC`

Wraps the platform that is being evaluated. This class provides methods to handle tested platform. The platform can be the device that Kenning is run on, a board running Kenning Zephyr Runtime or bare-metal IREE runtime or any remote or device running Kenning inference server.

deinit(measurements: Measurements | *None* = **None**)

Deinitializes platform.

Parameters

measurements : Optional[*Measurements*]
Measurements to which platform metrics can be added to.

get_default_protocol() → *Protocol*

Returns default protocol for given platform.

Returns
Default protocol for given platform.

Return type
Protocol

get_time() → float

Retrieves elapsed time from platform.

Returns
Elapsed time.

Return type
float

inference_step_callback()

Callback that is run every inference step.

init()

Initializes the platform.

post_init()

Called after the platform has been successfully setup, for cleanup or any other additional actions, that may be required by the platform.

read_data_from_platforms_yaml()

Retrieves platform data from specified platform definition files.

20.9 AutoML

The AutoML API is combined from two different elements - *AutoML model* and *AutoML flow mechanism*.

AutoML examples:

- AutoPyTorchModel and AutoPyTorchML

20.9.1 AutoML model

The `kenning.core.automl.AutoMLModel` represents a model that can be used for AutoML flow, and defines additional methods for registering and managing parameters. Moreover, the `arguments_structure` was extended with custom AutoML options, described in *Defining arguments for core classes*.

class kenning.core.automl.**AutoMLModel**

Bases: `ArgumentsHandler`, `ABC`

Base class representing AutoML-compatible model.

Should be used together with kenning.core.model.ModelWrapper class.

abstractmethod extract_model(network: *Any*) → Any

Extracts model from AutoML object.

Parameters

network : Any
AutoML object with optimized model.

Returns
Extracted model

Return type
Any

classmethod form_automl_schema() → dict

Gathers AutoML schema based on *arguments_structure* of class and all parent.

Returns
AutoML schema for the class.

Return type
Dict

Raises
InvalidSchemaError – If schema does not contain required fields.

static update_automl_defaults(arg_structure: *dict[str, dict]*, name: *str*)

Updates the default value of the AutoML parameter to make sure they fit into ranges.

Parameters

arg_structure : Dict[str, Dict]
The combined arguments structure of current class and its parents.

name : str
The name of parameter, should match with the *arguments_structure*.

classmethod update_automl_range(name: *str*, conf: *dict[str, tuple]*)

Updates the ranges of AutoML parameters.

Parameters

name : str
The name of parameter, should match with the *arguments_structure*.

conf : Dict[str, Tuple]
The dictionary with new parameters configuration.

Raises
InvalidArgumentsError – If parameter or its configuration does not exist.

20.9.2 AutoML flow mechanism

The `kenning.core.automl.AutoML` covers a standard AutoML flow: framework preparation, model searching and extraction to proper Kenning format.

class kenning.core.automl.**AutoML**(dataset: Dataset, platform: Platform, output_directory: *Path*, optimizers: *list[Optimizer]* = `[]`, runtime: Runtime | *None* = **None**, use_models: *list[str | dict[str, tuple]]* = `[]`, time_limit: *float* = `5.0`, optimize_metric: *str* = `'accuracy'`, n_best_models: *int* = `5`, application_size: *float | None* = **None**, skip_model_size_check: *bool* = **False**, callback_max_samples: *int* = `30`, seed: *int* = `1234`)

Bases: `ArgumentsHandler`, `ABC`

Base class describing AutoML flow: * preparing AutoML framework, * searching the best models, * generating Kenning configs based on found solutions.

classmethod from_argparse(dataset: Dataset | *None*, platform: Platform | *None*, args: *Namespace*) → *AutoML*

Constructor wrapper that takes the parameters from argparse args.

Parameters

dataset : Optional[*Dataset*]
The dataset object that is optionally used for optimization.

platform : Optional[*Platform*]
The platform on which generated models will be evaluated.

args : Namespace
Arguments from ArgumentParser object.

Returns
Object of class AutoML.

Return type
AutoML

classmethod from_json(json_dict: *dict*, dataset: Dataset | *None* = **None**, platform: Platform | *None* = **None**, optimizers: *list[*Optimizer*]* | *None* = **None**) → *AutoML*

Constructor wrapper that takes the parameters from json dict.

This function checks if the given dictionary is valid according to the *arguments_structure* defined. If it is then it invokes the constructor.

Parameters

json_dict : Dict
Arguments for the constructor.

dataset : Optional[*Dataset*]
The dataset object that is optionally used for optimization.

platform : Optional[*Platform*]
The platform on which generated models will be evaluated.

optimizers : Optional[List[*Optimizer*]]
The optional list with optimizers.

Returns
Object of class AutoML.

Return type
AutoML

abstractmethod get_best_configs() → Iterable[dict]

Extracts the best models and returns Kenning configuration for them.

Yields
Dict – Configuration for found models (from the best one).

abstractmethod get_statistics() → dict[str, int | float]

Returns statistic of the AutoML flow, like number of successful or crashed runs.

The created dictionary has to contain “general_info” - mapping of statisctics descriptions and values.

Optional fields:

- "trained_model_metrics" - mapping of models to dictionaries with datasets and metrics of trained models,
- "training_data" - mapping of models to losses from different parts of training, containing dictionaries with timestamps and loss values (averaged from batch or whole epoch). Possible parts of training: "training", "training_epoch", "validation", "validation_epoch", "test" and "test_epoch",
- "training_start_time" - mapping of models to a list of times, marking the beginning of trainings,
- "model_params" - mapping of models to dictionaries with parameters descriptions and values.

Returns
Dictionary with AutoML statistics, keys should describe given statistic.

Return type
Dict[str, Union[int, float]]

abstractmethod prepare_framework()
Prepares AutoML framework.

abstractmethod search()
Runs AutoML search.

20.10 Protocol

The `kenning.core.protocol.Protocol` is an abstract class that serves as a communication interface between the client (host) and the server (target platform).

For example it can be used to facilitate communication between Kenning client and Kenning Inference Server, but also between Kenning client and Kenning Zephyr Runtime Server.

It wraps the underlying communication logic, flow control and technology stack (for example KenningProtocol over TCP/IP) in abstract methods (to be overridden by its implementations), that can be called by client and server to communicate with each other.

20.10.1 Protocol implementations

`kenning.core.protocol.Protocol` implementations have to implement methods for:

- initialization (separate for target and host) - *initialize_server* and *initialize_client*,
- disconnecting (target and host) - *disconnect*,
- uploading a runtime (host) - *upload_runtime*,
- model input/output specification upload (host) - *upload_io_specification*,
- model upload (host) - *upload_model*,
- input data upload (host) - *upload_input*,
- requesting input data processing (host) - *request_processing*,
- downloading inference output (host) - *download_output*,

- downloading inference statistics (host) - *download_statistics*,
- uploading model optimizer configuration (host) - *upload_optimizers*,
- requesting model optimization and downloading an optimized model (host) - *request_optimization*,
- continuously receiving and processing log messages sent from the target (host) - *listen_to_server_logs*,
- capturing and sending to the client log messages generated by the server (target) - *start_sending_logs*, *stop_sending_logs*
- serving client requests (target) - *serve* (this method takes as arguments a number *server callbacks*, one for every type of client request)

If the implementation does not support one of the above methods, it must raise `kenning.core.exceptions.NotSupportedError` when called.

For detailed documentation of available `kenning.core.protocol.Protocol` implementations, see the *Kenning Protocols chapter*

20.10.2 Server callbacks

To use the `Protocol` on the server side, you need to implement a series of callback functions and pass them to the `serve` method:

`kenning.core.protocol.Protocol.`**`serve`**(self, upload_input_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_model_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, process_input_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, download_output_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, download_stats_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, upload_iospec_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_optimizers_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_unoptimized_model_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, download_optimized_model_callback: *Callable[[None], tuple[ServerStatus, bytes* | *None]]* | *None* = **None**, upload_runtime_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**)

Waits for requests from the other device (the client) and calls appropriate callbacks. Some callbacks take bytes as arguments, all return ServerStatus and some also return bytes. These responses are then sent back to the client.

Callbacks are guaranteed to be executed in the order requests were sent in.

This method is non-blocking.

Parameters

upload_input_callback : Optional[ServerUploadCallback]
Called, when the client uploads input ('upload_input' method below). Should upload model input into the runtime.

upload_model_callback : Optional[ServerUploadCallback]
Called. when the client uploads optimized model ('upload_model' method below). It should load the model and start inference session.

process_input_callback : Optional[ServerUploadCallback]
Called, when the client requests inference. Should return after inference is completed.

download_output_callback : Optional[ServerDownloadCallback]
Called, when the client requests inference output, should return status of the server and the output.

download_stats_callback : Optional[ServerDownloadCallback]
Called, when the client requests inference stats, should end the inference session and return status of the server and stats.

upload_iospec_callback : Optional[ServerUploadCallback]
Called to upload model input/output specifications (iospec).

upload_optimizers_callback : Optional[ServerUploadCallback]
Called to upload optimizer config (serialized JSON) - 'upload_optimizers' method call by the client.

upload_unoptimized_model_callback : Optional[ServerUploadCallback]
Called to upload an unoptimized ML model, should save it.

download_optimized_model_callback : Optional[ServerDownloadCallback]
Called, when client requests optimization of the model (uploaded with 'upload_unoptimized_model_callback'). Should optimize the model and return it.

upload_runtime_callback : Optional[ServerUploadCallback]
Called, when the client uploads runtime.

Implementations of the `Protocol` class should then implement the `serve` method to receive client requests (made for example by the client calling `upload_input` or `request_optimization`) and call appropriate callbacks (for example `upload_input_callback`).

Note that the `request_optimization` is a unique client request which should result in two server callbacks being called - first `upload_unoptimized_model_callback` and then `download_optimized_model_callback`.

There is no callback for the `upload_runtime` request, since this functionality is not supported by Kenning (only by certain other target platforms, like Kenning Zephyr Runtime).

Some callbacks (`ServerUploadCallback`) take `bytes` as an argument and return a `ServerStatus` object.

Other callbacks (`ServerDownloadCallback`) take no argument and return a tuple: `(ServerStatus, bytes)`.

`ServerStatus` is a class meant to store the status of a server (target) and convey it to the client.

It stores information on what the server was last doing (through the `ServerAction` enum) and whether that action succeeded or failed, or whether it's still ongoing.

Available values for ServerAction:

- `WAITING_FOR_CLIENT`
- `CLIENT_CONNECTED`
- `UPLOADING_IOSPEC`
- `UPLOADING_MODEL`
- `UPLOADING_UNOPTIMIZED_MODEL`
- `UPLOADING_INPUT`
- `PROCESSING_INPUT`
- `EXTRACTING_OUTPUT`
- `COMPUTING_STATISTICS`
- `OPTIMIZING_MODEL`
- `UPLOADING_OPTIMIZERS`
- `UPLOADING_RUNTIME`
- `IDLE`

class kenning.core.protocol.**ServerStatus**(last_action: *ServerAction* = ServerAction.WAITING_FOR_CLIENT, success: *bool* = **True**)

Bases: `object`

Class denoting status of an inference server (last executed action and information whether is succeeded or not).

finish_action(success: *bool*)

Updates the status with information about current action being finished and whether is succeeded or not.

Parameters

success : bool
True if action succeeded, False if not.

start_action(action: *ServerAction*)

Updates the status with information about new task started by the server. Sets the 'success' value to None (which means, that the action is in progress).

Parameters

action : ServerAction
Action being started.

20.10.3 Protocol methods

class kenning.core.protocol.**Protocol**(timeout: *int* = -1)

Bases: ArgumentsHandler, ABC

The interface for the communication protocol with the target devices.

The target device acts as a server in the communication.

The machine that runs the benchmark and collects the results is the client for the target device.

The inheriting classes for this class implement at least the client-side of the communication with the target device.

deduce_data_converter_from_io_spec(io_specification: *dict* | *Path* | *None* = **None**) → *DataConverter*

Function that reads model IO specification and then tries to deduce optimal data converter type.

Parameters

io_specification : Optional[Union[Dict, Path]]
Dictionary with IO specification or Path to file with it.

Returns
Optimal DataConverter type.

Return type
DataConverter

abstractmethod disconnect()

Ends connection with the other side.

abstractmethod download_output() → tuple[bool, Any | None]

Downloads the outputs from the target device.

Requests and downloads the latest inference output from the target device for quality measurements.

Returns
Tuple with download status (True if successful) and downloaded data.

Return type
Tuple[bool, Optional[Any]]

abstractmethod download_statistics(final: *bool* = **False**) → *Measurements*

Downloads inference statistics from the target device.

By default no statistics are gathered.

Parameters

final : bool
If the inference is finished

Returns
Inference statistics on target device.

Return type
Measurements

classmethod from_argparse(args: *Namespace*) → *Protocol*

Constructor wrapper that takes the parameters from argparse args.

Parameters

args : Namespace
Arguments from Protocol object.

Returns
Object of class Protocol.

Return type
Protocol

classmethod from_json(json_dict: *dict*) → *Protocol*

Constructor wrapper that takes the parameters from json dict.

This function checks if the given dictionary is valid according to the `arguments_structure` defined. If it is then it invokes the constructor.

Parameters

json_dict : Dict
Arguments for the constructor.

Returns
Object of class Protocol.

Return type
Protocol

abstractmethod initialize_client() → bool

Initializes client side of the protocol.

The client side is supposed to run on host testing the target hardware.

The parameters for the client should be provided in the constructor.

Returns
True if succeeded.

Return type
bool

abstractmethod initialize_server(client_connected_callback: *Callable[[Any], None] | None* = **None**, client_disconnected_callback: *Callable[[None], None] | None* = **None**) → bool

Initializes server side of the protocol.

The server side is supposed to run on target hardware.

The parameters for the server should be provided in the constructor.

Parameters

client_connected_callback : Optional[Callable[Any, None]]
Called when a client connects to the server. Either IP address or another distinguishing characteristic of the client will be passed to the callback (depending on the underlying protocol).

client_disconnected_callback : Optional[Callable[None, None]]
Called, when the current client disconnects from the server.

Returns
True if succeeded.

Return type
bool

abstractmethod listen_to_server_logs()
Starts continuously receiving and printing logs sent by the server.

abstractmethod listen_to_trace_data(tracedump_callback: *Callable[[bytes], None]*)
Starts continuously receiving batches of trace data from server. Calls a callback function with each received batch.

Parameters

tracedump_callback : Callable[[bytes], None]
Function, that will be called for each received batch of data.

abstractmethod request_optimization(model_path: ~pathlib.Path | None, get_time_func: ~typing.Callable[[], float] = <built-in function perf_counter>) → tuple[bool, bytes | None]
Request optimization of model.

Parameters

model_path : Optional[Path]
Path to the model for optimization. If None, empty bytestream will be sent.

get_time_func : Callable[[], float]
Function that returns current timestamp.

Returns
First element is equal to True if optimization finished successfully and the second element contains compiled model.

Return type
Tuple[bool, Optional[bytes]]

abstractmethod request_processing(get_time_func: ~typing.Callable[[], float] = <built-in function perf_counter>) → bool
Requests processing of input data and waits for acknowledgement.

This method triggers inference on target device and waits until the end of inference on target device is reached.

This method measures processing time on the target device from the level of the host.

Target may send its own measurements in the statistics.

Parameters

get_time_func : Callable[[], float]
Function that returns current timestamp.

Returns
True if inference finished successfully.

Return type
bool

abstractmethod serve(upload_input_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_model_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, process_input_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, download_output_callback: *Callable[[None], tuple[ServerStatus, bytes | None]]* | *None* = **None**, download_stats_callback: *Callable[[None], tuple[ServerStatus, bytes | None]]* | *None* = **None**, upload_iospec_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_optimizers_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, upload_unoptimized_model_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**, download_optimized_model_callback: *Callable[[None], tuple[ServerStatus, bytes | None]]* | *None* = **None**, upload_runtime_callback: *Callable[[bytes], ServerStatus]* | *None* = **None**)

Waits for requests from the other device (the client) and calls appropriate callbacks. Some callbacks take bytes as arguments, all return ServerStatus and some also return bytes. These responses are then sent back to the client.

Callbacks are guaranteed to be executed in the order requests were sent in.

This method is non-blocking.

Parameters

upload_input_callback : Optional[ServerUploadCallback]
Called, when the client uploads input ('upload_input' method below). Should upload model input into the runtime.

upload_model_callback : Optional[ServerUploadCallback]
Called. when the client uploads optimized model ('upload_model' method below). It should load the model and start inference session.

process_input_callback : Optional[ServerUploadCallback]
Called, when the client requests inference. Should return after inference is completed.

download_output_callback : Optional[ServerDownloadCallback]
Called, when the client requests inference output, should return status of the server and the output.

download_stats_callback : Optional[ServerDownloadCallback]
Called, when the client requests inference stats, should end the inference session and return status of the server and stats.

upload_iospec_callback : Optional[ServerUploadCallback]
Called to upload model input/output specifications (iospec).

upload_optimizers_callback : Optional[ServerUploadCallback]

Called to upload optimizer config (serialized JSON) - 'upload_optimizers' method call by the client.

upload_unoptimized_model_callback : Optional[ServerUploadCallback]
Called to upload an unoptimized ML model, should save it.

download_optimized_model_callback : Optional[ServerDownloadCallback]
Called, when client requests optimization of the model (uploaded with 'upload_unoptimized_model_callback'). Should optimize the model and return it.

upload_runtime_callback : Optional[ServerUploadCallback]
Called, when the client uploads runtime.

abstractmethod start_sending_logs()

Starts sending over the protocol all logs, except for the logs generated by the operation of sending logs itself (otherwise we would have an infinite logging recursion).

abstractmethod stop_sending_logs()

Stops sending logs over the protocol.

abstractmethod upload_input(data: *Any*) → bool

Uploads input to the target device and waits for acknowledgement.

This method should wait until the target device confirms the data is delivered and preprocessed for inference.

Parameters

data : Any
Input data for inference.

Returns
True if ready for inference.

Return type
bool

abstractmethod upload_io_specification(path: *Path* | *None*) → bool

Uploads input/output specification to the target device.

This method takes the specification in a json format from the given Path and sends it to the target device.

This method should receive the status of uploading the data to the target.

Parameters

path : Optional[Path]
Path to the json file. If None, empty bytestream will be sent.

Returns
True if data upload finished successfully.

Return type
bool

abstractmethod upload_model(path: *Path* | *None*) → bool

Uploads the model to the target device.

This method takes the model from given Path and sends it to the target device.

This method should receive the status of uploading the model from the target.

Parameters

path : Optional[Path]
Path to the model. If None, empty bytestream will be sent.

Returns
True if model upload finished successfully.

Return type
bool

abstractmethod upload_optimizers(optimizers_cfg: *dict[str, Any]*) → bool

Upload optimizers config to the target device.

Parameters

optimizers_cfg : Dict[str, Any]
Config JSON of optimizers.

Returns
True if data upload finished successfully.

Return type
bool

abstractmethod upload_runtime(path: *Path* | *None*) → bool

Uploads the runtime to the target device.

This method takes the binary from given Path and sends it to the target device.

This method should receive the status of runtime loading from the target.

Parameters

path : Optional[Path]
Path to the runtime binary. If None, empty bytestream will be sent.

Returns
True if runtime upload finished successfully.

Return type
bool

20.11 DataConverter

`kennning.core.dataconverter.DataConverter` - based classes are responsible for:

- converting data to the format expected by surrounding block,
- converting data from the surrounding block format to one previous block excepts.

The `DataConverter` objects are used by PipelineRunner during inference.

The available implementations of dataconverter are included in the `kenning.dataconverters` submodule. Example implementations:

- ModelWrapperDataConverter implement conversions by utilizing the *ModelWrapper* classes,
- ROS2DataConverter performs conversions by utilizing ROS2 action.

class kenning.core.dataconverter.**DataConverter**

Bases: `ArgumentsHandler`, `ABC`

Performs conversion of data between two surrounding blocks.

This class provides an API used by Runtimes during inference execution.

Each DataConverter should implement methods for:

- converting data from dataset to the format used by the surrounding block.
- converting data from format used by the surrounding block to the

inference output.

abstractmethod to_next_block(data: *Any*) → Any

Converts data to the format used by the surrounding block.

Parameters

data : Any
Data to be converted.

Returns
Converted data.

Return type
Any

abstractmethod to_previous_block(data: *Any*) → Any

Converts data from the format used by the surrounding block to one previous block expects.

Parameters

data : Any
Data to be converted.

Returns
Converted data.

Return type
Any

20.12 Measurements

The `kenning.core.measurements` module contains `Measurements` and `MeasurementsCollector` classes for collecting performance and quality metrics. `Measurements` is a dict-like object that provides various methods for adding performance metrics, adding values for time series, and updating existing values.

The dictionary held by `Measurements` requires serializable data, since most scripts save performance results in JSON format for later report generation. Module containing decorators for benchmark data gathering.

class kenning.core.measurements.**Measurements**

Stores benchmark measurements for later processing.

This is a dict-like object that wraps all processing results for later report generation.

The dictionary in Measurements has measurement type as a key, and list of values for given measurement type.

There can be other values assigned to a given measurement type than list, but it requires explicit initialization.

data

Dictionary storing lists of values.

Type
dict

accumulate(measurementtype: str, valuetoadd: ~typing.Any, initvaluefunc: ~typing.Callable[[], ~typing.Any] = <function Measurements.<lambda>>)

Adds given value to a measurement.

Deprecated since version 0.8.2: Method *accumulate* is deprecated, please use *add_measurement* instead.

This function adds given value (it can be integer, float, numpy array, or any type that implements iadd operator).

If it is the first assignment to a given measurement type, the first list element is initialized with the `initvaluefunc` (function returns the initial value).

Parameters

measurementtype : str
The name of the measurement.

valuetoadd : Any
New value to add to the measurement.

initvaluefunc : Callable[[], Any]
The initial value of the measurement, default 0.

add_measurement(measurementtype: str, value: ~typing.Any, initialvaluefunc: ~typing.Callable[[], ~typing.Any] = <function Measurements.<lambda>>)

Add new value to a given measurement type.

Parameters

measurementtype : str
The measurement type to be updated.

value : Any
The value to add.

initialvaluefunc : Callable[[], Any]
The initial value for the measurement.

clear()

Clears measurement data.

copy()

Makes copy of measurements data.

get_values(measurementtype: *str*) → list

Returns list of values for a given measurement type.

Parameters

measurementtype : str
The name of the measurement type.

Returns
List of values for a given measurement type.

Return type
List

initialize_measurement(measurement_type: *str*, value: *Any*)

Sets the initial value for a given measurement type.

By default, the initial values for every measurement are empty lists. Lists are meant to collect time series data and other probed measurements for further analysis.

In case the data is collected in a different container, it should be configured explicitly.

Parameters

measurement_type : str
The type (name) of the measurement.

value : Any
The initial value for the measurement type.

update_measurements(other: *dict* | Measurements)

Adds measurements of types given in the other object.

It requires another Measurements object, or a dictionary that has string keys and values that are lists of values. The lists from the other object are appended to the lists in this object.

Parameters

other : Union[Dict, 'Measurements']
A dictionary or another Measurements object that contains lists in every entry.

class kenning.core.measurements.**MeasurementsCollector**

It is a 'static' class collecting measurements from various sources.

classmethod clear()

Clears measurement data.

classmethod save_measurements(resultpath: *Path*)

Saves measurements to JSON file.

Parameters

resultpath : Path
Path to the saved JSON file.

classmethod set_unoptimized(optimized_measurementspath: *Path*, unoptimized_measurementspath: *Path*, remove_unoptimized_measurementsfile: *bool* = **True**)

Copies unoptimized model measurements to *UNOPTIMIZED* field of the optimized model measurements.

Parameters

optimized_measurementspath : Path
Path to the optimized model measurements.

unoptimized_measurementspath : Path
Path to the unoptimized model measurements.

remove_unoptimized_measurementsfile : bool
Determines whether the unoptimized model measurements should be deleted.

class kenning.core.measurements.**RemoteSystemStatsCollector**(prefix: *str*, protocol: *Any*, step: *float* = 0.1)

It is a separate thread used for collecting system statistics. From a remote machine.

It collects, if provided:

- CPU utilization,
- RAM utilization,
- GPU utilization,
- GPU Memory utilization.

It can be executed in parallel to another function to check its utilization of resources on the remote machine.

get_measurements() → *Measurements*

Returns measurements from the remote machine.

Collected measurements names are prefixed by the prefix given in the constructor.

The list of measurements:

- <*prefix*>*_cpus_percent*: gives per-core CPU utilization (%),
- <*prefix*>*_mem_percent*: gives overall memory usage (%),
- <*prefix*>*_gpu_utilization*: gives overall GPU utilization (%),
- <*prefix*>*_gpu_mem_utilization*: gives overall memory utilization (%),
- <*prefix*>*_timestamp*: gives the timestamp of above measurements (ns).

Returns
Measurements object.

Return type
Measurements

run()
Method representing the thread's activity.

You may override this method in a subclass. The standard run() method invokes the callable object passed to the object's constructor as the target argument, if any, with sequential and keyword arguments taken from the args and kwargs arguments, respectively.

class kenning.core.measurements.**SystemStatsCollector**(prefix: *str*, step: *float* = 0.1, pid: *int* = 14939)

It is a separate thread used for collecting system statistics.

It collects:

- CPU utilization,
- RAM utilization,
- GPU utilization,
- GPU Memory utilization.

It can be executed in parallel to another function to check its utilization of resources.

class NvidiaGpuProcessInfo(gpu_index: *int*, kenning_process: *Any*, process_newest_usage: *Any*)

A dataclass to store NVIDIA GPU process statistics of Kenning.

get_measurements() → *Measurements*
Returns measurements from the thread.

Collected measurements names are prefixed by the prefix given in the constructor.

The list of measurements:

- <*prefix*>*_cpus_percent*: gives per-core CPU utilization (%),
- <*prefix*>*_mem_percent*: gives overall memory usage (%),
- <*prefix*>*_gpu_utilization*: gives overall GPU utilization (%),
- <*prefix*>*_gpu_mem_utilization*: gives overall memory utilization (%),
- <*prefix*>*_timestamp*: gives the timestamp of above measurements (ns).

Returns
Measurements object.

Return type
Measurements

run()

Method representing the thread's activity.

You may override this method in a subclass. The standard run() method invokes the callable object passed to the object's constructor as the target argument, if any, with sequential and keyword arguments taken from the args and kwargs arguments, respectively.

kenning.core.measurements.**serverstatsmeasurements**(measurementname: *str*, host: *str*, port: *int* = 4371, step: *float* = 1.0) → Callable

Decorator for measuring memory usage of the function.

Check SystemStatsCollector.get_measurements for list of delivered measurements.

Parameters

measurementname : str
The name of the measurement type.

host : str
Host to connect to to get measurements.

port : int
Port on which measurements will be served

step : float
The step for the measurements, in seconds.

Returns
Decorated function.

Return type
Callable

kenning.core.measurements.**systemstatsmeasurements**(measurementname: *str*, step: *float* = 0.5) → Callable

Decorator for measuring memory usage of the function.

Check SystemStatsCollector.get_measurements for list of delivered measurements.

Parameters

measurementname : str
The name of the measurement type.

step : float
The step for the measurements, in seconds.

Returns
Decorated function.

Return type
Callable

kenning.core.measurements.**tagmeasurements**(tagname: *str*) → Callable

Decorator for adding tags for measurements and saving their timestamps.

Parameters

tagname : str
The name of tag.

Returns
Decorated function.

Return type
Callable

kenning.core.measurements.**timemeasurements**(measurementname: str, get_time_func: ~typing.Callable[[], float] = <built-in function perf_counter>) → Callable

Decorator for measuring time of the function.

The duration is given in nanoseconds.

Parameters

measurementname : str
The name of the measurement type.

get_time_func : Callable[[], float]
Function that returns current timestamp.

Returns
Decorated function.

Return type
Callable

20.13 DataProvider

The `DataProvider` classes are used during deployment to provide data for inference. They can provide data from such sources as a camera, video files, microphone data or a TCP connection.

The available `DataProvider` implementations are included in the `kenning.dataproviders` submodule. Example implementations:

- CameraDataProvider for capturing frames from camera.

class kenning.core.dataprovider.**DataProvider**(inputs_sources: *dict[str, tuple[int, str]]* = {}, inputs_specs: *dict[str, dict]* = {}, outputs: *dict[str, str]* = {})

Bases: Runner, ABC

A block that introduces data to Kenning flow.

abstractmethod detach_from_source()
Detaches from the source during shutdown.

abstractmethod fetch_input() → Any
Gets the sample from device.

Returns
Data to be processed by the model.

Return type
Any

prepare()

Prepares the source for data gathering depending on the source type.

This will for example initialize the camera and set the self.device to it.

preprocess_input(data: *Any*) → Any

Performs provider-specific preprocessing of inputs.

Parameters

data : Any
The data to be preprocessed.

Returns
Preprocessed data.

Return type
Any

20.14 OutputCollector

The `OutputCollector` classes are used during deployment for inference results receiving and processing. They can display the results, send them, or store them in a file.

The available output collector implementations are included in the `kenning.outputcollectors` submodule. Example implementations:

- DetectionVisualizer for visualizing detection model outputs,
- BaseRealTimeVisualizer base class for real time visualizers:
  - RealTimeDetectionVisualizer for visualizing detection model outputs,
  - RealTimeSegmentationVisualizer for visualizing segmentation model outputs,
  - RealTimeClassificationVisualizer for visualizing classification model outputs.

class kenning.core.outputcollector.**OutputCollector**(inputs_sources: *dict[str, tuple[int, str]]* = {}, inputs_specs: *dict[str, dict]* = {}, outputs: *dict[str, str]* = {})

Bases: Runner, ABC

Collects outputs from models running in the Kenning flow.

It performs final processing of data running in the Kenning flow. It can be used i.e. to display predictions, save them to file or send to other application.

abstractmethod detach_from_output()

Detaches from the output during shutdown.

abstractmethod process_output(input_data: *Any*, output_data: *Any*)

Returns the inferred data back to the specific place/device/connection.

Eg. it can save a video file with bounding boxes on objects or stream it via a TCP connection, or just show it on screen.

Parameters

input_data : Any
Data collected from Datacollector that was processed by the model.

output_data : Any
Data returned from the model.

abstractmethod should_close() → bool

Checks if a specific exit condition was reached.

This allows the OutputCollector to close gracefully if an exit condition was reached, eg. when a key was pressed.

Returns
True if exit condition was reached to break the loop.

Return type
bool

20.15 ArgumentsHandler

The `ArgumentsHandler` class is responsible for concatenating `arguments_structure` and creating parsers for command line and JSON config arguments.

In order to make some class being able to be instantiated from command line arguments or JSON config it is required to inherit from this class or its child class and implement `from_argparse` or `from_json` methods as described in *Defining arguments for core classes*.

class kenning.utils.args_manager.**ArgumentsHandler**

Bases: `ABC`

Class responsible for creating parsers for arguments from command line or json configs.

The child class should define its own *arguments_structure* and from_argparse/from_json methods so that it could be instantiated from command line arguments or json config.

classmethod add_argparse_argument(group: *_ArgumentGroup*, struct: *dict[str, dict]*, args: *Namespace*, *names: *str*, override_only: *bool* = **False**)

Adds arguments to the argparse group based on the given struct and names. If no names are given it uses all properties of the struct.

Note that the function modifies the given group.

If argument with name 'argparse_name' already exists in the group the existing argument is overridden with the new one.

Parameters

group : argparse._ArgumentGroup
Argparse group that is filled with new properties.

struct : Dict[str, Dict]
Struct with properties described.

args : argparse.Namespace
Arguments from ArgumentParser object.

***names : str**
Names of the properties that are to be added to the group. If empty every property in struct is used.

override_only : bool
True if parameter set in file configuration can be overridden from *argparse*.

Raises

- **KeyError** – Raised if there is a keyword that is not recognized.
- **InvalidArgumentsError** – Raised if the argument definition is invalid.

classmethod build_from_config(config: *dict[str, dict[str, Any]]*, **kwargs: *Any*) → Any

Constructor wrapper that takes the parameters from config dict.

This function matches parameters from the config, by name and type, to 'arguments_structure' and then calls the constructor with all parameters that match (after necessary type conversions), as well as additional direct arguments specified in the config.

Parameters

config : KenningBlockConfigDict
Configuration dictionary.

****kwargs : Any**
Additional class-dependent arguments.

Returns
Instance created from provided config.

Return type
Any

classmethod cast_complex_type(value: *Any*, target_type: *type | tuple[type]*) → Any

Casts a value, or a does a recursive cast, if the value is a container.

Parameters

value : Any
Value to be converted. Can be any object, or a list, union or tuple of objects (also recursively). For example types such as: Tuple[List[List[Tuple[int]]], Tuple[str]] are supported.

target_type : type | Tuple[type]
A type or multiple types (in case of value being a container), that the value should be converted to. When type is a container type, and value is another container type - the kind of container will be preserved (this function cannot change container type, it only casts values inside containers).

Returns
Converted value.

Return type
Any

Raises

- **ArgsManagerConvertError** – Raised if the value cannot be converted to the target type.
- **ValueError** – Casting is not possible.

classmethod convert_value(value: *Any*, spec: *dict*) → Any

Converts given value to type required by a given 'arguments_structure' argument.

Parameters

value : Any
Value to be converted. Can be any object, or a list, union or tuple of objects (also recursively). For example types such as: Union[List[List[Tuple[int]]], Tuple[str]] are supported, so are enums, None values etc.

spec : Dict
A value of the 'arguments_structure' dict, for the argument to be matched.

Returns
Converted value.

Return type
Any

Raises
ArgsManagerConvertError – Thrown when value cannot be safely converted.

classmethod form_argparse(args: *Namespace*, override_only: *bool* = **False**) → tuple[ArgumentParser, _ArgumentGroup | None]

Creates argparse parser based on *arguments_structure* of class and its all parent classes.

Parameters

args : argparse.Namespace
Arguments from ArgumentParser object.

override_only : bool
True if only parameters marked as *overridable* should be parsed.

Returns
Tuple with the argument parser object that can act as parent for program's argument parser, and the corresponding arguments' group pointer.

Return type
Tuple[argparse.ArgumentParser, Optional[argparse._ArgumentGroup]]

classmethod form_parameterschema() → dict

Creates parameter schema based on *arguments_structure* of class and its all parent classes.

Returns
Parameter schema for the class.

Return type
Dict

classmethod get_argparse_flag_from_argument_structure(argument_name: *str*, argument_spec: *dict[str, Any]*) → str

Retrieves the given parameter's flag (argparse) name, from its spec dict - or creates it based on the parameter's name, if it's not available.

Parameters

argument_name : str
Name of the argument.

argument_spec : Dict[str, Any]
Argument's dictionary from arguments_structure.

Returns
Proper argparse flag name for the given parameter.

Return type
str

classmethod get_full_arguments_structure() → dict

Takes 'arguments_structure' (dicts describing user-defined block parameters) from this class, and from all parent classes in the inheritance chain, and combines them. Settings from child always override settings from parent.

Returns
Combined 'arguments_structure' from the inheritance chain.

Return type
Dict

classmethod get_prefixed_parameter_variants(parameter: *str*, flag: *bool* = **False**) → tuple[str]

Appends block-specific an block-type-specific prefixes to the given parameter name.

Can be done either for a parameter in flag name format (–example-param to –nice-prefix:–example-param) or for a one in the normal parameter name format (example_param to nice_prefix:__example_param - the double underscore is added, so that it matches the way argparse changes it).

Parameters

parameter : str
Parameter name to extend.

flag : bool
True, if you need a flag name to add to a new argparse parser. False, if you need an argument name to retrieve a value from argparse.Namespace.

Returns
Parameter name with a block name prefix, and parameter name with a block type prefix - respectively.

Return type
Tuple[str]

classmethod match_parameters(config: *dict[str, dict[str, Any]]*, suppress_errors: *bool* = **True**) → tuple[dict[str, Any], list[str]]

Retrieves and matches arguments_structure parameters from the block configuration and from the unaffiliated flags section in KenningBlockConfigDict. Sets defaults for the missing ones, where appropriate.

Parameters

config : KenningBlockConfigDict
Configuration dictionary.

suppress_errors : bool
If true, no errors will be logged in this function (useful for preliminary matching/probing).

Returns
Dictionary of resolved parameters, and a list of names of all parameters matched from the config (excluding the ones for which defaults were used), for deconfliction purposes. A prefix in the format 'block_config:<block type>:<block>:<name>' is added in the deconfliction list for all parameters that have been found in the block-specific configuration (instead of unmatched flags) to differentiate them.

Return type
Tuple[Dict[str, Any], List[str]]

Raises
ConfigurationError – Thrown, when there's a missing required parameter.

classmethod process_argument_spec(argument_name: *str*, argument_spec: *dict[str, Any]*) → dict[str, Any]

Fills missing values in the given 'arguments_structure' parameter specification dict with defaults. For non-required arguments it checks if the default value provided conforms to the specified type, and if not - marks the argument as default, emitting a warning.

Parameters

argument_name : str
Name of the argument being processed.

argument_spec : Dict[str, Any]
Value of from the arguments_structure dict.

Returns
argument_spec – Validated and filled-out specification.

Return type
Dict[str, Any]

to_json() → dict[str, Any]

Convert object to JSON that contains its type and all parameters.

Returns
JSON config of given object.

Return type
Dict[str, Any]

classmethod verify_type(value: *Any*, spec: *dict*) → bool

Checks, if the given value is of type specified by the argument spec, or can be cast to it.

Parameters

value : Any
Value to be checked. Can be any object, or a list, union or tuple of objects (also recursively).

spec : Dict
A value of the 'arguments_structure' dict, for the argument to be matched.

Returns
True if the value matches the argument. False otherwise.

Return type
bool

20.16 ResourceManager

The ResourceManager is a singleton class which handles local and remote files for Kenning, such as datasets or models. It downloads missing files, provides paths to the available files, resolves custom URL schemes for and manages resource directories for Kenning, including cleanup.

class kenning.utils.resource_manager.**ResourceManager**(*args, **kwargs)

Bases: object

Download and cache resources used by Kenning.

add_custom_url_schemes(custom_url_schemes: *dict[str, str | Callable | None]*)

Add user defined URL schemes.

Parameters

custom_url_schemes : Dict[str, Optional[Union[str, Callable]]]
Dictionary with custom url schemes entries. Each entry consists of schema and corresponding conversion. Conversion can be None, string pattern or callable returning string.

check_for_uri(uri: *str*) → str | Path | None

A function that whether a input uri string is a valid supported URI string.

Parameters

uri : str
Resource URI.

Returns
A validated URI path or None when URI is invalid.

Return type
str | Path | None

clear_cache()

Remove all cached files.

get_resource(uri: *str*, output_path: *Path* | *None* = **None**) → Path

Retrieve file and return path to it.

If the uri points to remote resource, then it is downloaded (if not found in cache) and validated.

Parameters

uri : str
Resource URI.

output_path : Optional[Path]
Path to the output file. If not provided then the path is automatically created.

Returns
Path to the retrieved resource.

Return type
Path

Raises
ChecksumVerifyError – Raised when downloaded file has invalid checksum

list_cached_files() → list[Path]

Return list with cached files.

Returns
List of cached files.

Return type
List[Path]

set_cache_dir(cache_dir_path: *Path*)

Set the cache directory path and creates it if not exists.

Parameters

cache_dir_path : Path
Path to be set as cache directory.

set_max_cache_size(max_cache_size: *int*)

Set the max cache size.

Parameters

max_cache_size : int
Max cache size in bytes.

validate_resources_version()

Retrieve Kenning resources version and check if it is compatible with currently used Kenning.

20.17 ResourceURI

The `ResourceURI` class is a `pathlib.Path`-based object allowing the user to work with Kenning resources, both using regular paths and URI schemes supported by the *ResourceManager*.

class kenning.utils.resource_manager.**ResourceURI**(uri_or_path: *str* | *Path* | ResourceURI)

Bases: `Path`

Handle access to resource used in Kenning.

property origin : str

Original string passed to the constructor.

property parent : *ResourceURI*

Parent of the URI.

property uri : str | None

URI of the resource.

with_name(name: *str*) → *ResourceURI*

Return new URI with changed name.

Parameters

name : str
New name to be used.

Returns
URI with changed name.

Return type
ResourceURI

with_stem(stem: *str*) → *ResourceURI*

Return new URI with changed stem.

Parameters

stem : str
New stem to be used.

Returns
URI with changed stem.

Return type
ResourceURI

with_suffix(suffix: *str*) → *ResourceURI*

Return new URI with changed suffix.

Parameters

suffix : str
New suffix to be used.

Returns
URI with changed suffix.

Return type
ResourceURI

20.18 ModelConverter

`kenning.core.converter.ModelConverter` objects allow conversion between deep learning frameworks (e.g. PyTorch, Keras, ONNX, TVM and more). They allow alternative paths for optimizations, when *Optimizer* objects do not support current model format, or when direct optimization fails. For conversion to be possible, a converter needs to implement `to_<destination_format>` method.

Example model converters:

- OnnxConverter - wraps conversion from ONNX,
- KerasConverter - wraps conversion from Keras.

class kenning.core.converter.**ModelConverter**(source_model_path: *Path* | ResourceURI)

Bases: ABC

Loads or converts model to specific format.

to_ai8x(ai8x_model_path: *Path*, ai8x_tools: *Ai8xTools*, model: *Any* | *None* = **None**, **kwargs) → None

Converts model to ai8x format.

Parameters

ai8x_model_path : Path
Path where ai8x-compatible model will be saved.

ai8x_tools : Ai8xTools
Ai8X tools wrapper.

model : Optional[Any]
Optional model object. If not provided, model is loaded from file.

****kwargs**
Keyword arguments passed between conversions.

to_keras(model: *Any* | *None* = **None**, **kwargs) → tf.keras.Model

Converts model to Keras format.

Parameters

model : Optional[Any]
Optional model object. If not provided, model is loaded from file.

****kwargs**
Keyword arguments passed between conversions.

Return type
tf.keras.Model

to_litert(model: *Any* | *None* = **None**, **kwargs) → tf.lite.TFLiteConverter | ai_edge_litert.compiled_model.CompiledModel

Converts model to TensorFlowLite format.

Parameters

model : Optional[Any]
Optional model object. If not provided, model is loaded from file.

****kwargs**
Keyword arguments passed between conversions.

Returns
Either a LiteRT model, or a TFLiteConverer object.

Return type
Union[tf.lite.TFLiteConverter, ai_edge_litert.compiled_model.CompiledModel]

to_mlir(io_spec: *dict[str, list[dict]]*, model: *Any* | *None* = **None**, **kwargs) → str

Converts model to MLIR format.

Parameters

io_spec : Dict[str, List[Dict]]
Input and output specification.

model : Optional[Any]
Optional model object. If not provided, model is loaded from file.

****kwargs**
Keyword arguments passed between conversions.

Returns
Model code in a MLIR format.

Return type
str

to_onnx(model: *Any* | *None* = **None**, **kwargs) → onnx.ModelProto

Converts model to ONNX format.

Parameters

model : Optional[Any]
Optional model object. If not provided, model is loaded from file.

****kwargs**
Keyword arguments passed between conversions.

Returns
Loaded ONNX model.

Return type
onnx.ModelProto

to_torch(model: *Any* | *None* = **None**, **kwargs) → torch.nn.Module

Converts model to PyTorch format.

Parameters

model : Optional[Any]
Optional model object. If not provided, model is loaded from file.

****kwargs**
Keyword arguments passed between conversions.

Returns
Torch Model.

Return type
torch.nn.Module

to_tvm(io_spec: *dict[str, list[dict]]*, model: *Any | None* = **None**, ***kwargs*) → tuple[tvm.IRModule, dict | str]

Converts model to TVM format.

Parameters

io_spec : Dict[str, List[Dict]]
Input and output specification.

model : Optional[Any]
Optional model object. If not provided, model is loaded from file.

****kwargs**
Keyword arguments passed between conversions.

Returns

- **mod** (*tvm.IRModule*) – The relay module.
- **params** (*Union[Dict, str]*) – Parameters dictionary to be used by relay module.

to_tvm_relax(io_spec: *dict[str, list[dict]]*, model: *Any | None* = **None**, ***kwargs*) → tvm.IRModule

Converts model to TVM Relax module.

Parameters

io_spec : Dict[str, List[Dict]]
Input and output specification.

model : Optional[Any]
Optional model object. If not provided, model is loaded from file.

****kwargs**
Keyword arguments passed between conversions.

Returns
The Relax module, with weights embedded as constants.

Return type
tvm.IRModule

PYTHON MODULE INDEX

k

kenning.core.measurements, 307

INDEX

Symbols

_dataindex (*kenning.core.dataset.Dataset attribute*), 269

A

accept_client() (*kenning.protocols.network.NetworkProtocol method*), 245
accumulate() (*kenning.core.measurements.Measurements method*), 307
add_argparse_argument() (*kenning.utils.args_manager.ArgumentsHandler class method*), 314
add_custom_url_schemes() (*kenning.utils.resource_manager.ResourceManager method*), 319
add_measurement() (*kenning.core.measurements.Measurements method*), 307
ArgumentsHandler (*class in kenning.utils.args_manager*), 314
AutoML (*class in kenning.core.automl*), 294
AutoMLModel (*class in kenning.core.automl*), 293

B

batch_size (*kenning.core.dataset.Dataset attribute*), 269
begin_evaluate() (*kenning.core.dataset.Dataset method*), 270
build_from_config() (*kenning.utils.args_manager.ArgumentsHandler class method*), 315
BytesBasedProtocol (*class in kenning.protocols.bytes_based_protocol*), 226

C

calibration_dataset_generator() (*kenning.core.dataset.Dataset method*), 270
cast_complex_type() (*kenning.utils.args_manager.ArgumentsHandler class method*), 315
check_for_uri() (*kenning.utils.resource_manager.ResourceManager method*), 319
check_status() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 226
cleanup() (*kenning.core.runner.Runner method*), 267
clear() (*kenning.core.measurements.Measurements method*), 308
clear() (*kenning.core.measurements.MeasurementsCollector class method*), 308
clear_cache() (*kenning.utils.resource_manager.ResourceManager method*), 319
compile() (*kenning.core.optimizer.Optimizer method*), 282
convert_input_to_bytes() (*kenning.core.model.ModelWrapper method*), 276
convert_output_from_bytes() (*kenning.core.model.ModelWrapper method*), 276
convert_value() (*kenning.utils.args_manager.ArgumentsHandler class method*), 316
copy() (*kenning.core.measurements.Measurements method*), 308

D

data (*kenning.core.measurements.Measurements attribute*), 307
DataConverter (*class in kenning.core.dataconverter*), 306
DataProvider (*class in kenning.core.dataprovider*), 312

Dataset (*class in kenning.core.dataset*), 268
dataX (*kenning.core.dataset.Dataset attribute*), 268
dataXtest (*kenning.core.dataset.Dataset attribute*), 269
dataXtrain (*kenning.core.dataset.Dataset attribute*), 269
dataXval (*kenning.core.dataset.Dataset attribute*), 269
dataY (*kenning.core.dataset.Dataset attribute*), 269
dataYtest (*kenning.core.dataset.Dataset attribute*), 269
dataYtrain (*kenning.core.dataset.Dataset attribute*), 269
dataYval (*kenning.core.dataset.Dataset attribute*), 269
deduce_data_converter_from_io_spec() (*kenning.core.protocol.Protocol method*), 300
deduce_data_converter_from_io_spec() (*kenning.protocols.ros2.ROS2Protocol method*), 248
deinit() (*kenning.core.platform.Platform method*), 292
derive_io_spec_from_json_params() (*kenning.core.model.ModelWrapper class method*), 277
detach_from_output() (*kenning.core.outputcollector.OutputCollector method*), 313
detach_from_source() (*kenning.core.dataprovider.DataProvider method*), 312
disconnect() (*kenning.core.protocol.Protocol method*), 300
disconnect() (*kenning.protocols.network.NetworkProtocol method*), 246
disconnect() (*kenning.protocols.ros2.ROS2Protocol method*), 248
download_dataset_fun() (*kenning.core.dataset.Dataset method*), 270
download_output() (*kenning.core.protocol.Protocol method*), 300
download_output() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 226
download_output() (*kenning.protocols.ros2.ROS2Protocol method*), 248
download_statistics() (*kenning.core.protocol.Protocol method*), 300
download_statistics() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 227
download_statistics() (*kenning.protocols.ros2.ROS2Protocol method*), 249

E

end_evaluate() (*kenning.core.dataset.Dataset method*), 270
evaluate() (*kenning.core.dataset.Dataset method*), 270
event_active() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 227
event_active() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 237
extract_model() (*kenning.core.automl.AutoMLModel method*), 293
extract_output() (*kenning.core.runtime.Runtime method*), 286

F

fetch_input() (*kenning.core.dataprovider.DataProvider method*), 312
finish_action() (*kenning.core.protocol.ServerStatus method*), 299
finish_event() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 238
form_argparse() (*kenning.utils.args_manager.ArgumentsHandler class method*), 316
form_automl_schema() (*kenning.core.automl.AutoMLModel class method*), 294
form_parameterschema() (*kenning.core.flow.KenningFlow class method*), 266
form_parameterschema() (*kenning.utils.args_manager.ArgumentsHandler*

class method), 316
from_argparse() (kenning.core.automl.AutoML class method), 294
from_argparse() (kenning.core.dataset.Dataset class method), 271
from_argparse() (kenning.core.model.ModelWrapper class method), 277
from_argparse() (kenning.core.optimizer.Optimizer class method), 282
from_argparse() (kenning.core.protocol.Protocol class method), 300
from_argparse() (kenning.core.runtime.Runtime class method), 287
from_json() (kenning.core.automl.AutoML class method), 295
from_json() (kenning.core.dataset.Dataset class method), 271
from_json() (kenning.core.flow.KenningFlow class method), 266
from_json() (kenning.core.model.ModelWrapper class method), 277
from_json() (kenning.core.optimizer.Optimizer class method), 282
from_json() (kenning.core.protocol.Protocol class method), 301
from_json() (kenning.core.runtime.Runtime class method), 287

G

get_argparse_flag_from_argument_structure() (kenning.utils.args_manager.ArgumentsHandler class method), 317
get_available_ram() (kenning.core.runtime.Runtime static method), 287
get_best_configs() (kenning.core.automl.AutoML method), 295
get_class_names() (kenning.core.dataset.Dataset method), 271
get_data() (kenning.core.dataset.Dataset method), 271
get_data_unloaded() (kenning.core.dataset.Dataset method), 272
get_default_protocol() (kenning.core.platform.Platform method), 292
get_framework() (kenning.core.model.ModelWrapper class method), 278
get_framework() (kenning.core.optimizer.Optimizer method), 283
get_framework_version() (kenning.core.model.ModelWrapper class method), 278
get_framework_version() (kenning.core.optimizer.Optimizer class method), 283
get_full_arguments_structure() (kenning.utils.args_manager.ArgumentsHandler class method), 317
get_input_formats() (kenning.core.optimizer.Optimizer class method), 283
get_input_formats() (kenning.core.runtime.Runtime class method), 287
get_input_mean_std() (kenning.core.dataset.Dataset method), 272
get_input_type() (kenning.core.optimizer.Optimizer method), 283
get_io_spec_path() (kenning.core.runtime.Runtime static method), 288
get_io_specification() (kenning.core.model.ModelWrapper method), 278
get_io_specification_from_model() (kenning.core.model.ModelWrapper method), 278
get_measurements() (kenning.core.measurements.RemoteSystemStatsCollector method), 309
get_measurements() (kenning.core.measurements.SystemStatsCollector method), 310
get_model_class() (kenning.core.optimizer.Optimizer method), 283
get_model_size() (kenning.

ning.core.model.ModelWrapper method), 278
get_optimized_model_size() (*kenning.core.optimizer.Optimizer method*), 284
get_output_formats() (*kenning.core.model.ModelWrapper class method*), 279
get_output_formats() (*kenning.core.optimizer.Optimizer class method*), 284
get_path() (*kenning.core.model.ModelWrapper method*), 279
get_prefixed_parameter_variants() (*kenning.utils.args_manager.ArgumentsHandler class method*), 317
get_resource() (*kenning.utils.resource_manager.ResourceManager method*), 320
get_spec_path() (*kenning.core.optimizer.Optimizer static method*), 284
get_statistics() (*kenning.core.automl.AutoML method*), 295
get_time() (*kenning.core.platform.Platform method*), 293
get_time() (*kenning.core.runtime.Runtime method*), 288
get_values() (*kenning.core.measurements.Measurements method*), 308

I

infer() (*kenning.core.runtime.Runtime method*), 288
inference_session_end() (*kenning.core.runtime.Runtime method*), 288
inference_session_start() (*kenning.core.runtime.Runtime method*), 288
inference_step_callback() (*kenning.core.platform.Platform method*), 293
init() (*kenning.core.optimizer.Optimizer method*), 284
init() (*kenning.core.platform.Platform method*), 293
initialize_client() (*kenning.core.protocol.Protocol method*), 301
initialize_client() (*kenning.protocols.network.NetworkProtocol method*), 246
initialize_client() (*kenning.protocols.ros2.ROS2Protocol method*), 249
initialize_measurement() (*kenning.core.measurements.Measurements method*), 308
initialize_server() (*kenning.core.protocol.Protocol method*), 301
initialize_server() (*kenning.protocols.network.NetworkProtocol method*), 246
initialize_server() (*kenning.protocols.ros2.ROS2Protocol method*), 249
iter_test() (*kenning.core.dataset.Dataset method*), 272
iter_train() (*kenning.core.dataset.Dataset method*), 272
iter_val() (*kenning.core.dataset.Dataset method*), 272

K

kenning.core.measurements module, 307
KenningFlow (*class in kenning.core.flow*), 266
KenningProtocol (*class in kenning.protocols.kenning_protocol*), 237
kill_event() (*kenning.protocols.bytes_based_protocol.BytesBasedProtoco method*), 227
kill_event() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 238

L

list_cached_files() (*kenning.utils.resource_manager.ResourceManager method*), 320
listen() (*kenning.protocols.bytes_based_protocol.BytesBasedPr method*), 227
listen() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 238
listen_blocking() (*kenning.protocols.bytes_based_protocol.BytesBasedProtoco method*), 228

listen_blocking() (kenning.protocols.kenning_protocol.KenningProtocol method), 239
listen_to_server_logs() (kenning.core.protocol.Protocol method), 302
listen_to_server_logs() (kenning.protocols.bytes_based_protocol.BytesBasedProtocol method), 229
listen_to_server_logs() (kenning.protocols.ros2.ROS2Protocol method), 249
listen_to_trace_data() (kenning.core.protocol.Protocol method), 302
listen_to_trace_data() (kenning.protocols.bytes_based_protocol.BytesBasedProtocol method), 229
listen_to_trace_data() (kenning.protocols.ros2.ROS2Protocol method), 250
load_input() (kenning.core.runtime.Runtime method), 289
load_input_from_bytes() (kenning.core.runtime.Runtime method), 289
load_io_specification() (kenning.core.optimizer.Optimizer method), 284
load_model() (kenning.core.model.ModelWrapper method), 279

M

match_parameters() (kenning.utils.args_manager.ArgumentsHandler class method), 318
Measurements (class in kenning.core.measurements), 307
MeasurementsCollector (class in kenning.core.measurements), 308
ModelConverter (class in kenning.core.converter), 322
ModelWrapper (class in kenning.core.model), 276
module
    kenning.core.measurements, 307

N

NetworkProtocol (class in kenning.protocols.network), 245

O

Optimizer (class in kenning.core.optimizer), 281
origin (kenning.utils.resource_manager.ResourceURI property), 321
OutputCollector (class in kenning.core.outputcollector), 313

P

parent (kenning.utils.resource_manager.ResourceURI property), 321
parse_io_specification_from_json() (kenning.core.model.ModelWrapper class method), 279
Platform (class in kenning.core.platform), 292
post_init() (kenning.core.platform.Platform method), 293
postprocess_outputs() (kenning.core.model.ModelWrapper method), 279
prepare() (kenning.core.dataprovider.DataProvider method), 312
prepare() (kenning.core.dataset.Dataset method), 273
prepare_external_calibration_dataset() (kenning.core.dataset.Dataset method), 273
prepare_framework() (kenning.core.automl.AutoML method), 296
prepare_input_samples() (kenning.core.dataset.Dataset method), 273
prepare_io_specification() (kenning.core.runtime.Runtime method), 289
prepare_local() (kenning.core.runtime.Runtime method), 290
prepare_model() (kenning.core.model.ModelWrapper method), 280
prepare_model() (kenning.core.runtime.Runtime method), 290
prepare_output_samples() (kenning.core.dataset.Dataset method), 273
preprocess_input() (kenning.core.dataprovider.DataProvider

method), 313
preprocess_input() (*kenning.core.model.ModelWrapper method*), 280
preprocess_model_to_upload() (*kenning.core.runtime.Runtime method*), 290
process_argument_spec() (*kenning.utils.args_manager.ArgumentsHandler class method*), 318
process_output() (*kenning.core.outputcollector.OutputCollector method*), 313
Protocol (*class in kenning.core.protocol*), 300

R

read_data_from_platforms_yaml() (*kenning.core.platform.Platform method*), 293
read_io_specification() (*kenning.core.runtime.Runtime method*), 290
read_platform() (*kenning.core.model.ModelWrapper method*), 280
read_platform() (*kenning.core.optimizer.Optimizer method*), 285
read_platform() (*kenning.core.runtime.Runtime method*), 291
receive_data() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 240
receive_data() (*kenning.protocols.network.NetworkProtocol method*), 246
receive_message() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 240
receiver() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 240
RemoteSystemStatsCollector (*class in kenning.core.measurements*), 309
request() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 229
request() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 240
request_blocking() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 230
request_blocking() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 241
request_optimization() (*kenning.core.protocol.Protocol method*), 302
request_optimization() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 231
request_optimization() (*kenning.protocols.ros2.ROS2Protocol method*), 250
request_processing() (*kenning.core.protocol.Protocol method*), 302
request_processing() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 231
request_processing() (*kenning.protocols.ros2.ROS2Protocol method*), 250
ResourceManager (*class in kenning.utils.resource_manager*), 319
ResourceURI (*class in kenning.utils.resource_manager*), 321
ROS2Protocol (*class in kenning.protocols.ros2*), 248
run() (*kenning.core.flow.KenningFlow method*), 267
run() (*kenning.core.measurements.RemoteSystemStatsCollector method*), 310
run() (*kenning.core.measurements.SystemStatsCollector method*), 310
run() (*kenning.core.runner.Runner method*), 267
run() (*kenning.core.runtime.Runtime method*), 291
run_compatibility_checks() (*kenning.core.optimizer.Optimizer method*), 285
run_compatibility_checks() (*kenning.core.runtime.Runtime method*), 291
run_event() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 242
run_event_blocking() (*kenning.protocols.kenning_protocol.KenningProtocol*

method), 242
run_inference() (*kenning.core.model.ModelWrapper method*), 280
run_single_step() (*kenning.core.flow.KenningFlow method*), 267
run_test_inference() (*kenning.core.model.ModelWrapper method*), 281
Runner (*class in kenning.core.runner*), 267
Runtime (*class in kenning.core.runtime*), 286

S

save_dataset_checksum() (*kenning.core.dataset.Dataset method*), 274
save_io_specification() (*kenning.core.optimizer.Optimizer method*), 285
save_measurements() (*kenning.core.measurements.MeasurementsCollector class method*), 308
save_model() (*kenning.core.model.ModelWrapper method*), 281
save_to_onnx() (*kenning.core.model.ModelWrapper method*), 281
search() (*kenning.core.automl.AutoML method*), 296
send_data() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 243
send_data() (*kenning.protocols.network.NetworkProtocol method*), 247
send_message() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 243
send_messages() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 243
serve() (*in module kenning.core.protocol.Protocol*), 297
serve() (*kenning.core.protocol.Protocol method*), 303
serve() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 231
serve() (*kenning.protocols.ros2.ROS2Protocol method*), 250
serverstatsmeasurements() (*in module kenning.core.measurements*), 311
ServerStatus (*class in kenning.core.protocol*), 299
set_batch_size() (*kenning.core.dataset.Dataset method*), 274
set_cache_dir() (*kenning.utils.resource_manager.ResourceManager method*), 320
set_compiled_model_path() (*kenning.core.optimizer.Optimizer method*), 286
set_input_type() (*kenning.core.optimizer.Optimizer method*), 286
set_max_cache_size() (*kenning.utils.resource_manager.ResourceManager method*), 320
set_unoptimized() (*kenning.core.measurements.MeasurementsCollector class method*), 309
should_close() (*kenning.core.outputcollector.OutputCollector method*), 314
should_close() (*kenning.core.runner.Runner method*), 267
start() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 243
start_action() (*kenning.core.protocol.ServerStatus method*), 299
start_sending_logs() (*kenning.core.protocol.Protocol method*), 304
start_sending_logs() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 233
start_sending_logs() (*kenning.protocols.ros2.ROS2Protocol method*), 252
stop() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 244
stop_sending_logs() (*kenning.core.protocol.Protocol method*), 304
stop_sending_logs() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 233
stop_sending_logs() (*kenning.protocols.ros2.ROS2Protocol*

method), 252
SystemStatsCollector (*class in kenning.core.measurements*), 310
SystemStatsCollector.NvidiaGpuProcessInfo (*class in kenning.core.measurements*), 310
systemstatsmeasurements() (*in module kenning.core.measurements*), 311

T

tagmeasurements() (*in module kenning.core.measurements*), 311
test_subset_len() (*kenning.core.dataset.Dataset method*), 274
timemeasurements() (*in module kenning.core.measurements*), 312
to_ai8x() (*kenning.core.converter.ModelConverter method*), 322
to_json() (*kenning.utils.args_manager.ArgumentsHandler method*), 318
to_keras() (*kenning.core.converter.ModelConverter method*), 322
to_litert() (*kenning.core.converter.ModelConverter method*), 322
to_mlir() (*kenning.core.converter.ModelConverter method*), 323
to_next_block() (*kenning.core.dataconverter.DataConverter method*), 306
to_onnx() (*kenning.core.converter.ModelConverter method*), 323
to_previous_block() (*kenning.core.dataconverter.DataConverter method*), 306
to_torch() (*kenning.core.converter.ModelConverter method*), 323
to_tvm() (*kenning.core.converter.ModelConverter method*), 324
to_tvm_relax() (*kenning.core.converter.ModelConverter method*), 324
train_model() (*kenning.core.model.ModelWrapper method*), 281
train_subset_len() (*kenning.core.dataset.Dataset method*), 274
train_test_split_representations() (*kenning.core.dataset.Dataset method*), 274
transmit() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 233
transmit() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 244
transmit_blocking() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 233
transmit_blocking() (*kenning.protocols.kenning_protocol.KenningProtocol method*), 244

U

update_automl_defaults() (*kenning.core.automl.AutoMLModel static method*), 294
update_automl_range() (*kenning.core.automl.AutoMLModel class method*), 294
update_measurements() (*kenning.core.measurements.Measurements method*), 308
upload_input() (*kenning.core.protocol.Protocol method*), 304
upload_input() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 234
upload_input() (*kenning.protocols.ros2.ROS2Protocol method*), 252
upload_io_specification() (*kenning.core.protocol.Protocol method*), 304
upload_io_specification() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 234
upload_io_specification() (*kenning.protocols.ros2.ROS2Protocol method*), 252
upload_model() (*kenning.core.protocol.Protocol method*), 304

upload_model() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 234
upload_model() (*kenning.protocols.ros2.ROS2Protocol method*), 252
upload_optimizers() (*kenning.core.protocol.Protocol method*), 305
upload_optimizers() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 234
upload_optimizers() (*kenning.protocols.ros2.ROS2Protocol method*), 252
upload_output() (*kenning.core.runtime.Runtime method*), 291
upload_runtime() (*kenning.core.protocol.Protocol method*), 305
upload_runtime() (*kenning.protocols.bytes_based_protocol.BytesBasedProtocol method*), 235
upload_runtime() (*kenning.protocols.ros2.ROS2Protocol method*), 253
upload_stats() (*kenning.core.runtime.Runtime method*), 292
uri (*kenning.utils.resource_manager.ResourceURI property*), 321

V

val_subset_len() (*kenning.core.dataset.Dataset method*), 275
validate_resources_version() (*kenning.utils.resource_manager.ResourceManager method*), 320
verify_dataset_checksum() (*kenning.core.dataset.Dataset method*), 275
verify_type() (*kenning.utils.args_manager.ArgumentsHandler class method*), 319

W

with_name() (*kenning.utils.resource_manager.ResourceURI method*), 321
with_stem() (*kenning.utils.resource_manager.ResourceURI method*), 321
with_suffix() (*kenning.utils.resource_manager.ResourceURI method*), 321

Z

zpl_prepare_cmd_flags() (*kenning.core.optimizer.Optimizer method*), 286